

**TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN**



**ĐỒ ÁN MÔN HỌC
CÔNG NGHỆ PHẦN MỀM
(MSHP: 220055)**

**TÊN ĐỀ TÀI
XÂY DỰNG WEBSITE QUẢN LÝ ĐỒ ÁN SINH VIÊN**

Giảng viên hướng dẫn:

TS. Nguyễn Bảo Ân

Sinh viên thực hiện:

Phan Hoàng Dinh - 110123006 - DA23TTA

Huỳnh Dương Tấn Phát - 110123038 - DA23TTA

Thạch Thị Ngọc Trâm - 110123057 - DA23TTB

Vĩnh Long, tháng 8 năm 2026

[illegible]

Giáo viên hướng dẫn
(Ký tên và ghi rõ họ tên)

2

This image shows a full page of primary-ruled paper. It features approximately 28 horizontal dotted lines spaced evenly down the page, providing a guide for handwriting practice. The paper is otherwise blank, with no margins, text, or other markings.

Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến TS. Nguyễn Bảo Ân, giảng viên hướng dẫn học phần Công nghệ phần mềm, đã tận tình hướng dẫn, hỗ trợ và đóng góp nhiều ý kiến chuyên môn quý báu trong suốt quá trình thực hiện đề tài “Xây dựng website quản lý đồ án sinh viên”.

Nhờ sự hướng dẫn tận tâm của Thầy, nhóm đã có thể từng bước hoàn thiện đề tài từ quá trình khảo sát và phân tích yêu cầu, thiết kế hệ thống, xây dựng cơ sở dữ liệu, phát triển giao diện và các chức năng nghiệp vụ, đến kiểm thử và triển khai sản phẩm. Những nhận xét và góp ý của Thầy đã giúp nhóm kịp thời nhận ra các hạn chế, khắc phục những thiếu sót và nâng cao chất lượng của hệ thống.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ củng cố được những kiến thức đã học trong môn Công nghệ phần mềm mà còn tích lũy thêm nhiều kinh nghiệm thực tế về phân tích, thiết kế, lập trình, kiểm thử, triển khai hệ thống và phối hợp làm việc nhóm. Đây là những kiến thức và kỹ năng rất cần thiết đối với các thành viên trong quá trình học tập cũng như công việc sau này.

Nhóm chúng em cũng xin chân thành cảm ơn Đại học Trà Vinh, Trường Kỹ thuật và Công nghệ và Khoa Công nghệ Thông tin đã tạo điều kiện thuận lợi về môi trường học tập, cơ sở vật chất, kiến thức chuyên môn để nhóm có thể nghiên cứu và hoàn thành đề tài.

Bên cạnh đó, nhóm xin cảm ơn các bạn sinh viên, bạn bè và những người đã tham gia trải nghiệm, kiểm thử và đóng góp ý kiến cho hệ thống. Những phản hồi thực tế đã giúp nhóm điều chỉnh giao diện, hoàn thiện chức năng và nâng cao tính phù hợp của sản phẩm đối với nhu cầu quản lý đồ án sinh viên.

Mặc dù nhóm đã nỗ lực trong quá trình thực hiện, nhưng do thời gian, kiến thức và kinh nghiệm thực tế còn hạn chế nên báo cáo và sản phẩm khó tránh khỏi những thiếu sót. Nhóm rất mong tiếp tục nhận được những ý kiến nhận xét và đóng góp từ quý Thầy, Cô để đề tài được hoàn thiện hơn.

Một lần nữa, nhóm chúng em xin chân thành cảm ơn!

Ký tên

Sinh viên 1

Sinh viên 2

Sinh viên 3

Phan Hoàng Dinh

Huỳnh Dương Tấn Phát

Thạch Thị Ngọc Trâm

MỤC LỤC

Danh mục	Trang
LỜI CẢM ƠN	4
MỤC LỤC.....	6
CHƯƠNG 1: TỔNG QUAN.....	16
1.1 Đặt vấn đề	16
1.2 Cơ sở hình thành đề tài.....	17
1.3 Mục tiêu đề tài.....	18
1.4 Phạm vi nghiên cứu.....	20
1.4.1 Về chức năng hệ thống.....	20
1.4.2 Về công nghệ áp dụng.....	21
1.4.3 Về đối tượng sử dụng.....	22
1.4.4 Về thời gian thực hiện.....	23
1.4.5 Về tài nguyên thực hiện	24
1.5 Đối tượng nghiên cứu.....	24
1.5.1 Người dùng hệ thống.....	25
1.5.2 Quy trình quản lý đồ án sinh viên.....	25
1.5.3 Hệ thống phần mềm.....	26
1.5.4 Dữ liệu của hệ thống.....	27
1.5.5 Công nghệ và công cụ	28
1.6 Phương pháp nghiên cứu.....	29
1.6.1 Phương pháp khảo sát và phân tích yêu cầu.....	29
1.6.2 Phương pháp phát triển linh hoạt.....	29
1.6.3 Phương pháp thiết kế nguyên mẫu (Prototyping)	30
1.6.4 Phương pháp kiểm thử phần mềm (Testing)	31
1.6.5 Phương pháp kiểm thử phần mềm.....	32
1.6.6 Phương pháp triển khai và tích hợp liên tục.....	32
1.6.7 Phương pháp đánh giá và cải tiến.....	33
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	34
2.1 Các khái niệm cơ bản về công nghệ phần mềm	34
2.1.1 Khái niệm công nghệ phần mềm	34
2.1.2 Khái niệm phần mềm.....	35
2.1.3 Phát triển phần mềm.....	36
2.2 Các mô hình phát triển phần mềm.....	36
2.2.1 Mô hình thác nước.....	37
2.2.2 Mô hình chữ V.....	37
2.2.3 Mô hình gia tăng.....	38
2.2.4 Mô hình Agile.....	38

2.3 Phương pháp phát triển phần mềm Agile và Scrum.....	39
2.3.1 Khái niệm Agile	39
2.3.2 Đặc trưng của Agile.....	40
2.3.3 Các nguyên tắc của Agile.....	41
2.3.4 Ưu điểm và hạn chế của Agile	42
2.3.5 Các phương pháp Agile phổ biến	44
2.3.6 Phương pháp Scrum	44
2.4 Kiến trúc hệ thống web.....	46
2.4.1 Mô hình Client–Server.....	46
2.4.2 Kiến trúc frontend và backend tách biệt.....	48
2.4.3 Kiến trúc phân tầng của backend	49
2.4.4 RESTful API.....	50
2.4.5 Ứng dụng web một trang.....	51
2.4.6 Kiến trúc xử lý thông báo.....	52
2.5 Xác thực và phân quyền người dùng.....	53
2.5.1 Xác thực người dùng.....	53
2.5.2 Xác thực bằng JSON Web Token	54
2.5.3 Phân quyền theo vai trò.....	55
2.6 Cơ sở dữ liệu quan hệ và Microsoft SQL Server.....	56
2.6.1 Cơ sở dữ liệu quan hệ.....	56
2.6.2 Microsoft SQL Server	57
2.6.3 Tính toán vẹn và transaction	58
2.7 Hệ thống hàng đợi và xử lý bất đồng bộ	59
2.7.1 Xử lý đồng bộ và bất đồng bộ.....	60
2.7.2 RabbitMQ.....	60
2.7.3 Notification Worker và cơ chế Transactional Outbox	61
2.8 Docker và Docker Compose.....	62
2.8.1 Khái niệm Docker	62
2.8.2 Docker Image và Container	63
2.8.3 Docker Compose trong hệ thống.....	64
2.9 Tích hợp và triển khai liên tục.....	65
2.9.1 Khái niệm CI/CD	66
2.9.2 GitHub Actions	67
2.9.3 Quy trình CI/CD của hệ thống	67
2.10 Kiểm thử phần mềm	68
2.10.1 Kiểm thử đơn vị	69
2.10.2 Kiểm thử tích hợp và kiểm thử API.....	69
2.10.3 Kiểm thử giao diện và kiểm thử phân quyền	70
2.11 Swagger và tài liệu API.....	71

2.11.1 Khái niệm Swagger và OpenAPI	72
2.11.2 Ứng dụng Swagger trong hệ thống	72
2.11.3 Lợi ích của tài liệu API	73
2.12 Tổng kết chương.....	74
CHƯƠNG 3: PHÂN TÍCH YÊU CẦU.....	76
3.1 Xác định nhu cầu.....	76
3.2 Các chức năng của hệ thống.....	77
3.2.1 Các chức năng chính của hệ thống.....	77
3.3.2 Các yêu cầu phi chức năng.....	80
3.3 Quản lý dự án.....	83
3.3.1 Kế hoạch để phân tích và thiết kế giao diện	84
3.3.2 Kế hoạch xây dựng backend và RESTful API	85
3.3.3 Kế hoạch phát triển giao diện người dùng	87
3.3.4 Kế hoạch kiểm thử hệ thống	89
3.3.5 Kế hoạch triển khai hệ thống.....	91
3.3.6 Kế hoạch quản lý rủi ro	93
3.3.7 Kế hoạch tiến độ thực hiện dự án.....	95
3.4 Tổng kết chương.....	97
CHƯƠNG 4: THIẾT KẾ HỆ THỐNG	98
4.1 Kiến trúc tổng thể của hệ thống.....	98
4.1.1 Kiến trúc frontend	99
4.1.2 Kiến trúc backend	99
4.2 Xác định tác nhân của hệ thống.....	100
4.2.1 Quản trị viên.....	101
4.2.2 Giảng viên.....	101
4.2.3 Sinh viên	102
4.3 Thiết kế sơ đồ ca sử dụng.....	103
4.3.1 Sơ đồ ca sử dụng tổng quát	103
4.3.2 Sơ đồ ca sử dụng của quản trị viên	105
4.3.3 Sơ đồ ca sử dụng của giảng viên.....	107
4.3.4 Sơ đồ ca sử dụng của sinh viên	110
4.4 Thiết kế sơ đồ hoạt động	113
4.4.1 Sơ đồ hoạt động đăng nhập	113
4.4.2 Sơ đồ hoạt động đăng ký và xét duyệt đề tài.....	115
4.4.3 Sơ đồ hoạt động tạo yêu cầu nộp bài.....	117
4.4.4 Sơ đồ hoạt động nộp bài.....	119
4.4.5 Sơ đồ hoạt động chấm điểm và công bố kết quả.....	121
4.4.6 Sơ đồ hoạt động xử lý thông báo.....	124
4.5 Thiết kế sơ đồ tuần tự	126

4.5.1 Sơ đồ tuần tự đăng nhập hệ thống.....	126
4.5.2 Sơ đồ tuần tự đăng ký đề tài.....	128
4.5.3 Sơ đồ tuần tự xét duyệt đề tài.....	130
4.5.4 Sơ đồ tuần tự nộp bài.....	131
4.5.5 Sơ đồ tuần tự chấm điểm và công bố kết quả.....	133
4.6 Thiết kế cơ sở dữ liệu	135
4.6.1 Xác định các bảng dữ liệu chính	136
4.6.2 Thiết kế mối quan hệ giữa các bảng.....	137
4.6.3 Thiết kế bảng người dùng.....	140
4.6.4 Thiết kế bảng môn học và lớp học phần.....	141
4.6.5 Thiết kế bảng sinh viên trong lớp và nhóm sinh viên.....	142
4.6.6 Thiết kế các bảng quản lý đề tài	144
4.6.7 Thiết kế bảng tiến độ và yêu cầu nộp bài	147
4.6.8 Thiết kế bảng bài nộp và lịch sử nộp.....	149
4.6.9 Thiết kế bảng điểm và phản hồi	151
4.6.10 Thiết kế bảng thông báo và sự kiện Outbox.....	152
4.6.11 Thiết kế ràng buộc và chỉ mục cơ sở dữ liệu.....	154
4.7 Thiết kế RESTful API	156
4.7.1 Nguyên tắc thiết kế API	157
4.7.2 Xác thực và phân quyền API.....	158
4.7.3 Kiểm tra dữ liệu đầu vào	158
4.7.4 Thiết kế dữ liệu phản hồi.....	159
4.7.5 Mã trạng thái HTTP	159
4.7.6 Tài liệu hóa và kiểm thử API	160
4.8 Thiết kế giao diện người dùng.....	161
4.8.1 Nguyên tắc thiết kế giao diện.....	161
4.8.2 Thiết kế bố cục chung	162
4.8.3 Thiết kế giao diện đăng nhập	163
4.8.4 Thiết kế giao diện dành cho quản trị viên	164
4.8.4.1 Giao diện quản lý tài khoản.....	165
4.8.4.2 Giao diện quản lý môn học.....	166
4.8.4.3 Giao diện quản lý lớp học phần.....	166
4.8.5 Thiết kế giao diện dành cho giảng viên.....	167
4.8.5.1 Giao diện danh sách lớp phụ trách	167
4.8.5.2 Giao diện quản lý đề tài.....	168
4.8.5.3 Giao diện quản lý yêu cầu và bài nộp	169
4.8.6 Thiết kế giao diện dành cho sinh viên.....	170
4.8.6.1 Giao diện lớp học phần và nhóm sinh viên	170
4.8.6.2 Giao diện đăng ký đề tài.....	171

4.8.6.3 Giao diện yêu cầu và nộp bài	172
4.8.6.4 Giao diện xem điểm và phản hồi.....	173
4.8.7 Thiết kế giao diện thông báo	174
4.8.8 Thiết kế giao diện responsive và trạng thái hệ thống	175
4.9 Thiết kế triển khai hệ thống.....	177
4.9.1 Thiết kế môi trường triển khai	178
4.9.2 Thiết kế triển khai bằng Docker và Docker Compose.....	180
4.9.3 Thiết kế quy trình CI/CD bằng GitHub Actions	182
4.10 Tổng kết chương.....	184
CHƯƠNG 5: TRIỂN KHAI VÀ CÔNG NGHỆ SỬ DỤNG.....	186
5.1 Các công nghệ sử dụng	186
5.2 Xây dựng backend.....	188
5.2.1 Xây dựng tầng Routes và Middleware.....	188
5.2.2 Xây dựng tầng Controllers	188
5.2.3 Xây dựng tầng Services và Repositories.....	189
5.2.4 Xây dựng chức năng xác thực và phân quyền.....	191
5.2.5 Xây dựng và tài liệu hóa RESTful API	192
5.3 Xây dựng frontend.....	194
5.3.1 Xây dựng layout và điều hướng.....	194
5.3.2 Xây dựng giao diện dành cho quản trị viên.....	195
5.3.3 Xây dựng giao diện dành cho giảng viên	196
5.3.4 Xây dựng giao diện dành cho sinh viên	198
5.3.5 Xử lý trạng thái và lỗi trên giao diện.....	199
5.4 Xây dựng cơ sở dữ liệu.....	201
5.4.1 Khởi tạo cấu trúc và dữ liệu	201
5.4.2 Kết nối backend với SQL Server	203
5.5 Xây dựng chức năng thông báo bất đồng bộ.....	204
5.5.1 Xây dựng cơ chế Transactional Outbox.....	204
5.5.2 Tích hợp RabbitMQ	206
5.5.3 Xây dựng Notification Worker	207
5.6 Kiểm thử hệ thống.....	208
5.6.1 Kiểm thử backend và RESTful API	210
5.6.2 Kiểm thử giao diện và phân quyền người dùng.....	211
5.6.3 Kiểm thử dữ liệu và tính toàn vẹn cơ sở dữ liệu	213
5.6.4 Kiểm thử cơ chế thông báo bất đồng bộ.....	215
5.7 Triển khai hệ thống.....	217
5.7.1 Triển khai trong môi trường local bằng Docker Compose.....	218
5.7.2 Triển khai hệ thống lên môi trường cloud.....	219
5.7.3 Triển khai tự động bằng GitHub Actions	221

5.7.4 Kiểm tra hệ thống sau khi triển khai	223
5.8 Đánh giá kết quả xây dựng hệ thống.....	225
5.8.1 Kết quả đạt được	226
5.8.2 Hạn chế của hệ thống	228
5.8.3 Hướng phát triển	230
5.9 Tổng kết chương.....	231
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	232
6.1 Kết luận	232
6.2 Kết quả đạt được.....	233
6.2.1 Kết quả về mặt chức năng.....	233
6.2.2 Kết quả về mặt kỹ thuật	235
6.2.3 Kết quả về khả năng ứng dụng.....	236
6.3 Hạn chế của đề tài.....	236
6.4 Hướng phát triển.....	237
6.4.1 Hoàn thiện quy trình quản lý đồ án	237
6.4.2 Phát triển chức năng lưu trữ tệp	238
6.4.3 Mở rộng hệ thống thông báo	238
6.4.4 Xây dựng chức năng thống kê và cảnh báo.....	238
6.4.5 Nâng cao khả năng kiểm thử và bảo mật	239
6.4.6 Hoàn thiện giám sát và vận hành	239
6.4.7 Nghiên cứu tích hợp trí tuệ nhân tạo	240
6.5 Tổng kết chương.....	240
TÀI LIỆU THAM KHẢO.....	242
PHỤ LỤC.....	244

DANH MỤC HÌNH ẢNH

Danh mục	Trang
Hình 2.1 Quy trình phát triển phần mềm	35
Hình 2.2 Mô hình thác nước.....	37
Hình 2.3. Quy trình Scrum	45
Hình 2.4. Mô hình Client–Server của hệ thống	48
Hình 2.5. Kiến trúc phân tầng của backend	49
Hình 2.6. Kiến trúc xử lý thông báo của hệ thống	53
Hình 2.7. Quy trình xác thực bằng JWT	55
Hình 2.8. Mối quan hệ dữ liệu tổng quát của hệ thống	59
Hình 2.9. Mô hình truyền nhận thông điệp qua RabbitMQ	62
Hình 2.10. Kiến trúc Docker của hệ thống	65
Hình 2.11. Quy trình CI/CD của hệ thống.....	68
Hình 2.12. Quy trình kiểm tra API bằng Swagger.....	73
Hình 4.1. Kiến trúc tổng thể của hệ thống	100
Hình 4.2. Sơ đồ ca sử dụng tổng quát của hệ thống	104
Hình 4.3. Sơ đồ ca sử dụng của quản trị viên	106
Hình 4.4. Sơ đồ ca sử dụng của giảng viên	109
Hình 4.5. Sơ đồ ca sử dụng của sinh viên	112
Hình 4.6. Sơ đồ hoạt động đăng nhập hệ thống.....	115
Hình 4.7. Sơ đồ hoạt động đăng ký và xét duyệt đề tài	117
Hình 4.8. Sơ đồ hoạt động tạo yêu cầu nộp bài	119
Hình 4.9. Sơ đồ hoạt động nộp bài	121
Hình 4.10. Sơ đồ hoạt động chấm điểm và công bố kết quả	123
Hình 4.11. Sơ đồ hoạt động xử lý thông báo bất đồng bộ	125
Hình 4.12. Sơ đồ tuần tự đăng nhập hệ thống	128
Hình 4.13. Sơ đồ tuần tự đăng ký đề tài.....	130
Hình 4.14. Sơ đồ tuần tự xét duyệt đề tài.....	131
Hình 4.15. Sơ đồ tuần tự nộp bài.....	133
Hình 4.16. Sơ đồ tuần tự chấm điểm và công bố kết quả.....	135
Hình 4.17. Sơ đồ quan hệ cơ sở dữ liệu của hệ thống	139
Hình 4.18. Bố cục chung của giao diện hệ thống	163
Hình 4.19. Giao diện đăng nhập hệ thống.....	164
Hình 4.20. Giao diện quản lý tài khoản người dùng.....	166
Hình 4.21. Giao diện quản lý lớp học phần	167
Hình 4.22. Giao diện danh sách lớp học phần giảng viên phụ trách	168
Hình 4.23. Giao diện xét duyệt đề tài của giảng viên	169
Hình 4.24. Giao diện quản lý bài nộp và chấm điểm.....	170
Hình 4.25. Giao diện lớp học phần và nhóm sinh viên	171
Hình 4.26. Giao diện đăng ký và theo dõi trạng thái đề tài.....	172

Hình 4.27. Giao diện yêu cầu nộp bài và lịch sử nộp	173
Hình 4.28. Giao diện xem điểm và phản hồi của giảng viên	174
Hình 4.29. Giao diện danh sách thông báo	175
Hình 4.30. Giao diện hiển thị trên các kích thước màn hình khác nhau	177
Hình 4.31. Mô hình triển khai các thành phần của hệ thống.....	179
Hình 4.32. Mô hình các dịch vụ trong Docker Compose.....	181
Hình 4.33. Quy trình CI/CD của hệ thống.....	184
Hình 5.1. Cấu trúc thư mục của dự án.....	187
Hình 5.2. Cấu trúc phân tầng của backend	190
Hình 5.3. Mã nguồn middleware xác thực và phân quyền.....	192
Hình 5.4. Giao diện tài liệu RESTful API bằng Swagger	193
Hình 5.5. Cấu trúc layout và điều hướng của frontend	195
Hình 5.6. Giao diện quản lý người dùng của quản trị viên	196
Hình 5.7. Giao diện quản lý bài nộp và chấm điểm của giảng viên	197
Hình 5.8. Giao diện nộp bài và theo dõi lịch sử nộp của sinh viên	199
Hình 5.9. Giao diện xử lý trạng thái dữ liệu và thông báo lỗi.....	201
Hình 5.10. Danh sách các bảng trong cơ sở dữ liệu SQL Server	202
Hình 5.11. Kết quả kết nối backend với SQL Server	204
Hình 5.12. Dữ liệu sự kiện trong bảng NotificationOutbox.....	205
Hình 5.13. Giao diện quản lý queue trên RabbitMQ.....	207
Hình 5.14. Kết quả khởi động Notification Worker	208
Hình 5.15. Kết quả chạy bộ kiểm thử backend	211
Hình 5.16. Kết quả kiểm thử chức năng theo vai trò người dùng.....	213
Hình 5.17. Kết quả kiểm tra dữ liệu trong SQL Server.....	215
Hình 5.18. Kết quả xử lý thông báo từ RabbitMQ đến người dùng.....	217
Hình 5.19. Trạng thái các dịch vụ trong Docker Compose	219
Hình 5.20. Trạng thái các dịch vụ được triển khai trên môi trường cloud.....	221
Hình 5.21. Kết quả thực hiện quy trình CI/CD trên GitHub Actions.....	222
Hình 5.22. Kết quả kiểm tra hệ thống sau khi triển khai	225
Hình 5.23. Giao diện tổng thể của hệ thống sau khi hoàn thiện.....	228

DANH MỤC BẢNG BIỂU

Danh mục	Trang
<i>Bảng 2.1. So sánh một số mô hình phát triển phần mềm</i>	39
<i>Bảng 2.2. Ưu điểm và hạn chế của Agile</i>	43
<i>Bảng 2.3. Chức năng của các tầng trong backend</i>	50
<i>Bảng 2.4. Các phương thức HTTP thường được sử dụng</i>	51
<i>Bảng 2.5. Phạm vi quyền của các vai trò người dùng</i>	56
<i>Bảng 2.6. Các dịch vụ được tổ chức bằng Docker</i>	64
<i>Bảng 2.7. Các hình thức kiểm thử được áp dụng</i>	71
<i>Bảng 3.1. Tổng hợp các yêu cầu phi chức năng</i>	83
<i>Bảng 3.2. Kế hoạch phân tích yêu cầu và thiết kế hệ thống</i>	85
<i>Bảng 3.3. Kế hoạch xây dựng backend và RESTful API</i>	86
<i>Bảng 3.4. Kế hoạch phát triển giao diện người dùng</i>	88
<i>Bảng 3.5. Kế hoạch kiểm thử hệ thống</i>	90
<i>Bảng 3.6. Kế hoạch triển khai hệ thống</i>	92
<i>Bảng 3.7. Kế hoạch quản lý rủi ro</i>	94
<i>Bảng 4.1. Các ca sử dụng chính của quản trị viên</i>	106
<i>Bảng 4.2. Các ca sử dụng chính của giảng viên</i>	109
<i>Bảng 4.3. Các ca sử dụng chính của sinh viên</i>	112
<i>Bảng 4.4. Danh sách các bảng dữ liệu chính</i>	136
<i>Bảng 4.5. Cấu trúc bảng Users</i>	140
<i>Bảng 4.6. Cấu trúc bảng Courses</i>	141
<i>Bảng 4.7. Cấu trúc bảng CourseClasses</i>	142
<i>Bảng 4.8. Cấu trúc bảng CourseClassEnrollments</i>	143
<i>Bảng 4.9. Cấu trúc bảng StudentGroups</i>	143
<i>Bảng 4.10. Cấu trúc bảng StudentGroupMembers</i>	144
<i>Bảng 4.11. Cấu trúc bảng Topics</i>	145
<i>Bảng 4.12. Cấu trúc bảng TopicRegistrations</i>	145
<i>Bảng 4.13. Cấu trúc bảng TopicReviews</i>	146
<i>Bảng 4.14. Cấu trúc bảng ProgressMilestones</i>	147
<i>Bảng 4.15. Cấu trúc bảng SubmissionRequirements</i>	148
<i>Bảng 4.16. Cấu trúc bảng AssignmentSubmissions</i>	149
<i>Bảng 4.17. Cấu trúc bảng SubmissionAttempts</i>	150
<i>Bảng 4.18. Cấu trúc bảng Grades</i>	151
<i>Bảng 4.19. Cấu trúc bảng Notifications</i>	153
<i>Bảng 4.21. Các ràng buộc và chỉ mục quan trọng</i>	155
<i>Bảng 4.22. Các phương thức HTTP được sử dụng trong hệ thống</i>	157
<i>Bảng 4.23. Các mã trạng thái HTTP được sử dụng trong hệ thống</i>	159
<i>Bảng 4.24. Các thành phần triển khai bằng Docker</i>	181
<i>Bảng 4.25. Các bước chính trong quy trình CI/CD</i>	183

<i>Bảng 5.1. Môi trường và công cụ phát triển hệ thống</i>	<i>186</i>
<i>Bảng 5.2. Cách xử lý các trạng thái trên frontend.....</i>	<i>200</i>
<i>Bảng 5.3. Các nhóm kiểm thử được thực hiện</i>	<i>209</i>
<i>Bảng 5.4. Kết quả kiểm thử theo vai trò</i>	<i>212</i>
<i>Bảng 5.5. Nội dung kiểm tra tính toàn vẹn dữ liệu</i>	<i>214</i>
<i>Bảng 5.6. Kết quả kiểm thử chức năng thông báo</i>	<i>216</i>
<i>Bảng 5.7. Trạng thái các dịch vụ trong môi trường Docker Compose</i>	<i>218</i>
<i>Bảng 5.8. Các thành phần được triển khai trên môi trường cloud</i>	<i>220</i>
<i>Bảng 5.9. Nội dung kiểm tra hệ thống sau triển khai</i>	<i>224</i>
<i>Bảng 5.10. Tổng hợp kết quả đạt được của hệ thống</i>	<i>227</i>
<i>Bảng 5.11. Các hạn chế hiện tại của hệ thống</i>	<i>229</i>

CHƯƠNG 1: TỔNG QUAN

1.1 Đặt vấn đề

Trong quá trình đào tạo tại các trường đại học, đồ án môn học và đồ án chuyên ngành giữ vai trò quan trọng trong việc giúp sinh viên vận dụng kiến thức lý thuyết vào thực tiễn. Thông qua quá trình thực hiện đồ án, sinh viên có cơ hội rèn luyện kỹ năng phân tích yêu cầu, thiết kế hệ thống, lập trình, kiểm thử, viết báo cáo, làm việc nhóm và trình bày sản phẩm.

Tuy nhiên, việc quản lý đồ án sinh viên tại nhiều lớp học phần hiện nay vẫn còn được thực hiện bằng các phương pháp thủ công hoặc thông qua nhiều công cụ riêng lẻ như bảng tính, thư điện tử, nhóm trò chuyện và dịch vụ lưu trữ trực tuyến. Giảng viên thường phải tổng hợp danh sách sinh viên, nhóm thực hiện, đề tài đăng ký, tiến độ, bài nộp và điểm số từ nhiều nguồn khác nhau. Điều này làm mất nhiều thời gian, khó kiểm soát dữ liệu và dễ xảy ra sai sót trong quá trình theo dõi.

Đối với sinh viên, việc đăng ký đề tài, theo dõi yêu cầu của giảng viên, nộp bài, nộp lại sản phẩm, xem phản hồi và kiểm tra điểm số cũng gặp nhiều bất tiện nếu không có một hệ thống quản lý tập trung. Sinh viên có thể bỏ lỡ thời hạn nộp bài, không nắm rõ trạng thái đề tài hoặc khó theo dõi lịch sử các lần nộp.

Bên cạnh đó, việc lưu trữ dữ liệu bằng nhiều công cụ khác nhau khiến thông tin bị phân tán, khó tìm kiếm và không đảm bảo sự nhất quán. Khi số lượng sinh viên, lớp học phần và đồ án tăng lên, công tác quản lý càng trở nên phức tạp. Vì vậy, cần có một hệ thống phần mềm hỗ trợ quản lý tập trung toàn bộ quy trình thực hiện đồ án.

Từ thực tế trên, nhóm lựa chọn đề tài “Xây dựng website quản lý đồ án sinh viên”. Hệ thống được xây dựng nhằm hỗ trợ quản trị viên, giảng viên và sinh viên quản lý các hoạt động liên quan đến đồ án trên một nền tảng thống nhất. Website cho phép quản lý người dùng, môn học, lớp học phần, nhóm sinh viên, đăng ký và duyệt đề tài, tiến độ thực hiện, yêu cầu nộp bài, bài nộp, điểm số, phản hồi và thông báo.

Ngoài việc giải quyết nhu cầu thực tế, đề tài còn giúp nhóm vận dụng các kiến thức của học phần Công nghệ phần mềm vào một sản phẩm hoàn chỉnh. Quá trình thực hiện đề

tài bao gồm các giai đoạn khảo sát, phân tích yêu cầu, thiết kế hệ thống, xây dựng cơ sở dữ liệu, phát triển frontend và backend, kiểm thử, đóng gói bằng Docker và triển khai trên môi trường cloud.

1.2 Cơ sở hình thành đề tài

Trong môi trường đại học, mỗi học kỳ có thể có nhiều lớp học phân tổ chức thực hiện đồ án. Một giảng viên có thể phụ trách nhiều lớp, trong khi một sinh viên có thể tham gia nhiều học phần khác nhau. Mỗi lớp có danh sách sinh viên, nhóm thực hiện, đề tài, mốc tiến độ, yêu cầu nộp bài và phương thức đánh giá riêng.

Nếu quản lý bằng bảng tính hoặc tin nhắn, giảng viên phải thực hiện nhiều thao tác thủ công như:

- Tổng hợp danh sách sinh viên theo từng lớp học phần.
- Theo dõi thành viên của từng nhóm.
- Kiểm tra đề tài đã đăng ký hay chưa.
- Duyệt, từ chối hoặc yêu cầu chỉnh sửa đề tài.
- Tạo các mốc tiến độ và yêu cầu nộp bài.
- Kiểm tra số lần nộp và thời gian nộp.
- Chấm điểm và gửi phản hồi.
- Thông báo cho sinh viên khi có thay đổi.

Các công việc trên có mối quan hệ chặt chẽ với nhau nhưng thường bị phân tán trên nhiều công cụ. Việc này gây khó khăn trong quá trình tra cứu, đối chiếu và bảo đảm dữ liệu đúng theo từng lớp học phần.

Về phía sinh viên, nhu cầu sử dụng một hệ thống tập trung cũng rất cần thiết. Sinh viên cần biết mình đang tham gia lớp học phần nào, thuộc nhóm nào, đề tài đã được duyệt hay chưa, cần nộp những nội dung gì, hạn nộp là khi nào và kết quả đánh giá ra sao.

Từ những nhu cầu đó, nhóm nhận thấy việc xây dựng một website quản lý đồ án sinh viên là cần thiết. Hệ thống giúp số hóa quy trình quản lý, giảm thao tác thủ công, tăng tính minh bạch và hỗ trợ giảng viên theo dõi tiến độ của sinh viên một cách thuận tiện hơn.

Đồng thời, đề tài phù hợp với yêu cầu của học phần Công nghệ phần mềm vì có đầy đủ các thành phần của một hệ thống thực tế, bao gồm giao diện người dùng, RESTful API, xác thực và phân quyền, cơ sở dữ liệu, kiểm thử, tài liệu API, Docker và quy trình triển khai tự động.

1.3 Mục tiêu đề tài

Đề tài “Xây dựng website quản lý đồ án sinh viên” được thực hiện với mục tiêu xây dựng một hệ thống quản lý tập trung, hỗ trợ toàn bộ quá trình tổ chức, thực hiện và đánh giá đồ án trong các lớp học phần. Hệ thống hướng đến việc thay thế các phương pháp quản lý thủ công, giảm tình trạng dữ liệu bị phân tán giữa bảng tính, thư điện tử, nhóm trò chuyện và các dịch vụ lưu trữ riêng lẻ.

Website được xây dựng để phục vụ ba nhóm người dùng chính gồm quản trị viên, giảng viên và sinh viên. Mỗi nhóm người dùng được cung cấp các chức năng và quyền truy cập phù hợp với nhiệm vụ của mình. Quản trị viên chịu trách nhiệm quản lý tài khoản, môn học, lớp học phần và dữ liệu chung của hệ thống. Giảng viên có thể quản lý các lớp đang phụ trách, theo dõi sinh viên, nhóm thực hiện, đăng ký đề tài, tiến độ, bài nộp và kết quả đánh giá. Sinh viên có thể theo dõi lớp học phần, tham gia nhóm, đăng ký đề tài, nộp bài, xem phản hồi và kiểm tra điểm số.

Một trong những mục tiêu quan trọng của đề tài là hỗ trợ quản lý xuyên suốt quy trình thực hiện đồ án. Quy trình bắt đầu từ việc tạo môn học và lớp học phần, phân công giảng viên, thêm sinh viên vào lớp, thành lập nhóm, đăng ký đề tài, duyệt đề tài, theo dõi tiến độ, tổ chức các đợt nộp bài, chấm điểm và gửi phản hồi. Dữ liệu của từng hoạt động được liên kết với đúng lớp học phần, giúp hạn chế nhầm lẫn giữa các lớp, các nhóm và các đề tài khác nhau.

Hệ thống cũng hướng đến việc nâng cao khả năng theo dõi và kiểm soát quá trình thực hiện đồ án. Giảng viên có thể tạo yêu cầu nộp bài, thiết lập thời hạn, giới hạn số lần

nộp, cho phép hoặc không cho phép nộp lại, đồng thời kiểm tra lịch sử các lần nộp của sinh viên. Sinh viên có thể chủ động theo dõi thời hạn, trạng thái đề tài, tiến độ thực hiện và kết quả đánh giá ngay trên hệ thống.

Về mặt kỹ thuật, đề tài đặt mục tiêu xây dựng frontend và backend thành hai dự án riêng biệt, giao tiếp với nhau thông qua RESTful API. Backend được tổ chức theo kiến trúc phân tầng gồm routes, controllers, services và repositories nhằm phân tách rõ trách nhiệm của từng thành phần, tạo điều kiện thuận lợi cho việc bảo trì, kiểm thử và mở rộng hệ thống.

Đề tài sử dụng cơ chế xác thực và phân quyền bằng JSON Web Token để bảo vệ các chức năng và dữ liệu theo từng vai trò người dùng. Mật khẩu được mã hóa trước khi lưu trữ, các API quan trọng được kiểm tra quyền truy cập và dữ liệu được giới hạn theo đúng người dùng, lớp học phần hoặc nhóm sinh viên có liên quan.

Bên cạnh các chức năng nghiệp vụ, hệ thống còn được xây dựng tài liệu API bằng Swagger để hỗ trợ kiểm tra và tra cứu các endpoint. RabbitMQ và notification worker được sử dụng để xử lý các tác vụ thông báo theo hướng bất đồng bộ. Cơ chế transactional outbox giúp hạn chế nguy cơ mất sự kiện khi xảy ra lỗi giữa quá trình ghi dữ liệu và gửi thông báo.

Một mục tiêu khác của đề tài là bảo đảm hệ thống có thể được đóng gói và triển khai trong nhiều môi trường khác nhau. Frontend, backend, cơ sở dữ liệu, RabbitMQ và notification worker được cấu hình bằng Docker và Docker Compose. Việc container hóa giúp giảm sự khác biệt giữa môi trường phát triển, kiểm thử và triển khai, đồng thời giúp quá trình cài đặt hệ thống thuận tiện hơn.

Ngoài ra, nhóm thiết lập quy trình tích hợp và triển khai liên tục bằng GitHub Actions. Khi mã nguồn được cập nhật lên nhánh chính, hệ thống tự động chạy kiểm thử backend, kiểm tra mã nguồn frontend, thực hiện quá trình build và chỉ triển khai lên môi trường cloud khi các bước kiểm tra đều thành công. Qua đó, quy trình phát triển được tự động hóa, hạn chế lỗi và nâng cao tính ổn định của sản phẩm.

Thông qua đề tài, nhóm không chỉ hướng đến việc tạo ra một website đáp ứng nhu cầu quản lý đồ án sinh viên mà còn mong muốn vận dụng đầy đủ kiến thức của học phần Công nghệ phần mềm vào một dự án thực tế. Quá trình thực hiện giúp các thành viên rèn luyện kỹ năng phân tích yêu cầu, thiết kế hệ thống, xây dựng cơ sở dữ liệu, phát triển phần mềm, kiểm thử, quản lý mã nguồn, triển khai cloud và phối hợp làm việc nhóm.

1.4 Phạm vi nghiên cứu

Đề tài “Xây dựng website quản lý đồ án sinh viên” được thực hiện trong phạm vi một hệ thống quản lý đồ án dành cho môi trường đào tạo đại học. Hệ thống tập trung hỗ trợ các hoạt động chính từ khâu quản lý lớp học phần, tổ chức nhóm sinh viên, đăng ký đề tài, theo dõi tiến độ, nộp bài đến đánh giá kết quả. Phạm vi của đề tài được xác định dựa trên chức năng hệ thống, công nghệ áp dụng, đối tượng sử dụng, thời gian thực hiện và các tài nguyên phục vụ quá trình phát triển.

1.4.1 Về chức năng hệ thống

Hệ thống được xây dựng để phục vụ ba nhóm người dùng chính gồm quản trị viên, giảng viên và sinh viên. Mỗi nhóm người dùng có các quyền truy cập và chức năng riêng, phù hợp với vai trò trong quá trình quản lý và thực hiện đồ án.

Đối với quản trị viên, hệ thống hỗ trợ quản lý thông tin người dùng, bao gồm tài khoản quản trị viên, giảng viên và sinh viên. Quản trị viên có thể quản lý môn học, lớp học phần, phân công giảng viên phụ trách, thêm sinh viên vào từng lớp và theo dõi dữ liệu tổng thể của hệ thống. Các chức năng này giúp chuẩn hóa thông tin đầu vào trước khi giảng viên và sinh viên sử dụng hệ thống.

Đối với giảng viên, hệ thống cho phép xem các lớp học phần được phân công phụ trách, quản lý danh sách sinh viên, theo dõi nhóm thực hiện và kiểm tra đăng ký đề tài. Giảng viên có thể duyệt đề tài, từ chối hoặc yêu cầu sinh viên chỉnh sửa nội dung chưa phù hợp. Sau khi đề tài được duyệt, hệ thống khóa nội dung đăng ký nhằm bảo đảm tính thống nhất trong quá trình thực hiện.

Giảng viên còn có thể tạo các mốc tiến độ và yêu cầu nộp bài cho từng lớp học phần. Mỗi yêu cầu có thể được thiết lập thời hạn, trạng thái hoạt động, số lần nộp và các điều

kiện liên quan. Sau khi sinh viên nộp bài, giảng viên có thể xem nội dung, kiểm tra lịch sử nộp, nhập điểm và phản hồi trực tiếp trên hệ thống.

Đối với sinh viên, hệ thống hỗ trợ xem các lớp học phần đang tham gia, theo dõi thông tin nhóm và thành viên, đăng ký đề tài, chỉnh sửa nội dung khi được giảng viên yêu cầu và kiểm tra trạng thái xét duyệt. Sinh viên có thể xem các mốc tiến độ, yêu cầu nộp bài, thời hạn thực hiện và tiến hành nộp hoặc nộp lại bài theo quy định của giảng viên.

Bên cạnh đó, sinh viên có thể xem lịch sử các lần nộp, phản hồi của giảng viên, điểm số đã được công bố và các thông báo liên quan. Việc tập trung các thông tin này trong một hệ thống giúp sinh viên chủ động hơn trong quá trình thực hiện đồ án và hạn chế tình trạng bỏ sót yêu cầu.

Trong phạm vi hiện tại, hệ thống chưa tập trung phát triển ứng dụng dành riêng cho thiết bị di động, chưa hỗ trợ đăng nhập bằng tài khoản Google hoặc các mạng xã hội, chưa tích hợp lưu trữ tệp trên các dịch vụ object storage chuyên dụng và chưa sử dụng trí tuệ nhân tạo để đánh giá tự động nội dung đồ án. Những nội dung này có thể được nghiên cứu và bổ sung trong các giai đoạn phát triển tiếp theo.

1.4.2 Về công nghệ áp dụng

Phần giao diện người dùng của hệ thống được xây dựng bằng ReactJS kết hợp với Vite. Việc sử dụng ReactJS giúp giao diện được tổ chức thành các thành phần độc lập, có khả năng tái sử dụng và dễ dàng mở rộng. Vite hỗ trợ quá trình phát triển và đóng gói ứng dụng nhanh hơn, phù hợp với mô hình frontend tách biệt.

Backend được xây dựng bằng Node.js và ExpressJS, cung cấp các RESTful API để frontend trao đổi dữ liệu. Mã nguồn backend được tổ chức theo kiến trúc phân tầng gồm routes, controllers, services và repositories. Cách tổ chức này giúp tách biệt nhiệm vụ giữa tiếp nhận yêu cầu, xử lý nghiệp vụ và truy xuất dữ liệu, từ đó nâng cao khả năng bảo trì và kiểm thử.

Hệ thống sử dụng Microsoft SQL Server làm hệ quản trị cơ sở dữ liệu. Dữ liệu được tổ chức theo mô hình quan hệ, phù hợp với các đối tượng có liên kết chặt chẽ như

người dùng, môn học, lớp học phân, nhóm sinh viên, đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Cơ chế xác thực được thực hiện bằng JSON Web Token. Sau khi đăng nhập thành công, người dùng nhận được token dùng để truy cập các API được bảo vệ. Hệ thống kiểm tra vai trò của người dùng trước khi cho phép thực hiện chức năng, đồng thời giới hạn dữ liệu theo đúng lớp học phân, nhóm hoặc tài khoản có liên quan. Mật khẩu người dùng được mã hóa trước khi lưu vào cơ sở dữ liệu.

Swagger được sử dụng để mô tả các API của hệ thống, hỗ trợ kiểm tra endpoint, tham số đầu vào và kết quả trả về. Đây cũng là tài liệu giúp quá trình tích hợp frontend với backend trở nên rõ ràng và thuận tiện hơn.

Đối với chức năng thông báo, hệ thống sử dụng RabbitMQ để xử lý các sự kiện theo hướng bất đồng bộ. Notification worker tiếp nhận sự kiện từ hàng đợi và xử lý việc tạo thông báo cho người dùng. Cơ chế transactional outbox được áp dụng nhằm bảo đảm sự kiện được lưu trữ trước khi gửi đến RabbitMQ, qua đó hạn chế nguy cơ mất thông báo khi kết nối gặp sự cố.

Docker và Docker Compose được sử dụng để đóng gói và tổ chức các thành phần của hệ thống. Việc container hóa giúp frontend, backend, SQL Server, RabbitMQ và notification worker có thể chạy trong môi trường thống nhất, giảm sự khác biệt giữa các máy phát triển.

Git và GitHub được sử dụng để quản lý mã nguồn. GitHub Actions được cấu hình để tự động chạy kiểm thử backend, kiểm tra frontend, thực hiện quá trình build và triển khai hệ thống. Frontend, backend và worker được triển khai trên Render; cơ sở dữ liệu SQL Server được đặt trên MonsterASP; RabbitMQ được cung cấp thông qua CloudAMQP.

1.4.3 Về đối tượng sử dụng

Đối tượng sử dụng chính của hệ thống là quản trị viên, giảng viên và sinh viên trong các lớp học phân có tổ chức thực hiện đồ án.

Quản trị viên là người chịu trách nhiệm chuẩn bị và quản lý dữ liệu chung. Nhóm người dùng này cần có kiến thức cơ bản về cấu trúc tổ chức môn học, lớp học phần và tài khoản người dùng để thực hiện việc cấu hình hệ thống.

Giảng viên là người trực tiếp quản lý quá trình thực hiện đồ án. Hệ thống giúp giảng viên giảm thời gian tổng hợp dữ liệu, thuận tiện theo dõi từng lớp, từng nhóm và từng bài nộp. Mỗi giảng viên chỉ được truy cập các lớp học phần thuộc phạm vi phụ trách.

Sinh viên là đối tượng trực tiếp đăng ký đề tài và thực hiện các yêu cầu của đồ án. Sinh viên chỉ được xem và thao tác trên dữ liệu liên quan đến tài khoản, lớp học phần và nhóm của mình. Cách giới hạn này góp phần bảo vệ dữ liệu và tránh nhầm lẫn giữa các lớp học phần.

Trong phạm vi hiện tại, hệ thống được xây dựng chủ yếu để phục vụ nhu cầu học tập và trình diễn trong môi trường Trường Đại học Trà Vinh. Hệ thống chưa được thiết kế để sử dụng ngay cho các tổ chức có quy mô lớn hoặc có quy trình quản lý đồ án quá khác biệt.

1.4.4 Về thời gian thực hiện

Quá trình thực hiện đề tài được chia thành nhiều giai đoạn liên tiếp. Giai đoạn đầu tập trung khảo sát nhu cầu, xác định yêu cầu chức năng, yêu cầu phi chức năng và xây dựng kế hoạch phát triển. Sau đó, nhóm tiến hành thiết kế kiến trúc hệ thống, cơ sở dữ liệu, API và giao diện người dùng.

Giai đoạn phát triển tập trung xây dựng backend, frontend và tích hợp các chức năng theo từng vai trò. Các chức năng được triển khai theo từng nhóm nghiệp vụ, bắt đầu từ xác thực người dùng, quản lý môn học và lớp học phần, sau đó mở rộng sang quản lý nhóm, đăng ký đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Sau khi hoàn thiện các chức năng chính, nhóm tiến hành kiểm thử, sửa lỗi, bổ sung dữ liệu mẫu và cải thiện giao diện. Giai đoạn cuối tập trung vào Docker hóa, thiết lập CI/CD, triển khai hệ thống trên môi trường cloud và hoàn thiện báo cáo.

Thời gian thực hiện đề tài được giới hạn trong thời gian của học phần Công nghệ phần mềm. Vì vậy, nhóm ưu tiên hoàn thành các chức năng cốt lõi và bảo đảm hệ thống có thể chạy ổn định, thay vì mở rộng quá nhiều tính năng nâng cao.

1.4.5 Về tài nguyên thực hiện

Nhóm sử dụng máy tính cá nhân để lập trình, kiểm thử và chạy hệ thống trong môi trường local. Visual Studio Code được sử dụng làm công cụ chỉnh sửa mã nguồn. SQL Server Management Studio hỗ trợ kiểm tra và quản lý cơ sở dữ liệu SQL Server.

Docker Desktop được sử dụng để chạy các container của hệ thống trong quá trình phát triển. Postman và Swagger hỗ trợ kiểm tra RESTful API. GitHub được dùng để lưu trữ mã nguồn, theo dõi lịch sử thay đổi và thực hiện quy trình tích hợp, triển khai tự động.

Về hạ tầng trực tuyến, nhóm tận dụng các gói dịch vụ phù hợp cho mục đích học tập và trình diễn. Render được sử dụng để triển khai frontend, backend và notification worker. MonsterASP cung cấp cơ sở dữ liệu SQL Server trên môi trường cloud, trong khi CloudAMQP cung cấp dịch vụ RabbitMQ.

Các dữ liệu được sử dụng trong hệ thống chủ yếu là dữ liệu mẫu do nhóm xây dựng nhằm phục vụ kiểm thử và trình diễn. Nhóm không sử dụng dữ liệu cá nhân nhạy cảm của sinh viên trong môi trường thực tế. Các tài khoản và mật khẩu truy cập dịch vụ được lưu trong biến môi trường, không được đưa trực tiếp vào mã nguồn hoặc kho lưu trữ GitHub.

Nhìn chung, phạm vi nghiên cứu của đề tài được xác định phù hợp với thời gian, nguồn lực và yêu cầu của học phần. Hệ thống tập trung giải quyết đầy đủ quy trình quản lý đồ án sinh viên ở mức cơ bản và tạo nền tảng để tiếp tục mở rộng trong tương lai.

1.5 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài “Xây dựng website quản lý đồ án sinh viên” bao gồm người dùng tham gia vào quá trình quản lý đồ án, quy trình nghiệp vụ quản lý đồ án, hệ thống phần mềm và các công nghệ được sử dụng để xây dựng, kiểm thử và triển khai hệ thống. Việc xác định rõ các đối tượng nghiên cứu giúp nhóm phân tích đúng nhu cầu, thiết kế chức năng phù hợp và bảo đảm hệ thống đáp ứng được yêu

cầu của từng nhóm người dùng.

1.5.1 Người dùng hệ thống

Đối tượng nghiên cứu đầu tiên là những người trực tiếp sử dụng và tương tác với hệ thống, bao gồm quản trị viên, giảng viên và sinh viên. Mỗi nhóm người dùng có nhiệm vụ, quyền hạn và phạm vi truy cập dữ liệu khác nhau.

Quản trị viên là người chịu trách nhiệm quản lý các dữ liệu nền tảng của hệ thống. Các dữ liệu này bao gồm tài khoản người dùng, thông tin môn học, lớp học phần, giảng viên phụ trách và danh sách sinh viên tham gia lớp. Quản trị viên có quyền truy cập dữ liệu tổng thể nhằm bảo đảm hệ thống được tổ chức và vận hành đúng cấu trúc.

Giảng viên là người trực tiếp quản lý quá trình thực hiện đồ án của sinh viên trong các lớp học phần được phân công. Giảng viên có nhu cầu theo dõi danh sách sinh viên, nhóm thực hiện, đề tài đăng ký, tiến độ, yêu cầu nộp bài, lịch sử nộp và kết quả đánh giá. Vì vậy, hệ thống cần cung cấp cho giảng viên công cụ quản lý tập trung, đồng thời giới hạn dữ liệu đúng theo các lớp học phần mà giảng viên phụ trách.

Sinh viên là đối tượng trực tiếp thực hiện đồ án và sử dụng phần lớn các chức năng liên quan đến đăng ký đề tài, theo dõi tiến độ và nộp bài. Sinh viên cần biết mình đang tham gia lớp học phần nào, thuộc nhóm nào, đề tài đang ở trạng thái nào, có những yêu cầu nộp bài nào và kết quả đánh giá ra sao. Hệ thống chỉ cho phép sinh viên truy cập dữ liệu liên quan đến tài khoản, lớp học phần và nhóm của mình.

Việc nghiên cứu đặc điểm của từng nhóm người dùng là cơ sở để xây dựng cơ chế xác thực, phân quyền và giới hạn dữ liệu. Điều này giúp bảo đảm mỗi người dùng chỉ được thực hiện những chức năng phù hợp với vai trò và trách nhiệm của mình.

1.5.2 Quy trình quản lý đồ án sinh viên

Đối tượng nghiên cứu tiếp theo là quy trình quản lý đồ án sinh viên trong một lớp học phần. Đây là quy trình gồm nhiều hoạt động liên quan chặt chẽ với nhau, bắt đầu từ việc tổ chức lớp cho đến khi hoàn thành đánh giá kết quả.

Quy trình bắt đầu khi quản trị viên tạo môn học, lớp học phần, phân công giảng viên và thêm sinh viên vào lớp. Sau đó, sinh viên được tổ chức thành các nhóm thực hiện. Mỗi nhóm có thể đăng ký đề tài và gửi nội dung đề tài đến giảng viên phụ trách để xem xét.

Giảng viên tiến hành kiểm tra nội dung đăng ký và lựa chọn một trong các trạng thái xử lý như chờ duyệt, yêu cầu chỉnh sửa, từ chối hoặc phê duyệt. Khi đề tài đã được phê duyệt, hệ thống khóa nội dung đăng ký nhằm tránh việc sinh viên tự ý thay đổi đề tài trong quá trình thực hiện.

Trong giai đoạn thực hiện, giảng viên có thể tạo các mốc tiến độ và yêu cầu nộp bài. Sinh viên căn cứ vào nội dung và thời hạn của từng yêu cầu để chuẩn bị sản phẩm, tài liệu hoặc báo cáo cần thiết. Hệ thống lưu lại các lần nộp để giảng viên có thể theo dõi lịch sử và kiểm tra quá trình chỉnh sửa của sinh viên.

Sau khi nhận bài, giảng viên tiến hành đánh giá, nhập điểm và phản hồi. Kết quả chỉ được hiển thị cho sinh viên khi giảng viên công bố. Đồng thời, hệ thống tạo các thông báo liên quan để người dùng kịp thời nắm bắt những thay đổi trong quá trình thực hiện đồ án.

Việc nghiên cứu đầy đủ quy trình này giúp nhóm xác định các chức năng, trạng thái dữ liệu và mối liên hệ giữa các thành phần trong hệ thống. Đây cũng là cơ sở để thiết kế cơ sở dữ liệu và xây dựng các API xử lý nghiệp vụ.

1.5.3 Hệ thống phần mềm

Sản phẩm trung tâm của đề tài là một website quản lý đồ án sinh viên được xây dựng theo mô hình tách biệt giữa frontend và backend. Frontend chịu trách nhiệm cung cấp giao diện và tiếp nhận thao tác của người dùng, trong khi backend xử lý nghiệp vụ, xác thực, phân quyền và truy xuất dữ liệu.

Frontend được thiết kế theo từng vai trò người dùng. Sau khi đăng nhập, quản trị viên, giảng viên và sinh viên sẽ được chuyển đến các khu vực chức năng khác nhau. Dữ liệu hiển thị trên giao diện được lấy từ backend thông qua RESTful API.

Backend được tổ chức theo kiến trúc phân tầng gồm routes, controllers, services và repositories. Mỗi tầng đảm nhiệm một vai trò riêng. Routes định nghĩa đường dẫn API, controllers tiếp nhận yêu cầu, services xử lý nghiệp vụ và repositories thực hiện truy vấn cơ sở dữ liệu. Việc phân tầng giúp hệ thống dễ bảo trì, mở rộng và kiểm thử hơn.

Cơ sở dữ liệu của hệ thống lưu trữ các thông tin như người dùng, vai trò, môn học, lớp học phần, sinh viên tham gia lớp, nhóm, thành viên nhóm, đăng ký đề tài, mốc tiến độ, yêu cầu nộp bài, bài nộp, lịch sử nộp, điểm số và thông báo. Các bảng dữ liệu được liên kết với nhau nhằm bảo đảm tính toàn vẹn và hạn chế dữ liệu bị trùng lặp hoặc sai phạm vi.

Ngoài frontend, backend và cơ sở dữ liệu, hệ thống còn có notification worker để xử lý thông báo. Worker hoạt động độc lập với backend và nhận các sự kiện thông qua RabbitMQ. Cơ chế này giúp các tác vụ thông báo không làm ảnh hưởng trực tiếp đến tốc độ phản hồi của API.

Báo cáo mẫu cũng xác định hệ thống phần mềm, frontend, backend, cơ sở dữ liệu, API và các thành phần hỗ trợ là những đối tượng nghiên cứu quan trọng trong quá trình thực hiện đề tài.

1.5.4 Dữ liệu của hệ thống

Dữ liệu là một trong những đối tượng quan trọng cần được nghiên cứu trong đề tài. Các chức năng của hệ thống đều phụ thuộc vào tính chính xác, đầy đủ và nhất quán của dữ liệu hệ thống.

Thông tin người dùng được tổ chức theo ba vai trò chính là quản trị viên, giảng viên và sinh viên. Mỗi tài khoản được liên kết với các dữ liệu phù hợp với vai trò của mình. Giảng viên được liên kết với lớp học phần phụ trách, trong khi sinh viên được liên kết với các lớp học phần và nhóm đang tham gia.

Dữ liệu môn học và lớp học phần là cơ sở để phân chia phạm vi quản lý. Các thông tin như nhóm, đề tài, yêu cầu nộp bài, bài nộp và điểm số đều phải thuộc về một lớp học phần cụ thể. Nhờ đó, hệ thống có thể ngăn chặn tình trạng dữ liệu của lớp này bị hiển thị hoặc thao tác trong lớp khác.

Dữ liệu bài nộp không chỉ lưu kết quả hiện tại mà còn lưu lịch sử các lần nộp. Điều này giúp giảng viên theo dõi quá trình chỉnh sửa và đánh giá mức độ tiến bộ của sinh viên. Dữ liệu điểm số cũng được quản lý theo trạng thái công bố hoặc chưa công bố nhằm bảo đảm sinh viên chỉ xem được kết quả khi giảng viên cho phép.

Bên cạnh dữ liệu nghiệp vụ, hệ thống còn lưu thông báo và trạng thái xử lý sự kiện. Các thông tin này hỗ trợ kiểm tra việc gửi thông báo đã thành công hay chưa và hạn chế tình trạng sự kiện bị thất lạc trong quá trình xử lý.

1.5.5 Công nghệ và công cụ

Đề tài nghiên cứu và áp dụng nhiều công nghệ phục vụ cho từng giai đoạn phát triển hệ thống. ReactJS và Vite được sử dụng để xây dựng giao diện người dùng. Node.js và ExpressJS được sử dụng để phát triển backend và các RESTful API.

Microsoft SQL Server được sử dụng để tổ chức và lưu trữ dữ liệu quan hệ. JSON Web Token được áp dụng để xác thực và phân quyền người dùng, trong khi bcrypt được sử dụng để mã hóa mật khẩu trước khi lưu vào cơ sở dữ liệu.

Swagger hỗ trợ xây dựng tài liệu và kiểm tra API. RabbitMQ được sử dụng làm hệ thống hàng đợi, kết hợp với notification worker và cơ chế transactional outbox để xử lý thông báo theo hướng bất đồng bộ.

Docker và Docker Compose được sử dụng để đóng gói và chạy các thành phần của hệ thống trong môi trường thống nhất. Git và GitHub hỗ trợ quản lý mã nguồn, theo dõi lịch sử thay đổi và phối hợp phát triển. GitHub Actions được sử dụng để tự động chạy kiểm thử, build và triển khai hệ thống.

Trong quá trình triển khai, Render được sử dụng cho frontend, backend và notification worker. MonsterASP cung cấp cơ sở dữ liệu SQL Server trên cloud, còn CloudAMQP cung cấp dịch vụ RabbitMQ. Các công cụ như Postman, Swagger và SQL Server Management Studio được sử dụng để kiểm tra API, truy vấn dữ liệu và hỗ trợ phát hiện lỗi.

Việc nghiên cứu và kết hợp các công nghệ trên giúp nhóm xây dựng được một hệ thống hoàn chỉnh, từ giao diện, xử lý nghiệp vụ, lưu trữ dữ liệu đến kiểm thử và triển khai thực tế.

1.6 Phương pháp nghiên cứu

1.6.1 Phương pháp khảo sát và phân tích yêu cầu

Trước khi xây dựng hệ thống, nhóm tiến hành tìm hiểu quy trình quản lý đồ án trong các lớp học phần. Nội dung khảo sát tập trung vào những công việc thường xuyên của quản trị viên, giảng viên và sinh viên như quản lý danh sách lớp, tổ chức nhóm, đăng ký đề tài, theo dõi tiến độ, tạo yêu cầu nộp bài, chấm điểm và gửi thông báo.

Qua quá trình khảo sát, nhóm nhận thấy việc quản lý bằng bảng tính, thư điện tử, nhóm trò chuyện và các dịch vụ lưu trữ riêng lẻ làm cho dữ liệu bị phân tán, khó theo dõi và dễ xảy ra nhầm lẫn. Giảng viên phải mất nhiều thời gian tổng hợp thông tin từ nhiều nguồn, trong khi sinh viên có thể bỏ sót yêu cầu hoặc không nắm được trạng thái hiện tại của đề tài và bài nộp.

Từ những vấn đề đã xác định, nhóm tiến hành phân tích yêu cầu theo từng vai trò người dùng. Quản trị viên cần quản lý người dùng, môn học và lớp học phần; giảng viên cần quản lý nhóm, đề tài, tiến độ, bài nộp và điểm số; sinh viên cần theo dõi lớp học, đăng ký đề tài, nộp bài và xem kết quả.

Các yêu cầu sau khi thu thập được phân loại thành yêu cầu chức năng và yêu cầu phi chức năng. Yêu cầu chức năng mô tả những nghiệp vụ mà hệ thống cần thực hiện, trong khi yêu cầu phi chức năng tập trung vào bảo mật, phân quyền, hiệu năng, khả năng sử dụng, khả năng bảo trì và triển khai hệ thống.

1.6.2 Phương pháp phát triển linh hoạt

Nhóm áp dụng phương pháp phát triển phần mềm linh hoạt theo hướng lặp và tăng trưởng. Thay vì xây dựng toàn bộ hệ thống trong một lần, đề tài được chia thành nhiều giai đoạn nhỏ, mỗi giai đoạn tập trung hoàn thiện một nhóm chức năng cụ thể.

Ở giai đoạn đầu, nhóm xây dựng chức năng đăng nhập, xác thực và phân quyền người dùng. Tiếp theo, nhóm phát triển các chức năng quản lý môn học, lớp học phần, giảng viên và sinh viên. Sau khi phân dữ liệu nền tảng hoạt động ổn định, nhóm tiếp tục xây dựng chức năng quản lý nhóm, đăng ký đề tài, tiến độ, yêu cầu nộp bài, lịch sử nộp, chấm điểm và thông báo.

Sau mỗi giai đoạn phát triển, nhóm tiến hành chạy thử, kiểm tra chức năng và ghi nhận các lỗi phát sinh. Những lỗi hoặc điểm chưa phù hợp được sửa trước khi chuyển sang giai đoạn tiếp theo. Phương pháp này giúp nhóm dễ kiểm soát tiến độ, hạn chế lỗi tích lũy và linh hoạt điều chỉnh khi yêu cầu thay đổi.

Trong quá trình thực hiện, mã nguồn được quản lý bằng Git và GitHub. Các thay đổi được chia thành từng nhóm chức năng và lưu lại thông qua các commit. Nhờ đó, nhóm có thể theo dõi lịch sử phát triển, kiểm tra lại các thay đổi và khôi phục khi xảy ra lỗi.

1.6.3 Phương pháp thiết kế nguyên mẫu (Prototyping)

Để kiểm tra cách tổ chức giao diện và luồng thao tác của người dùng, nhóm áp dụng phương pháp thiết kế nguyên mẫu. Trước khi hoàn thiện giao diện chính thức, nhóm xây dựng các màn hình cơ bản cho từng vai trò và kiểm tra khả năng sử dụng trong các tình huống thực tế.

Đối với quản trị viên, giao diện được thiết kế theo hướng dễ quản lý danh sách người dùng, môn học và lớp học phần. Đối với giảng viên, các chức năng được tổ chức theo từng lớp phụ trách để thuận tiện theo dõi sinh viên, nhóm, đề tài, yêu cầu nộp bài và kết quả. Đối với sinh viên, giao diện tập trung vào lớp học phần, nhóm, đề tài, tiến độ, bài nộp và thông báo.

Trong quá trình xây dựng nguyên mẫu, nhóm kiểm tra cách bố trí thanh điều hướng, bảng dữ liệu, thẻ thông tin, bộ lọc, hộp thoại và biểu mẫu nhập liệu. Những vấn đề như nội dung bị tràn, chữ quá dài, bố cục thiếu cân đối, bộ lọc khó sử dụng hoặc thông tin chưa rõ ràng được ghi nhận và điều chỉnh.

Giao diện tiếp tục được cải tiến dựa trên quá trình sử dụng thử. Nhóm bổ sung trạng thái không có dữ liệu, thông báo lỗi, trạng thái đang tải, xác nhận thao tác và các nhãn mô tả phù hợp. Việc xây dựng nguyên mẫu giúp giảm sai sót trước khi hoàn thiện toàn bộ hệ thống.

1.6.4 Phương pháp kiểm thử phần mềm (Testing)

Hệ thống được thiết kế theo mô hình tách biệt giữa frontend và backend. Frontend chịu trách nhiệm hiển thị giao diện và tiếp nhận thao tác của người dùng, trong khi backend xử lý nghiệp vụ, xác thực, phân quyền và truy xuất cơ sở dữ liệu. Hai thành phần trao đổi dữ liệu thông qua RESTful API.

Backend được tổ chức theo kiến trúc phân tầng gồm routes, controllers, services và repositories. Routes khai báo các đường dẫn API; controllers tiếp nhận yêu cầu từ phía người dùng; services xử lý các quy tắc nghiệp vụ; repositories thực hiện truy vấn và cập nhật dữ liệu trong SQL Server. Cách tổ chức này giúp mã nguồn rõ ràng, hạn chế sự phụ thuộc giữa các thành phần và thuận tiện khi kiểm thử.

Cơ sở dữ liệu được thiết kế theo mô hình quan hệ. Các bảng dữ liệu được liên kết thông qua khóa chính và khóa ngoại nhằm bảo đảm tính toàn vẹn. Dữ liệu nghiệp vụ được giới hạn theo lớp học phần để tránh tình trạng giảng viên hoặc sinh viên truy cập thông tin không thuộc phạm vi của mình.

Đối với chức năng thông báo, nhóm sử dụng RabbitMQ kết hợp với notification worker. Các sự kiện phát sinh từ backend được lưu vào bảng transactional outbox trước khi chuyển đến hàng đợi. Notification worker tiếp nhận và xử lý sự kiện để tạo thông báo cho người dùng. Cách triển khai này giúp giảm nguy cơ mất sự kiện khi hệ thống gặp lỗi kết nối.

1.6.5 Phương pháp kiểm thử phần mềm

Kiểm thử được thực hiện trong suốt quá trình phát triển thay vì chỉ thực hiện khi hệ thống đã hoàn thành. Sau mỗi nhóm chức năng, nhóm tiến hành kiểm tra dữ liệu đầu vào, kết quả đầu ra, quyền truy cập và các trường hợp lỗi có thể xảy ra.

Đối với backend, nhóm xây dựng các trường hợp kiểm thử cho những nghiệp vụ quan trọng như đăng nhập, xác thực, phân quyền, quản lý lớp học phần, đăng ký đề tài, nộp bài, chấm điểm và thông báo. Bộ kiểm thử backend hiện tại đạt kết quả *58/58 trường hợp kiểm thử thành công*.

Các RESTful API được kiểm tra bằng Swagger và Postman. Việc kiểm thử tập trung vào đường dẫn API, phương thức HTTP, tham số đầu vào, mã trạng thái phản hồi, nội dung dữ liệu trả về và quyền truy cập của từng vai trò.

Đối với frontend, nhóm kiểm tra các luồng thao tác thực tế của quản trị viên, giảng viên và sinh viên. Các nội dung được kiểm tra bao gồm đăng nhập, điều hướng, tìm kiếm, lọc dữ liệu, hiển thị danh sách, gửi biểu mẫu, thông báo lỗi và khả năng hiển thị trên nhiều kích thước màn hình.

Ngoài ra, nhóm xây dựng một bộ dữ liệu mẫu lớn để kiểm tra khả năng hoạt động của hệ thống khi có nhiều người dùng và nhiều dữ liệu liên kết. Dữ liệu mẫu bao gồm quản trị viên, giảng viên, sinh viên, lớp học phần, nhóm, đăng ký đề tài, yêu cầu nộp bài, bài nộp, lịch sử nộp, điểm và thông báo. Việc kiểm tra được thực hiện trên nhiều tài khoản thuộc từng vai trò nhằm bảo đảm các màn hình đều hiển thị đúng dữ liệu theo phạm vi truy cập của từng vai trò.

1.6.6 Phương pháp triển khai và tích hợp liên tục

Nhóm sử dụng Docker để đóng gói các thành phần của hệ thống thành các container riêng biệt. Trong môi trường phát triển cục bộ, frontend, backend, SQL Server, RabbitMQ và notification worker có thể được khởi chạy thông qua Docker Compose. Điều này giúp các thành viên sử dụng cùng một môi trường và hạn chế sự khác biệt về cấu hình máy tính.

GitHub Actions được sử dụng để xây dựng quy trình tích hợp và triển khai liên tục. Khi mã nguồn được cập nhật lên nhánh chính, workflow tự động cài đặt các thư viện cần thiết, chạy kiểm thử backend, kiểm tra mã nguồn frontend và thực hiện quá trình build.

Chỉ khi các bước kiểm tra đều thành công, hệ thống mới thực hiện triển khai lên môi trường cloud. Frontend, backend và notification worker được triển khai trên Render. Cơ sở dữ liệu SQL Server được đặt trên MonsterASP, còn RabbitMQ được cung cấp bởi CloudAMQP.

Việc áp dụng CI/CD giúp nhóm phát hiện lỗi sớm trước khi triển khai, giảm các thao tác thủ công và bảo đảm phiên bản trên môi trường cloud được xây dựng từ mã nguồn đã vượt qua các bước kiểm tra cần thiết.

1.6.7 Phương pháp đánh giá và cải tiến

Sau mỗi giai đoạn phát triển, nhóm tiến hành đánh giá mức độ hoàn thành chức năng, khả năng sử dụng của giao diện và tính ổn định của hệ thống. Quá trình đánh giá dựa trên kết quả kiểm thử, phản hồi trong quá trình sử dụng và những lỗi được phát hiện khi chạy hệ thống với dữ liệu thực tế.

Các vấn đề được phân loại theo mức độ ảnh hưởng. Những lỗi liên quan đến dữ liệu, phân quyền và nghiệp vụ được ưu tiên xử lý trước. Sau đó, nhóm tiếp tục cải thiện giao diện, khả năng tìm kiếm, bộ lọc, bố cục hiển thị và nội dung thông báo.

Một số nội dung đã được cải tiến trong quá trình hoàn thiện gồm giới hạn dữ liệu theo lớp học phần, khóa đăng ký đề tài sau khi được duyệt, sửa lỗi tràn nội dung, điều chỉnh bộ lọc, bổ sung lịch sử nộp bài và tăng độ phủ dữ liệu mẫu.

Thông qua quá trình đánh giá và cải tiến liên tục, hệ thống từng bước đáp ứng tốt hơn nhu cầu quản lý đồ án sinh viên của đề tài. Đồng thời, phương pháp này giúp nhóm phát hiện những hạn chế còn tồn tại để làm cơ sở đề xuất hướng phát triển trong các giai đoạn tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Các khái niệm cơ bản về công nghệ phần mềm

2.1.1 Khái niệm công nghệ phần mềm

Công nghệ phần mềm là lĩnh vực nghiên cứu và áp dụng các nguyên tắc, phương pháp, quy trình và công cụ kỹ thuật vào quá trình xây dựng, vận hành và bảo trì phần mềm. Mục tiêu của công nghệ phần mềm không chỉ là tạo ra một chương trình có thể chạy được mà còn bảo đảm chương trình đáp ứng đúng nhu cầu của người dùng, có độ tin cậy cao, dễ sử dụng, dễ bảo trì và có khả năng mở rộng trong tương lai.

Một sản phẩm phần mềm thường trải qua nhiều giai đoạn khác nhau, bao gồm khảo sát nhu cầu, phân tích yêu cầu, thiết kế hệ thống, lập trình, kiểm thử, triển khai và bảo trì. Mỗi giai đoạn đều có vai trò riêng và có mối quan hệ chặt chẽ với các giai đoạn còn lại. Nếu yêu cầu được phân tích không đầy đủ hoặc thiết kế hệ thống chưa phù hợp thì quá trình lập trình và kiểm thử sẽ gặp nhiều khó khăn.

Trong đề tài “Xây dựng website quản lý đồ án sinh viên”, các kiến thức của công nghệ phần mềm được áp dụng xuyên suốt quá trình phát triển. Nhóm tiến hành xác định yêu cầu theo từng vai trò người dùng, thiết kế kiến trúc frontend và backend, tổ chức cơ sở dữ liệu, xây dựng API, kiểm thử chức năng, đóng gói ứng dụng bằng Docker và triển khai hệ thống lên môi trường cloud.



Hình 2.1 Quy trình phát triển phần mềm

2.1.2 Khái niệm phần mềm

Phần mềm là tập hợp các chương trình, dữ liệu, tài liệu và quy trình được xây dựng nhằm giúp máy tính thực hiện một hoặc nhiều nhiệm vụ cụ thể. Khác với phần cứng là các thiết bị vật lý, phần mềm tồn tại dưới dạng mã nguồn, chương trình thực thi, cơ sở dữ liệu và các tệp cấu hình.

Phần mềm có thể được chia thành nhiều nhóm. Phần mềm hệ thống hỗ trợ quản lý và điều khiển phần cứng, tiêu biểu như hệ điều hành và trình điều khiển thiết bị. Phần mềm ứng dụng phục vụ trực tiếp nhu cầu của người dùng, chẳng hạn như hệ thống quản lý sinh viên, phần mềm kế toán hoặc website thương mại điện tử. Phần mềm lập trình bao gồm các công cụ hỗ trợ phát triển như trình biên dịch, trình soạn thảo mã nguồn và môi trường phát triển tích hợp.

Website quản lý đồ án sinh viên thuộc nhóm phần mềm ứng dụng. Hệ thống được xây dựng nhằm hỗ trợ quản trị viên, giảng viên và sinh viên thực hiện các nghiệp vụ liên quan đến lớp học phần, nhóm sinh viên, đề tài, tiến độ, bài nộp, điểm số và thông báo.

2.1.3 Phát triển phần mềm

Vòng đời phát triển phần mềm, thường được gọi là Software Development Life Cycle, mô tả các giai đoạn mà một sản phẩm phần mềm trải qua từ khi hình thành ý tưởng đến khi được đưa vào sử dụng và bảo trì.

Giai đoạn đầu tiên là lập kế hoạch và khảo sát yêu cầu. Nhóm phát triển cần xác định vấn đề thực tế, đối tượng sử dụng và mục tiêu của sản phẩm. Tiếp theo là phân tích yêu cầu để mô tả rõ các chức năng và điều kiện mà hệ thống phải đáp ứng.

Sau khi yêu cầu được xác định, nhóm tiến hành thiết kế kiến trúc, cơ sở dữ liệu, giao diện và các luồng xử lý dữ liệu. Giai đoạn lập trình chuyển các bản thiết kế thành mã nguồn thực tế. Sản phẩm sau đó được kiểm thử để phát hiện lỗi và kiểm tra mức độ đáp ứng yêu cầu.

Khi hệ thống hoạt động ổn định, phần mềm được triển khai lên môi trường sử dụng. Trong quá trình vận hành, nhóm tiếp tục theo dõi, sửa lỗi, tối ưu và bổ sung chức năng mới. Các giai đoạn này không phải lúc nào cũng diễn ra hoàn toàn tuần tự mà có thể được lặp lại nhiều lần tùy theo mô hình phát triển được lựa chọn.

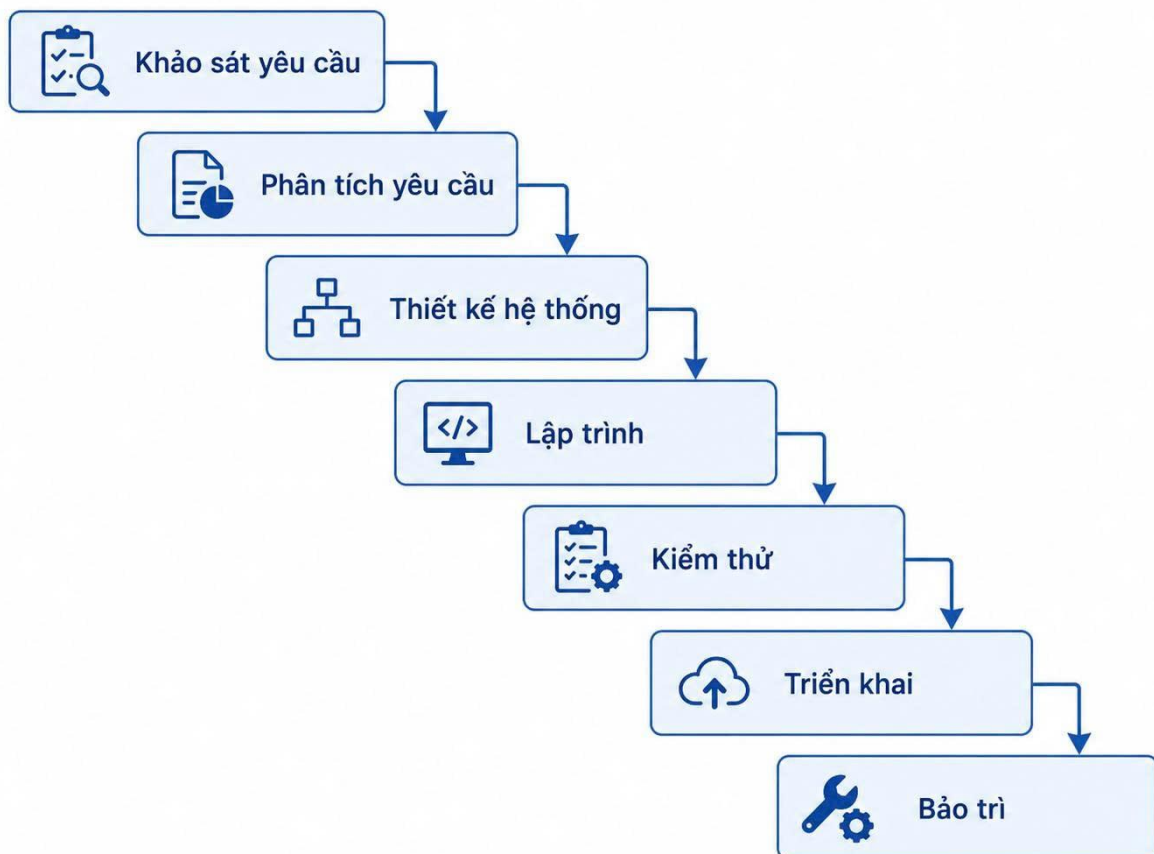
2.2 Các mô hình phát triển phần mềm

Mô hình phát triển phần mềm là cách tổ chức các hoạt động trong quá trình xây dựng sản phẩm. Việc lựa chọn mô hình phù hợp giúp nhóm kiểm soát tiến độ, chất lượng, chi phí và khả năng thay đổi yêu cầu.

2.2.1 Mô hình thác nước

Mô hình thác nước tổ chức quá trình phát triển phần mềm theo một chuỗi giai đoạn tuần tự. Một giai đoạn chỉ bắt đầu khi giai đoạn trước đã hoàn thành. Các bước thường bao gồm thu thập yêu cầu, phân tích, thiết kế, lập trình, kiểm thử, triển khai và bảo trì.

Ưu điểm của mô hình này là dễ theo dõi, tài liệu rõ ràng và phù hợp với những dự án có yêu cầu ổn định. Tuy nhiên, mô hình thác nước khó thích nghi khi yêu cầu thay đổi trong quá trình thực hiện. Nếu lỗi được phát hiện ở giai đoạn cuối thì chi phí chỉnh sửa thường cao.



Hình 2.2 Mô hình thác nước

2.2.2 Mô hình chữ V

Mô hình chữ V là sự mở rộng của mô hình thác nước, trong đó mỗi giai đoạn phát triển được liên kết với một giai đoạn kiểm thử tương ứng. Ví dụ, yêu cầu hệ thống được

kiểm tra bằng kiểm thử chấp nhận; thiết kế kiến trúc được kiểm tra bằng kiểm thử hệ thống; thiết kế module được kiểm tra bằng kiểm thử tích hợp; mã nguồn được kiểm tra bằng kiểm thử đơn vị.

Mô hình chữ V nhấn mạnh việc lập kế hoạch kiểm thử ngay từ đầu. Điều này giúp nâng cao chất lượng sản phẩm và giảm nguy cơ bỏ sót các trường hợp kiểm thử. Tuy nhiên, mô hình vẫn có tính tuần tự cao và không phù hợp khi yêu cầu thường xuyên thay đổi.

2.2.3 Mô hình gia tăng

Mô hình gia tăng chia sản phẩm thành nhiều phần nhỏ. Mỗi phần được phân tích, thiết kế, lập trình và kiểm thử độc lập trước khi tích hợp vào hệ thống chung. Sau mỗi giai đoạn, nhóm có thể tạo ra một phiên bản phần mềm có thêm chức năng mới.

Mô hình này giúp người dùng sớm tiếp cận sản phẩm và cung cấp phản hồi trong quá trình phát triển. Nhóm cũng có thể ưu tiên xây dựng các chức năng quan trọng trước, sau đó tiếp tục mở rộng những chức năng còn lại.

Đề tài quản lý đồ án sinh viên được phát triển gần với mô hình gia tăng. Nhóm lần lượt hoàn thiện chức năng xác thực, quản lý người dùng, lớp học phần, nhóm sinh viên, đăng ký đề tài, yêu cầu nộp bài, chấm điểm và thông báo.

2.2.4 Mô hình Agile

Agile là phương pháp phát triển phần mềm theo hướng linh hoạt, lặp và tăng trưởng. Dự án được chia thành những chu kỳ ngắn, trong đó nhóm thực hiện các công việc phân tích, thiết kế, lập trình và kiểm thử để tạo ra một phần sản phẩm có thể sử dụng.

Agile không yêu cầu toàn bộ yêu cầu phải cố định ngay từ đầu. Nhóm phát triển có thể tiếp nhận phản hồi, điều chỉnh chức năng và cải tiến sản phẩm sau mỗi chu kỳ. Phương pháp này phù hợp với những dự án có yêu cầu thay đổi hoặc cần triển khai sản phẩm trong thời gian ngắn.

Bảng 2.1. So sánh một số mô hình phát triển phần mềm

Tiêu chí	Mô hình thác nước	Mô hình chữ V	Mô hình gia tăng	Mô hình Agile
Cách thực hiện	Thực hiện tuần tự theo từng giai đoạn	Thực hiện tuần tự, mỗi giai đoạn phát triển có một giai đoạn kiểm thử tương ứng	Chia hệ thống thành nhiều phần và phát triển theo từng phiên bản	Phát triển lặp lại theo các chu kỳ ngắn
Khả năng thay đổi yêu cầu	Thấp, khó thay đổi khi dự án đã bắt đầu	Thấp, việc thay đổi có thể ảnh hưởng đến nhiều giai đoạn	Trung bình, có thể thay đổi trong các phần chưa phát triển	Cao, dễ tiếp nhận và điều chỉnh yêu cầu
Thời điểm kiểm thử	Chủ yếu sau khi hoàn thành lập trình	Kiểm thử được lập kế hoạch song song với từng giai đoạn phát triển	Kiểm thử sau mỗi phần chức năng được hoàn thành	Kiểm thử liên tục trong mỗi chu kỳ phát triển
Thời điểm có sản phẩm	Cuối quá trình phát triển	Cuối quá trình phát triển	Có sản phẩm sau mỗi lần gia tăng	Có sản phẩm sau mỗi Sprint
Mức độ tham gia của người dùng	Thấp	Thấp	Trung bình	Cao, người dùng thường xuyên đóng góp phản hồi

2.3 Phương pháp phát triển phần mềm Agile và Scrum

2.3.1 Khái niệm Agile

Agile là phương pháp phát triển phần mềm theo hướng linh hoạt, lặp và tăng trưởng. Thay vì xây dựng toàn bộ hệ thống trong một quy trình kéo dài rồi mới đưa ra kiểm thử, Agile chia dự án thành nhiều chu kỳ phát triển ngắn. Trong mỗi chu kỳ, nhóm thực hiện các hoạt động như lập kế hoạch, phân tích yêu cầu, thiết kế, lập trình và kiểm thử để tạo ra một phần sản phẩm có thể hoạt động.

Mỗi chu kỳ phát triển trong Agile thường kéo dài từ một đến bốn tuần. Sau khi hoàn thành một chu kỳ, nhóm tiến hành đánh giá kết quả, tiếp nhận phản hồi và điều chỉnh kế hoạch cho chu kỳ tiếp theo. Nhờ đó, sản phẩm được cải tiến liên tục và có khả năng thích ứng tốt khi yêu cầu thay đổi.

Khác với mô hình thác nước, Agile không yêu cầu tất cả yêu cầu phải được xác định hoàn toàn ngay từ đầu. Các yêu cầu có thể được bổ sung, điều chỉnh hoặc ưu tiên lại trong quá trình phát triển. Điều này phù hợp với các dự án phần mềm có yêu cầu thường xuyên thay đổi hoặc cần nhận phản hồi sớm từ người sử dụng.

Trong đề tài “Xây dựng website quản lý đồ án sinh viên”, phương pháp Agile được áp dụng bằng cách chia hệ thống thành từng nhóm chức năng. Nhóm lần lượt xây dựng chức năng xác thực, quản lý người dùng, lớp học phần, nhóm sinh viên, đăng ký đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo. Sau mỗi giai đoạn, hệ thống được kiểm tra và điều chỉnh trước khi tiếp tục phát triển chức năng mới.

2.3.2 Đặc trưng của Agile

Đặc trưng đầu tiên của Agile là tính lặp. Quá trình phát triển không diễn ra một lần duy nhất mà được thực hiện qua nhiều chu kỳ. Trong mỗi chu kỳ, nhóm đều tiến hành phân tích, thiết kế, lập trình và kiểm thử. Những kết quả của chu kỳ trước được sử dụng làm nền tảng cho chu kỳ tiếp theo.

Agile còn có tính tăng trưởng. Sau mỗi chu kỳ, hệ thống được bổ sung thêm một phần chức năng mới và vẫn có khả năng hoạt động. Sản phẩm được hoàn thiện dần theo thời gian thay vì chỉ được đưa ra sử dụng khi toàn bộ dự án kết thúc.

Một đặc trưng khác của Agile là khả năng thích nghi. Khi người dùng đưa ra yêu cầu mới hoặc nhóm phát hiện một chức năng chưa phù hợp, kế hoạch phát triển có thể được điều chỉnh mà không cần xây dựng lại toàn bộ hệ thống. Các yêu cầu quan trọng hơn có thể được ưu tiên thực hiện trước.

Agile cũng đề cao sự phối hợp giữa các thành viên. Nhóm phát triển thường xuyên trao đổi về tiến độ, các khó khăn gặp phải và hướng giải quyết. Việc trao đổi liên tục giúp phát hiện vấn đề sớm và tránh tình trạng mỗi thành viên phát triển một phần riêng biệt nhưng khó tích hợp với nhau.

Bên cạnh đó, Agile chú trọng đến sản phẩm có thể hoạt động. Tài liệu vẫn cần thiết nhưng không nên thay thế cho việc xây dựng và kiểm thử sản phẩm thực tế. Sau mỗi chu kỳ, nhóm cần tạo ra một phần chức năng có thể trình diễn hoặc kiểm tra được.

2.3.3 Các nguyên tắc của Agile

Agile ưu tiên việc tạo ra phần mềm có giá trị và có thể sử dụng. Nhóm phát triển cần thường xuyên bàn giao các chức năng đã hoàn thiện thay vì chờ đến cuối dự án mới đưa ra toàn bộ sản phẩm.

Các thay đổi về yêu cầu được xem là một phần bình thường của quá trình phát triển. Ngay cả khi dự án đã bắt đầu, nhóm vẫn có thể tiếp nhận yêu cầu mới nếu sự thay đổi đó giúp sản phẩm phù hợp hơn với nhu cầu thực tế.

Nhóm phát triển và người sử dụng cần duy trì sự trao đổi thường xuyên. Phản hồi từ người dùng giúp nhóm hiểu rõ những điểm chưa phù hợp trong giao diện, nghiệp vụ hoặc cách trình bày dữ liệu.

Agile cũng nhấn mạnh việc duy trì tốc độ phát triển ổn định. Nhóm cần lập kế hoạch phù hợp với thời gian và nguồn lực hiện có, tránh xây dựng quá nhiều chức năng trong một giai đoạn nhưng không bảo đảm chất lượng.

Chất lượng kỹ thuật và thiết kế tốt là nền tảng để hệ thống có thể mở rộng. Mã nguồn cần được tổ chức rõ ràng, giảm sự phụ thuộc giữa các thành phần và có các trường hợp kiểm thử cho những nghiệp vụ quan trọng.

Sau mỗi chu kỳ, nhóm cần xem xét lại cách làm việc, đánh giá những điểm đạt được và những hạn chế còn tồn tại. Từ đó, nhóm điều chỉnh phương pháp thực hiện để nâng cao hiệu quả trong chu kỳ tiếp theo.

2.3.4 Ưu điểm và hạn chế của Agile

2.3.4.1 Ưu điểm của Agile

Agile có khả năng thích ứng tốt với sự thay đổi. Khi yêu cầu của người dùng thay đổi, nhóm có thể điều chỉnh nội dung của các chu kỳ tiếp theo mà không ảnh hưởng quá lớn đến toàn bộ dự án.

Việc chia dự án thành các chu kỳ ngắn giúp nhóm dễ dàng theo dõi tiến độ. Sau mỗi chu kỳ, nhóm có thể xác định rõ chức năng nào đã hoàn thành, chức năng nào còn lỗi và công việc nào cần ưu tiên.

Agile giúp phát hiện lỗi sớm vì hoạt động kiểm thử được thực hiện trong từng chu kỳ. Các lỗi nghiệp vụ, lỗi giao diện hoặc lỗi tích hợp có thể được phát hiện và khắc phục trước khi hệ thống trở nên quá phức tạp.

Người sử dụng có thể sớm tiếp cận các phiên bản của sản phẩm và đưa ra phản hồi. Những phản hồi này giúp nhóm điều chỉnh hệ thống theo đúng nhu cầu thực tế, hạn chế việc xây dựng chức năng không phù hợp.

Phương pháp Agile cũng tăng cường sự phối hợp giữa các thành viên. Việc thường xuyên trao đổi và cập nhật tiến độ giúp nhóm nắm rõ tình hình chung của dự án và hỗ trợ nhau khi gặp khó khăn.

2.3.4.2 Hạn chế của Agile

Agile yêu cầu các thành viên phải thường xuyên trao đổi và phối hợp chặt chẽ. Nếu nhóm thiếu chủ động hoặc không cập nhật tiến độ đầy đủ, công việc có thể bị chồng chéo hoặc bỏ sót.

Do yêu cầu có thể thay đổi trong quá trình thực hiện, phạm vi dự án đôi khi khó xác định chính xác ngay từ đầu. Nếu không kiểm soát tốt, nhóm có thể bỏ sung quá nhiều chức năng và làm ảnh hưởng đến thời gian hoàn thành.

Agile cũng đòi hỏi người sử dụng hoặc người hướng dẫn cung cấp phản hồi thường xuyên. Khi phản hồi đến chậm, nhóm có thể tiếp tục phát triển theo một hướng chưa phù hợp và phải chỉnh sửa nhiều ở giai đoạn sau.

Đối với các nhóm chưa có nhiều kinh nghiệm, việc ước lượng thời gian cho từng chu kỳ có thể chưa chính xác. Một số công việc tưởng đơn giản nhưng khi thực hiện lại phát sinh thêm nhiều vấn đề kỹ thuật.

Bên cạnh đó, nếu chỉ tập trung vào việc hoàn thiện chức năng mà bỏ qua tài liệu, hệ thống sẽ khó bảo trì. Vì vậy, nhóm vẫn cần xây dựng các tài liệu cần thiết như đặc tả yêu cầu, sơ đồ hệ thống, tài liệu API và hướng dẫn triển khai.

Bảng 2.2. Ưu điểm và hạn chế của Agile

Tiêu chí	Ưu điểm	Hạn chế
Khả năng thích ứng	Dễ dàng điều chỉnh khi yêu cầu thay đổi trong quá trình phát triển	Nếu thay đổi quá nhiều sẽ làm phạm vi dự án khó kiểm soát
Tiến độ phát triển	Dự án được chia thành các chu kỳ ngắn nên dễ theo dõi tiến độ	Việc ước lượng thời gian cho từng chu kỳ có thể chưa chính xác
Kiểm thử	Kiểm thử được thực hiện thường xuyên, giúp phát hiện và sửa lỗi sớm	Yêu cầu nhóm phải kiểm tra liên tục trong suốt quá trình phát triển
Sản phẩm	Sau mỗi chu kỳ có thể tạo ra một phần sản phẩm có khả năng hoạt động	Sản phẩm giữa các chu kỳ có thể chưa đầy đủ toàn bộ chức năng
Phản hồi người dùng	Người dùng có thể tham gia đánh giá và đóng góp ý kiến thường xuyên	Hiệu quả phụ thuộc vào mức độ tham gia và phản hồi của người dùng
Làm việc nhóm	Tăng cường sự trao đổi, phối hợp và hỗ trợ giữa các thành viên	Thành viên phải chủ động, có trách nhiệm và thường xuyên cập nhật công việc
Chất lượng phần mềm	Sản phẩm được cải tiến liên tục qua từng chu kỳ	Chất lượng có thể bị ảnh hưởng nếu nhóm chạy theo tiến độ và bỏ qua kỹ thuật
Tài liệu	Giảm thời gian xây dựng những tài liệu không thật sự cần thiết	Có nguy cơ thiếu tài liệu nếu nhóm chỉ tập trung vào lập trình

2.3.5 Các phương pháp Agile phổ biến

Agile là một định hướng phát triển phần mềm, bên trong bao gồm nhiều phương pháp khác nhau. Mỗi phương pháp có cách tổ chức công việc và phạm vi áp dụng riêng.

Scrum là phương pháp phổ biến, tổ chức dự án thành các chu kỳ ngắn gọi là Sprint. Mỗi Sprint có mục tiêu, danh sách công việc và kết quả cần đạt được.

Kanban quản lý công việc thông qua bảng trực quan. Các nhiệm vụ thường được chia thành những trạng thái như cần thực hiện, đang thực hiện, đang kiểm tra và đã hoàn thành. Kanban giúp nhóm dễ theo dõi tiến độ và nhận biết công việc đang bị tồn đọng.

Extreme Programming tập trung vào chất lượng mã nguồn và kỹ thuật lập trình. Phương pháp này khuyến khích kiểm thử thường xuyên, lập trình theo cặp, tích hợp liên tục và cải tiến mã nguồn.

Lean Software Development chú trọng việc loại bỏ những hoạt động không tạo ra giá trị, tối ưu thời gian thực hiện và sử dụng nguồn lực hiệu quả.

Trong phạm vi đề tài, nhóm chủ yếu áp dụng các nguyên tắc của Scrum kết hợp với bảng theo dõi công việc gần với Kanban. Công việc được chia thành từng nhóm chức năng và theo dõi theo trạng thái thực hiện.

2.3.6 Phương pháp Scrum

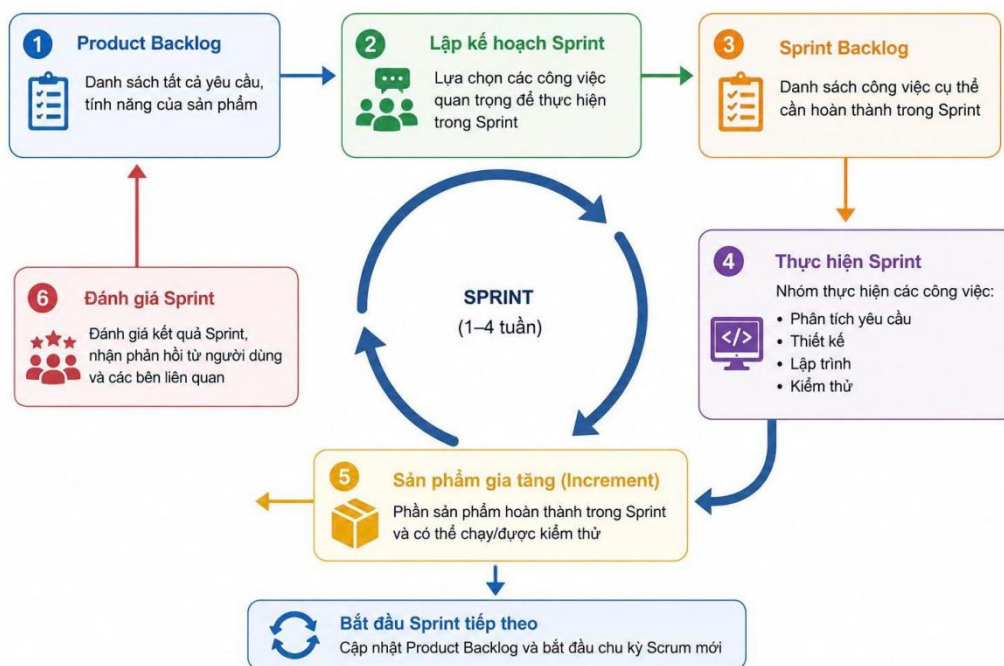
Scrum là một phương pháp quản lý và phát triển sản phẩm thuộc Agile. Scrum chia dự án thành các chu kỳ ngắn gọi là Sprint. Mỗi Sprint thường kéo dài từ một đến bốn tuần và tập trung hoàn thành một nhóm công việc cụ thể.

Trước khi bắt đầu Sprint, nhóm lựa chọn các công việc quan trọng từ danh sách yêu cầu chung của sản phẩm. Những công việc này được phân tích, chia nhỏ và đưa vào kế hoạch của Sprint. Trong quá trình thực hiện, các thành viên thường xuyên cập nhật tiến độ và xử lý những vấn đề phát sinh.

Cuối mỗi Sprint, nhóm đánh giá những chức năng đã hoàn thành, kiểm tra sản phẩm và ghi nhận phản hồi. Sau đó, nhóm tổ chức đánh giá lại quá trình làm việc để xác định những điểm tốt, những hạn chế và phương pháp cải thiện cho Sprint tiếp theo.

Scrum thường có ba vai trò chính. Product Owner chịu trách nhiệm xác định và ưu tiên yêu cầu của sản phẩm. Scrum Master hỗ trợ nhóm tuân thủ quy trình và giải quyết các trở ngại của đề tài. Development Team là nhóm trực tiếp phân tích, thiết kế, lập trình và kiểm thử sản phẩm.

Các tài liệu chính trong Scrum gồm Product Backlog, Sprint Backlog và Increment. Product Backlog là danh sách toàn bộ yêu cầu của sản phẩm. Sprint Backlog là các công việc được lựa chọn để thực hiện trong Sprint hiện tại. Increment là phần sản phẩm hoàn chỉnh được tạo ra sau Sprint.



Hình 2.3. Quy trình Scrum

Trong đề tài, Scrum được áp dụng ở mức phù hợp với quy mô nhóm. Nhóm không tổ chức đầy đủ tất cả vai trò như trong một dự án doanh nghiệp nhưng vẫn vận dụng cách chia công việc thành các giai đoạn ngắn, xác định mục tiêu cho từng giai đoạn và đánh giá kết quả sau khi hoàn thành.

Giai đoạn đầu tập trung khảo sát yêu cầu, thiết kế cơ sở dữ liệu và xây dựng chức năng xác thực. Giai đoạn tiếp theo phát triển chức năng quản lý người dùng, môn học và lớp học phần. Sau đó, nhóm hoàn thiện quản lý nhóm sinh viên, đăng ký đề tài, tiến độ và yêu cầu nộp bài.

Các giai đoạn cuối tập trung vào bài nộp, lịch sử nộp, chấm điểm, thông báo, kiểm thử, Docker và triển khai cloud. Sau mỗi giai đoạn, nhóm chạy thử hệ thống, phát hiện lỗi và điều chỉnh chức năng trước khi chuyển sang nội dung tiếp theo.

Việc áp dụng Agile và Scrum giúp nhóm kiểm soát tiến độ tốt hơn, phát hiện lỗi sớm và linh hoạt điều chỉnh hệ thống theo phản hồi trong quá trình phát triển. Phương pháp này phù hợp với đề tài vì các chức năng có mối liên hệ với nhau nhưng vẫn có thể chia thành nhiều nhóm nghiệp vụ để xây dựng và kiểm thử riêng biệt.

2.4 Kiến trúc hệ thống web

Kiến trúc hệ thống mô tả cách các thành phần phần mềm được tổ chức, kết nối và phối hợp với nhau trong quá trình hoạt động. Việc lựa chọn kiến trúc phù hợp giúp hệ thống dễ phát triển, kiểm thử, bảo trì và mở rộng.

Đối với đề tài “Xây dựng website quản lý đồ án sinh viên”, hệ thống được xây dựng theo mô hình Client–Server, trong đó frontend và backend được tách thành hai thành phần riêng biệt. Phần backend tiếp tục được tổ chức theo kiến trúc phân tầng nhằm phân chia rõ trách nhiệm xử lý của từng thành phần.

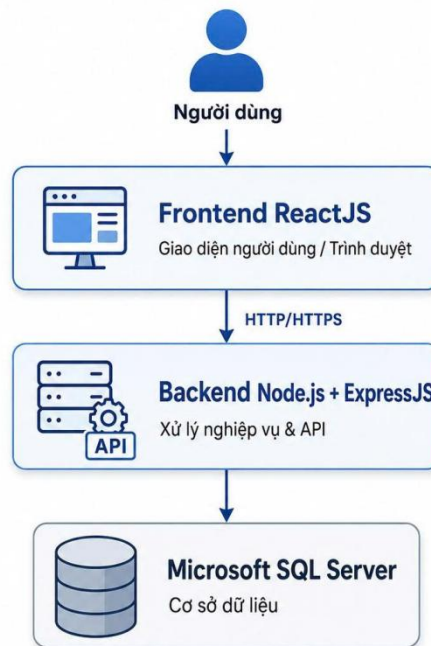
2.4.1 Mô hình Client–Server

Client–Server là mô hình kiến trúc trong đó hệ thống được chia thành hai phía chính là client và server. Client có nhiệm vụ cung cấp giao diện và tiếp nhận thao tác của người dùng, còn server chịu trách nhiệm xử lý nghiệp vụ hệ thống, truy xuất dữ liệu và trả kết quả cho client.

Trong hệ thống quản lý đồ án sinh viên, frontend được xây dựng bằng ReactJS và đóng vai trò client. Người dùng thực hiện các thao tác như đăng nhập, xem lớp học phần, đăng ký đề tài, nộp bài hoặc chấm điểm thông qua giao diện website.

Backend được xây dựng bằng Node.js và ExpressJS, đóng vai trò server. Backend tiếp nhận yêu cầu từ frontend thông qua giao thức HTTP hoặc HTTPS, sau đó kiểm tra thông tin xác thực, phân quyền người dùng, xử lý nghiệp vụ và truy vấn dữ liệu trong SQL Server. Kết quả xử lý được trả về frontend dưới định dạng JSON.

Mô hình Client–Server giúp tách biệt phần giao diện với phần xử lý dữ liệu. Nhờ đó, frontend và backend có thể được phát triển, kiểm thử và triển khai tương đối độc lập. Trong tương lai, backend cũng có thể được tái sử dụng để phục vụ ứng dụng di động hoặc những hệ thống khác.



Hình 2.4. Mô hình Client–Server của hệ thống

2.4.2 Kiến trúc frontend và backend tách biệt

Trong kiến trúc frontend và backend tách biệt, hai thành phần được xây dựng thành các dự án riêng nhưng phối hợp với nhau thông qua RESTful API.

Frontend chịu trách nhiệm hiển thị giao diện, điều hướng giữa các trang, tiếp nhận dữ liệu nhập từ người dùng và gửi yêu cầu đến backend. Frontend không truy cập trực tiếp vào cơ sở dữ liệu mà chỉ nhận dữ liệu do backend cung cấp.

Backend chịu trách nhiệm xác thực người dùng, phân quyền, kiểm tra dữ liệu đầu vào, xử lý nghiệp vụ và truy xuất cơ sở dữ liệu. Mọi quy tắc quan trọng như kiểm tra giảng viên có phụ trách lớp hay không, sinh viên có thuộc nhóm hay không hoặc đề tài đã được duyệt hay chưa đều được thực hiện tại backend.

Việc tách biệt hai thành phần giúp tăng tính bảo mật vì thông tin kết nối cơ sở dữ liệu không được đưa lên phía trình duyệt. Đồng thời, cách tổ chức này giúp nhóm dễ dàng thay đổi giao diện mà không ảnh hưởng trực tiếp đến logic nghiệp vụ và dữ liệu.

Frontend và backend cũng có thể được triển khai độc lập. Khi chỉ thay đổi giao diện, nhóm có thể cập nhật frontend mà không cần triển khai lại backend. Ngược lại, các thay

đổi về API hoặc nghiệp vụ có thể được thực hiện tại backend mà không cần sửa toàn bộ giao diện.

2.4.3 Kiến trúc phân tầng của backend

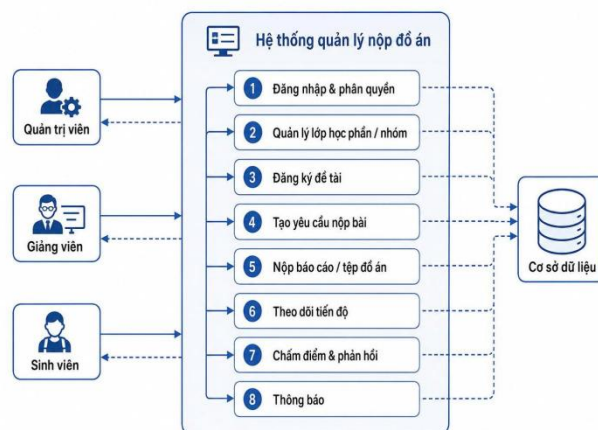
Backend của hệ thống được tổ chức theo kiến trúc phân tầng gồm Routes, Controllers, Services và Repositories. Mỗi tầng đảm nhận một nhiệm vụ riêng nhằm giúp mã nguồn rõ ràng, dễ kiểm tra và thuận tiện khi bảo trì.

Tầng Routes chịu trách nhiệm khai báo các đường dẫn API, phương thức HTTP và middleware cần sử dụng. Tại đây, hệ thống có thể kiểm tra token, vai trò người dùng và dữ liệu đầu vào trước khi chuyển yêu cầu sang controller.

Tầng Controllers tiếp nhận yêu cầu từ routes, lấy tham số và nội dung request, sau đó gọi service phù hợp. Controller chủ yếu đóng vai trò điều phối và trả kết quả về cho frontend.

Tầng Services chứa các quy tắc nghiệp vụ chính của hệ thống. Ví dụ, service kiểm tra quyền truy cập lớp học phần, trạng thái đăng ký đề tài, điều kiện nộp bài hoặc quyền công bố điểm.

Tầng Repositories chịu trách nhiệm truy vấn và cập nhật dữ liệu trong SQL Server. Việc tách riêng repository giúp phần nghiệp vụ không phụ thuộc trực tiếp vào câu lệnh trong SQL.



Hình 2.5. Kiến trúc phân tầng của backend

Bảng 2.3. Chức năng của các tầng trong backend

Tầng	Chức năng chính	Nội dung thực hiện trong hệ thống
Routes	Định nghĩa đường dẫn API	Khai báo các endpoint, phương thức HTTP và middleware xác thực, phân quyền người dùng.
Controllers	Tiếp nhận và điều phối yêu cầu	Nhận dữ liệu từ request, gọi service xử lý và trả kết quả về frontend.
Services	Xử lý nghiệp vụ	Kiểm tra quyền truy cập, trạng thái đề tài, điều kiện nộp bài, chấm điểm và các quy tắc nghiệp vụ của hệ thống.
Repositories	Truy xuất dữ liệu	Thực hiện các thao tác thêm, sửa, xóa, tìm kiếm và truy vấn dữ liệu trong SQL Server.
Database	Lưu trữ dữ liệu	Lưu thông tin người dùng, môn học, lớp học phân, nhóm sinh viên, đề tài, bài nộp, điểm số và thông báo.

Kiến trúc phân tầng giúp hạn chế việc trộn lẫn giữa xử lý yêu cầu, nghiệp vụ và truy vấn dữ liệu. Khi cần thay đổi quy tắc nghiệp vụ, nhóm chủ yếu chỉnh sửa tại service; khi cần tối ưu câu lệnh SQL, nhóm có thể tập trung thay đổi tại repository mà ít ảnh hưởng đến các tầng còn lại.

2.4.4 RESTful API

RESTful API là phương thức giao tiếp giữa frontend và backend dựa trên giao thức HTTP. Trong mô hình này, mỗi nhóm dữ liệu hoặc chức năng của hệ thống được biểu diễn bằng một endpoint riêng. Frontend gửi yêu cầu đến backend thông qua endpoint, sau đó backend xử lý và trả kết quả về dưới định dạng JSON.

Hệ thống sử dụng các phương thức HTTP phổ biến như GET để lấy dữ liệu, POST để tạo mới, PUT hoặc PATCH để cập nhật và DELETE để xóa dữ liệu. Việc sử dụng đúng phương thức giúp các API có cấu trúc rõ ràng, thống nhất và dễ kiểm tra.

Trong website quản lý đồ án sinh viên, RESTful API được sử dụng cho các chức năng như đăng nhập, quản lý người dùng, môn học, lớp học phần, nhóm sinh viên, đăng ký đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Mỗi API đều được kiểm tra thông tin xác thực và quyền truy cập. Backend không chỉ kiểm tra vai trò của người dùng mà còn kiểm tra phạm vi dữ liệu. Chẳng hạn, giảng viên chỉ được truy cập các lớp học phần do mình phụ trách, còn sinh viên chỉ được xem dữ liệu thuộc lớp và nhóm mà mình tham gia.

Bảng 2.4. Các phương thức HTTP thường được sử dụng

Phương thức	Chức năng chính	Ví dụ trong hệ thống
GET	Lấy hoặc xem dữ liệu từ hệ thống	Xem danh sách lớp học phần, sinh viên, đề tài hoặc bài nộp.
POST	Tạo mới dữ liệu	Tạo tài khoản, lớp học phần, nhóm sinh viên hoặc yêu cầu nộp bài.
PUT	Cập nhật toàn bộ thông tin của một đối tượng	Cập nhật thông tin người dùng hoặc lớp học phần.
PATCH	Cập nhật một phần thông tin của đối tượng	Thay đổi trạng thái đăng ký đề tài hoặc trạng thái công bố điểm.
DELETE	Xóa dữ liệu khỏi hệ thống	Xóa lớp học phần, nhóm hoặc yêu cầu nộp bài khi được phép.

RESTful API giúp frontend và backend hoạt động độc lập tương đối. Frontend không cần biết cách dữ liệu được lưu trong SQL Server mà chỉ cần gửi đúng yêu cầu theo cấu trúc API đã quy định. Đồng thời, backend có thể được tái sử dụng cho các ứng dụng khác trong tương lai.

2.4.5 Ứng dụng web một trang

Ứng dụng web một trang, còn được gọi là Single Page Application, là mô hình trong đó phần giao diện chính được tải một lần khi người dùng truy cập website. Sau đó, nội dung trên trang được cập nhật động mà không cần tải lại toàn bộ website sau mỗi thao tác.

Frontend của hệ thống được xây dựng bằng ReactJS theo mô hình Single Page Application. Giao diện được chia thành nhiều component độc lập như thanh điều hướng, bảng dữ liệu, biểu mẫu, thẻ thông tin, hộp thoại và khu vực thông báo.

React Router được sử dụng để quản lý các đường dẫn trong frontend. Sau khi đăng nhập, người dùng được chuyển đến khu vực phù hợp với vai trò của mình. Quản trị viên, giảng viên và sinh viên có các đường dẫn và màn hình chức năng riêng.

Khi người dùng chuyển từ trang danh sách lớp học phần sang trang chi tiết lớp, trình duyệt không cần tải lại toàn bộ ứng dụng. React chỉ thay đổi phần nội dung cần thiết và lấy dữ liệu mới từ backend thông qua RESTful API.

Mô hình Single Page Application giúp tốc độ chuyển trang nhanh hơn và tạo trải nghiệm sử dụng liền mạch. Tuy nhiên, frontend cần quản lý tốt trạng thái đăng nhập, token, dữ liệu đang tải, lỗi từ API và quyền truy cập của từng vai trò.

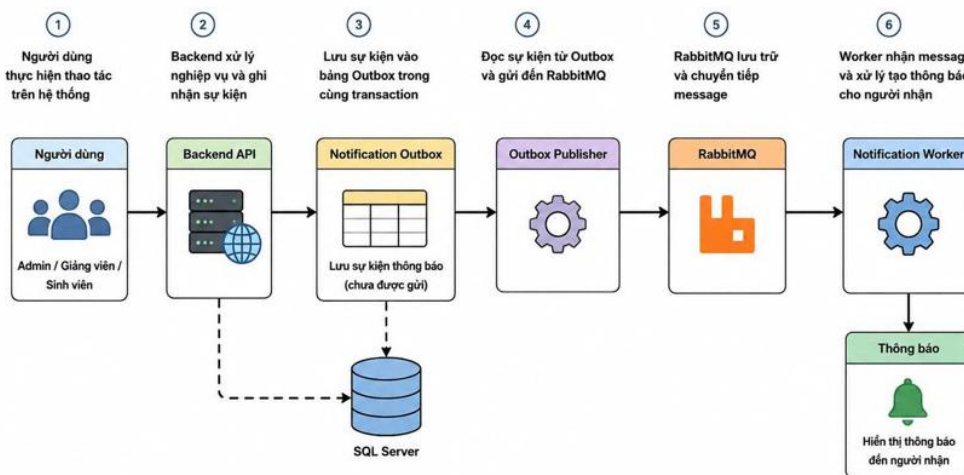
2.4.6 Kiến trúc xử lý thông báo

Hệ thống sử dụng kiến trúc xử lý thông báo theo hướng bất đồng bộ. Thay vì thực hiện toàn bộ việc tạo thông báo ngay trong quá trình xử lý yêu cầu của người dùng, backend ghi nhận sự kiện và chuyển công việc cho notification worker xử lý.

Khi xảy ra một nghiệp vụ quan trọng như đề tài được duyệt, giảng viên yêu cầu chỉnh sửa, sinh viên nộp bài hoặc điểm được công bố, backend lưu dữ liệu nghiệp vụ và sự kiện thông báo vào cơ sở dữ liệu.

Sự kiện được lưu trong bảng NotificationOutbox theo cơ chế Transactional Outbox. Việc lưu dữ liệu nghiệp vụ và sự kiện trong cùng một transaction giúp bảo đảm rằng nếu nghiệp vụ thành công thì sự kiện thông báo cũng được ghi nhận.

Sau đó, tiến trình phát sự kiện đọc các bản ghi chưa được gửi trong bảng outbox và chuyển chúng đến RabbitMQ. Notification worker tiếp nhận message từ RabbitMQ, xử lý nội dung và tạo thông báo cho đúng người nhận.



Hình 2.6. Kiến trúc xử lý thông báo của hệ thống

Việc xử lý thông báo bất đồng bộ giúp backend trả kết quả nhanh hơn vì người dùng không phải chờ toàn bộ quá trình tạo thông báo hoàn thành. Notification worker có thể hoạt động độc lập và tiếp tục xử lý các sự kiện trong hàng đợi.

Cơ chế Transactional Outbox còn giúp hạn chế mất thông báo khi RabbitMQ tạm thời không hoạt động. Sự kiện vẫn được lưu trong SQL Server và có thể được gửi lại khi kết nối được khôi phục.

2.5 Xác thực và phân quyền người dùng

Xác thực và phân quyền là hai thành phần quan trọng trong việc bảo vệ dữ liệu và kiểm soát quyền truy cập của người dùng. Trong hệ thống quản lý đồ án sinh viên, mỗi tài khoản phải được xác minh trước khi sử dụng các chức năng. Sau khi đăng nhập thành công, hệ thống tiếp tục kiểm tra vai trò và phạm vi dữ liệu trước khi cho phép người dùng thực hiện thao tác.

2.5.1 Xác thực người dùng

Xác thực là quá trình kiểm tra danh tính của người dùng thông qua thông tin đăng nhập. Người dùng cung cấp tài khoản và mật khẩu trên giao diện, sau đó frontend gửi dữ liệu đến API đăng nhập của backend.

Backend tìm kiếm tài khoản trong cơ sở dữ liệu và kiểm tra trạng thái hoạt động. Nếu tài khoản tồn tại, hệ thống sử dụng bcrypt để so sánh mật khẩu người dùng nhập với mật khẩu đã được mã hóa trong SQL Server.

Mật khẩu không được lưu trực tiếp dưới dạng văn bản thông thường. Khi tạo tài khoản hoặc đổi mật khẩu, hệ thống sử dụng thuật toán băm để tạo ra một chuỗi không thể chuyển ngược về mật khẩu ban đầu. Cách xử lý này giúp hạn chế nguy cơ lộ mật khẩu khi dữ liệu bị truy cập trái phép.

Nếu thông tin đăng nhập hợp lệ, backend tạo token và trả về cho frontend. Trường hợp tài khoản không tồn tại, mật khẩu không đúng hoặc tài khoản không còn hoạt động, hệ thống từ chối đăng nhập và trả về thông báo phù hợp.

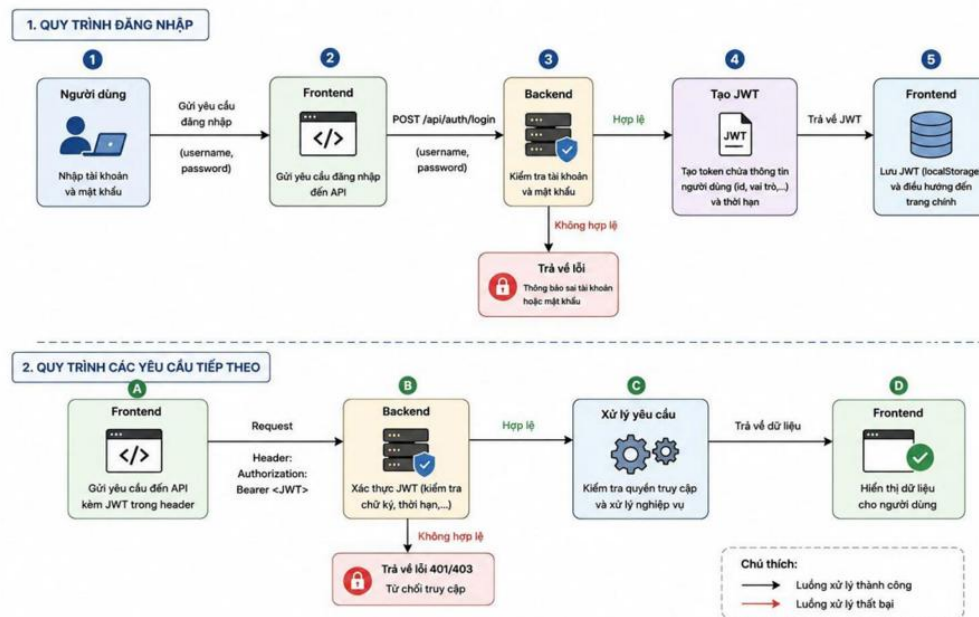
2.5.2 Xác thực bằng JSON Web Token

JSON Web Token, viết tắt là JWT, là tiêu chuẩn được sử dụng để truyền thông tin xác thực giữa frontend và backend. Sau khi người dùng đăng nhập thành công, backend tạo một token chứa các thông tin cần thiết như mã người dùng, vai trò và thời hạn sử dụng.

JWT gồm ba thành phần chính là Header, Payload và Signature. Header chứa thông tin về loại token và thuật toán ký. Payload chứa các dữ liệu xác định người dùng. Signature được tạo từ Header, Payload và khóa bí mật, giúp backend kiểm tra token có bị thay đổi hay không.

Frontend gửi token kèm theo các yêu cầu đến API được bảo vệ. Thông thường, token được đặt trong phần Authorization của HTTP Header theo cấu trúc Bearer Token. Backend kiểm tra chữ ký và thời hạn token trước khi xử lý yêu cầu.

Khi token hợp lệ, thông tin người dùng được chuyển đến các middleware và tầng xử lý nghiệp vụ. Nếu token bị thiếu, hết hạn hoặc không đúng chữ ký, backend trả về lỗi xác thực và không cho phép truy cập dữ liệu.



Hình 2.7. Quy trình xác thực bằng JWT

2.5.3 Phân quyền theo vai trò

Sau khi xác thực danh tính, hệ thống thực hiện phân quyền để xác định người dùng được phép truy cập những chức năng nào. Website hiện có ba vai trò chính gồm quản trị viên, giảng viên và sinh viên.

Quản trị viên có quyền quản lý dữ liệu chung như tài khoản người dùng, môn học, lớp học phần và phân công giảng viên. Giảng viên có quyền quản lý những lớp học phần được phân công, theo dõi sinh viên, xử lý đăng ký đề tài, tạo yêu cầu nộp bài, chấm điểm và gửi phản hồi. Sinh viên được phép xem lớp đang tham gia, quản lý thông tin nhóm, đăng ký đề tài, nộp bài và xem kết quả.

Việc phân quyền được thực hiện tại backend thông qua middleware và các quy tắc nghiệp vụ trong service. Hệ thống không chỉ kiểm tra vai trò mà còn kiểm tra phạm vi dữ liệu. Một giảng viên không được truy cập lớp học phần của giảng viên khác, trong khi sinh viên không được xem bài nộp hoặc điểm số của nhóm không liên quan.

Frontend cũng sử dụng thông tin vai trò để hiển thị menu và chức năng phù hợp. Tuy nhiên, việc ẩn nút trên giao diện không được xem là biện pháp bảo mật chính.

Backend vẫn phải kiểm tra quyền đối với mọi API nhằm ngăn người dùng gửi yêu cầu trực tiếp đến hệ thống.

Bảng 2.5. Phạm vi quyền của các vai trò người dùng

Vai trò	Phạm vi sử dụng	Chức năng chính
Quản trị viên	Quản lý toàn bộ dữ liệu chung của hệ thống	Quản lý tài khoản người dùng, môn học, lớp học phần, phân công giảng viên và danh sách sinh viên.
Giảng viên	Quản lý các lớp học phần được phân công phụ trách	Quản lý sinh viên, nhóm, đăng ký đề tài, yêu cầu nộp bài, bài nộp, điểm số và phản hồi.
Sinh viên	Truy cập các lớp học phần và nhóm mà sinh viên tham gia	Đăng ký đề tài, theo dõi tiến độ, xem yêu cầu nộp bài, nộp bài, xem phản hồi, điểm số và thông báo.

Cơ chế xác thực bằng JWT kết hợp với phân quyền theo vai trò giúp hệ thống bảo vệ các API và giới hạn dữ liệu phù hợp với từng người dùng. Đây là nền tảng quan trọng để bảo đảm tính riêng tư, an toàn và chính xác trong quá trình quản lý đồ án sinh viên.

2.6 Cơ sở dữ liệu quan hệ và Microsoft SQL Server

Cơ sở dữ liệu là thành phần quan trọng dùng để lưu trữ, quản lý và truy xuất thông tin của hệ thống. Đối với website quản lý đồ án sinh viên, dữ liệu có nhiều mối liên hệ chặt chẽ như người dùng, môn học, lớp học phần, nhóm sinh viên, đề tài, bài nộp, điểm số và thông báo. Vì vậy, nhóm lựa chọn mô hình cơ sở dữ liệu quan hệ và sử dụng Microsoft SQL Server làm hệ quản trị cơ sở dữ liệu chính.

2.6.1 Cơ sở dữ liệu quan hệ

Cơ sở dữ liệu quan hệ tổ chức dữ liệu thành các bảng. Mỗi bảng gồm nhiều dòng và nhiều cột, trong đó một dòng biểu diễn một bản ghi, còn một cột biểu diễn một thuộc tính của đối tượng.

Các bảng trong cơ sở dữ liệu liên kết với nhau thông qua khóa chính và khóa ngoại. Khóa chính dùng để xác định duy nhất từng bản ghi trong bảng. Khóa ngoại dùng để tham chiếu đến bản ghi thuộc bảng khác, qua đó thể hiện mối quan hệ giữa các đối tượng.

Trong hệ thống quản lý đồ án sinh viên, bảng người dùng lưu thông tin tài khoản của quản trị viên, giảng viên và sinh viên. Bảng lớp học phần liên kết với môn học và giảng viên phụ trách. Danh sách sinh viên tham gia lớp được lưu trong bảng trung gian giữa người dùng và lớp học phần.

Các nhóm sinh viên được liên kết với từng lớp học phần, trong khi thành viên nhóm được lưu trong một bảng riêng. Đăng ký đề tài, yêu cầu nộp bài, bài nộp và điểm số đều được liên kết với lớp học phần hoặc nhóm tương ứng. Cách tổ chức này giúp dữ liệu có cấu trúc rõ ràng và hạn chế tình trạng trùng lặp.

Cơ sở dữ liệu quan hệ còn hỗ trợ các ràng buộc nhằm bảo đảm tính chính xác của dữ liệu. Ví dụ, một bài nộp không thể tồn tại nếu yêu cầu nộp bài tương ứng không tồn tại. Một thành viên nhóm phải thuộc về một nhóm hợp lệ và một sinh viên chỉ được truy cập dữ liệu của lớp học phần mà mình tham gia.

2.6.2 Microsoft SQL Server

Microsoft SQL Server là hệ quản trị cơ sở dữ liệu quan hệ do Microsoft phát triển. SQL Server hỗ trợ lưu trữ dữ liệu, thực hiện truy vấn, quản lý transaction, tạo chỉ mục và thiết lập các ràng buộc toàn vẹn.

Hệ thống sử dụng SQL Server để lưu trữ toàn bộ dữ liệu nghiệp vụ như tài khoản người dùng, môn học, lớp học phần, danh sách sinh viên, nhóm, đề tài, yêu cầu nộp bài, bài nộp, lịch sử nộp, điểm số và thông báo.

Dữ liệu được truy xuất thông qua các câu lệnh SQL tại tầng repository của backend. Khi service cần lấy danh sách lớp học phần, kiểm tra thành viên nhóm hoặc lưu bài nộp mới, repository thực hiện truy vấn đến SQL Server và trả kết quả cho tầng xử lý nghiệp vụ.

Trong môi trường phát triển cục bộ, SQL Server được chạy bằng Docker nhằm tạo môi trường thống nhất giữa các thành viên. Khi triển khai hệ thống trực tuyến, cơ sở dữ

liệu được đặt trên dịch vụ SQL Server của MonsterASP để backend có thể truy cập qua Internet.

SQL Server phù hợp với đề tài vì hỗ trợ tốt dữ liệu quan hệ, transaction và các ràng buộc khóa ngoại. Đây là những yếu tố cần thiết để bảo đảm tính nhất quán của dữ liệu trong quá trình nhiều người dùng cùng thao tác trên hệ thống.

2.6.3 Tính toàn vẹn và transaction

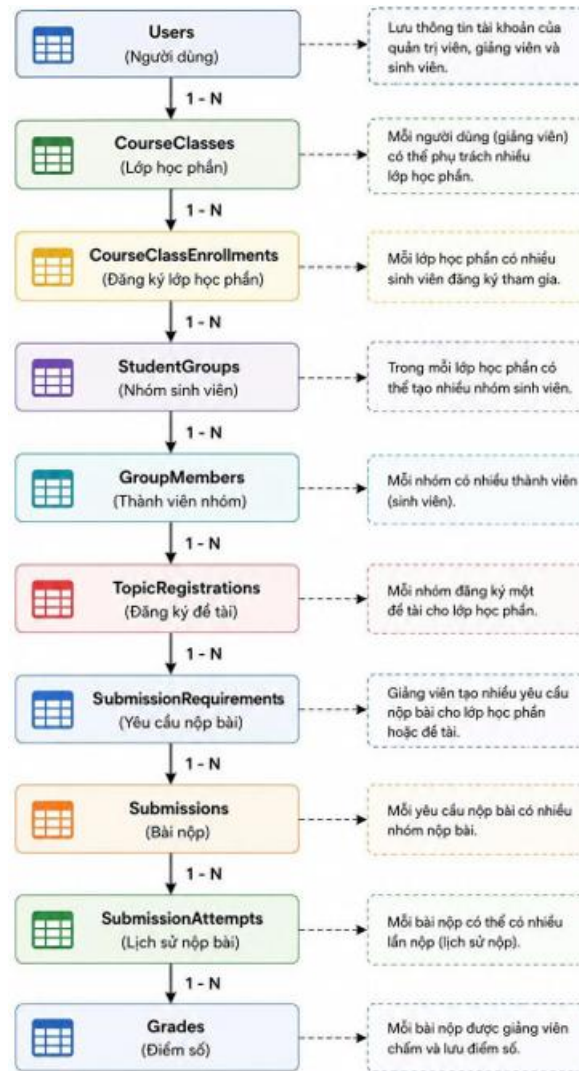
Tính toàn vẹn dữ liệu thể hiện mức độ chính xác, đầy đủ và nhất quán của thông tin trong cơ sở dữ liệu. Hệ thống sử dụng khóa chính, khóa ngoại, kiểu dữ liệu và các điều kiện kiểm tra để hạn chế dữ liệu không hợp lệ.

Toàn vẹn thực thể bảo đảm mỗi bản ghi có một giá trị định danh duy nhất. Toàn vẹn tham chiếu bảo đảm khóa ngoại luôn tham chiếu đến bản ghi tồn tại. Toàn vẹn miền giá trị giúp giới hạn dữ liệu theo phạm vi phù hợp, chẳng hạn trạng thái đề tài chỉ được nhận các giá trị đã định nghĩa.

Transaction là tập hợp nhiều thao tác cơ sở dữ liệu được xử lý như một đơn vị thống nhất. Nếu tất cả thao tác thành công, transaction được xác nhận. Nếu có một thao tác thất bại, toàn bộ thay đổi có thể được hoàn tác để tránh dữ liệu bị thiếu hoặc không đồng bộ.

Ví dụ, khi giảng viên công bố điểm, hệ thống có thể cập nhật trạng thái điểm và đồng thời tạo một sự kiện thông báo. Hai thao tác này cần được thực hiện trong cùng transaction. Nếu việc lưu sự kiện thất bại, hệ thống không nên chỉ cập nhật điểm mà bỏ mất thông báo liên quan.

Transaction cũng được sử dụng trong quá trình seed dữ liệu mẫu. Nhiều bản ghi liên quan được thêm vào cơ sở dữ liệu theo đúng thứ tự và được kiểm tra tính toàn vẹn. Nếu phát sinh lỗi, transaction giúp ngăn việc chỉ tạo thành công một phần dữ liệu.



Hình 2.8. Mối quan hệ dữ liệu tổng quát của hệ thống

Việc sử dụng cơ sở dữ liệu quan hệ kết hợp với Microsoft SQL Server giúp hệ thống quản lý dữ liệu có cấu trúc, bảo đảm các mối liên kết nghiệp vụ và hỗ trợ truy xuất thông tin chính xác theo từng người dùng, lớp học phần và nhóm sinh viên.

2.7 Hệ thống hàng đợi và xử lý bất đồng bộ

Trong một hệ thống phần mềm, một số tác vụ không nhất thiết phải hoàn thành ngay trong lúc người dùng gửi yêu cầu. Chẳng hạn, sau khi giảng viên duyệt đề tài hoặc công bố điểm, hệ thống cần tạo thông báo cho sinh viên. Nếu tất cả công việc đều được xử lý trực tiếp trong cùng một yêu cầu, thời gian phản hồi có thể kéo dài và ảnh hưởng đến trải nghiệm của người dùng.

Để khắc phục vấn đề này, website quản lý đồ án sinh viên áp dụng cơ chế xử lý bất đồng bộ thông qua RabbitMQ và Notification Worker. Backend tập trung xử lý nghiệp vụ chính, còn các tác vụ tạo thông báo được chuyển sang hàng đợi để xử lý riêng.

2.7.1 Xử lý đồng bộ và bất đồng bộ

Xử lý đồng bộ là cách thực hiện trong đó các công việc được xử lý lần lượt. Người dùng phải chờ toàn bộ quá trình hoàn thành trước khi nhận được kết quả. Phương pháp này dễ triển khai nhưng có thể làm tăng thời gian phản hồi khi tác vụ xử lý phức tạp hoặc phụ thuộc vào dịch vụ khác.

Ví dụ, khi giảng viên công bố điểm, nếu backend vừa cập nhật điểm, vừa tạo thông báo và thực hiện các tác vụ liên quan trong cùng một luồng xử lý thì người dùng phải chờ tất cả công việc hoàn thành. Nếu dịch vụ thông báo gặp lỗi, nghiệp vụ công bố điểm cũng có thể bị ảnh hưởng.

Xử lý bất đồng bộ cho phép backend hoàn thành nghiệp vụ chính và ghi nhận công việc cần xử lý sau. Những công việc này được chuyển đến hàng đợi để một tiến trình độc lập tiếp nhận và thực hiện.

Trong hệ thống, sau khi dữ liệu nghiệp vụ được lưu thành công, sự kiện thông báo được chuyển đến RabbitMQ. Notification Worker nhận sự kiện và tạo thông báo cho người dùng. Nhờ đó, backend có thể trả kết quả nhanh hơn và không phải chờ quá trình xử lý thông báo hoàn thành.

2.7.2 RabbitMQ

RabbitMQ là hệ thống trung gian truyền nhận thông điệp, thường được sử dụng để kết nối các thành phần phần mềm theo hướng bất đồng bộ. Các thông điệp được gửi vào hàng đợi và được tiến trình nhận xử lý theo thứ tự phù hợp.

Trong RabbitMQ, thành phần gửi thông điệp được gọi là Producer, còn thành phần nhận và xử lý thông điệp được gọi là Consumer. Queue là nơi lưu trữ tạm thời các thông điệp trong thời gian chờ xử lý.

Trong đề tài, backend hoặc tiến trình Outbox Publisher đóng vai trò Producer. Khi phát sinh một sự kiện như đề tài được duyệt, bài nộp được cập nhật hoặc điểm được công bố, sự kiện được gửi đến RabbitMQ. Notification Worker đóng vai trò Consumer, nhận thông điệp và tạo thông báo cho người dùng liên quan.

RabbitMQ giúp giảm sự phụ thuộc trực tiếp giữa backend và Notification Worker. Backend không cần biết cụ thể thông báo được xử lý như thế nào mà chỉ cần phát đi sự kiện. Worker có thể được sửa đổi hoặc mở rộng mà không làm thay đổi nhiều đến phần nghiệp vụ chính.

Ngoài ra, các thông điệp chưa được xử lý có thể tiếp tục nằm trong hàng đợi. Khi Notification Worker tạm thời ngừng hoạt động và được khởi động lại, worker có thể tiếp tục nhận các sự kiện còn lại.

2.7.3 Notification Worker và cơ chế Transactional Outbox

Notification Worker là một tiến trình chạy độc lập, có nhiệm vụ nhận sự kiện từ RabbitMQ và tạo thông báo cho người dùng. Worker theo dõi hàng đợi, đọc nội dung message, xác định người nhận và ghi thông báo vào cơ sở dữ liệu.

Ví dụ, khi giảng viên yêu cầu chỉnh sửa đề tài, worker tạo thông báo cho các sinh viên thuộc nhóm thực hiện. Khi điểm được công bố, worker tạo thông báo cho sinh viên hoặc nhóm có bài nộp tương ứng.

Để hạn chế nguy cơ dữ liệu nghiệp vụ được lưu thành công nhưng sự kiện thông báo không được gửi, hệ thống áp dụng cơ chế Transactional Outbox. Backend không gửi trực tiếp message đến RabbitMQ ngay trong lúc cập nhật dữ liệu. Thay vào đó, sự kiện được ghi vào bảng NotificationOutbox trong cùng transaction với dữ liệu nghiệp vụ.

Khi transaction thành công, cả dữ liệu nghiệp vụ và sự kiện thông báo đều được lưu. Sau đó, Outbox Publisher đọc các sự kiện đang chờ, gửi chúng đến RabbitMQ và cập nhật trạng thái tương ứng.

Các trạng thái của sự kiện có thể bao gồm đang chờ xử lý, đang xử lý, đã gửi hoặc gửi thất bại. Việc lưu trạng thái giúp hệ thống theo dõi quá trình phát sự kiện và hỗ trợ gửi lại khi xảy ra lỗi.

Cơ chế kết hợp RabbitMQ, Notification Worker và Transactional Outbox giúp hệ thống nâng cao khả năng ổn định. Nghiệp vụ chính không phụ thuộc trực tiếp vào thời điểm xử lý thông báo, đồng thời các sự kiện vẫn được lưu lại khi RabbitMQ hoặc worker tạm thời gặp sự cố.



Hình 2.9. Mô hình truyền nhận thông điệp qua RabbitMQ

2.8 Docker và Docker Compose

Docker được sử dụng trong đề tài nhằm đóng gói các thành phần của hệ thống thành những môi trường chạy độc lập. Việc áp dụng Docker giúp hạn chế sự khác biệt về phiên bản phần mềm, thư viện và cấu hình giữa các máy tính của thành viên. Hệ thống có thể được khởi chạy nhất quán trên môi trường phát triển, kiểm thử và triển khai.

2.8.1 Khái niệm Docker

Docker là nền tảng hỗ trợ đóng gói ứng dụng cùng với thư viện, cấu hình và môi trường cần thiết vào các container. Mỗi container hoạt động như một môi trường độc lập nhưng vẫn sử dụng chung tài nguyên của hệ điều hành máy chủ.

Trong quá trình phát triển phần mềm theo phương pháp truyền thống, ứng dụng có thể hoạt động đúng trên máy của một thành viên nhưng gặp lỗi khi chạy trên máy khác do khác phiên bản Node.js, SQL Server hoặc các thư viện liên quan. Docker giúp giải quyết vấn đề này bằng cách xác định rõ môi trường chạy của từng thành phần.

Đối với website quản lý đồ án sinh viên, Docker được sử dụng để đóng gói frontend, backend, SQL Server, RabbitMQ và Notification Worker. Các thành phần được kết nối với nhau thông qua mạng nội bộ của Docker.

Việc sử dụng Docker giúp nhóm dễ dàng khởi động, dừng và kiểm tra toàn bộ hệ thống mà không phải cài đặt thủ công từng dịch vụ. Khi cần triển khai trên một môi trường mới, nhóm chỉ cần sử dụng các tệp cấu hình Docker và thiết lập biến môi trường phù hợp.

2.8.2 Docker Image và Container

Docker Image là một mẫu đóng gói chứa mã nguồn, thư viện, môi trường chạy và các chỉ dẫn cần thiết để khởi tạo ứng dụng. Image thường được xây dựng dựa trên nội dung của tệp Dockerfile.

Docker Container là một phiên bản đang hoạt động của Docker Image. Một image có thể được sử dụng để tạo ra nhiều container khác nhau. Các container được cách ly tương đối với nhau nhưng có thể giao tiếp thông qua mạng Docker.

Trong đề tài, frontend và backend có cấu hình Docker riêng. Frontend được build thành các tệp tĩnh và phục vụ giao diện cho người dùng. Backend chạy trên môi trường Node.js, cung cấp RESTful API và kết nối đến SQL Server.

Notification Worker cũng được chạy trong một container riêng. Worker kết nối đến RabbitMQ để nhận các message và ghi thông báo vào cơ sở dữ liệu. SQL Server và RabbitMQ được chạy dưới dạng các dịch vụ độc lập nhằm tạo môi trường kiểm thử gần giống với môi trường triển khai thực tế.

Việc chia các thành phần thành nhiều container giúp dễ xác định lỗi. Khi một dịch vụ gặp sự cố, nhóm có thể kiểm tra log hoặc khởi động lại riêng container đó mà không cần dừng toàn bộ hệ thống.

2.8.3 Docker Compose trong hệ thống

Docker Compose là công cụ dùng để khai báo và quản lý nhiều container trong cùng một hệ thống. Các dịch vụ, mạng, cổng kết nối, biến môi trường, volume và mối quan hệ phụ thuộc được mô tả trong tệp `docker-compose.yml`.

Trong đề tài, Docker Compose giúp khởi chạy đồng thời các dịch vụ cần thiết. Backend có thể phụ thuộc vào SQL Server và RabbitMQ, trong khi Notification Worker cần kết nối đến cả RabbitMQ và cơ sở dữ liệu.

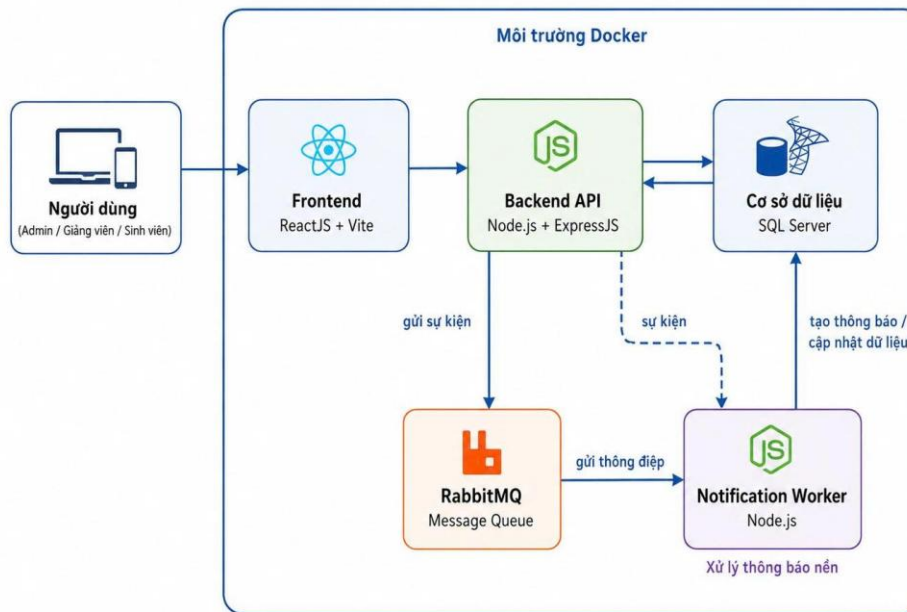
Docker Compose còn tạo mạng nội bộ để các container giao tiếp thông qua tên dịch vụ. Backend không cần sử dụng địa chỉ IP cố định của SQL Server mà có thể kết nối thông qua tên dịch vụ được khai báo trong tệp cấu hình.

Đối với dữ liệu SQL Server, hệ thống sử dụng volume để lưu trữ dữ liệu bên ngoài vòng đời của container. Khi container được khởi động lại hoặc tạo mới, dữ liệu vẫn có thể được giữ lại nếu volume không bị xóa.

Bảng 2.6. Các dịch vụ được tổ chức bằng Docker

Dịch vụ	Công nghệ sử dụng	Chức năng trong hệ thống
Frontend	ReactJS, Vite	Cung cấp giao diện cho quản trị viên, giảng viên và sinh viên.
Backend	Node.js, ExpressJS	Xử lý RESTful API, xác thực, phân quyền và các nghiệp vụ của hệ thống.
Database	Microsoft SQL Server	Lưu trữ dữ liệu người dùng, môn học, lớp học phần, nhóm, đề tài, bài nộp, điểm số và thông báo.
Message Queue	RabbitMQ	Tiếp nhận, lưu trữ tạm thời và chuyển tiếp các sự kiện thông báo.

Dịch vụ	Công nghệ sử dụng	Chức năng trong hệ thống
Notification Worker	Node.js	Nhận message từ RabbitMQ, xử lý sự kiện và tạo thông báo cho người dùng.



Hình 2.10. Kiến trúc Docker của hệ thống

Việc sử dụng Docker và Docker Compose giúp quá trình cài đặt hệ thống trở nên đơn giản, đồng nhất và dễ kiểm soát hơn. Đây cũng là cơ sở để nhóm thực hiện kiểm thử, triển khai và tích hợp hệ thống trên các nền tảng cloud.

2.9 Tích hợp và triển khai liên tục

Trong quá trình phát triển phần mềm, việc kiểm tra và triển khai mã nguồn theo phương pháp thủ công có thể mất nhiều thời gian, đồng thời dễ phát sinh sai sót. Để nâng cao chất lượng sản phẩm và tự động hóa quá trình triển khai, nhóm áp dụng quy trình tích hợp và triển khai liên tục thông qua GitHub Actions.

Quy trình này giúp hệ thống tự động kiểm tra mã nguồn mỗi khi có thay đổi được đẩy lên GitHub. Chỉ những phiên bản vượt qua các bước kiểm thử, kiểm tra mã nguồn và build thành công mới được phép triển khai lên môi trường cloud.

2.9.1 Khái niệm CI/CD

CI/CD là tập hợp các phương pháp tự động hóa quá trình tích hợp, kiểm thử và triển khai phần mềm. CI là viết tắt của Continuous Integration, nghĩa là tích hợp liên tục. CD có thể được hiểu là Continuous Delivery hoặc Continuous Deployment, nghĩa là phân phối hoặc triển khai liên tục.

Continuous Integration cho phép các thành viên thường xuyên đưa mã nguồn mới vào kho lưu trữ chung. Sau mỗi lần cập nhật, hệ thống tự động cài đặt thư viện, kiểm tra mã nguồn, chạy kiểm thử và thực hiện quá trình build. Nếu một bước xảy ra lỗi, quy trình dừng lại và thông báo để nhóm khắc phục.

Continuous Delivery bảo đảm phiên bản phần mềm sau khi vượt qua các bước kiểm tra luôn ở trạng thái sẵn sàng triển khai. Continuous Deployment tiếp tục tự động đưa phiên bản đó lên môi trường hoạt động mà không cần thực hiện nhiều thao tác thủ công.

Việc áp dụng CI/CD giúp nhóm phát hiện lỗi sớm, giảm rủi ro khi triển khai và bảo đảm phiên bản trên môi trường cloud được xây dựng từ mã nguồn đã qua kiểm tra.

2.9.2 GitHub Actions

GitHub Actions là dịch vụ tự động hóa được tích hợp trực tiếp trong GitHub. Các quy trình được mô tả bằng tệp cấu hình YAML đặt trong thư mục `.github/workflows` của dự án.

Mỗi workflow gồm một hoặc nhiều công việc, được gọi là job. Trong mỗi job có nhiều bước thực hiện như tải mã nguồn, cài đặt thư viện, chạy kiểm thử, kiểm tra frontend, build ứng dụng và gọi dịch vụ triển khai.

Trong đề tài, GitHub Actions được kích hoạt khi mã nguồn được đẩy lên nhánh `main` hoặc khi người quản lý chạy workflow thủ công. Backend và frontend được kiểm tra trong các job riêng biệt nhằm giúp quy trình rõ ràng và dễ xác định lỗi.

Đối với backend, workflow cài đặt các thư viện cần thiết và chạy toàn bộ bộ kiểm thử tự động. Đối với frontend, workflow thực hiện kiểm tra mã nguồn bằng lint và tiến hành build ứng dụng. Chỉ khi các job đều thành công, bước triển khai mới được thực hiện.

Các thông tin nhạy cảm như địa chỉ Deploy Hook, mật khẩu cơ sở dữ liệu hoặc khóa truy cập không được ghi trực tiếp trong tệp workflow. Những giá trị này được lưu trong GitHub Secrets và chỉ được sử dụng khi quy trình chạy.

2.9.3 Quy trình CI/CD của hệ thống

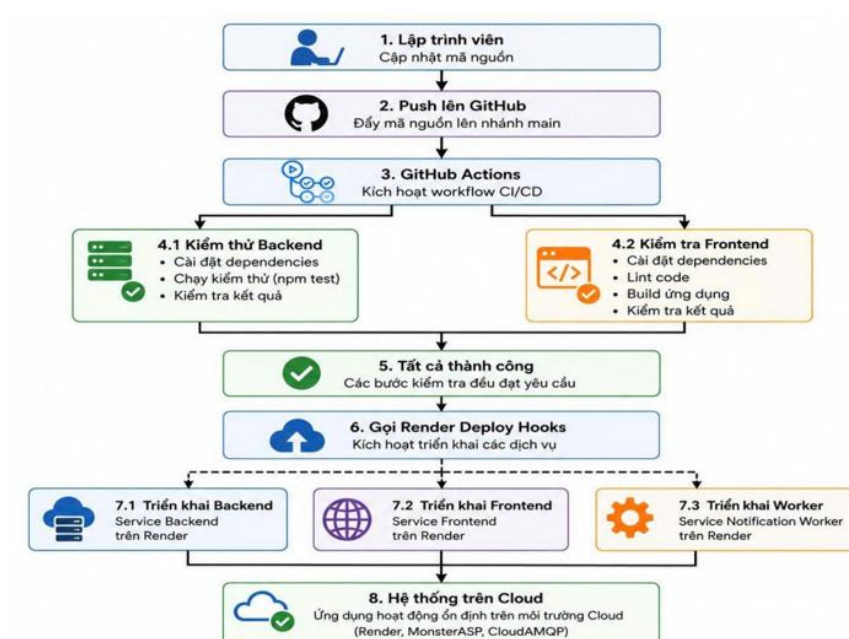
Quy trình CI/CD bắt đầu khi mã nguồn mới được đẩy lên kho lưu trữ GitHub. GitHub Actions tiếp nhận sự kiện và khởi chạy các job kiểm tra backend và frontend.

Backend được cài đặt thư viện và chạy bộ kiểm thử nhằm xác nhận các nghiệp vụ vẫn hoạt động đúng. Frontend được kiểm tra lỗi mã nguồn và thực hiện build để bảo đảm ứng dụng có thể tạo phiên bản triển khai.

Nếu một bước kiểm tra thất bại, quy trình dừng lại và không triển khai phiên bản mới. Điều này giúp ngăn mã nguồn bị lỗi được đưa lên môi trường sử dụng.

Khi toàn bộ bước kiểm tra hoàn tất thành công, workflow gọi các Render Deploy Hook tương ứng. Backend, frontend và Notification Worker được triển khai thành những dịch vụ riêng trên Render.

Cơ sở dữ liệu SQL Server được đặt trên MonsterASP, còn RabbitMQ được cung cấp bởi CloudAMQP. Các dịch vụ kết nối với nhau thông qua biến môi trường được cấu hình riêng trên nền tảng triển khai.



Hình 2.11. Quy trình CI/CD của hệ thống

Quy trình CI/CD giúp giảm các thao tác triển khai thủ công và bảo đảm chỉ những phiên bản đạt yêu cầu mới được đưa lên môi trường cloud. Đây là một trong những yếu tố góp phần nâng cao tính ổn định và chuyên nghiệp của hệ thống.

2.10 Kiểm thử phần mềm

Kiểm thử phần mềm là quá trình kiểm tra hệ thống nhằm phát hiện lỗi và đánh giá mức độ đáp ứng các yêu cầu đã đặt ra. Hoạt động kiểm thử giúp xác định các chức năng có hoạt động đúng hay không, dữ liệu có được xử lý chính xác hay không và người dùng có được truy cập đúng phạm vi quyền hạn hay không.

Trong đề tài “Xây dựng website quản lý đồ án sinh viên”, kiểm thử được thực hiện xuyên suốt quá trình phát triển. Sau khi hoàn thành từng nhóm chức năng, nhóm tiến hành chạy thử, kiểm tra kết quả và sửa lỗi trước khi tích hợp với các chức năng khác. Việc kiểm thử sớm giúp hạn chế lỗi tích lũy và nâng cao tính ổn định của hệ thống.

2.10.1 Kiểm thử đơn vị

Kiểm thử đơn vị, còn được gọi là Unit Test, tập trung kiểm tra từng hàm, phương thức hoặc module nhỏ trong hệ thống. Mục tiêu của kiểm thử đơn vị là xác nhận mỗi thành phần hoạt động đúng khi được kiểm tra độc lập.

Trong backend, kiểm thử đơn vị được áp dụng chủ yếu cho tầng service vì đây là nơi chứa các quy tắc nghiệp vụ quan trọng. Các repository hoặc thành phần bên ngoài có thể được mô phỏng để kiểm tra logic mà không cần truy cập cơ sở dữ liệu thật.

Một số nghiệp vụ cần được kiểm thử gồm kiểm tra quyền truy cập lớp học phân, xử lý trạng thái đăng ký đề tài, điều kiện nộp bài, giới hạn số lần nộp và công bố điểm. Ví dụ, khi đăng ký đề tài đã được duyệt, hệ thống phải từ chối mọi yêu cầu chỉnh sửa hoặc thay đổi trạng thái tiếp theo.

Kiểm thử đơn vị giúp nhóm phát hiện lỗi tại vị trí cụ thể, giảm thời gian tìm nguyên nhân và hạn chế việc sửa một chức năng làm ảnh hưởng đến chức năng khác.

2.10.2 Kiểm thử tích hợp và kiểm thử API

Kiểm thử tích hợp được sử dụng để kiểm tra khả năng phối hợp giữa nhiều thành phần trong hệ thống. Một chức năng có thể liên quan đến routes, controllers, services, repositories và cơ sở dữ liệu. Mặc dù từng thành phần hoạt động đúng khi kiểm tra riêng, lỗi vẫn có thể xuất hiện trong quá trình trao đổi dữ liệu giữa các tầng.

Trong đề tài, kiểm thử tích hợp được thực hiện để kiểm tra luồng xử lý từ khi frontend hoặc công cụ kiểm thử gửi yêu cầu đến backend, cho đến khi dữ liệu được truy vấn hoặc cập nhật trong SQL Server.

Kiểm thử API tập trung kiểm tra các endpoint RESTful. Các nội dung cần kiểm tra gồm phương thức HTTP, dữ liệu đầu vào, mã trạng thái, cấu trúc dữ liệu trả về và quyền truy cập của người dùng.

Ngoài trường hợp xử lý thành công, API còn được kiểm tra với các trường hợp như thiếu token, token hết hạn, không đúng vai trò, dữ liệu không hợp lệ, đối tượng không tồn tại hoặc người dùng cố gắng truy cập dữ liệu ngoài phạm vi được phép.

Swagger và Postman được sử dụng để gửi yêu cầu và kiểm tra phản hồi của API. Việc kiểm thử này giúp bảo đảm backend cung cấp dữ liệu đúng và xử lý lỗi rõ ràng trước khi tích hợp hoàn chỉnh với frontend.

2.10.3 Kiểm thử giao diện và kiểm thử phân quyền

Kiểm thử giao diện được thực hiện để đánh giá khả năng hiển thị và phản hồi của website khi người dùng thao tác. Các nội dung kiểm tra bao gồm điều hướng giữa các trang, nhập dữ liệu vào biểu mẫu, tìm kiếm, lọc dữ liệu, hiển thị bảng, hộp thoại, trạng thái đang tải và thông báo lỗi.

Giao diện được kiểm tra trên các kích thước màn hình khác nhau nhằm hạn chế tình trạng tràn chữ, chồng lấn nội dung hoặc bố cục không cân đối. Những trường hợp dữ liệu dài như tên môn học, tên lớp, tên đề tài và tên tệp cũng được kiểm tra để bảo đảm giao diện hiển thị ổn định.

Kiểm thử phân quyền được thực hiện với ba vai trò quản trị viên, giảng viên và sinh viên. Mỗi vai trò được đăng nhập bằng nhiều tài khoản khác nhau để kiểm tra menu, chức năng và phạm vi dữ liệu.

Quản trị viên phải truy cập được các chức năng quản lý chung. Giảng viên chỉ được xem và thao tác trên những lớp học phân được phân công. Sinh viên chỉ được truy cập dữ liệu của các lớp và nhóm mà mình tham gia.

Nhóm cũng sử dụng bộ dữ liệu mẫu lớn để kiểm tra các màn hình khi có nhiều bản ghi. Việc này giúp đánh giá khả năng hiển thị danh sách, bộ lọc, phân nhóm dữ liệu và tính chính xác của phạm vi truy cập trong điều kiện gần với thực tế.

Bảng 2.7. Các hình thức kiểm thử được áp dụng

Hình thức kiểm thử	Mục đích	Nội dung áp dụng trong hệ thống
Kiểm thử đơn vị	Kiểm tra từng hàm hoặc module độc lập	Kiểm tra các service, điều kiện nghiệp vụ, trạng thái đề tài và quyền thao tác.
Kiểm thử tích hợp	Kiểm tra sự phối hợp giữa nhiều thành phần	Kiểm tra luồng xử lý qua routes, controllers, services, repositories và SQL Server.
Kiểm thử API	Kiểm tra hoạt động của các endpoint RESTful	Kiểm tra phương thức HTTP, dữ liệu đầu vào, mã trạng thái và dữ liệu trả về.
Kiểm thử giao diện	Kiểm tra khả năng hiển thị và thao tác của người dùng	Kiểm tra điều hướng, biểu mẫu, bảng dữ liệu, bộ lọc và khả năng responsive.
Kiểm thử phân quyền	Kiểm tra phạm vi truy cập theo từng vai trò	Kiểm tra quyền của quản trị viên, giảng viên và sinh viên đối với chức năng và dữ liệu.

Các nội dung trên là cơ sở lý thuyết cho quá trình kiểm thử thực tế được trình bày chi tiết ở chương kiểm thử của báo cáo.

2.11 Swagger và tài liệu API

Trong hệ thống có frontend và backend được phát triển tách biệt, tài liệu API giữ vai trò quan trọng trong việc mô tả cách các thành phần trao đổi dữ liệu với nhau. Tài liệu API giúp nhóm xác định rõ đường dẫn, phương thức HTTP, tham số đầu vào, dữ liệu trả về và các mã trạng thái có thể xuất hiện.

Đối với đề tài “Xây dựng website quản lý đồ án sinh viên”, Swagger được sử dụng để xây dựng và hiển thị tài liệu cho các RESTful API của backend. Thông qua giao diện Swagger, nhóm có thể theo dõi danh sách endpoint, kiểm tra yêu cầu và gửi thử dữ liệu mà không cần sử dụng trực tiếp giao diện frontend.

2.11.1 Khái niệm Swagger và OpenAPI

OpenAPI là một tiêu chuẩn dùng để mô tả RESTful API theo cấu trúc thống nhất. Nội dung mô tả có thể bao gồm đường dẫn API, phương thức HTTP, tham số, dữ liệu trong request body, cấu trúc response và các yêu cầu xác thực.

Swagger là tập hợp công cụ hỗ trợ xây dựng, hiển thị và kiểm tra tài liệu API dựa trên tiêu chuẩn OpenAPI. Một trong những công cụ phổ biến là Swagger UI, cung cấp giao diện trực quan để người phát triển xem và thử nghiệm các endpoint.

Mỗi API được mô tả với những thông tin cơ bản như tên chức năng, mục đích sử dụng, phương thức HTTP, đường dẫn, dữ liệu đầu vào và kết quả trả về. Nhờ đó, người phát triển frontend có thể biết chính xác cách gửi yêu cầu đến backend.

Swagger cũng cho phép khai báo các mã trạng thái HTTP. Ví dụ, mã 200 thể hiện yêu cầu được xử lý thành công, mã 201 thể hiện dữ liệu được tạo mới, mã 400 thể hiện dữ liệu đầu vào không hợp lệ, mã 401 thể hiện người dùng chưa được xác thực và mã 403 thể hiện người dùng không có quyền truy cập.

2.11.2 Ứng dụng Swagger trong hệ thống

Trong website quản lý đồ án sinh viên, Swagger được sử dụng để mô tả các nhóm API như xác thực người dùng, quản lý tài khoản, môn học, lớp học phần, nhóm sinh viên, đăng ký đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Đối với các API yêu cầu xác thực, Swagger hỗ trợ nhập JWT vào phần Authorization. Sau khi cung cấp token hợp lệ, người phát triển có thể gửi yêu cầu đến những endpoint được bảo vệ và kiểm tra dữ liệu trả về theo từng vai trò.

Ví dụ, khi kiểm tra API lấy danh sách lớp học phần của giảng viên, người kiểm thử nhập token của tài khoản giảng viên và gửi yêu cầu trực tiếp trên Swagger. Backend xác thực token, kiểm tra vai trò và trả về các lớp học phần thuộc phạm vi phụ trách của tài khoản đó.

Tương tự, Swagger có thể được sử dụng để kiểm tra API đăng ký đề tài, thay đổi trạng thái đề tài, tạo yêu cầu nộp bài hoặc xem điểm số. Việc thử nghiệm trực tiếp giúp nhóm phát hiện lỗi về tham số, quyền truy cập và cấu trúc dữ liệu trước khi tích hợp với frontend.



Hình 2.12. Quy trình kiểm tra API bằng Swagger

2.11.3 Lợi ích của tài liệu API

Tài liệu API giúp quá trình phối hợp giữa frontend và backend trở nên rõ ràng hơn. Frontend có thể dựa vào tài liệu để biết endpoint cần gọi, dữ liệu cần gửi và cách xử lý kết quả trả về.

Swagger còn giúp giảm thời gian kiểm tra API. Thay vì phải viết mã giao diện để thử một chức năng, nhóm có thể gửi yêu cầu trực tiếp từ trình duyệt và xem kết quả ngay trên giao diện tài liệu.

Khi API được thay đổi, tài liệu có thể được cập nhật để phản ánh cấu trúc mới. Điều này hạn chế tình trạng frontend sử dụng sai đường dẫn, sai tên thuộc tính hoặc sai phương thức HTTP.

Tài liệu API cũng hỗ trợ việc bảo trì hệ thống. Thành viên mới hoặc người tiếp tục phát triển sản phẩm có thể nhanh chóng tìm hiểu các chức năng backend mà không cần đọc toàn bộ mã nguồn.

Ngoài ra, Swagger giúp kiểm tra tính nhất quán giữa các endpoint. Các API có thể được tổ chức theo nhóm, sử dụng cùng cách đặt tên, cùng cấu trúc phản hồi và cùng quy tắc xác thực. Qua đó, hệ thống trở nên dễ hiểu và thuận tiện hơn khi mở rộng.

2.12 Tổng kết chương

Chương 2 đã trình bày những cơ sở lý thuyết và công nghệ quan trọng được sử dụng trong quá trình xây dựng website quản lý đồ án sinh viên. Nội dung đầu tiên tập trung vào khái niệm công nghệ phần mềm, vòng đời phát triển phần mềm và các mô hình phát triển phổ biến như mô hình thác nước, mô hình chữ V, mô hình gia tăng và Agile.

Trong quá trình thực hiện đề tài, nhóm áp dụng phương pháp Agile kết hợp với Scrum ở mức phù hợp với quy mô dự án. Các chức năng được chia thành nhiều giai đoạn nhỏ, sau mỗi giai đoạn nhóm tiến hành kiểm tra, tiếp nhận phản hồi và tiếp tục cải tiến sản phẩm.

Hệ thống được xây dựng theo mô hình Client–Server, trong đó frontend và backend được tách thành hai thành phần riêng biệt. Backend được tổ chức theo kiến trúc phân tầng gồm Routes, Controllers, Services và Repositories. Cách tổ chức này giúp phân chia rõ trách nhiệm của từng thành phần, thuận tiện cho việc kiểm thử, bảo trì và mở rộng hệ thống.

Cơ chế xác thực bằng JSON Web Token và phân quyền theo vai trò được áp dụng để bảo vệ các chức năng và dữ liệu. Ba nhóm người dùng gồm quản trị viên, giảng viên và sinh viên có phạm vi truy cập khác nhau. Ngoài việc kiểm tra vai trò, backend còn giới hạn dữ liệu theo lớp học phần, nhóm sinh viên và tài khoản có liên quan.

Microsoft SQL Server được sử dụng để lưu trữ dữ liệu quan hệ của hệ thống. Các ràng buộc khóa chính, khóa ngoại và transaction góp phần bảo đảm tính toàn vẹn và nhất quán của dữ liệu. RabbitMQ, Notification Worker và cơ chế Transactional Outbox được sử

dụng để xử lý thông báo theo hướng bất đồng bộ, giúp hạn chế mất sự kiện và giảm thời gian phản hồi của backend.

Docker và Docker Compose hỗ trợ đóng gói, cấu hình và vận hành các thành phần trong một môi trường thống nhất. GitHub Actions được sử dụng để tự động kiểm thử, kiểm tra mã nguồn, build và triển khai frontend, backend cùng Notification Worker lên môi trường cloud.

Bên cạnh đó, hệ thống sử dụng nhiều hình thức kiểm thử như kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử API, kiểm thử giao diện và kiểm thử phân quyền. Swagger được sử dụng để xây dựng tài liệu API, hỗ trợ kiểm tra endpoint và tạo điều kiện thuận lợi cho quá trình phối hợp giữa frontend với backend.

Những cơ sở lý thuyết được trình bày trong chương này là nền tảng để nhóm tiến hành phân tích yêu cầu, xác định chức năng, thiết kế cơ sở dữ liệu và xây dựng kiến trúc tổng thể của hệ thống. Các nội dung phân tích yêu cầu và quản lý quá trình phát triển đề tài sẽ được trình bày trong chương tiếp theo.

CHƯƠNG 3: PHÂN TÍCH YÊU CẦU

3.1 Xác định nhu cầu

Trong quá trình tổ chức các học phần có đồ án, giảng viên cần quản lý nhiều thông tin như danh sách sinh viên, lớp học phần, nhóm thực hiện, đề tài đăng ký, tiến độ, yêu cầu nộp bài, bài nộp và điểm số. Khi các dữ liệu này được lưu trữ trên nhiều công cụ khác nhau như bảng tính, thư điện tử, nhóm trò chuyện và dịch vụ lưu trữ trực tuyến, việc theo dõi và tổng hợp trở nên khó khăn.

Giảng viên có thể phải kiểm tra nhiều nguồn để xác định nhóm nào đã đăng ký đề tài, nhóm nào chưa nộp bài, sinh viên đã nộp bao nhiêu lần hoặc điểm đã được công bố hay chưa. Khi số lượng lớp học phần và sinh viên tăng lên, phương pháp quản lý thủ công dễ dẫn đến nhầm lẫn, bỏ sót và mất nhiều thời gian.

Đối với sinh viên, việc sử dụng nhiều công cụ riêng lẻ cũng gây bất tiện. Sinh viên có thể không nắm rõ trạng thái đề tài, không biết yêu cầu nào đang mở, hạn nộp vào thời điểm nào hoặc giảng viên đã phản hồi ra sao. Lịch sử các lần nộp cũng khó được theo dõi nếu tệp được gửi qua thư điện tử hoặc nhóm trò chuyện.

Quản trị viên cần một công cụ để quản lý tập trung tài khoản người dùng, môn học, lớp học phần, giảng viên phụ trách và danh sách sinh viên. Nếu những dữ liệu nền tảng này không được tổ chức thống nhất, các nghiệp vụ phía sau như tạo nhóm, đăng ký đề tài và chấm điểm sẽ gặp khó khăn.

Từ những vấn đề trên, hệ thống cần đáp ứng các nhu cầu chính sau:

- Quản lý dữ liệu tập trung theo môn học và lớp học phần.
- Phân quyền rõ ràng cho quản trị viên, giảng viên và sinh viên.
- Quản lý nhóm sinh viên và đăng ký đề tài.
- Theo dõi trạng thái xét duyệt đề tài.
- Quản lý tiến độ và yêu cầu nộp bài.

- Lưu trữ bài nộp và lịch sử các lần nộp.
- Hỗ trợ chấm điểm, phản hồi và công bố kết quả.
- Gửi thông báo khi có thay đổi quan trọng.
- Giới hạn dữ liệu theo đúng lớp học phần và người dùng liên quan.

Hệ thống được xây dựng nhằm số hóa quy trình quản lý đồ án, giám thao tác thủ công, tăng khả năng theo dõi và cung cấp một nền tảng thống nhất cho toàn bộ người dùng.

3.2 Các chức năng của hệ thống

3.2.1 Các chức năng chính của hệ thống

Hệ thống được tổ chức theo ba nhóm chức năng tương ứng với ba vai trò người dùng.

- Chức năng của quản trị viên

Quản trị viên chịu trách nhiệm quản lý các dữ liệu nền tảng của hệ thống. Người dùng thuộc vai trò này có thể quản lý tài khoản, môn học, lớp học phần, giảng viên phụ trách và danh sách sinh viên.

Các chức năng chính của quản trị viên gồm:

- Đăng nhập và truy cập khu vực quản trị.
- Xem, tìm kiếm và quản lý tài khoản người dùng.
- Tạo tài khoản quản trị viên, giảng viên và sinh viên.
- Cập nhật thông tin và trạng thái hoạt động của tài khoản.
- Quản lý danh sách môn học.
- Tạo và cập nhật lớp học phần.

- Phân công giảng viên phụ trách lớp học phần.
- Thêm hoặc loại sinh viên khỏi lớp học phần.
- Theo dõi dữ liệu tổng thể của hệ thống.

Thông qua các chức năng này, quản trị viên chuẩn bị dữ liệu ban đầu để giảng viên và sinh viên có thể sử dụng hệ thống đúng phạm vi.

- Chức năng của giảng viên

Giảng viên là người trực tiếp quản lý quá trình thực hiện đồ án trong các lớp học phần được phân công. Mỗi giảng viên chỉ được truy cập dữ liệu thuộc phạm vi phụ trách.

Các chức năng chính của giảng viên gồm:

- Xem danh sách lớp học phần đang phụ trách.
- Xem danh sách sinh viên trong từng lớp.
- Theo dõi nhóm sinh viên và thành viên của từng nhóm.
- Xem các đăng ký đề tài.
- Duyệt, từ chối hoặc yêu cầu chỉnh sửa đề tài.
- Theo dõi trạng thái đề tài theo từng lớp học phần.
- Tạo các mốc tiến độ cho quá trình thực hiện đồ án.
- Tạo và quản lý yêu cầu nộp bài.
- Thiết lập thời hạn, trạng thái và số lần nộp.
- Xem bài nộp và lịch sử các lần nộp.
- Nhập điểm và phản hồi cho sinh viên.
- Công bố hoặc thu hồi trạng thái công bố điểm.

- Theo dõi các thông báo liên quan đến lớp phụ trách.

Khi đề tài đã được duyệt, hệ thống khóa nội dung đăng ký để tránh việc thay đổi không được kiểm soát. Đây là một quy tắc nghiệp vụ quan trọng nhằm bảo đảm tính thống nhất trong quá trình thực hiện.

- Chức năng của sinh viên

Sinh viên sử dụng hệ thống để theo dõi lớp học phần, tham gia nhóm, đăng ký đề tài, thực hiện yêu cầu và xem kết quả đánh giá.

- Các chức năng chính của sinh viên gồm:
- Đăng nhập và xem thông tin tài khoản.
- Xem các lớp học phần đang tham gia.
- Xem thông tin nhóm và thành viên trong nhóm.
- Đăng ký đề tài cho nhóm.
- Chỉnh sửa đề tài khi giảng viên yêu cầu.
- Theo dõi trạng thái xét duyệt đề tài.
- Xem các mốc tiến độ.
- Xem danh sách yêu cầu nộp bài.
- Theo dõi thời hạn và điều kiện của từng yêu cầu.
- Nộp bài hoặc nộp lại khi được phép.
- Xem lịch sử các lần nộp.
- Xem phản hồi của giảng viên.
- Xem điểm số đã được công bố.

- Xem và đánh dấu thông báo đã đọc.

Sinh viên chỉ được truy cập dữ liệu của các lớp học phần và nhóm mà mình tham gia. Việc giới hạn phạm vi giúp bảo vệ dữ liệu và tránh hiển thị nhầm thông tin giữa các lớp.

3.3.2 Các yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm bảo đảm an toàn dữ liệu, khả năng vận hành ổn định, hiệu quả sử dụng và thuận tiện trong quá trình bảo trì, triển khai. Các yêu cầu này được xem xét xuyên suốt từ giai đoạn thiết kế, phát triển, kiểm thử đến khi hệ thống được đưa lên môi trường cloud.

- Yêu cầu về bảo mật:

- + Người dùng phải đăng nhập trước khi truy cập các chức năng được bảo vệ. Hệ thống sử dụng JSON Web Token để xác thực yêu cầu và xác định vai trò của người dùng. Sau khi đăng nhập thành công, token được gửi kèm trong các yêu cầu tiếp theo để backend kiểm tra trước khi cho phép truy cập dữ liệu.

- + Mật khẩu người dùng phải được băm trước khi lưu vào cơ sở dữ liệu. Các thông tin nhạy cảm như mật khẩu SQL Server, khóa bí mật JWT, địa chỉ kết nối RabbitMQ và Render Deploy Hook được lưu trong biến môi trường, không được ghi trực tiếp vào mã nguồn hoặc đưa lên kho lưu trữ GitHub.

- + Backend phải kiểm tra quyền truy cập đối với từng API. Ngoài việc kiểm tra vai trò, hệ thống còn xác định phạm vi dữ liệu mà người dùng được phép thao tác. Giảng viên chỉ được quản lý các lớp học phần được phân công, trong khi sinh viên chỉ được truy cập dữ liệu thuộc lớp và nhóm mà mình tham gia.

- Yêu cầu về hiệu năng:

- + Hệ thống cần phản hồi trong thời gian hợp lý khi người dùng thực hiện các thao tác như đăng nhập, xem lớp học phần, tìm kiếm sinh viên, lọc đề tài hoặc tải danh sách bài

nộp. Các truy vấn dữ liệu cần được giới hạn theo lớp học phần, người dùng và trạng thái để tránh lấy những bản ghi không cần thiết.

+ Đối với các tác vụ không cần hoàn thành ngay trong luồng yêu cầu chính, hệ thống sử dụng RabbitMQ và Notification Worker để xử lý bất đồng bộ. Cách thực hiện này giúp backend trả kết quả sớm hơn và giảm ảnh hưởng của quá trình tạo thông báo đến thời gian phản hồi của API.

- Yêu cầu về tính toàn vẹn dữ liệu:

+ Dữ liệu trong hệ thống phải được liên kết thông qua khóa chính và khóa ngoại. Các bản ghi chỉ được tạo khi những đối tượng liên quan tồn tại hợp lệ. Ví dụ, một bài nộp phải thuộc về một yêu cầu nộp bài, thành viên nhóm phải thuộc một nhóm sinh viên và nhóm phải nằm trong một lớp học phần cụ thể.

+ Các nghiệp vụ gồm nhiều thao tác cập nhật cần được thực hiện trong transaction. Khi một thao tác thất bại, toàn bộ thay đổi liên quan phải được hoàn tác nhằm tránh tình trạng dữ liệu chỉ được lưu thành công một phần.

+ Hệ thống cũng cần kiểm tra các quy tắc nghiệp vụ trước khi cập nhật dữ liệu. Đề tài đã được duyệt không được tự ý chỉnh sửa, điểm chưa công bố không được hiển thị cho sinh viên và bài nộp chỉ được tiếp nhận khi yêu cầu vẫn còn hiệu lực.

- Yêu cầu về khả năng sử dụng:

+ Giao diện phải rõ ràng, dễ hiểu và phù hợp với từng vai trò người dùng. Quản trị viên, giảng viên và sinh viên được cung cấp menu và chức năng tương ứng với nhiệm vụ của mình.

+ Các màn hình danh sách cần hỗ trợ tìm kiếm, lọc dữ liệu và hiển thị trạng thái bằng nội dung dễ hiểu. Những trường hợp không có dữ liệu, đang tải hoặc xảy ra lỗi cần được thông báo rõ ràng, tránh hiển thị trực tiếp các thông tin kỹ thuật khó hiểu cho người dùng.

+ Website phải có khả năng hiển thị phù hợp trên nhiều kích thước màn hình. Những nội dung dài như tên môn học, tên lớp học phần, tên đề tài và tên tệp không được làm tràn hoặc phá vỡ bố cục giao diện.

- Yêu cầu về độ tin cậy:

+ Hệ thống cần xử lý phù hợp các trường hợp mất kết nối, token hết hạn, tài nguyên không tồn tại hoặc người dùng không có quyền truy cập. Mỗi trường hợp phải trả về mã trạng thái và thông báo rõ ràng để frontend có thể hiển thị cho người dùng.

+ Đối với chức năng thông báo, sự kiện được ghi vào bảng NotificationOutbox trước khi gửi đến RabbitMQ. Cơ chế Transactional Outbox giúp hạn chế việc dữ liệu nghiệp vụ đã được lưu nhưng sự kiện thông báo bị mất khi dịch vụ hàng đợi gặp sự cố.

+ Hệ thống cũng lưu trạng thái xử lý của sự kiện, bao gồm đang chờ, đang xử lý, đã gửi hoặc thất bại. Điều này giúp theo dõi và hỗ trợ xử lý lại khi cần thiết.

- Yêu cầu về khả năng bảo trì:

+ Backend được tổ chức theo các tầng Routes, Controllers, Services và Repositories. Mỗi tầng có trách nhiệm riêng nhằm hạn chế việc trộn lẫn giữa xử lý yêu cầu, nghiệp vụ và truy vấn cơ sở dữ liệu.

+ RESTful API được mô tả bằng Swagger để hỗ trợ kiểm tra và tra cứu. Mã nguồn được quản lý bằng Git và GitHub, giúp theo dõi lịch sử thay đổi và phối hợp phát triển.

+ Trước khi triển khai, các thay đổi quan trọng phải vượt qua những bước kiểm tra gồm backend test, frontend lint, frontend build và kiểm tra lỗi định dạng Git. Cách tổ chức này giúp giảm nguy cơ đưa mã nguồn lỗi lên môi trường hoạt động.

- Yêu cầu về triển khai:

+ Các thành phần của hệ thống được đóng gói bằng Docker để tạo môi trường chạy thống nhất. Docker Compose được sử dụng để khởi chạy frontend, backend, SQL Server, RabbitMQ và Notification Worker trong môi trường phát triển cục bộ.

+ GitHub Actions được cấu hình để tự động kiểm thử và build hệ thống khi mã nguồn được cập nhật lên nhánh chính. Chỉ khi các bước kiểm tra đều thành công, workflow mới kích hoạt quá trình triển khai frontend, backend và Notification Worker lên Render.

+ Cơ sở dữ liệu SQL Server được triển khai trên MonsterASP, trong khi RabbitMQ được cung cấp bởi CloudAMQP. Thông tin kết nối giữa các dịch vụ được quản lý bằng biến môi trường riêng trên từng nền tảng.

Bảng 3.1. Tổng hợp các yêu cầu phi chức năng

Nhóm yêu cầu	Nội dung chính
Bảo mật	Xác thực bằng JWT, phân quyền theo vai trò và phạm vi dữ liệu, băm mật khẩu và bảo vệ các biến môi trường.
Hiệu năng	Tối ưu truy vấn theo lớp học phần, người dùng và trạng thái; xử lý thông báo bất đồng bộ bằng RabbitMQ.
Toàn vẹn dữ liệu	Sử dụng khóa chính, khóa ngoại, transaction và kiểm tra các quy tắc nghiệp vụ trước khi cập nhật.
Khả năng sử dụng	Giao diện rõ ràng, hỗ trợ tìm kiếm, bộ lọc, trạng thái dữ liệu và hiển thị responsive.
Độ tin cậy	Xử lý lỗi rõ ràng, sử dụng Transactional Outbox và lưu trạng thái xử lý sự kiện.
Khả năng bảo trì	Tổ chức backend theo kiến trúc phân tầng, sử dụng Swagger, Git, test, lint và build.
Triển khai	Sử dụng Docker, Docker Compose, GitHub Actions, Render, MonsterASP và CloudAMQP.

3.3 Quản lý dự án

Quản lý dự án là quá trình lập kế hoạch, phân chia công việc, theo dõi tiến độ và kiểm soát chất lượng trong suốt quá trình xây dựng hệ thống. Đối với đề tài “Xây dựng website quản lý đồ án sinh viên”, nhóm chia công việc thành nhiều giai đoạn nhỏ tương ứng với từng nhóm chức năng.

Trong mỗi giai đoạn, nhóm xác định mục tiêu cần hoàn thành, các chức năng liên quan, kết quả đầu ra và những công việc cần kiểm tra. Cách tổ chức này giúp hạn chế tình trạng phát triển dàn trải, đồng thời tạo điều kiện để nhóm kiểm thử và cải tiến hệ thống sau từng giai đoạn.

Mã nguồn của dự án được quản lý bằng Git và lưu trữ trên GitHub. Các thay đổi được ghi nhận thông qua commit, giúp nhóm theo dõi lịch sử phát triển, kiểm tra nội dung đã thực hiện và hạn chế mất mã nguồn. Trước khi triển khai, hệ thống được kiểm tra bằng backend test, frontend lint, frontend build và quy trình GitHub Actions.

3.3.1 Kế hoạch để phân tích và thiết kế giao diện

Giai đoạn đầu của dự án tập trung vào việc tìm hiểu quy trình quản lý đồ án sinh viên và xác định những vấn đề cần giải quyết. Nhóm tiến hành phân tích nhu cầu của ba nhóm người dùng gồm quản trị viên, giảng viên và sinh viên.

Đối với quản trị viên, hệ thống cần hỗ trợ quản lý tài khoản, môn học, lớp học phần, giảng viên phụ trách và danh sách sinh viên. Đối với giảng viên, hệ thống cần cung cấp các chức năng quản lý nhóm, đề tài, tiến độ, yêu cầu nộp bài, bài nộp và điểm số. Đối với sinh viên, hệ thống cần hỗ trợ theo dõi lớp học phần, tham gia nhóm, đăng ký đề tài, nộp bài và xem kết quả.

Sau khi tổng hợp nhu cầu, nhóm xác định các yêu cầu chức năng, yêu cầu phi chức năng và các quy tắc nghiệp vụ quan trọng. Một số quy tắc được chú trọng gồm giới hạn dữ liệu theo lớp học phần, khóa đề tài sau khi được duyệt, kiểm tra thời hạn nộp bài, lưu lịch sử các lần nộp và chỉ hiển thị điểm khi giảng viên đã công bố.

Trên cơ sở các yêu cầu đã xác định, nhóm tiến hành thiết kế kiến trúc tổng thể của hệ thống. Frontend và backend được tách thành hai dự án riêng biệt. Backend được tổ chức theo các tầng Routes, Controllers, Services và Repositories. Cơ sở dữ liệu được thiết kế bằng SQL Server, còn chức năng thông báo được xử lý bằng RabbitMQ và Notification Worker.

Giao diện được định hướng theo từng vai trò người dùng. Mỗi vai trò có thanh điều hướng, trang tổng quan và các màn hình nghiệp vụ riêng. Bố cục được thiết kế theo hướng đơn giản, dễ theo dõi, hỗ trợ tìm kiếm, lọc dữ liệu và hiển thị trên nhiều kích thước màn hình.

Bảng 3.2. Kế hoạch phân tích yêu cầu và thiết kế hệ thống

Công việc	Nội dung thực hiện	Kết quả đầu ra
Khảo sát nhu cầu	Tìm hiểu quy trình quản lý lớp học phần, nhóm sinh viên, đăng ký đề tài, tiến độ, bài nộp và điểm số.	Danh sách các vấn đề thực tế và nhu cầu của người dùng.
Xác định vai trò	Phân tích nhiệm vụ, quyền hạn và phạm vi sử dụng của quản trị viên, giảng viên và sinh viên.	Danh sách chức năng và quyền truy cập của từng vai trò.
Phân tích yêu cầu	Xác định yêu cầu chức năng, yêu cầu phi chức năng và các quy tắc nghiệp vụ của hệ thống.	Tài liệu yêu cầu hệ thống làm cơ sở cho quá trình phát triển.
Thiết kế kiến trúc	Xác định mô hình Client–Server, kiến trúc frontend–backend tách biệt, backend phân tầng và Notification Worker.	Sơ đồ kiến trúc tổng thể của hệ thống.
Thiết kế cơ sở dữ liệu	Xác định các bảng, thuộc tính, khóa chính, khóa ngoại và mối quan hệ giữa các đối tượng.	Mô hình cơ sở dữ liệu và sơ đồ quan hệ dữ liệu.
Thiết kế giao diện	Xây dựng bố cục, luồng thao tác và các màn hình chính theo từng vai trò người dùng.	Nguyên mẫu giao diện dành cho quản trị viên, giảng viên và sinh viên.

3.3.2 Kế hoạch xây dựng backend và RESTful API

Sau khi hoàn thành phân tích yêu cầu và thiết kế hệ thống, nhóm tiến hành xây dựng backend bằng Node.js và ExpressJS. Quá trình phát triển backend được chia thành nhiều nhóm nghiệp vụ để thuận tiện cho việc kiểm tra và tích hợp.

Nhóm chức năng đầu tiên là xác thực và phân quyền người dùng. Backend xử lý đăng nhập, kiểm tra mật khẩu, tạo JWT và xác định vai trò của tài khoản. Các middleware được xây dựng để bảo vệ endpoint và giới hạn quyền truy cập.

Nhóm chức năng tiếp theo tập trung vào dữ liệu nền tảng gồm người dùng, môn học, lớp học phần và danh sách sinh viên. Đây là các dữ liệu cần thiết để triển khai những nghiệp vụ như tạo nhóm, đăng ký đề tài và yêu cầu nộp bài.

Sau khi dữ liệu nền tảng hoạt động ổn định, nhóm tiếp tục phát triển API quản lý nhóm sinh viên và đăng ký đề tài. Các API phải kiểm tra sinh viên có thuộc lớp học phần hay không, nhóm có hợp lệ hay không và đề tài đã được duyệt hay chưa.

Tiếp theo, nhóm xây dựng API quản lý mốc tiến độ, yêu cầu nộp bài, bài nộp và lịch sử nộp. Các quy tắc như thời hạn, số lần nộp, trạng thái yêu cầu và quyền nộp lại được xử lý tại tầng service.

Nhóm chức năng cuối cùng gồm chấm điểm, công bố kết quả và thông báo. Khi xảy ra các sự kiện quan trọng, backend ghi nhận sự kiện vào NotificationOutbox để chuyển đến RabbitMQ và Notification Worker xử lý.

Các endpoint được mô tả bằng Swagger và kiểm tra bằng Postman. Sau mỗi nhóm chức năng, nhóm chạy kiểm thử để phát hiện lỗi về nghiệp vụ, phân quyền, dữ liệu đầu vào và mã trạng thái trả về.

Bảng 3.3. Kế hoạch xây dựng backend và RESTful API

Giai đoạn	Nội dung thực hiện	Kết quả cần đạt
Xác thực và phân quyền	Xây dựng chức năng đăng nhập, tạo JWT, kiểm tra token và middleware phân quyền theo vai trò.	Người dùng đăng nhập thành công và chỉ truy cập được các chức năng phù hợp với quyền hạn.
Quản lý dữ liệu nền tảng	Xây dựng API quản lý người dùng, môn học, lớp học phần, giảng viên phụ trách và danh	Các dữ liệu cơ bản được quản lý đầy đủ, chính xác và làm nền tảng cho các nghiệp vụ tiếp theo.

Giai đoạn	Nội dung thực hiện	Kết quả cần đạt
	sách sinh viên.	
Quản lý nhóm và đề tài	Xây dựng API tạo nhóm, quản lý thành viên, đăng ký đề tài và xử lý trạng thái đề tài.	Nhóm sinh viên và đề tài được quản lý đúng theo từng lớp học phần.
Quản lý tiến độ và yêu cầu nộp bài	Xây dựng API tạo mốc tiến độ, yêu cầu nộp bài, thời hạn, số lần nộp và trạng thái hoạt động.	Giảng viên có thể tổ chức và theo dõi tiến độ thực hiện đồ án của sinh viên.
Quản lý bài nộp và lịch sử nộp	Xây dựng API tiếp nhận bài nộp, nộp lại và lưu lịch sử các lần nộp.	Sinh viên nộp bài đúng quy định, giảng viên theo dõi được toàn bộ quá trình nộp.
Quản lý điểm và phản hồi	Xây dựng API nhập điểm, cập nhật phản hồi, công bố hoặc thu hồi điểm.	Giảng viên đánh giá được bài nộp và sinh viên xem được kết quả sau khi điểm được công bố.
Xử lý thông báo	Tích hợp NotificationOutbox, RabbitMQ và Notification Worker để xử lý sự kiện bất đồng bộ.	Thông báo được tạo đúng người nhận, hạn chế mất sự kiện khi dịch vụ tạm thời gặp lỗi.
Kiểm thử và tài liệu API	Kiểm tra các endpoint bằng Unit Test, Swagger và Postman; xử lý các trường hợp lỗi và phân quyền.	RESTful API hoạt động đúng nghiệp vụ, trả mã trạng thái phù hợp và có tài liệu rõ ràng.

3.3.3 Kế hoạch phát triển giao diện người dùng

Sau khi các API chính của backend được xây dựng, nhóm tiến hành phát triển giao diện người dùng bằng ReactJS và Vite. Giao diện được tổ chức theo từng vai trò gồm quản trị viên, giảng viên và sinh viên nhằm bảo đảm mỗi người dùng chỉ nhìn thấy những chức năng phù hợp với quyền hạn của mình.

Giai đoạn đầu tập trung xây dựng các thành phần dùng chung như trang đăng nhập, thanh điều hướng, bố cục chính, thông báo lỗi, trạng thái tải dữ liệu và chức năng đăng

xuất. Sau khi người dùng đăng nhập thành công, hệ thống dựa vào vai trò trong JWT để chuyển đến khu vực chức năng tương ứng.

Đối với quản trị viên, nhóm xây dựng các màn hình quản lý tài khoản, môn học, lớp học phần, giảng viên phụ trách và danh sách sinh viên. Các màn hình danh sách được bổ sung chức năng tìm kiếm, lọc dữ liệu, thêm mới, cập nhật và hiển thị trạng thái hoạt động.

Đối với giảng viên, giao diện tập trung vào các lớp học phần đang phụ trách. Từ mỗi lớp, giảng viên có thể xem danh sách sinh viên, nhóm thực hiện, đăng ký đề tài, yêu cầu nộp bài, bài nộp, lịch sử nộp và điểm số. Các bộ lọc theo lớp học phần và trạng thái được sử dụng để giúp giảng viên dễ tìm kiếm dữ liệu.

Đối với sinh viên, nhóm phát triển các màn hình xem lớp học phần, thông tin nhóm, đăng ký đề tài, tiến độ, yêu cầu nộp bài, bài cuối kỳ, điểm số và thông báo. Giao diện được thiết kế theo hướng đơn giản để sinh viên dễ theo dõi những công việc cần thực hiện.

Trong quá trình phát triển, nhóm kiểm tra khả năng hiển thị trên nhiều kích thước màn hình. Những nội dung dài như tên môn học, lớp học phần, đề tài và tên tệp được xử lý để không làm tràn hoặc phá vỡ bố cục. Các trạng thái không có dữ liệu, đang tải hoặc lỗi kết nối cũng được bổ sung để giao diện thân thiện hơn.

Bảng 3.4. Kế hoạch phát triển giao diện người dùng

Giai đoạn	Nội dung thực hiện	Kết quả cần đạt
Xây dựng giao diện dùng chung	Phát triển trang đăng nhập, thanh điều hướng, bố cục chính, trạng thái tải dữ liệu và thông báo lỗi.	Giao diện có cấu trúc thống nhất và hỗ trợ điều hướng phù hợp theo từng vai trò.
Giao diện quản trị viên	Xây dựng các màn hình quản lý người dùng, môn học, lớp học phần, giảng viên và danh sách sinh viên.	Quản trị viên có thể quản lý đầy đủ các dữ liệu nền tảng của hệ thống.
Giao diện giảng viên	Xây dựng các màn hình quản lý lớp phụ trách, sinh viên,	Giảng viên có thể theo dõi và quản lý toàn bộ quá trình thực

Giai đoạn	Nội dung thực hiện	Kết quả cần đạt
	nhóm, đề tài, yêu cầu nộp bài, bài nộp và điểm số.	hiện đồ án.
Giao diện sinh viên	Xây dựng các màn hình lớp học phần, nhóm, đề tài, tiến độ, yêu cầu nộp bài, điểm số và thông báo.	Sinh viên có thể chủ động theo dõi công việc, bài nộp và kết quả đánh giá.
Tìm kiếm và bộ lọc	Bổ sung chức năng tìm kiếm và lọc dữ liệu theo lớp học phần, từ khóa và trạng thái.	Người dùng có thể tìm kiếm và tra cứu thông tin nhanh chóng, chính xác.
Hoàn thiện giao diện responsive	Kiểm tra bố cục, bảng dữ liệu, nội dung dài và khả năng hiển thị trên nhiều kích thước màn hình.	Giao diện hiển thị ổn định, không bị tràn hoặc chồng lấn nội dung.
Tích hợp RESTful API	Kết nối các màn hình frontend với backend và xử lý dữ liệu trả về.	Frontend hiển thị đúng dữ liệu và xử lý phù hợp các trường hợp thành công hoặc lỗi.

3.3.4 Kế hoạch kiểm thử hệ thống

Kiểm thử được thực hiện song song với quá trình phát triển nhằm phát hiện lỗi sớm và hạn chế ảnh hưởng giữa các chức năng. Sau khi hoàn thành một nhóm nghiệp vụ, nhóm tiến hành kiểm tra logic backend, RESTful API, giao diện và quyền truy cập của người dùng.

Đối với backend, nhóm xây dựng các trường hợp kiểm thử cho những nghiệp vụ quan trọng như đăng nhập, phân quyền, quản lý lớp học phần, đăng ký đề tài, nộp bài, chấm điểm và xử lý thông báo. Các test được chạy tự động để bảo đảm những thay đổi mới không làm ảnh hưởng đến chức năng đã hoàn thành trước đó.

Các endpoint được kiểm tra bằng Swagger và Postman. Nhóm kiểm tra cả trường hợp thành công và thất bại, bao gồm dữ liệu không hợp lệ, thiếu token, token hết hạn, không đúng vai trò hoặc truy cập dữ liệu ngoài phạm vi cho phép.

Đối với frontend, nhóm kiểm tra các luồng thao tác của quản trị viên, giảng viên và sinh viên. Các nội dung được kiểm tra gồm đăng nhập, chuyển trang, tìm kiếm, lọc dữ liệu, gửi biểu mẫu, hiển thị thông báo và xử lý lỗi từ backend.

Kiểm thử phân quyền được thực hiện bằng nhiều tài khoản thuộc từng vai trò. Giảng viên chỉ được truy cập các lớp học phần được phân công, trong khi sinh viên chỉ được xem dữ liệu của lớp và nhóm mà mình tham gia.

Nhóm cũng tạo bộ dữ liệu demo có số lượng lớn để kiểm tra các màn hình danh sách, bảng, nhóm dữ liệu và lịch sử. Việc kiểm tra với nhiều bản ghi giúp phát hiện các lỗi về bộ lọc, phân trang, hiển thị nội dung dài và phạm vi dữ liệu.

Kết quả kiểm thử backend tại thời điểm hoàn thiện hệ thống đạt 58/58 trường hợp kiểm thử thành công. Các lỗi phát hiện trong quá trình kiểm thử được sửa và kiểm tra lại trước khi đưa mã nguồn lên môi trường triển khai.

Bảng 3.5. Kế hoạch kiểm thử hệ thống

Nội dung kiểm thử	Phương pháp thực hiện	Kết quả cần đạt
Kiểm thử nghiệp vụ backend	Chạy các trường hợp Unit Test và Integration Test cho những nghiệp vụ quan trọng.	Các quy tắc nghiệp vụ hoạt động chính xác và không phát sinh lỗi hồi quy.
Kiểm thử RESTful API	Sử dụng Swagger, Postman và các trường hợp kiểm thử tự động.	API trả về đúng dữ liệu, mã trạng thái và thông báo lỗi phù hợp.
Kiểm thử xác thực	Kiểm tra đăng nhập, JWT, token hết hạn, token thiếu hoặc không hợp lệ.	Chỉ tài khoản hợp lệ mới có thể truy cập các chức năng được bảo vệ.
Kiểm thử phân quyền	Đăng nhập bằng tài khoản quản	Mỗi người dùng chỉ được

Nội dung kiểm thử	Phương pháp thực hiện	Kết quả cần đạt
	trị viên, giảng viên và sinh viên để kiểm tra quyền truy cập.	truy cập chức năng và dữ liệu thuộc phạm vi cho phép.
Kiểm thử giao diện	Kiểm tra điều hướng, biểu mẫu, bảng dữ liệu, bộ lọc, thông báo và trạng thái tải.	Giao diện hoạt động ổn định, rõ ràng và thuận tiện khi sử dụng.
Kiểm thử responsive	Kiểm tra hệ thống trên nhiều kích thước màn hình và với nội dung dài.	Bố cục không bị tràn, chồng lấn hoặc mất nội dung.
Kiểm thử với dữ liệu lớn	Sử dụng bộ dữ liệu demo gồm nhiều tài khoản, lớp học phân, nhóm, bài nộp và điểm số.	Các màn hình hiển thị đúng dữ liệu và vẫn hoạt động ổn định.
Kiểm thử hồi quy	Chạy lại toàn bộ bộ kiểm thử sau mỗi thay đổi quan trọng.	Các chức năng đã hoàn thành trước đó không bị ảnh hưởng bởi thay đổi mới.

3.3.5 Kế hoạch triển khai hệ thống

Sau khi hoàn thành các chức năng chính và kiểm thử hệ thống, nhóm tiến hành triển khai sản phẩm lên môi trường cloud để người dùng có thể truy cập thông qua Internet. Việc triển khai được thực hiện riêng cho frontend, backend, Notification Worker, cơ sở dữ liệu và hệ thống hàng đợi thông báo.

Frontend ReactJS được build thành các tệp tĩnh và triển khai trên Render. Backend Node.js và Notification Worker cũng được triển khai thành các dịch vụ riêng biệt. Việc tách các thành phần giúp nhóm dễ dàng theo dõi trạng thái hoạt động, cập nhật phiên bản và xử lý lỗi khi một dịch vụ gặp sự cố.

Cơ sở dữ liệu SQL Server được triển khai trên MonsterASP. Backend kết nối đến cơ sở dữ liệu thông qua chuỗi kết nối được lưu trong biến môi trường. Các thông tin như địa chỉ máy chủ, tên cơ sở dữ liệu, tài khoản và mật khẩu không được ghi trực tiếp vào mã nguồn.

RabbitMQ được cung cấp thông qua CloudAMQP để tiếp nhận và phân phối các sự kiện thông báo. Notification Worker kết nối đến RabbitMQ, nhận thông điệp từ hàng đợi và tạo thông báo cho người dùng tương ứng.

Các thành phần của hệ thống được đóng gói bằng Docker nhằm bảo đảm môi trường chạy nhất quán. Trong môi trường phát triển cục bộ, Docker Compose hỗ trợ khởi chạy đồng thời frontend, backend, SQL Server, RabbitMQ và Notification Worker.

Quy trình triển khai được tích hợp với GitHub Actions. Khi mã nguồn được cập nhật lên nhánh chính, workflow tự động chạy kiểm thử backend, kiểm tra frontend và thực hiện build. Chỉ khi toàn bộ bước kiểm tra thành công, hệ thống mới kích hoạt quá trình triển khai phiên bản mới.

Bảng 3.6. Kế hoạch triển khai hệ thống

Thành phần	Nội dung triển khai	Kết quả cần đạt
Frontend	Build ứng dụng ReactJS và triển khai trên Render.	Người dùng có thể truy cập giao diện hệ thống thông qua trình duyệt.
Backend	Đóng gói dịch vụ Node.js và triển khai thành Web Service trên Render.	RESTful API hoạt động ổn định và kết nối được với các dịch vụ liên quan.
Notification Worker	Triển khai worker thành dịch vụ riêng để tiếp nhận và xử lý thông điệp.	Các sự kiện thông báo được xử lý bất đồng bộ.
Cơ sở dữ liệu	Triển khai SQL Server trên MonsterASP và cấu hình chuỗi kết nối.	Dữ liệu được lưu trữ tập trung và backend có thể truy cập an toàn.
RabbitMQ	Tạo dịch vụ RabbitMQ trên CloudAMQP và cấu hình hàng đợi.	Backend và worker có thể gửi, nhận sự kiện thông báo.
Docker	Xây dựng Dockerfile và cấu hình Docker Compose cho môi trường	Các thành phần chạy trong môi trường thống nhất và dễ

Thành phần	Nội dung triển khai	Kết quả cần đạt
	local.	cài đặt.
CI/CD	Cấu hình GitHub Actions để test, lint, build và kích hoạt triển khai.	Phiên bản mới chỉ được triển khai khi các bước kiểm tra thành công.
Biến môi trường	Cấu hình khóa JWT, chuỗi kết nối, RabbitMQ URL và Deploy Hook.	Thông tin bí mật được bảo vệ và không xuất hiện trong mã nguồn.

3.3.6 Kế hoạch quản lý rủi ro

Trong quá trình phát triển hệ thống, nhóm có thể gặp các rủi ro liên quan đến yêu cầu nghiệp vụ, tiến độ, dữ liệu, tích hợp API và môi trường triển khai. Việc xác định trước các rủi ro giúp nhóm chuẩn bị phương án xử lý và hạn chế ảnh hưởng đến kết quả của đề tài.

Một trong những rủi ro thường gặp là yêu cầu thay đổi trong quá trình phát triển. Khi phát hiện chức năng chưa phù hợp với quy trình thực tế, nhóm cần đánh giá mức độ ảnh hưởng trước khi chỉnh sửa. Các thay đổi quan trọng phải được cập nhật đồng thời ở cơ sở dữ liệu, backend, frontend và tài liệu API.

Rủi ro về phân quyền có thể khiến người dùng truy cập dữ liệu không thuộc phạm vi quản lý. Để hạn chế vấn đề này, backend phải kiểm tra cả vai trò và phạm vi dữ liệu tại từng nghiệp vụ. Nhóm cũng thực hiện kiểm thử bằng nhiều tài khoản quản trị viên, giảng viên và sinh viên.

Quá trình tích hợp frontend với backend có thể phát sinh lỗi do sai đường dẫn, phương thức HTTP, tên thuộc tính hoặc cấu trúc dữ liệu trả về. Swagger được sử dụng làm tài liệu thống nhất để frontend và backend phối hợp trong quá trình phát triển.

Đối với cơ sở dữ liệu, lỗi kết nối hoặc sai cấu trúc bảng có thể làm hệ thống ngừng hoạt động. Nhóm cần quản lý phiên bản câu lệnh SQL, kiểm tra khóa ngoại, sao lưu dữ liệu quan trọng và hạn chế thay đổi trực tiếp trên môi trường triển khai.

RabbitMQ hoặc Notification Worker có thể tạm thời mất kết nối. Để giảm nguy cơ mất thông báo, sự kiện được lưu trong NotificationOutbox trước khi được gửi đến hàng đợi. Những sự kiện xử lý thất bại có thể được theo dõi và thực hiện lại.

Môi trường cloud cũng có thể gặp tình trạng khởi động chậm, giới hạn tài nguyên hoặc gián đoạn dịch vụ. Nhóm cần theo dõi log của từng thành phần, kiểm tra trạng thái kết nối và chuẩn bị môi trường local bằng Docker để phục vụ kiểm tra khi cần thiết.

Bảng 3.7. Kế hoạch quản lý rủi ro

Rủi ro	Ảnh hưởng có thể xảy ra	Biện pháp xử lý
Thay đổi yêu cầu	Chức năng đã xây dựng phải chỉnh sửa, ảnh hưởng đến tiến độ.	Phân tích mức độ ảnh hưởng, ưu tiên chức năng chính và cập nhật tài liệu trước khi thực hiện.
Chậm tiến độ	Một số chức năng không hoàn thành đúng kế hoạch.	Chia công việc thành giai đoạn nhỏ, theo dõi tiến độ và ưu tiên nghiệp vụ quan trọng.
Lỗi phân quyền	Người dùng truy cập chức năng hoặc dữ liệu không thuộc quyền hạn.	Kiểm tra vai trò và phạm vi dữ liệu tại backend, đồng thời kiểm thử bằng nhiều tài khoản.
Lỗi tích hợp API	Frontend hiển thị sai dữ liệu hoặc không gửi được yêu cầu.	Sử dụng Swagger, thống nhất cấu trúc request và response, kiểm tra mã trạng thái HTTP.
Lỗi cơ sở dữ liệu	Mất kết nối, dữ liệu không nhất quán hoặc phát sinh bản ghi không hợp lệ.	Sử dụng khóa ngoại, transaction, sao lưu dữ liệu và kiểm tra câu lệnh trước khi triển khai.
RabbitMQ mất kết nối	Thông báo không được gửi hoặc xử lý đúng thời điểm.	Sử dụng Transactional Outbox, lưu trạng thái sự kiện và hỗ trợ xử lý lại.
Lỗi triển khai	Dịch vụ không khởi động hoặc không kết nối được với thành phần khác.	Kiểm tra log, biến môi trường, Dockerfile và trạng thái của từng dịch vụ.
Lộ thông tin bí	Mật khẩu hoặc khóa truy	Lưu thông tin trong biến môi trường

Rủi ro	Ảnh hưởng có thể xảy ra	Biện pháp xử lý
mật	cập bị công khai trong mã nguồn.	và không đưa tệp cấu hình bí mật lên GitHub.
Lỗi hồi quy	Thay đổi mới làm hỏng chức năng đã hoàn thành.	Chạy lại toàn bộ test, lint và build trước mỗi lần triển khai.

3.3.7 Kế hoạch tiến độ thực hiện dự án

Để kiểm soát tiến độ, quá trình xây dựng website quản lý đồ án sinh viên được chia thành nhiều giai đoạn. Mỗi giai đoạn tập trung vào một nhóm công việc cụ thể và có kết quả đầu ra rõ ràng.

Giai đoạn đầu tập trung khảo sát quy trình quản lý đồ án, xác định đối tượng sử dụng, yêu cầu chức năng, yêu cầu phi chức năng và các quy tắc nghiệp vụ. Kết quả của giai đoạn này là tài liệu yêu cầu làm cơ sở cho việc thiết kế hệ thống.

Sau khi hoàn thành phân tích, nhóm tiến hành thiết kế kiến trúc tổng thể, cơ sở dữ liệu, RESTful API và giao diện cho từng vai trò. Các bảng dữ liệu, mối quan hệ và phạm vi truy cập được xác định trước khi phát triển chức năng.

Giai đoạn xây dựng backend được thực hiện theo từng nhóm nghiệp vụ, bắt đầu từ xác thực, phân quyền và dữ liệu nền tảng. Sau đó, nhóm tiếp tục phát triển các chức năng quản lý nhóm, đề tài, tiến độ, bài nộp, điểm số và thông báo.

Frontend được xây dựng song song với quá trình hoàn thiện API. Các màn hình được tích hợp lần lượt theo vai trò quản trị viên, giảng viên và sinh viên. Sau mỗi nhóm chức năng, nhóm kiểm tra dữ liệu hiển thị, luồng thao tác và xử lý lỗi.

Khi các chức năng chính hoàn thành, hệ thống được kiểm thử tổng thể bằng dữ liệu demo. Các lỗi liên quan đến nghiệp vụ, phân quyền, giao diện và tích hợp được ghi nhận, sửa chữa và kiểm tra lại.

Giai đoạn cuối tập trung đóng gói bằng Docker, cấu hình GitHub Actions và triển khai các thành phần lên môi trường cloud. Sau khi triển khai, nhóm tiếp tục kiểm tra hệ thống trong môi trường thực tế và hoàn thiện tài liệu báo cáo.

Bảng 3.8. Kế hoạch tiến độ thực hiện dự án

Giai đoạn	Công việc thực hiện	Kết quả đầu ra
Khảo sát và phân tích	Tìm hiểu quy trình quản lý đồ án, xác định vai trò, chức năng và quy tắc nghiệp vụ.	Tài liệu yêu cầu chức năng và phi chức năng.
Thiết kế hệ thống	Thiết kế kiến trúc, cơ sở dữ liệu, RESTful API và giao diện.	Sơ đồ kiến trúc, mô hình dữ liệu và nguyên mẫu giao diện.
Xây dựng backend	Phát triển xác thực, phân quyền và các API nghiệp vụ.	Backend hoạt động và cung cấp đầy đủ các endpoint cần thiết.
Xây dựng frontend	Phát triển giao diện cho quản trị viên, giảng viên và sinh viên.	Các màn hình được tích hợp với backend và hiển thị đúng dữ liệu.
Tích hợp thông báo	Xây dựng NotificationOutbox, RabbitMQ và Notification Worker.	Thông báo được xử lý theo cơ chế bất đồng bộ.
Kiểm thử hệ thống	Kiểm thử nghiệp vụ, API, phân quyền, giao diện và dữ liệu lớn.	Các lỗi được phát hiện, sửa chữa và kiểm tra lại.
Triển khai	Đóng gói bằng Docker, cấu hình CI/CD và triển khai lên cloud.	Hệ thống có thể truy cập và vận hành qua Internet.
Hoàn thiện báo cáo	Tổng hợp kết quả, hình ảnh, bảng biểu và đánh giá hệ thống.	Báo cáo đề tài được hoàn thiện đầy đủ.

3.4 Tổng kết chương

Chương 3 đã trình bày quá trình phân tích yêu cầu và lập kế hoạch phát triển website quản lý đồ án sinh viên. Các nhu cầu thực tế của quản trị viên, giảng viên và sinh viên được xác định làm cơ sở xây dựng các chức năng của hệ thống.

Hệ thống cung cấp những nhóm chức năng chính gồm quản lý tài khoản, môn học, lớp học phân, nhóm sinh viên, đăng ký đề tài, tiến độ, yêu cầu nộp bài, bài nộp, điểm số và thông báo. Mỗi chức năng được giới hạn theo vai trò và phạm vi dữ liệu của người dùng.

Bên cạnh yêu cầu chức năng, chương này cũng xác định các yêu cầu phi chức năng liên quan đến bảo mật, hiệu năng, tính toàn vẹn dữ liệu, khả năng sử dụng, độ tin cậy, khả năng bảo trì và triển khai.

Quá trình phát triển được chia thành các kế hoạch cụ thể gồm phân tích và thiết kế, xây dựng backend, phát triển frontend, kiểm thử, triển khai và quản lý rủi ro. Việc chia dự án thành nhiều giai đoạn giúp nhóm dễ theo dõi tiến độ, kiểm soát chất lượng và xử lý những vấn đề phát sinh.

Những yêu cầu và kế hoạch được trình bày trong chương này là cơ sở để tiến hành thiết kế chi tiết cơ sở dữ liệu, kiến trúc hệ thống, sơ đồ chức năng và giao diện người dùng trong chương tiếp theo.

CHƯƠNG 4: THIẾT KẾ HỆ THỐNG

4.1 Kiến trúc tổng thể của hệ thống

Kiến trúc tổng thể của hệ thống được xây dựng theo mô hình Client–Server. Người dùng truy cập hệ thống thông qua trình duyệt và tương tác với giao diện frontend. Frontend gửi các yêu cầu HTTP đến backend để đăng nhập, truy xuất dữ liệu hoặc thực hiện các nghiệp vụ quản lý đồ án.

Backend tiếp nhận yêu cầu, kiểm tra JWT, xác định vai trò và phạm vi dữ liệu của người dùng. Sau khi yêu cầu được xác thực, backend xử lý nghiệp vụ và thực hiện truy vấn đến cơ sở dữ liệu SQL Server.

Đối với các sự kiện cần tạo thông báo, backend ghi dữ liệu nghiệp vụ và sự kiện vào cơ sở dữ liệu. Sự kiện trong NotificationOutbox sau đó được gửi đến RabbitMQ. Notification Worker nhận thông điệp từ hàng đợi và tạo thông báo cho người dùng tương ứng.

Các thành phần chính của hệ thống gồm:

- Trình duyệt của người dùng.
- Frontend ReactJS.
- Backend Node.js và ExpressJS.
- Cơ sở dữ liệu SQL Server.
- NotificationOutbox.
- RabbitMQ.
- Notification Worker.
- Swagger phục vụ mô tả và kiểm tra API.

4.1.1 Kiến trúc frontend

Frontend được xây dựng bằng ReactJS và Vite, chịu trách nhiệm hiển thị giao diện cho ba vai trò gồm quản trị viên, giảng viên và sinh viên.

Các màn hình được tổ chức thành nhiều thành phần để thuận tiện cho việc tái sử dụng và bảo trì. Những thành phần dùng chung gồm thanh điều hướng, bố cục trang, bảng dữ liệu, biểu mẫu, hộp thoại xác nhận, trạng thái tải và thông báo lỗi.

Frontend thực hiện các nhiệm vụ chính như:

- Tiếp nhận thao tác của người dùng.
- Kiểm tra dữ liệu cơ bản trước khi gửi.
- Gửi yêu cầu đến RESTful API.
- Đính kèm JWT vào các yêu cầu được bảo vệ.
- Hiển thị dữ liệu được backend trả về.
- Điều hướng người dùng theo vai trò.
- Xử lý trạng thái tải, lỗi và không có dữ liệu.
- Hiển thị thông báo và kết quả thao tác.

Frontend chỉ kiểm soát việc hiển thị chức năng trên giao diện. Quyền truy cập thực tế vẫn được backend kiểm tra để tránh trường hợp người dùng gửi yêu cầu trực tiếp đến API.

4.1.2 Kiến trúc backend

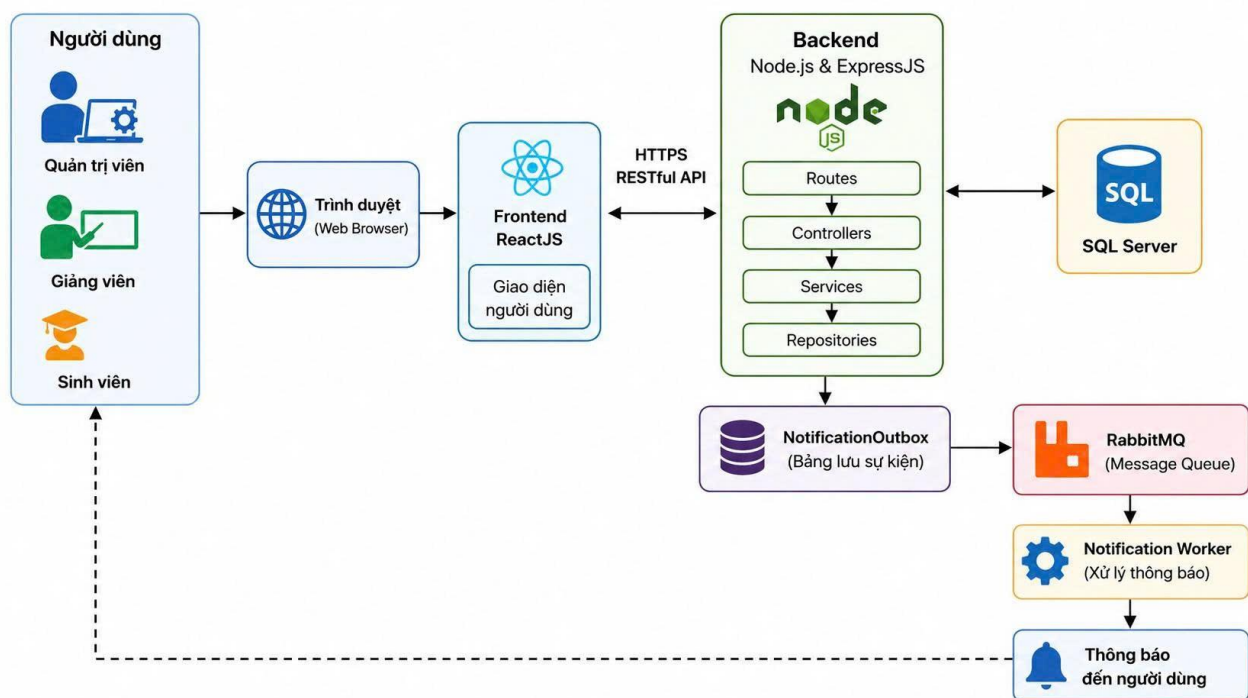
Backend được phát triển bằng Node.js và ExpressJS, đồng thời được tổ chức theo kiến trúc phân tầng gồm Routes, Controllers, Services và Repositories.

Routes chịu trách nhiệm khai báo đường dẫn API, phương thức HTTP và các middleware cần sử dụng. Controllers tiếp nhận request, gọi tầng service và trả response về frontend.

Services chứa các quy tắc nghiệp vụ chính như kiểm tra quyền truy cập, xác định phạm vi lớp học phần, kiểm tra trạng thái đề tài, thời hạn nộp bài và điều kiện công bố điểm.

Repositories chịu trách nhiệm làm việc trực tiếp với SQL Server. Tầng này thực hiện các thao tác truy vấn, thêm, cập nhật và xóa dữ liệu theo yêu cầu từ tầng service.

Việc chia backend thành nhiều tầng giúp hạn chế trộn lẫn giữa xử lý HTTP, nghiệp vụ và truy vấn dữ liệu. Khi cần thay đổi một nghiệp vụ, nhóm có thể chỉnh sửa tại tầng service mà không ảnh hưởng lớn đến những thành phần còn lại.



Hình 4.1. Kiến trúc tổng thể của hệ thống

4.2 Xác định tác nhân của hệ thống

Tác nhân là người hoặc thành phần bên ngoài có tương tác với hệ thống. Website quản lý đề án sinh viên có ba tác nhân chính gồm quản trị viên, giảng viên và sinh viên.

4.2.1 Quản trị viên

Quản trị viên chịu trách nhiệm quản lý dữ liệu nền tảng và duy trì hoạt động chung của hệ thống. Các chức năng chính của quản trị viên gồm:

- Đăng nhập hệ thống.
- Quản lý tài khoản người dùng.
- Tạo tài khoản giảng viên và sinh viên.
- Cập nhật trạng thái tài khoản.
- Quản lý môn học.
- Quản lý lớp học phần.
- Phân công giảng viên phụ trách.
- Quản lý danh sách sinh viên trong lớp.
- Theo dõi dữ liệu tổng thể.

Quản trị viên có phạm vi quản lý rộng nhất nhưng không trực tiếp thực hiện các nghiệp vụ chuyên môn như đăng ký đề tài, nộp bài hoặc chấm điểm.

4.2.2 Giảng viên

Giảng viên là tác nhân quản lý trực tiếp quá trình thực hiện đề án trong các lớp học phần được phân công. Các chức năng chính gồm:

- Xem lớp học phần phụ trách.
- Xem danh sách sinh viên.
- Theo dõi nhóm sinh viên.
- Xem và xét duyệt đăng ký đề tài.

- Yêu cầu sinh viên chỉnh sửa đề tài.
- Tạo mốc tiến độ.
- Tạo yêu cầu nộp bài.
- Xem bài nộp và lịch sử nộp.
- Nhập điểm và phản hồi.
- Công bố hoặc thu hồi điểm.
- Theo dõi thông báo liên quan.
- Giảng viên chỉ được truy cập dữ liệu thuộc các lớp học phần mà mình được phân công.

4.2.3 Sinh viên

Sinh viên là tác nhân trực tiếp tham gia thực hiện đồ án. Các chức năng chính gồm:

- Đăng nhập hệ thống.
- Xem lớp học phần đang tham gia.
- Xem thông tin nhóm.
- Đăng ký đề tài.
- Chỉnh sửa đề tài khi được yêu cầu.
- Theo dõi trạng thái xét duyệt.
- Xem các mốc tiến độ.
- Xem yêu cầu nộp bài.
- Nộp bài và nộp lại khi được phép.

- Xem lịch sử các lần nộp.
- Xem phản hồi của giảng viên.
- Xem điểm đã được công bố.
- Xem và đánh dấu thông báo đã đọc.
- Sinh viên chỉ được truy cập dữ liệu của lớp học phần và nhóm mà mình tham gia.

4.3 Thiết kế sơ đồ ca sử dụng

Sơ đồ ca sử dụng, hay Use Case Diagram, được dùng để mô tả mối quan hệ giữa các tác nhân và những chức năng mà họ có thể thực hiện trong hệ thống. Sơ đồ giúp xác định phạm vi chức năng, quyền hạn và sự tương tác của từng nhóm người dùng.

Website quản lý đồ án sinh viên có ba tác nhân chính gồm quản trị viên, giảng viên và sinh viên. Các tác nhân đều phải thực hiện chức năng đăng nhập trước khi sử dụng những chức năng được bảo vệ.

4.3.1 Sơ đồ ca sử dụng tổng quát

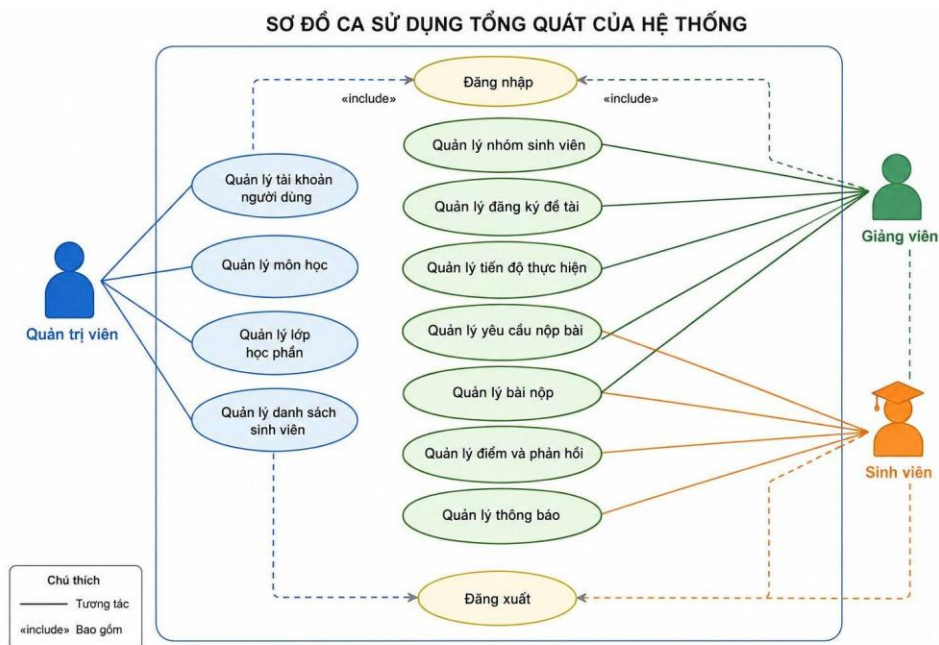
Sơ đồ ca sử dụng tổng quát thể hiện những nhóm chức năng chính của toàn bộ hệ thống. Sau khi đăng nhập, người dùng được chuyển đến khu vực tương ứng với vai trò của mình.

Quản trị viên thực hiện các chức năng quản lý dữ liệu nền tảng như tài khoản, môn học, lớp học phần và danh sách sinh viên. Giảng viên quản lý quá trình thực hiện đồ án trong các lớp học phần được phân công. Sinh viên tham gia nhóm, đăng ký đề tài, nộp bài và theo dõi kết quả.

Các nhóm chức năng chính trong sơ đồ ca sử dụng tổng quát gồm:

- Đăng nhập và đăng xuất.
- Quản lý tài khoản người dùng.

- Quản lý môn học.
- Quản lý lớp học phần.
- Quản lý danh sách sinh viên.
- Quản lý nhóm sinh viên.
- Quản lý đăng ký đề tài.
- Quản lý tiến độ thực hiện.
- Quản lý yêu cầu nộp bài.
- Quản lý bài nộp.
- Quản lý điểm và phản hồi.
- Quản lý thông báo.



Hình 4.2. Sơ đồ ca sử dụng tổng quát của hệ thống

Trong sơ đồ tổng quát, tác nhân quản trị viên được liên kết với các chức năng quản lý dữ liệu nền tảng. Tác nhân giảng viên được liên kết với các chức năng quản lý lớp phụ

trách, đề tài, tiến độ, bài nộp và điểm số. Tác nhân sinh viên được liên kết với các chức năng liên quan đến quá trình thực hiện đồ án.

4.3.2 Sơ đồ ca sử dụng của quản trị viên

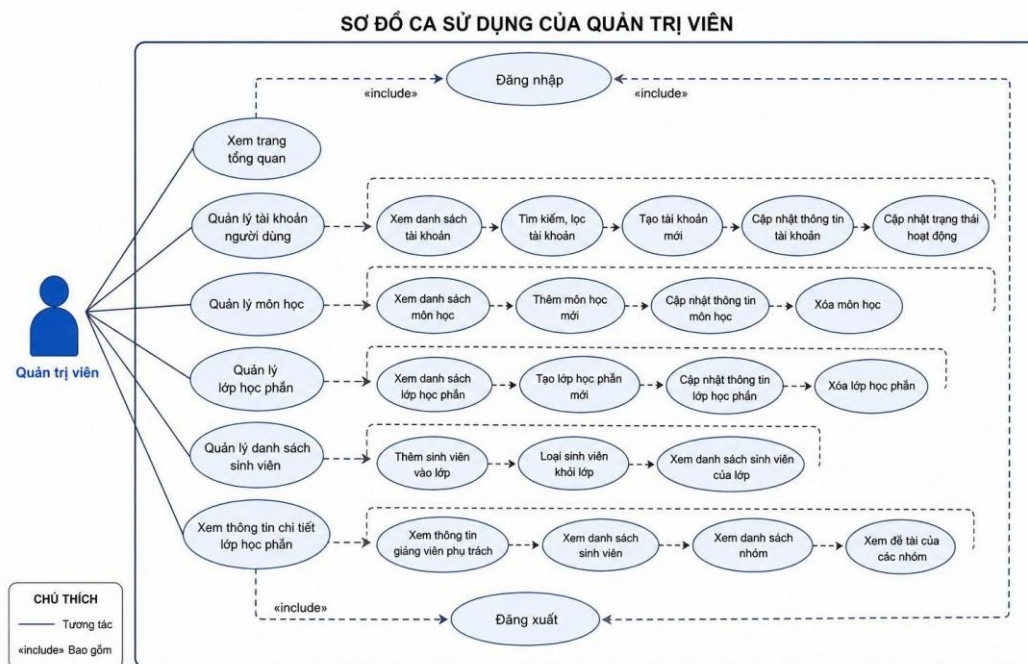
Quản trị viên là người chịu trách nhiệm thiết lập và quản lý các dữ liệu cần thiết để hệ thống hoạt động. Sau khi đăng nhập, quản trị viên có thể truy cập khu vực quản trị và thực hiện các chức năng thuộc phạm vi toàn hệ thống.

Các ca sử dụng của quản trị viên gồm:

- Đăng nhập hệ thống.
- Xem trang tổng quan.
- Xem danh sách tài khoản.
- Tìm kiếm và lọc tài khoản.
- Tạo tài khoản mới.
- Cập nhật thông tin tài khoản.
- Cập nhật trạng thái hoạt động.
- Quản lý môn học.
- Quản lý lớp học phần.
- Phân công giảng viên phụ trách.
- Thêm sinh viên vào lớp học phần.
- Loại sinh viên khỏi lớp học phần.
- Xem thông tin chi tiết lớp học phần.
- Đăng xuất hệ thống.

Khi tạo hoặc cập nhật tài khoản, quản trị viên phải nhập đầy đủ những thông tin bắt buộc. Backend kiểm tra tính hợp lệ của dữ liệu, quyền của người thực hiện và sự tồn tại của các thông tin liên quan trước khi lưu vào cơ sở dữ liệu.

Khi quản lý lớp học phần, quản trị viên có thể lựa chọn môn học, giảng viên phụ trách và danh sách sinh viên. Một sinh viên chỉ được thêm vào lớp khi tài khoản tồn tại và có vai trò phù hợp.



Hình 4.3. Sơ đồ ca sử dụng của quản trị viên

Bảng 4.1. Các ca sử dụng chính của quản trị viên

Mã ca sử dụng	Tên ca sử dụng	Mô tả
UC-AD01	Đăng nhập	Xác thực tài khoản quản trị viên và cấp quyền truy cập hệ thống.
UC-AD02	Quản lý tài khoản	Xem, tìm kiếm, tạo mới và cập nhật tài khoản người dùng.
UC-AD03	Quản lý môn học	Thêm, cập nhật và theo dõi danh sách môn học.
UC-AD04	Quản lý lớp học phần	Tạo và cập nhật thông tin lớp

Mã ca sử dụng	Tên ca sử dụng	Mô tả
		học phần.
UC-AD05	Phân công giảng viên	Chỉ định giảng viên phụ trách lớp học phần.
UC-AD06	Quản lý sinh viên trong lớp	Thêm hoặc loại sinh viên khỏi lớp học phần.
UC-AD07	Xem dữ liệu tổng quan	Theo dõi số lượng tài khoản, môn học và lớp học phần.
UC-AD08	Đăng xuất	Kết thúc phiên làm việc của quản trị viên.

4.3.3 Sơ đồ ca sử dụng của giảng viên

Giảng viên sử dụng hệ thống để quản lý quá trình thực hiện đồ án của sinh viên trong các lớp học phần được phân công. Trước khi thực hiện nghiệp vụ, hệ thống kiểm tra tài khoản giảng viên và phạm vi lớp học phần.

Các ca sử dụng của giảng viên gồm:

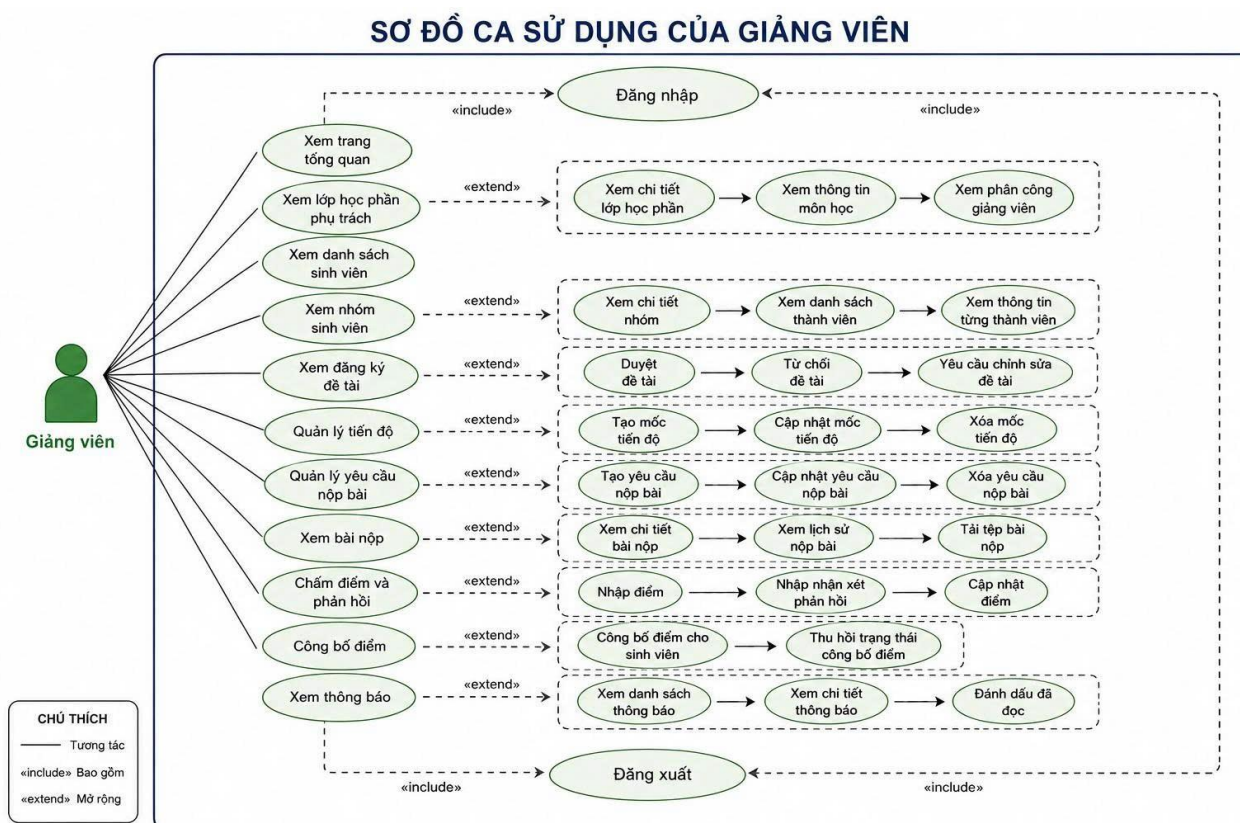
- Đăng nhập hệ thống.
- Xem trang tổng quan.
- Xem danh sách lớp học phần phụ trách.
- Xem danh sách sinh viên.
- Xem nhóm sinh viên.
- Theo dõi thành viên của từng nhóm.
- Xem đăng ký đề tài.
- Duyệt đề tài.
- Từ chối đề tài.

- Yêu cầu chỉnh sửa đề tài.
- Tạo và quản lý mốc tiến độ.
- Tạo yêu cầu nộp bài.
- Cập nhật thời hạn và số lần nộp.
- Xem danh sách bài nộp.
- Xem lịch sử các lần nộp.
- Nhập điểm và phản hồi.
- Công bố điểm.
- Thu hồi trạng thái công bố điểm.
- Xem thông báo.
- Đăng xuất hệ thống.

Giảng viên chỉ được thao tác với các lớp học phần được phân công. Khi giảng viên gửi yêu cầu đến backend, hệ thống kiểm tra mã lớp học phần và đối chiếu với thông tin phân công trước khi thực hiện nghiệp vụ.

Đối với đăng ký đề tài, giảng viên có thể duyệt, từ chối hoặc yêu cầu nhóm sinh viên chỉnh sửa. Khi đề tài đã được duyệt, nội dung đăng ký được khóa để bảo đảm tính thống nhất.

Đối với bài nộp, giảng viên có thể xem nội dung, lịch sử nộp và thời điểm nộp của từng nhóm. Sau khi đánh giá, giảng viên nhập điểm, phản hồi và quyết định thời điểm công bố kết quả cho sinh viên.



Hình 4.4. Sơ đồ ca sử dụng của giảng viên

Bảng 4.2. Các ca sử dụng chính của giảng viên

Mã ca sử dụng	Tên ca sử dụng	Mô tả
UC-LE01	Xem lớp phụ trách	Hiển thị các lớp học phân được phân công cho giảng viên.
UC-LE02	Quản lý nhóm sinh viên	Theo dõi nhóm và thành viên trong từng lớp học phân.
UC-LE03	Xét duyệt đề tài	Duyệt, từ chối hoặc yêu cầu chỉnh sửa đề tài.
UC-LE04	Quản lý tiến độ	Tạo và cập nhật các mốc thực hiện đồ án.
UC-LE05	Quản lý yêu cầu nộp bài	Tạo yêu cầu, thiết lập thời hạn và số lần nộp.
UC-LE06	Xem bài nộp	Xem bài nộp và lịch sử các lần nộp của sinh viên.
UC-LE07	Chấm điểm và phản hồi	Nhập điểm và nhận xét cho bài nộp.

Mã ca sử dụng	Tên ca sử dụng	Mô tả
UC-LE08	Công bố điểm	Cho phép sinh viên xem kết quả đánh giá.
UC-LE09	Xem thông báo	Theo dõi các thông báo liên quan đến lớp học phần.
UC-LE10	Đăng xuất	Kết thúc phiên làm việc của giảng viên.

4.3.4 Sơ đồ ca sử dụng của sinh viên

Sinh viên là tác nhân trực tiếp tham gia quá trình thực hiện đồ án. Sau khi đăng nhập, sinh viên có thể xem thông tin lớp học phần, theo dõi nhóm, đăng ký đề tài, thực hiện các yêu cầu nộp bài và xem kết quả đánh giá.

Hệ thống chỉ cho phép sinh viên truy cập dữ liệu thuộc các lớp học phần mà mình đang tham gia. Khi sinh viên gửi yêu cầu đến backend, hệ thống kiểm tra tài khoản, lớp học phần và nhóm sinh viên trước khi xử lý.

Các ca sử dụng chính của sinh viên gồm:

- Đăng nhập hệ thống.
- Xem trang tổng quan.
- Xem danh sách lớp học phần đang tham gia.
- Xem thông tin chi tiết lớp học phần.
- Xem thông tin nhóm sinh viên.
- Xem danh sách thành viên trong nhóm.
- Đăng ký đề tài.
- Cập nhật đề tài khi được yêu cầu chỉnh sửa.
- Theo dõi trạng thái xét duyệt đề tài.

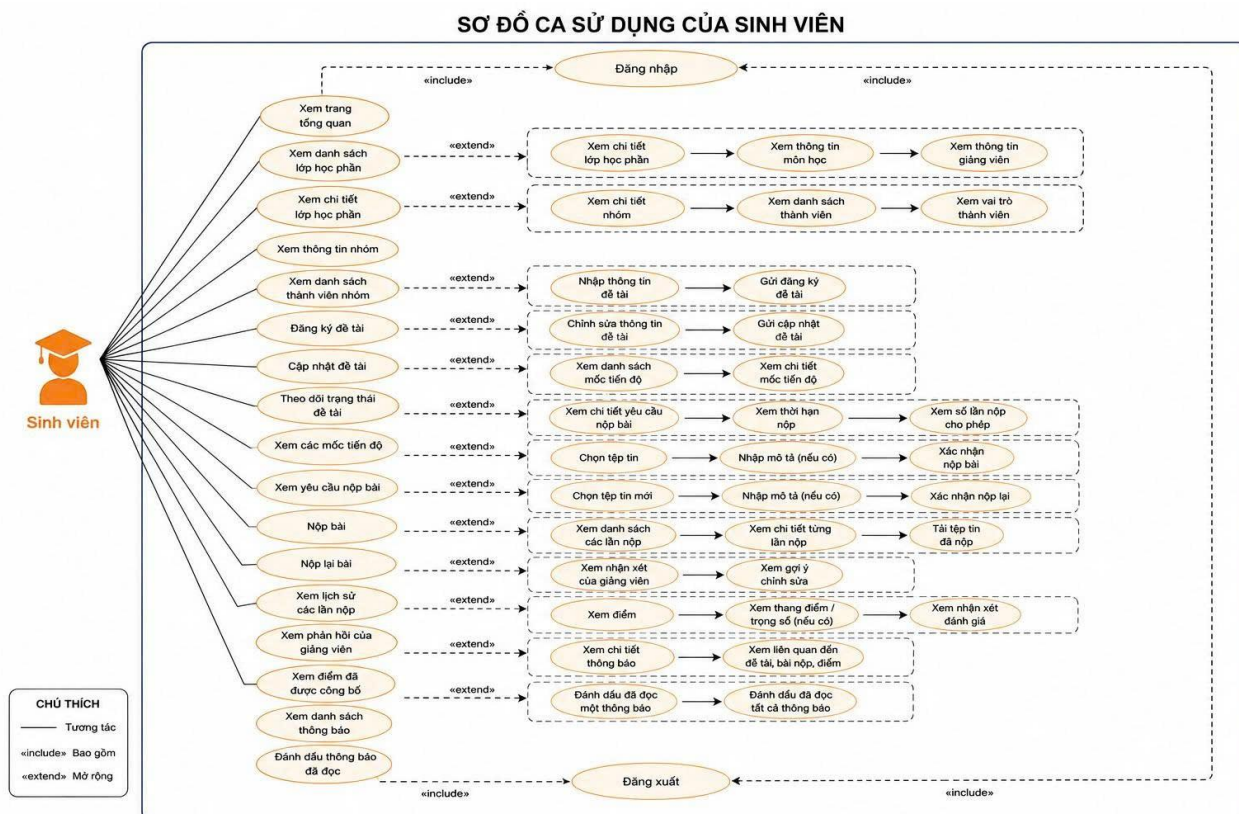
- Xem các mốc tiến độ.
- Xem yêu cầu nộp bài.
- Xem thời hạn và số lần nộp cho phép.
- Nộp bài.
- Nộp lại bài khi được phép.
- Xem lịch sử các lần nộp.
- Xem phản hồi của giảng viên.
- Xem điểm đã được công bố.
- Xem danh sách thông báo.
- Đánh dấu thông báo đã đọc.
- Đăng xuất hệ thống.

Đối với chức năng đăng ký đề tài, sinh viên đại diện nhóm nhập tên đề tài, mô tả và các thông tin liên quan. Sau khi gửi đăng ký, đề tài được chuyển sang trạng thái chờ giảng viên xét duyệt.

Nếu giảng viên yêu cầu chỉnh sửa, sinh viên có thể cập nhật nội dung và gửi lại. Khi đề tài đã được duyệt, hệ thống khóa chức năng chỉnh sửa để bảo đảm nội dung thực hiện không bị thay đổi tùy ý.

Đối với chức năng nộp bài, sinh viên chỉ được nộp khi yêu cầu đang hoạt động, chưa vượt quá thời hạn và số lần nộp vẫn còn hợp lệ. Mỗi lần nộp được lưu thành một phiên bản để giảng viên có thể xem lịch sử.

Sinh viên chỉ được xem điểm sau khi giảng viên đã thực hiện công bố. Điểm chưa công bố và các nhận xét nội bộ không được hiển thị trên giao diện sinh viên.



Hình 4.5. Sơ đồ ca sử dụng của sinh viên

Bảng 4.3. Các ca sử dụng chính của sinh viên

Mã ca sử dụng	Tên ca sử dụng	Mô tả
UC-ST01	Xem lớp học phần	Hiển thị các lớp học phần mà sinh viên đang tham gia.
UC-ST02	Xem thông tin nhóm	Hiển thị nhóm và danh sách thành viên của sinh viên.
UC-ST03	Đăng ký đề tài	Gửi tên đề tài, mô tả và thông tin liên quan để giảng viên xét duyệt.
UC-ST04	Cập nhật đề tài	Chỉnh sửa đề tài khi giảng viên yêu cầu và đề tài chưa được duyệt.
UC-ST05	Theo dõi trạng thái đề tài	Xem đề tài đang chờ duyệt, được duyệt, bị từ chối hoặc cần chỉnh sửa.
UC-ST06	Xem tiến độ	Theo dõi các mốc tiến độ do giảng viên tạo.

Mã ca sử dụng	Tên ca sử dụng	Mô tả
UC-ST07	Xem yêu cầu nộp bài	Xem nội dung yêu cầu, thời hạn và số lần nộp cho phép.
UC-ST08	Nộp bài	Gửi bài theo yêu cầu khi còn thời hạn và đủ điều kiện.
UC-ST09	Nộp lại bài	Gửi phiên bản mới khi số lần nộp vẫn còn hợp lệ.
UC-ST10	Xem lịch sử nộp	Xem các phiên bản bài đã nộp và thời gian của từng lần nộp.
UC-ST11	Xem điểm và phản hồi	Xem điểm cùng nhận xét sau khi giảng viên công bố.
UC-ST12	Xem thông báo	Theo dõi các thông báo liên quan đến đề tài, bài nộp và điểm số.
UC-ST13	Đánh dấu đã đọc	Cập nhật trạng thái thông báo đã được sinh viên xem.
UC-ST14	Đăng xuất	Kết thúc phiên làm việc của sinh viên.

4.4 Thiết kế sơ đồ hoạt động

Sơ đồ hoạt động, hay Activity Diagram, được sử dụng để mô tả trình tự xử lý của một chức năng trong hệ thống. Sơ đồ thể hiện điểm bắt đầu, các bước thực hiện, điều kiện rẽ nhánh và kết quả kết thúc của quy trình.

Đối với website quản lý đồ án sinh viên, sơ đồ hoạt động được xây dựng cho những nghiệp vụ quan trọng như đăng nhập, đăng ký và xét duyệt đề tài, tạo yêu cầu nộp bài, nộp bài, chấm điểm và gửi thông báo.

4.4.1 Sơ đồ hoạt động đăng nhập

Quy trình đăng nhập bắt đầu khi người dùng truy cập trang đăng nhập và nhập thông tin tài khoản. Frontend kiểm tra các trường bắt buộc trước khi gửi yêu cầu đến backend.

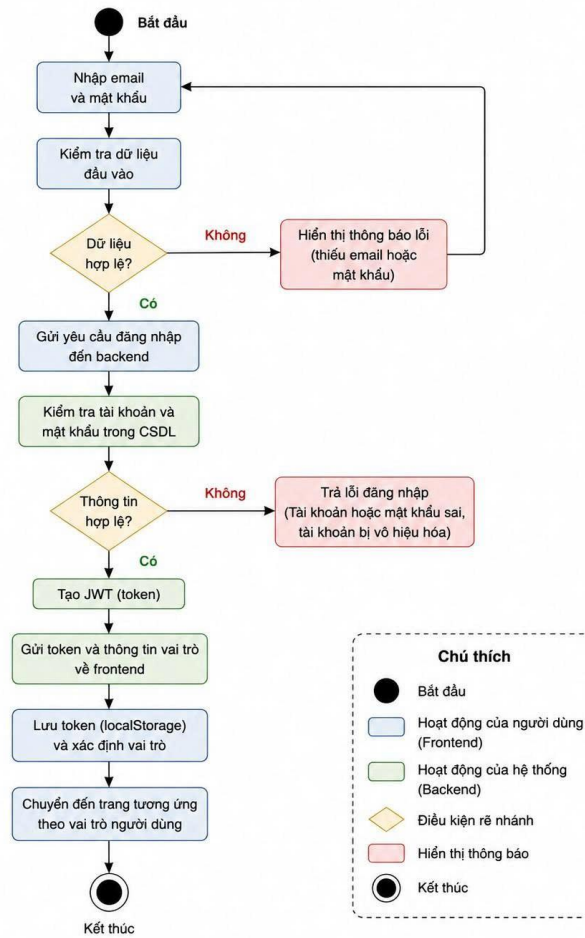
Backend tiếp nhận email và mật khẩu, sau đó tìm kiếm tài khoản tương ứng trong cơ sở dữ liệu. Nếu tài khoản không tồn tại, mật khẩu không chính xác hoặc tài khoản đã bị vô hiệu hóa, hệ thống trả về thông báo đăng nhập thất bại.

Khi thông tin hợp lệ, backend tạo JWT chứa mã người dùng và vai trò. Token được gửi về frontend để lưu trữ và sử dụng trong những yêu cầu tiếp theo.

Frontend dựa vào vai trò của người dùng để chuyển đến khu vực phù hợp. Quản trị viên được chuyển đến trang quản trị, giảng viên được chuyển đến trang quản lý lớp phụ trách và sinh viên được chuyển đến trang học tập cá nhân.

Quy trình đăng nhập được thực hiện theo các bước sau:

- Người dùng nhập email và mật khẩu.
- Frontend kiểm tra dữ liệu đầu vào.
- Frontend gửi yêu cầu đăng nhập đến backend.
- Backend tìm kiếm tài khoản.
- Backend kiểm tra trạng thái tài khoản và mật khẩu.
- Nếu thông tin không hợp lệ, hệ thống trả về thông báo lỗi.
- Nếu thông tin hợp lệ, backend tạo JWT.
- Frontend lưu token và xác định vai trò.
- Người dùng được chuyển đến trang tương ứng.



Hình 4.6. Sơ đồ hoạt động đăng nhập hệ thống

4.4.2 Sơ đồ hoạt động đăng ký và xét duyệt đề tài

Quy trình đăng ký đề tài có sự tham gia của sinh viên và giảng viên. Sinh viên đại diện nhóm thực hiện đăng ký, trong khi giảng viên phụ trách lớp học phần có nhiệm vụ xem xét và xử lý đề tài.

Trước khi cho phép đăng ký, hệ thống kiểm tra sinh viên có thuộc lớp học phần và nhóm hợp lệ hay không. Hệ thống cũng kiểm tra nhóm đã có đề tài đang chờ xử lý hoặc đã được duyệt hay chưa.

Nếu đủ điều kiện, sinh viên nhập tên đề tài, mô tả và gửi đăng ký. Backend lưu đề tài với trạng thái chờ duyệt, đồng thời tạo sự kiện thông báo cho giảng viên phụ trách.

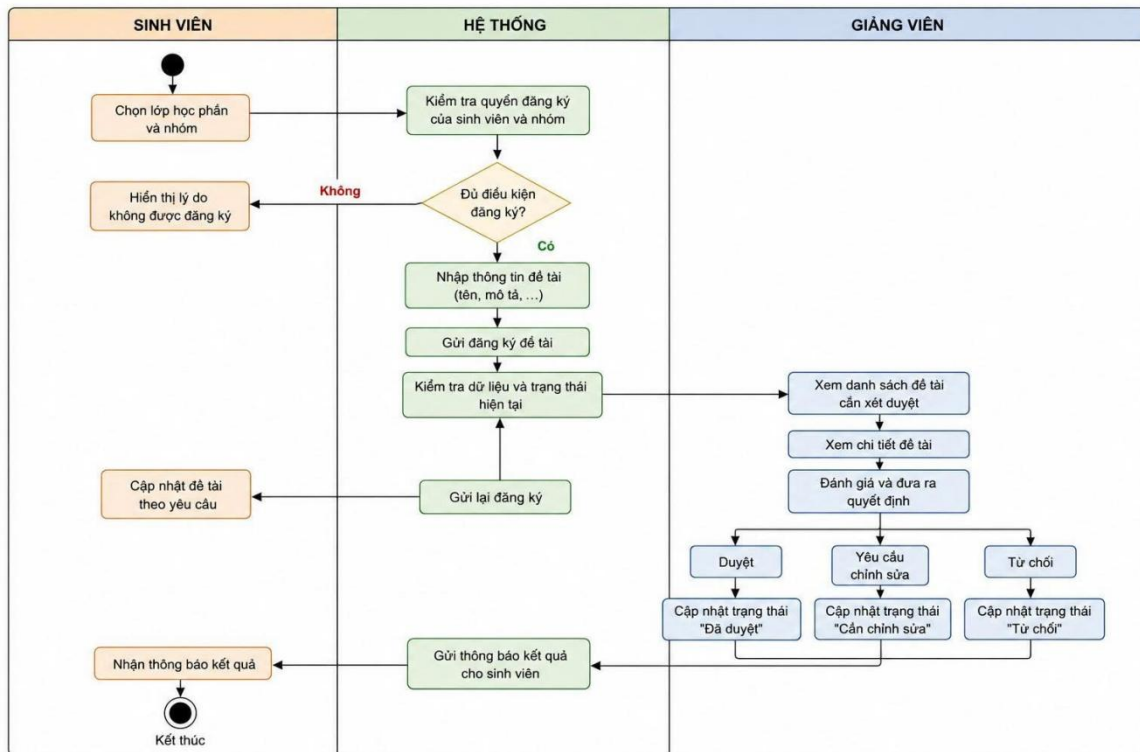
Giảng viên mở danh sách đề tài của lớp học phần và xem nội dung đăng ký. Sau khi đánh giá, giảng viên có thể lựa chọn một trong ba kết quả: duyệt đề tài, từ chối đề tài hoặc yêu cầu chỉnh sửa.

Khi đề tài được duyệt, hệ thống cập nhật trạng thái và khóa nội dung đăng ký. Sinh viên không thể tự ý thay đổi đề tài sau thời điểm này.

Nếu giảng viên yêu cầu chỉnh sửa, sinh viên được phép cập nhật nội dung và gửi lại để tiếp tục xét duyệt. Nếu đề tài bị từ chối, nhóm có thể đăng ký đề tài khác theo quy định của lớp học phần.

Quy trình được thực hiện theo các bước sau:

- Sinh viên chọn lớp học phần và nhóm.
- Hệ thống kiểm tra quyền đăng ký.
- Sinh viên nhập thông tin đề tài.
- Backend kiểm tra dữ liệu và trạng thái hiện tại.
- Đề tài được lưu với trạng thái chờ duyệt.
- Hệ thống tạo thông báo cho giảng viên.
- Giảng viên xem thông tin đề tài.
- Giảng viên lựa chọn duyệt, từ chối hoặc yêu cầu chỉnh sửa.
- Hệ thống cập nhật trạng thái đề tài.
- Sinh viên nhận thông báo kết quả.



Hình 4.7. Sơ đồ hoạt động đăng ký và xét duyệt đề tài

4.4.3 Sơ đồ hoạt động tạo yêu cầu nộp bài

Chức năng tạo yêu cầu nộp bài được thực hiện bởi giảng viên phụ trách lớp học phần. Yêu cầu nộp bài giúp giảng viên xác định nội dung sinh viên cần thực hiện, thời hạn nộp, số lần được phép nộp và trạng thái hoạt động của yêu cầu.

Quy trình bắt đầu khi giảng viên chọn một lớp học phần đang phụ trách và mở chức năng quản lý yêu cầu nộp bài. Hệ thống kiểm tra quyền truy cập của giảng viên đối với lớp học phần trước khi hiển thị biểu mẫu tạo mới.

Giảng viên nhập các thông tin cần thiết như tiêu đề, nội dung yêu cầu, thời gian bắt đầu, thời hạn nộp, số lần nộp tối đa và trạng thái hoạt động. Frontend kiểm tra các trường bắt buộc và định dạng dữ liệu trước khi gửi yêu cầu đến backend.

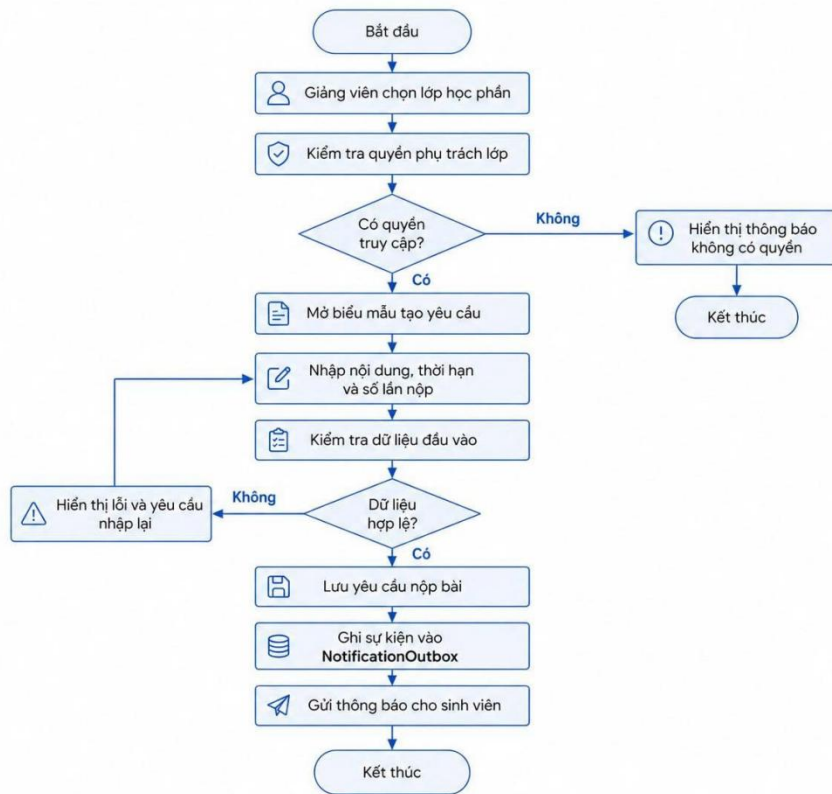
Backend tiếp tục kiểm tra quyền của giảng viên, sự tồn tại của lớp học phần và tính hợp lệ của thời gian. Thời hạn kết thúc phải lớn hơn thời gian bắt đầu và số lần nộp phải lớn hơn không.

Nếu dữ liệu không hợp lệ, hệ thống trả về thông báo lỗi để giảng viên chỉnh sửa.
Nếu dữ liệu hợp lệ, yêu cầu nộp bài được lưu vào cơ sở dữ liệu.

Sau khi lưu thành công, hệ thống tạo sự kiện thông báo cho các sinh viên thuộc lớp học phần. Sự kiện được ghi vào NotificationOutbox và được Notification Worker xử lý để tạo thông báo cho người nhận.

Quy trình tạo yêu cầu nộp bài gồm các bước:

- Giảng viên chọn lớp học phần.
- Hệ thống kiểm tra quyền quản lý lớp.
- Giảng viên mở biểu mẫu tạo yêu cầu.
- Giảng viên nhập nội dung, thời hạn và số lần nộp.
- Frontend kiểm tra dữ liệu đầu vào.
- Backend kiểm tra quyền và quy tắc nghiệp vụ.
- Nếu dữ liệu không hợp lệ, hệ thống trả thông báo lỗi.
- Nếu dữ liệu hợp lệ, hệ thống lưu yêu cầu nộp bài.
- Hệ thống tạo sự kiện thông báo.
- Sinh viên nhận thông báo về yêu cầu mới.



Hình 4.8. Sơ đồ hoạt động tạo yêu cầu nộp bài

4.4.4 Sơ đồ hoạt động nộp bài

Chức năng nộp bài cho phép sinh viên gửi kết quả thực hiện theo yêu cầu của giảng viên. Trước khi tiếp nhận bài nộp, hệ thống phải kiểm tra quyền của sinh viên, trạng thái yêu cầu, thời hạn và số lần đã nộp.

Quy trình bắt đầu khi sinh viên chọn một yêu cầu nộp bài trong lớp học phần đang tham gia. Hệ thống hiển thị nội dung yêu cầu, thời hạn, số lần nộp tối đa và số lần sinh viên đã thực hiện.

Khi sinh viên chọn tệp và xác nhận nộp, frontend kiểm tra tệp đã được chọn hay chưa và dữ liệu có đầy đủ hay không. Yêu cầu sau đó được gửi đến backend cùng với JWT của người dùng.

Backend kiểm tra sinh viên có thuộc lớp học phần và nhóm liên quan hay không. Hệ thống tiếp tục kiểm tra yêu cầu có đang hoạt động, đã đến thời gian cho phép nộp và chưa vượt quá thời hạn hay không.

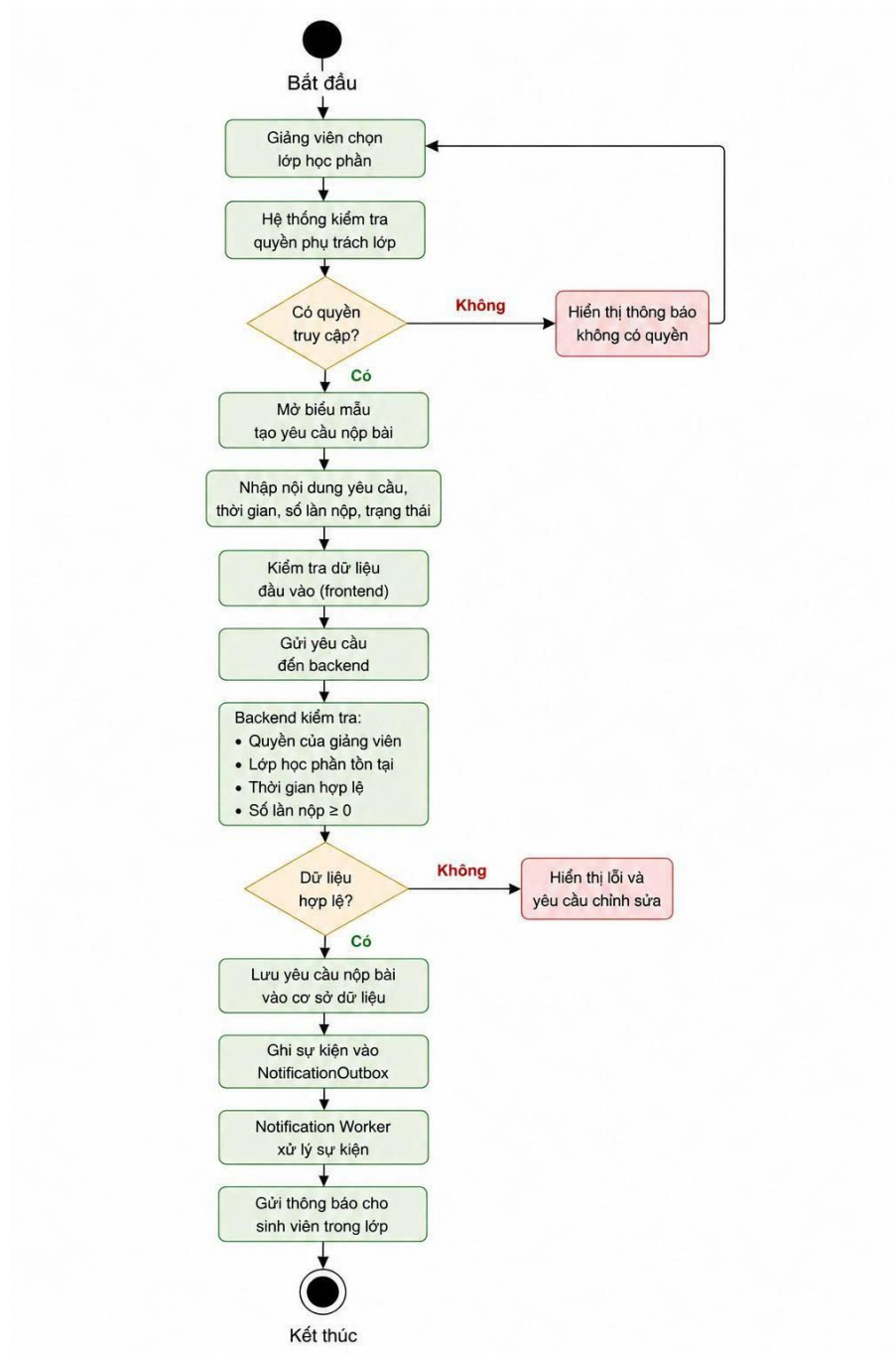
Số lần nộp hiện tại cũng được đối chiếu với giới hạn do giảng viên thiết lập. Nếu đã vượt quá số lần cho phép, hệ thống từ chối tiếp nhận bài nộp mới.

Khi tất cả điều kiện đều hợp lệ, hệ thống lưu thông tin bài nộp và tạo một bản ghi lịch sử cho lần nộp hiện tại. Việc lưu lịch sử giúp giảng viên theo dõi các phiên bản đã được gửi trước đó.

Sau khi bài nộp được lưu thành công, hệ thống tạo thông báo cho giảng viên phụ trách. Sinh viên nhận được thông báo xác nhận và có thể xem lại lần nộp trong lịch sử.

Quy trình nộp bài gồm các bước:

- Sinh viên chọn yêu cầu nộp bài.
- Hệ thống hiển thị nội dung và điều kiện nộp.
- Sinh viên chọn tệp cần gửi.
- Frontend kiểm tra dữ liệu.
- Backend kiểm tra quyền của sinh viên.
- Hệ thống kiểm tra trạng thái và thời hạn.
- Hệ thống kiểm tra số lần nộp.
- Nếu không đủ điều kiện, hệ thống trả về lý do từ chối.
- Nếu đủ điều kiện, hệ thống lưu bài nộp.
- Hệ thống lưu lịch sử lần nộp.
- Hệ thống thông báo cho giảng viên.
- Sinh viên nhận xác nhận nộp bài thành công.



Hình 4.9. Sơ đồ hoạt động nộp bài

4.4.5 Sơ đồ hoạt động chấm điểm và công bố kết quả

Chức năng chấm điểm được thực hiện bởi giảng viên phụ trách lớp học phần. Giảng viên có thể xem bài nộp, lịch sử các lần nộp, nhập điểm và phản hồi cho sinh viên. Kết quả chỉ được hiển thị cho sinh viên sau khi giảng viên thực hiện công bố điểm.

Quy trình bắt đầu khi giảng viên chọn lớp học phần và yêu cầu nộp bài cần đánh giá. Hệ thống kiểm tra giảng viên có quyền phụ trách lớp học phần hay không trước khi hiển thị danh sách bài nộp.

Giảng viên chọn bài nộp của một nhóm hoặc sinh viên, sau đó xem thông tin chi tiết, tệp đã gửi và lịch sử các lần nộp. Dựa trên kết quả đánh giá, giảng viên nhập điểm và nội dung phản hồi.

Frontend kiểm tra các trường bắt buộc và giới hạn điểm trước khi gửi dữ liệu đến backend. Backend tiếp tục kiểm tra quyền của giảng viên, sự tồn tại của bài nộp và tính hợp lệ của điểm số.

Nếu dữ liệu không hợp lệ, hệ thống trả về thông báo để giảng viên chỉnh sửa. Nếu dữ liệu hợp lệ, điểm và phản hồi được lưu vào cơ sở dữ liệu. Ở thời điểm này, kết quả có thể ở trạng thái chưa công bố nên sinh viên chưa thể xem.

Khi giảng viên chọn công bố điểm, backend cập nhật trạng thái công bố và tạo thông báo cho sinh viên. Nếu phát hiện sai sót, giảng viên có thể thu hồi trạng thái công bố để chỉnh sửa trước khi công bố lại.

- Quy trình chấm điểm và công bố kết quả gồm các bước:

- Giảng viên chọn lớp học phần.

- Hệ thống kiểm tra quyền phụ trách lớp.

- Giảng viên chọn yêu cầu nộp bài.

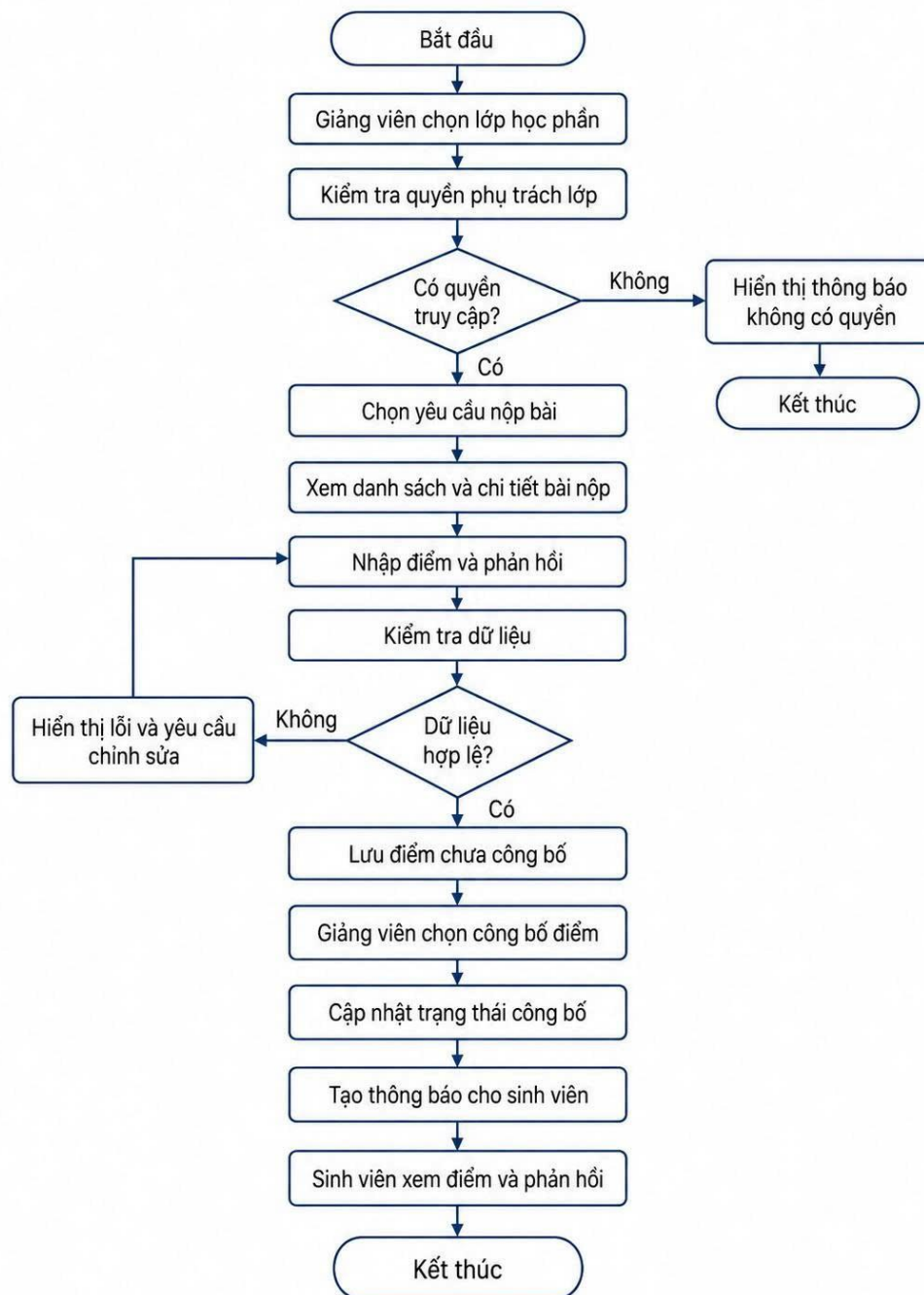
- Hệ thống hiển thị danh sách bài nộp.

- Giảng viên xem chi tiết và lịch sử nộp.

- Giảng viên nhập điểm và phản hồi.

- Backend kiểm tra dữ liệu.

- Hệ thống lưu điểm ở trạng thái chưa công bố.
- Giảng viên lựa chọn công bố điểm.
- Hệ thống cập nhật trạng thái và thông báo cho sinh viên.
- Sinh viên xem điểm và phản hồi.



Hình 4.10. Sơ đồ hoạt động chấm điểm và công bố kết quả

4.4.6 Sơ đồ hoạt động xử lý thông báo

Chức năng thông báo được sử dụng để cung cấp cho người dùng những thay đổi quan trọng liên quan đến đề tài, yêu cầu nộp bài, bài nộp và điểm số. Hệ thống xử lý thông báo theo cơ chế bất đồng bộ thông qua NotificationOutbox, RabbitMQ và Notification Worker.

Quy trình bắt đầu khi một nghiệp vụ quan trọng được thực hiện thành công. Ví dụ, sinh viên gửi đăng ký đề tài, giảng viên tạo yêu cầu nộp bài, sinh viên nộp bài hoặc giảng viên công bố điểm.

Backend lưu dữ liệu nghiệp vụ và sự kiện thông báo trong cùng một transaction. Sự kiện được ghi vào bảng NotificationOutbox với trạng thái chờ xử lý. Việc này giúp hạn chế trường hợp dữ liệu nghiệp vụ đã được lưu nhưng sự kiện thông báo bị mất.

Tiến trình xử lý Outbox lấy các sự kiện đang chờ và gửi chúng đến RabbitMQ. Khi gửi thành công, trạng thái sự kiện được cập nhật. Nếu quá trình gửi thất bại, sự kiện được lưu lại để theo dõi hoặc xử lý lại.

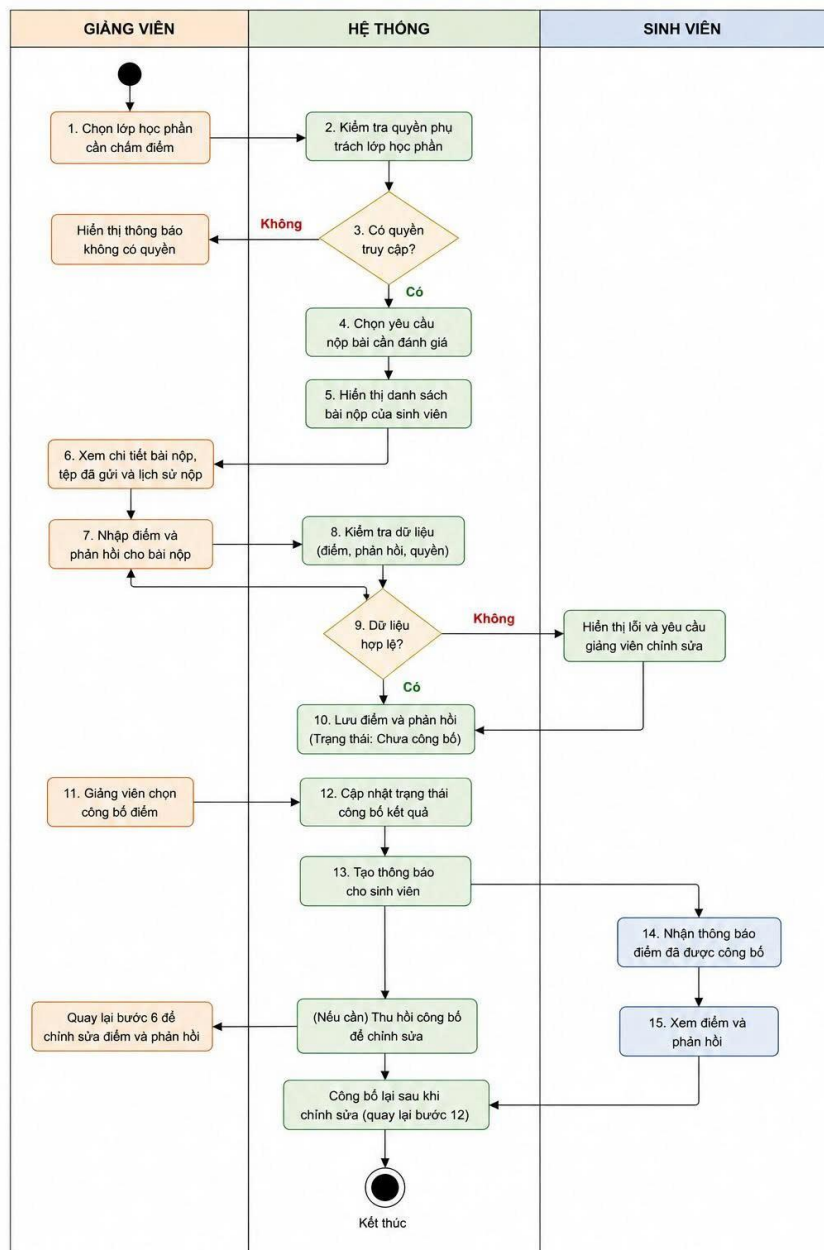
Notification Worker nhận thông điệp từ RabbitMQ, xác định người nhận và tạo bản ghi thông báo trong cơ sở dữ liệu. Sau khi xử lý thành công, worker xác nhận thông điệp để RabbitMQ loại bỏ khỏi hàng đợi.

Người dùng mở khu vực thông báo để xem danh sách nội dung liên quan. Khi người dùng đọc một thông báo, hệ thống cập nhật trạng thái từ chưa đọc thành đã đọc.

Quy trình xử lý thông báo gồm các bước:

- Nghiệp vụ chính được thực hiện thành công.
- Backend tạo sự kiện thông báo.
- Sự kiện được lưu vào NotificationOutbox.
- Tiến trình Outbox gửi sự kiện đến RabbitMQ.

- Notification Worker nhận thông điệp.
- Worker xác định người nhận.
- Hệ thống tạo thông báo trong cơ sở dữ liệu.
- Người dùng xem danh sách thông báo.
- Người dùng đánh dấu thông báo đã đọc.



Hình 4.11. Sơ đồ hoạt động xử lý thông báo bất đồng bộ

4.5 Thiết kế sơ đồ tuần tự

Sơ đồ tuần tự, hay Sequence Diagram, được sử dụng để mô tả trình tự trao đổi thông điệp giữa người dùng và các thành phần của hệ thống theo thời gian. Sơ đồ giúp làm rõ đối tượng nào gửi yêu cầu, thành phần nào xử lý nghiệp vụ và dữ liệu được lưu hoặc trả về như thế nào.

Trong website quản lý đồ án sinh viên, các thành phần thường tham gia vào sơ đồ tuần tự gồm:

- Tác nhân sử dụng hệ thống.
- Giao diện frontend.
- Routes và Controllers.
- Services.
- Repositories.
- Cơ sở dữ liệu SQL Server.
- NotificationOutbox.
- RabbitMQ và Notification Worker.

4.5.1 Sơ đồ tuần tự đăng nhập hệ thống

Quy trình đăng nhập bắt đầu khi người dùng nhập email và mật khẩu trên giao diện. Frontend kiểm tra dữ liệu cơ bản, sau đó gửi yêu cầu đăng nhập đến API của backend.

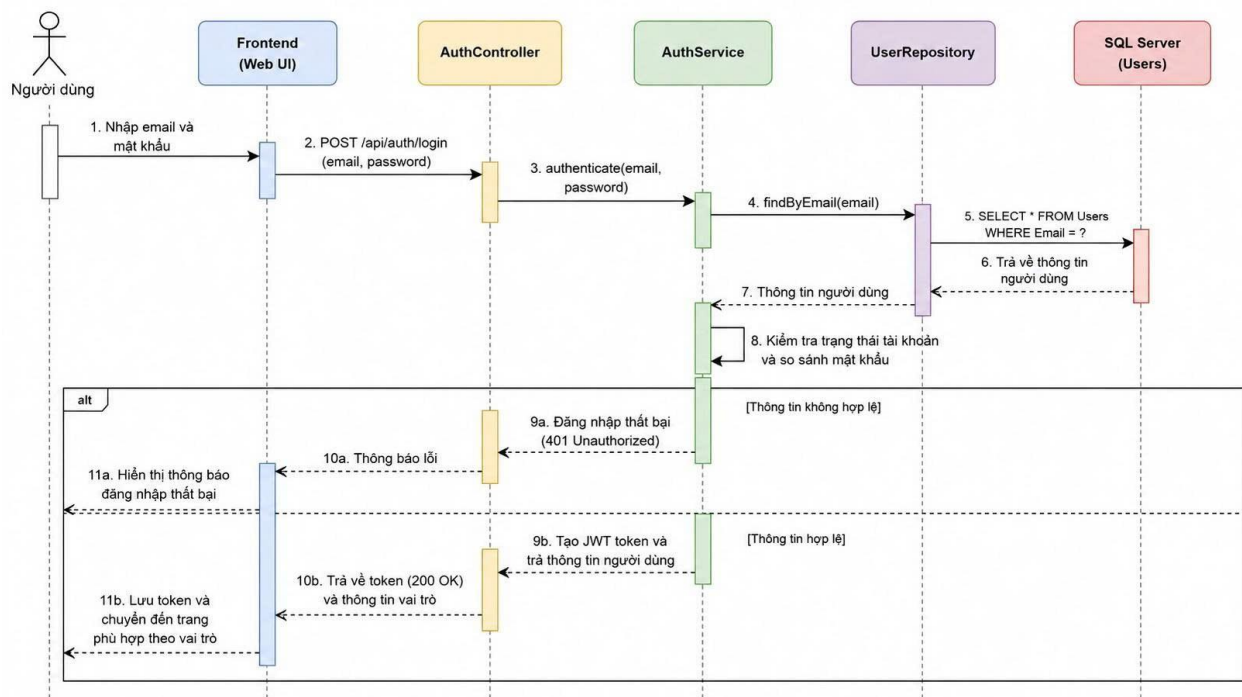
Controller tiếp nhận yêu cầu và chuyển dữ liệu đến AuthService. AuthService gọi Repository để tìm kiếm tài khoản trong SQL Server. Khi tìm thấy tài khoản, service kiểm tra trạng thái hoạt động và so sánh mật khẩu đã nhập với mật khẩu được băm trong cơ sở dữ liệu.

Nếu thông tin không hợp lệ, backend trả về mã trạng thái lỗi cùng thông báo đăng nhập thất bại. Frontend tiếp nhận kết quả và hiển thị thông báo để người dùng kiểm tra lại thông tin.

Nếu thông tin hợp lệ, AuthService tạo JWT chứa mã người dùng và vai trò. Token cùng thông tin tài khoản được trả về frontend. Frontend lưu token, xác định vai trò và chuyển người dùng đến trang tương ứng.

Trình tự xử lý đăng nhập gồm:

- Người dùng nhập email và mật khẩu.
- Frontend gửi yêu cầu đăng nhập.
- Controller gọi AuthService.
- AuthService yêu cầu Repository tìm tài khoản.
- Repository truy vấn SQL Server.
- Cơ sở dữ liệu trả về thông tin tài khoản.
- Service kiểm tra trạng thái và mật khẩu.
- Backend tạo JWT khi thông tin hợp lệ.
- Controller trả token và thông tin vai trò.
- Frontend lưu token và chuyển trang.



Hình 4.12. Sơ đồ tuần tự đăng nhập hệ thống

4.5.2 Sơ đồ tuần tự đăng ký đề tài

Quy trình đăng ký đề tài bắt đầu khi sinh viên mở biểu mẫu và nhập tên đề tài, mô tả cùng các thông tin liên quan. Frontend gửi dữ liệu đăng ký đến backend kèm theo JWT.

Middleware xác thực token và xác định tài khoản sinh viên. Controller tiếp nhận yêu cầu và gọi TopicRegistrationService để xử lý nghiệp vụ.

Service kiểm tra sinh viên có thuộc lớp học phần và nhóm hợp lệ hay không. Hệ thống cũng kiểm tra nhóm đã có đề tài được duyệt hoặc đang chờ xét duyệt hay chưa.

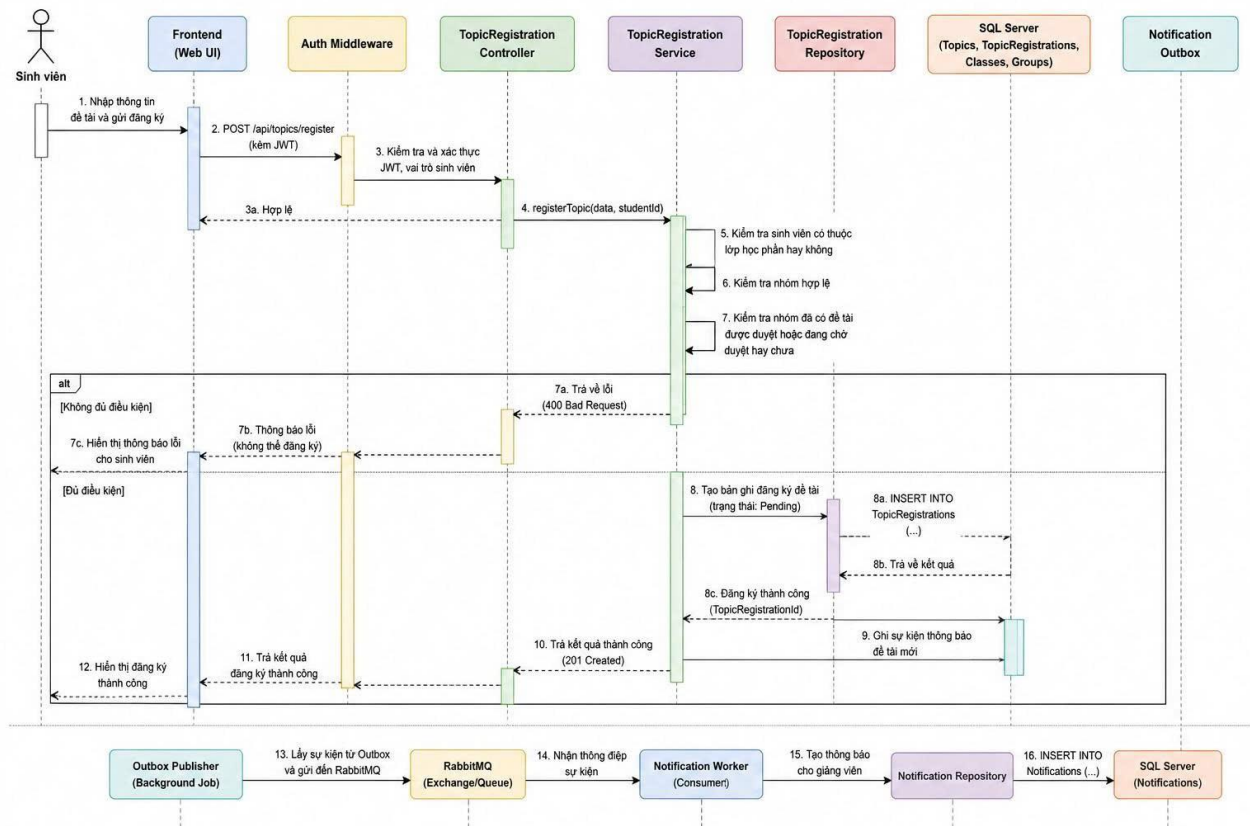
Nếu sinh viên không đủ điều kiện, backend trả về thông báo lỗi. Nếu đủ điều kiện, service yêu cầu Repository tạo bản ghi đăng ký đề tài với trạng thái chờ duyệt.

Trong cùng transaction, hệ thống ghi thêm sự kiện thông báo vào NotificationOutbox. Sau khi transaction hoàn tất, backend trả kết quả đăng ký thành công cho frontend.

Tiến trình Outbox tiếp tục gửi sự kiện đến RabbitMQ. Notification Worker nhận thông điệp, xác định giảng viên phụ trách lớp học phần và tạo thông báo trong cơ sở dữ liệu.

Trình tự xử lý đăng ký đề tài gồm:

- Sinh viên nhập thông tin đề tài.
- Frontend gửi yêu cầu đăng ký kèm JWT.
- Middleware xác thực sinh viên.
- Controller gọi service đăng ký đề tài.
- Service kiểm tra lớp học phần và nhóm.
- Repository kiểm tra trạng thái đề tài hiện tại.
- Repository lưu đề tài với trạng thái chờ duyệt.
- Hệ thống ghi sự kiện vào NotificationOutbox.
- Backend trả kết quả cho frontend.
- RabbitMQ và Notification Worker xử lý thông báo cho giảng viên.



Hình 4.13. Sơ đồ tuần tự đăng ký đề tài

4.5.3 Sơ đồ tuần tự xét duyệt đề tài

Quy trình xét duyệt bắt đầu khi giảng viên mở danh sách các đề tài đang chờ xử lý trong lớp học phần được phân công. Frontend gửi yêu cầu lấy danh sách đến backend.

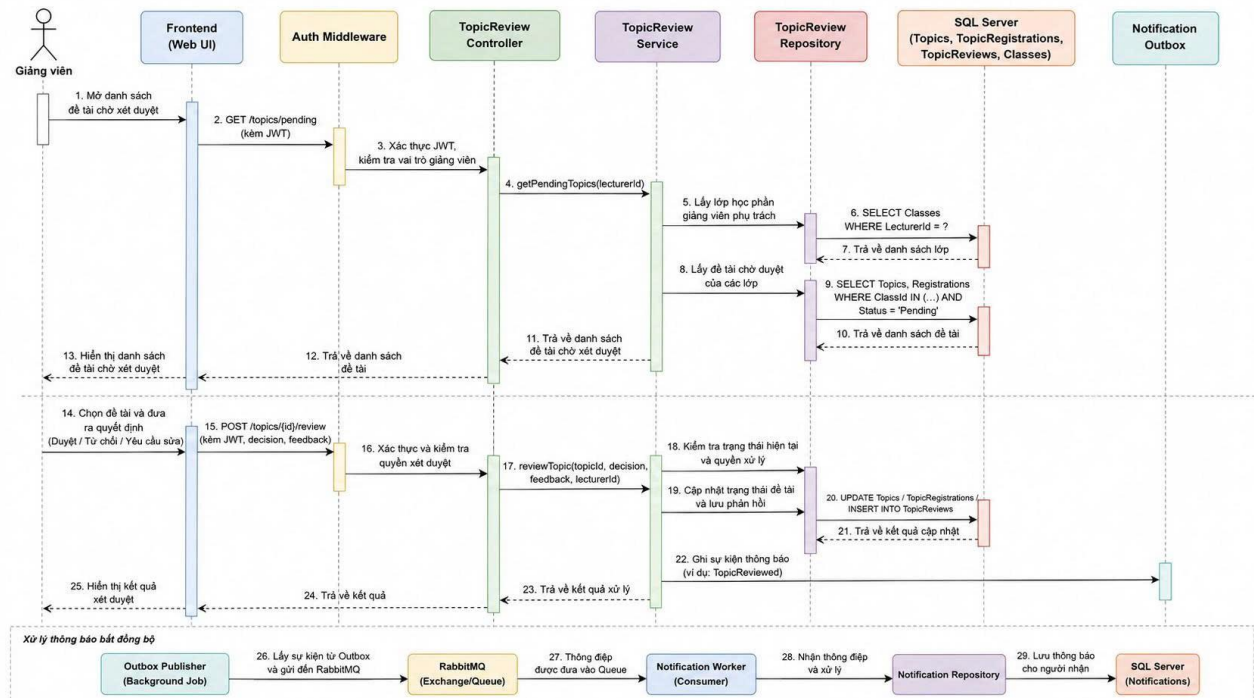
Backend kiểm tra JWT, vai trò giảng viên và quyền phụ trách lớp học phần. Sau khi xác thực thành công, hệ thống truy vấn danh sách đề tài và trả dữ liệu về giao diện.

Giảng viên chọn một đề tài và đưa ra quyết định gồm duyệt, từ chối hoặc yêu cầu chỉnh sửa. Frontend gửi quyết định cùng nội dung phản hồi đến backend.

Service kiểm tra trạng thái hiện tại của đề tài và quyền xử lý của giảng viên. Nếu hợp lệ, Repository cập nhật trạng thái đề tài trong cơ sở dữ liệu.

Khi đề tài được duyệt, hệ thống khóa nội dung đăng ký để sinh viên không thể tự ý chỉnh sửa. Nếu giảng viên yêu cầu chỉnh sửa, đề tài được chuyển sang trạng thái cần chỉnh sửa và sinh viên được phép cập nhật lại.

Sau khi cập nhật trạng thái, hệ thống ghi sự kiện thông báo vào NotificationOutbox. Notification Worker xử lý sự kiện và tạo thông báo kết quả cho các thành viên trong nhóm.



Hình 4.14. Sơ đồ tuần tự xét duyệt đề tài

4.5.4 Sơ đồ tuần tự nộp bài

Quy trình nộp bài bắt đầu khi sinh viên mở một yêu cầu nộp bài trong lớp học phần đang tham gia. Frontend gửi yêu cầu lấy thông tin chi tiết để hiển thị nội dung, thời hạn, trạng thái và số lần nộp còn lại.

Khi sinh viên chọn tệp và xác nhận nộp, frontend kiểm tra dữ liệu đầu vào rồi gửi yêu cầu đến backend kèm theo JWT. Auth Middleware xác thực token và chuyển thông tin sinh viên đến SubmissionController.

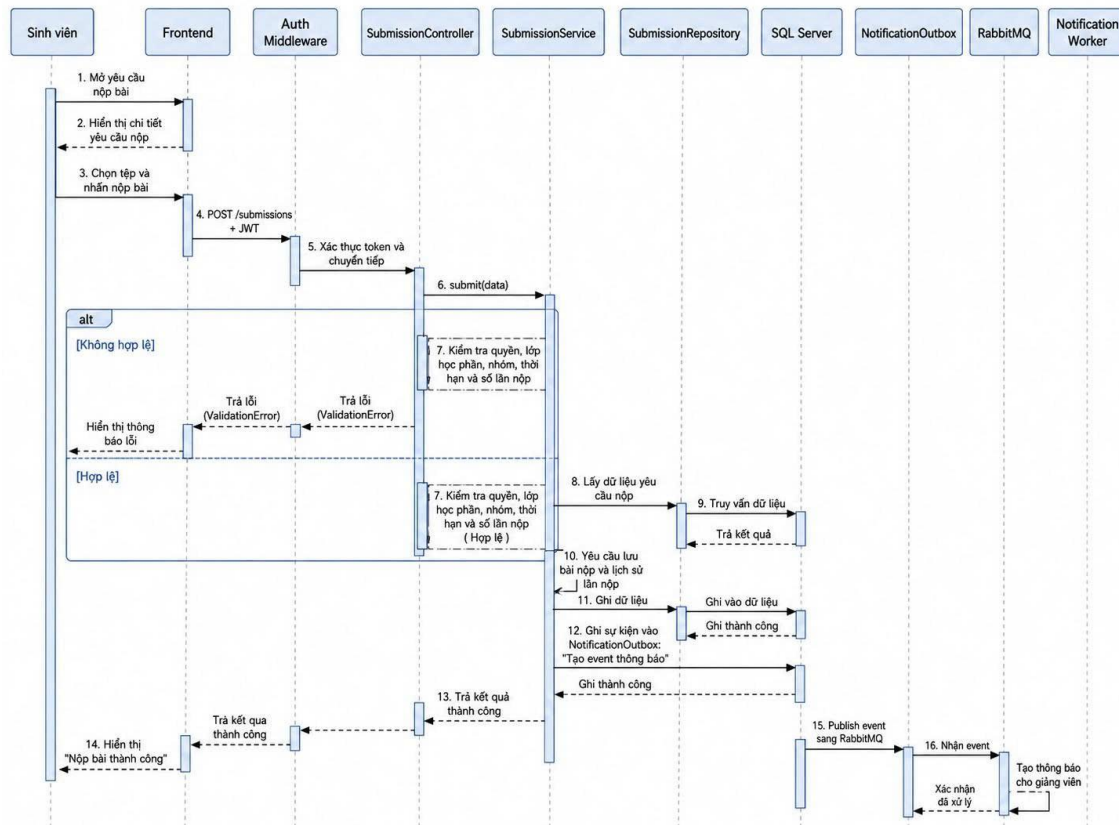
SubmissionService kiểm tra sinh viên có thuộc lớp học phần và nhóm liên quan hay không. Hệ thống tiếp tục kiểm tra trạng thái yêu cầu, thời gian cho phép nộp và số lần nộp tối đa.

Nếu không đủ điều kiện, backend trả về thông báo lỗi để frontend hiển thị lý do. Nếu hợp lệ, SubmissionRepository lưu thông tin bài nộp và tạo bản ghi lịch sử cho lần nộp hiện tại.

Trong cùng quá trình xử lý, hệ thống ghi một sự kiện vào NotificationOutbox. Sự kiện sau đó được chuyển qua RabbitMQ để Notification Worker tạo thông báo cho giảng viên phụ trách.

Trình tự xử lý gồm:

- Sinh viên mở yêu cầu nộp bài.
- Frontend lấy thông tin chi tiết yêu cầu.
- Sinh viên chọn tệp và xác nhận nộp.
- Middleware xác thực JWT.
- Controller gọi SubmissionService.
- Service kiểm tra lớp học phần và nhóm.
- Service kiểm tra thời hạn và số lần nộp.
- Repository lưu bài nộp.
- Repository lưu lịch sử lần nộp.
- Hệ thống ghi sự kiện thông báo.
- Backend trả kết quả thành công.
- Notification Worker tạo thông báo cho giảng viên.



Hình 4.15. Sơ đồ tuần tự nộp bài

Sau khi frontend nhận kết quả thành công, giao diện cập nhật danh sách lịch sử nộp và hiển thị thông báo xác nhận cho sinh viên.

4.5.5 Sơ đồ tuần tự chấm điểm và công bố kết quả

Quy trình chấm điểm bắt đầu khi giảng viên mở danh sách bài nộp của một yêu cầu trong lớp học phân được phân công. Frontend gửi yêu cầu đến backend để lấy dữ liệu bài nộp và lịch sử các lần nộp.

Backend xác thực JWT, kiểm tra vai trò giảng viên và quyền phụ trách lớp học phân. Khi quyền truy cập hợp lệ, hệ thống truy vấn danh sách bài nộp và trả dữ liệu về frontend.

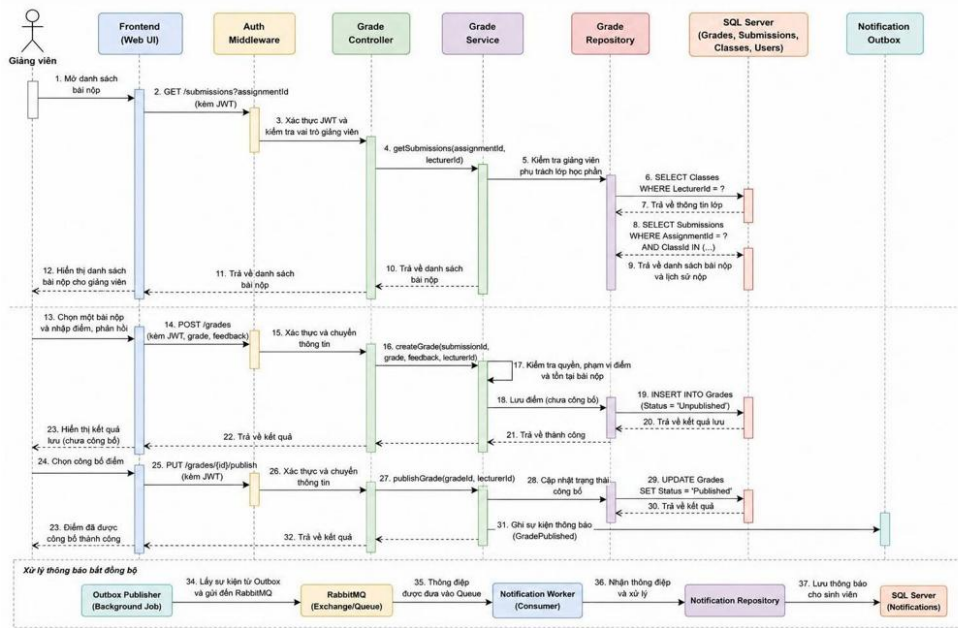
Giảng viên chọn một bài nộp, nhập điểm và nội dung phản hồi. GradeService kiểm tra quyền xử lý, phạm vi điểm và sự tồn tại của bài nộp trước khi lưu kết quả với trạng thái chưa công bố.

Ở trạng thái này, sinh viên chưa thể xem điểm. Khi giảng viên chọn công bố, frontend gửi một yêu cầu khác đến backend. Hệ thống cập nhật trạng thái công bố và ghi sự kiện vào NotificationOutbox.

Notification Worker xử lý sự kiện và tạo thông báo cho sinh viên. Sau đó, sinh viên có thể mở giao diện điểm số để xem điểm và phản hồi của giảng viên.

Trình tự xử lý gồm:

- Giảng viên mở danh sách bài nộp.
- Backend kiểm tra quyền phụ trách lớp.
- Repository lấy danh sách bài nộp.
- Giảng viên chọn bài cần đánh giá.
- Frontend gửi điểm và phản hồi.
- Service kiểm tra dữ liệu.
- Repository lưu điểm ở trạng thái chưa công bố.
- Giảng viên chọn công bố điểm.
- Backend cập nhật trạng thái công bố.
- Hệ thống ghi sự kiện thông báo.
- Notification Worker tạo thông báo cho sinh viên.
- Sinh viên xem điểm và phản hồi.



Hình 4.16. Sơ đồ tuần tự chấm điểm và công bố kết quả

4.6 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của website quản lý đồ án sinh viên được xây dựng trên Microsoft SQL Server theo mô hình quan hệ. Dữ liệu được tổ chức thành các bảng có khóa chính, khóa ngoại và các ràng buộc nhằm bảo đảm tính toàn vẹn trong quá trình thêm, cập nhật và truy xuất.

Thiết kế cơ sở dữ liệu phải đáp ứng các nghiệp vụ chính gồm quản lý tài khoản, môn học, lớp học phần, sinh viên trong lớp, nhóm sinh viên, đăng ký đề tài, yêu cầu nộp bài, bài nộp, lịch sử nộp, điểm số và thông báo.

Dữ liệu nghiệp vụ được giới hạn chủ yếu theo lớp học phần. Trường CourseClassId được sử dụng trong nhiều bảng để xác định dữ liệu thuộc lớp nào, từ đó hỗ trợ kiểm tra quyền truy cập của giảng viên và sinh viên.

4.6.1 Xác định các bảng dữ liệu chính

Các bảng trong hệ thống được chia thành những nhóm dữ liệu sau:

- Nhóm quản lý người dùng và phân quyền.
- Nhóm quản lý môn học và lớp học phần.
- Nhóm quản lý sinh viên và nhóm thực hiện.
- Nhóm quản lý đề tài.
- Nhóm quản lý yêu cầu và bài nộp.
- Nhóm quản lý điểm số.
- Nhóm quản lý thông báo.
- Nhóm lưu sự kiện xử lý bất đồng bộ.

Bảng 4.4. Danh sách các bảng dữ liệu chính

Tên bảng	Nội dung lưu trữ
Users	Lưu tài khoản, họ tên, email, mật khẩu đã băm, vai trò và trạng thái hoạt động của người dùng.
Courses	Lưu thông tin môn học như mã môn, tên môn và mô tả.
CourseClasses	Lưu thông tin lớp học phần, học kỳ, năm học, môn học và giảng viên phụ trách.
CourseClassEnrollments	Lưu danh sách sinh viên tham gia từng lớp học phần.
StudentGroups	Lưu thông tin các nhóm sinh viên trong lớp học phần.
StudentGroupMembers	Lưu danh sách thành viên và vai trò của từng sinh viên trong nhóm.

Tên bảng	Nội dung lưu trữ
Topics	Lưu thông tin đề tài như tên đề tài, mô tả và các nội dung liên quan.
TopicRegistrations	Lưu việc nhóm sinh viên đăng ký đề tài và trạng thái xét duyệt.
TopicReviews	Lưu quyết định duyệt, từ chối hoặc yêu cầu chỉnh sửa của giảng viên.
ProgressMilestones	Lưu các mốc tiến độ thực hiện đồ án do giảng viên thiết lập.
SubmissionRequirements	Lưu yêu cầu nộp bài, nội dung, thời hạn, số lần nộp và trạng thái hoạt động.
AssignmentSubmissions	Lưu thông tin bài nộp hiện tại của nhóm hoặc sinh viên.
SubmissionAttempts	Lưu lịch sử từng lần nộp, thời gian nộp và thông tin tệp.
Grades	Lưu điểm, phản hồi của giảng viên và trạng thái công bố kết quả.
Notifications	Lưu nội dung thông báo, người nhận và trạng thái đã đọc.
NotificationOutbox	Lưu các sự kiện thông báo đang chờ, đã xử lý hoặc xử lý thất bại.

Bảng AssignmentSubmissions lưu trạng thái hiện tại của bài nộp, trong khi bảng SubmissionAttempts lưu toàn bộ lịch sử các lần nộp. Việc tách hai bảng giúp hệ thống vừa truy xuất nhanh bài nộp mới nhất, vừa giữ được các phiên bản trước đó.

Tương tự, bảng Notifications lưu thông báo dành cho người dùng, còn bảng NotificationOutbox lưu sự kiện trung gian trước khi được chuyển đến RabbitMQ và Notification Worker.

4.6.2 Thiết kế mối quan hệ giữa các bảng

Bảng Users là bảng trung tâm quản lý tài khoản của quản trị viên, giảng viên và sinh viên. Vai trò của người dùng được xác định thông qua thuộc tính Role.

Một môn học trong bảng Courses có thể được mở thành nhiều lớp học phần trong bảng CourseClasses. Mỗi lớp học phần thuộc một môn học và được phân công cho một giảng viên phụ trách.

Mối quan hệ giữa sinh viên và lớp học phần là quan hệ nhiều – nhiều. Một sinh viên có thể tham gia nhiều lớp học phần, đồng thời một lớp học phần có nhiều sinh viên. Quan hệ này được biểu diễn thông qua bảng trung gian CourseClassEnrollments.

Một lớp học phần có thể có nhiều nhóm sinh viên. Mỗi nhóm thuộc duy nhất một lớp học phần. Quan hệ giữa nhóm và sinh viên cũng là quan hệ nhiều – nhiều và được biểu diễn thông qua bảng StudentGroupMembers.

Mỗi nhóm có thể thực hiện đăng ký đề tài. Thông tin đề tài được lưu trong bảng Topics, còn trạng thái đăng ký và xét duyệt được lưu trong bảng TopicRegistrations. Các quyết định của giảng viên được ghi nhận trong bảng TopicReviews.

Một lớp học phần có thể có nhiều mốc tiến độ và nhiều yêu cầu nộp bài. Mỗi yêu cầu nộp bài thuộc một lớp học phần và có thể tiếp nhận nhiều bài nộp từ các nhóm sinh viên.

Một bài nộp có thể có nhiều lần nộp. Bảng AssignmentSubmissions lưu thông tin bài nộp chính, còn bảng SubmissionAttempts lưu từng phiên bản được gửi lên hệ thống.

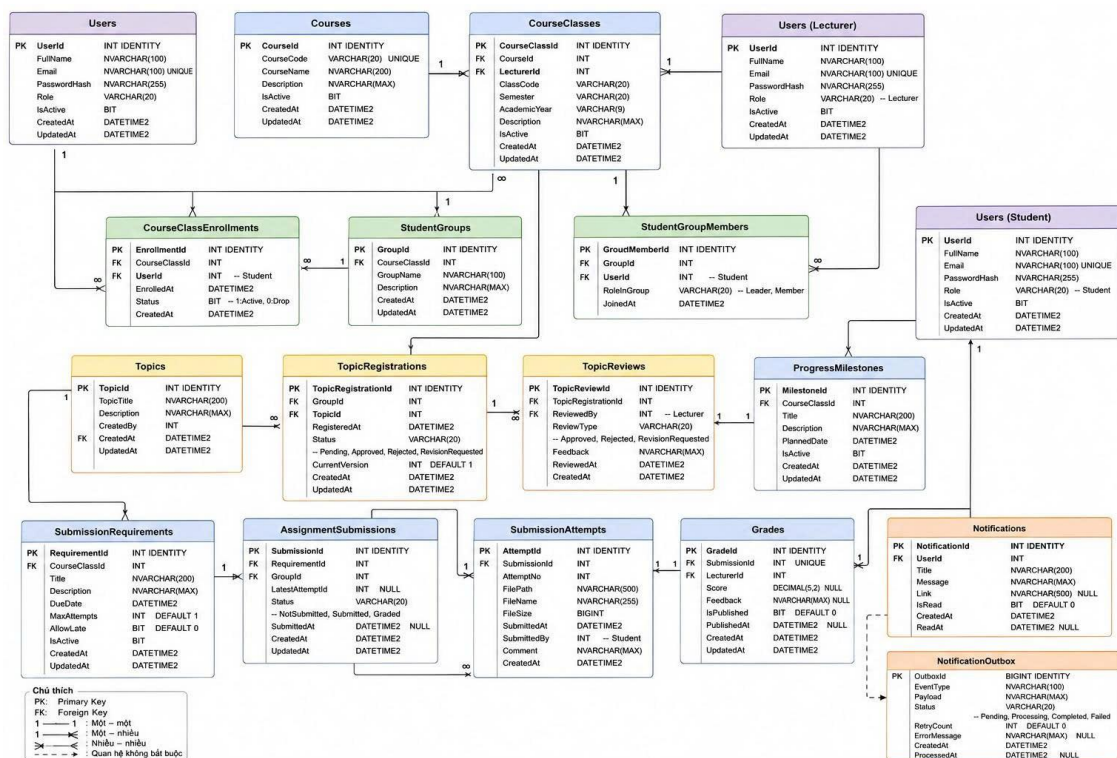
Mỗi bài nộp có thể có một kết quả đánh giá trong bảng Grades. Điểm chỉ được hiển thị cho sinh viên khi thuộc tính trạng thái công bố được cập nhật.

Một người dùng có thể nhận nhiều thông báo trong bảng Notifications. Trạng thái đọc được lưu riêng cho từng thông báo để người dùng có thể theo dõi các nội dung mới.

Các quan hệ chính của cơ sở dữ liệu gồm:

- Một môn học có nhiều lớp học phần.

- Một giảng viên có thể phụ trách nhiều lớp học phần.
- Một lớp học phần có nhiều sinh viên.
- Một lớp học phần có nhiều nhóm sinh viên.
- Một nhóm có nhiều thành viên.
- Một nhóm có thể đăng ký một đề tài tại một thời điểm.
- Một lớp học phần có nhiều yêu cầu nộp bài.
- Một yêu cầu có nhiều bài nộp.
- Một bài nộp có nhiều lần nộp.
- Một bài nộp có thể có một kết quả đánh giá.
- Một người dùng có thể nhận nhiều thông báo.



Hình 4.17. Sơ đồ quan hệ cơ sở dữ liệu của hệ thống

4.6.3 Thiết kế bảng người dùng

Bảng Users được sử dụng để lưu trữ thông tin tài khoản của quản trị viên, giảng viên và sinh viên. Các loại người dùng được phân biệt thông qua trường Role, thay vì thiết kế thành nhiều bảng tài khoản riêng biệt.

Email được sử dụng làm thông tin đăng nhập và không được trùng giữa các tài khoản. Mật khẩu không được lưu dưới dạng văn bản thông thường mà phải được băm trước khi đưa vào cơ sở dữ liệu.

Trường IsActive xác định tài khoản còn được phép sử dụng hệ thống hay không. Khi tài khoản bị vô hiệu hóa, người dùng không thể đăng nhập nhưng dữ liệu liên quan vẫn được giữ lại.

Bảng 4.5. Cấu trúc bảng Users

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
UserId	INT	Khóa chính, tự tăng	Mã định danh của người dùng.
FullName	NVARCHAR(100)	Không rỗng	Họ và tên người dùng.
Email	NVARCHAR(100)	Không rỗng, duy nhất	Email đăng nhập hệ thống.
PasswordHash	NVARCHAR(255)	Không rỗng	Mật khẩu đã được băm.
Role	VARCHAR(20)	Không rỗng	Vai trò gồm ADMIN, LECTURER hoặc STUDENT.
IsActive	BIT	Giá trị mặc định là 1	Trạng thái hoạt động của tài khoản.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo tài khoản.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật tài khoản gần nhất.

Các giá trị của trường Role cần được kiểm soát để tránh xuất hiện vai trò không hợp lệ. Backend cũng phải kiểm tra vai trò trong JWT trước khi cho phép người dùng truy cập chức năng.

4.6.4 Thiết kế bảng môn học và lớp học phần

Bảng Courses lưu thông tin môn học được sử dụng trong hệ thống. Một môn học có thể được mở thành nhiều lớp học phần khác nhau theo từng học kỳ và năm học.

Bảng CourseClasses lưu thông tin cụ thể của mỗi lớp học phần. Mỗi lớp thuộc một môn học và được phân công cho một giảng viên phụ trách.

Quan hệ giữa hai bảng là quan hệ một – nhiều. Một bản ghi trong Courses có thể liên kết với nhiều bản ghi trong CourseClasses, nhưng mỗi lớp học phần chỉ thuộc một môn học.

Bảng 4.6. Cấu trúc bảng Courses

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
CourseId	INT	Khóa chính, tự tăng	Mã định danh môn học.
CourseCode	VARCHAR(20)	Không rỗng, duy nhất	Mã môn học.
CourseName	NVARCHAR(200)	Không rỗng	Tên môn học.
Description	NVARCHAR(MAX)	Có thể rỗng	Nội dung mô tả môn học.
IsActive	BIT	Giá trị mặc định là 1	Trạng thái sử dụng của môn học.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo môn học.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật gần nhất.

Bảng 4.7. Cấu trúc bảng CourseClasses

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
CourseClassId	INT	Khóa chính, tự tăng	Mã định danh lớp học phần.
CourseId	INT	Khóa ngoại đến Courses	Môn học của lớp học phần.
LecturerId	INT	Khóa ngoại đến Users	Giảng viên phụ trách lớp.
ClassCode	VARCHAR(20)	Không rỗng	Mã lớp học phần.
Semester	VARCHAR(20)	Không rỗng	Học kỳ mở lớp.
AcademicYear	VARCHAR(20)	Không rỗng	Năm học của lớp học phần.
Description	NVARCHAR(MAX)	Có thể rỗng	Mô tả bổ sung về lớp học phần.
IsActive	BIT	Giá trị mặc định là 1	Trạng thái hoạt động của lớp.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo lớp học phần.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật gần nhất.

Trường LecturerId phải tham chiếu đến tài khoản có vai trò giảng viên. Backend cần kiểm tra điều kiện này trước khi thực hiện phân công.

4.6.5 Thiết kế bảng sinh viên trong lớp và nhóm sinh viên

Quan hệ giữa sinh viên và lớp học phần được lưu trong bảng CourseClassEnrollments. Bảng trung gian này cho phép một sinh viên tham gia nhiều lớp học phần và một lớp có nhiều sinh viên.

Bảng StudentGroups lưu thông tin các nhóm thực hiện đồ án trong từng lớp học phần. Mỗi nhóm chỉ thuộc một lớp nhưng một lớp có thể có nhiều nhóm.

Danh sách thành viên của nhóm được lưu trong bảng StudentGroupMembers. Bảng này liên kết người dùng với nhóm và xác định vai trò của thành viên như trưởng nhóm hoặc thành viên.

Bảng 4.8. Cấu trúc bảng CourseClassEnrollments

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
EnrollmentId	INT	Khóa chính, tự tăng	Mã đăng ký tham gia lớp.
CourseClassId	INT	Khóa ngoại đến CourseClasses	Lớp học phần mà sinh viên tham gia.
UserId	INT	Khóa ngoại đến Users	Tài khoản sinh viên.
EnrolledAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm sinh viên được thêm vào lớp.
Status	VARCHAR(20)	Không rỗng	Trạng thái tham gia lớp.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo bản ghi.

Cặp giá trị CourseClassId và UserId cần được thiết lập duy nhất để tránh một sinh viên được thêm nhiều lần vào cùng một lớp học phần.

Bảng 4.9. Cấu trúc bảng StudentGroups

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
GroupId	INT	Khóa chính, tự tăng	Mã định danh nhóm sinh viên.
CourseClassId	INT	Khóa ngoại đến CourseClasses	Lớp học phần chứa nhóm.
GroupName	NVARCHAR(100)	Không rỗng	Tên của nhóm sinh viên.
Description	NVARCHAR(MAX)	Có thể rỗng	Mô tả bổ sung về nhóm.
CreatedAt	DATETIME2	Giá trị mặc định là thời	Thời điểm tạo nhóm.

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
		gian hiện tại	
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật nhóm gần nhất.

Tên nhóm có thể được đặt duy nhất trong phạm vi một lớp học phần để tránh nhầm lẫn giữa các nhóm trong cùng lớp.

Bảng 4.10. Cấu trúc bảng StudentGroupMembers

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
GroupMemberId	INT	Khóa chính, tự tăng	Mã thành viên nhóm.
GroupId	INT	Khóa ngoại đến StudentGroups	Nhóm mà sinh viên tham gia.
UserId	INT	Khóa ngoại đến Users	Tài khoản sinh viên.
RoleInGroup	VARCHAR(20)	Không rỗng	Vai trò trong nhóm như LEADER hoặc MEMBER.
JoinedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm sinh viên tham gia nhóm.

Cặp GroupId và UserId phải duy nhất để tránh một sinh viên xuất hiện nhiều lần trong cùng một nhóm. Hệ thống cũng cần kiểm tra sinh viên thuộc lớp học phần của nhóm trước khi thêm vào danh sách thành viên.

4.6.6 Thiết kế các bảng quản lý đề tài

Thông tin đề tài được tổ chức trong ba bảng gồm Topics, TopicRegistrations và TopicReviews. Cách thiết kế này giúp tách riêng nội dung đề tài, quá trình đăng ký của nhóm và kết quả xét duyệt của giảng viên.

Bảng Topics lưu nội dung chính của đề tài. Bảng TopicRegistrations liên kết đề tài với nhóm sinh viên và lưu trạng thái xử lý hiện tại. Mỗi lần giảng viên đưa ra quyết định được ghi nhận trong bảng TopicReviews để lưu lịch sử xét duyệt.

Bảng 4.11. Cấu trúc bảng Topics

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
TopicId	INT	Khóa chính, tự tăng	Mã định danh đề tài.
TopicTitle	NVARCHAR(200)	Không rỗng	Tên đề tài.
Description	NVARCHAR(MAX)	Có thể rỗng	Nội dung mô tả chi tiết của đề tài.
CreatedBy	INT	Khóa ngoại đến Users	Người tạo đề tài.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo đề tài.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật nội dung gần nhất.

Tên đề tài phải được kiểm tra trước khi lưu để tránh dữ liệu rỗng hoặc vượt quá độ dài cho phép. Khi đề tài đã được duyệt, hệ thống không cho phép sinh viên tự ý thay đổi nội dung.

Bảng 4.12. Cấu trúc bảng TopicRegistrations

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
TopicRegistrationId	INT	Khóa chính, tự tăng	Mã đăng ký đề tài.
GroupId	INT	Khóa ngoại đến StudentGroups	Nhóm sinh viên đăng ký đề tài.
TopicId	INT	Khóa ngoại đến Topics	Đề tài được nhóm đăng ký.
RegisteredAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm gửi đăng ký.

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
Status	VARCHAR(30)	Không rỗng	Trạng thái xét duyệt của đăng ký.
CurrentVersion	INT	Giá trị mặc định là 1	Phiên bản hiện tại của nội dung đăng ký.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo bản ghi.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật gần nhất.

Trường Status có thể nhận các giá trị như:

- PENDING: đang chờ giảng viên xét duyệt.
- APPROVED: đề tài đã được duyệt.
- REJECTED: đề tài bị từ chối.
- REVISION_REQUESTED: giảng viên yêu cầu chỉnh sửa.

Một nhóm không được có nhiều đăng ký đề tài đang chờ xử lý hoặc đã được duyệt trong cùng một lớp học phần. Backend phải kiểm tra điều kiện này trước khi tạo bản ghi mới.

Bảng 4.13. Cấu trúc bảng TopicReviews

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
TopicReviewId	INT	Khóa chính, tự tăng	Mã lần xét duyệt đề tài.
TopicRegistrationId	INT	Khóa ngoại đến TopicRegistrations	Đăng ký đề tài được xét duyệt.
ReviewedBy	INT	Khóa ngoại đến Users	Giảng viên thực hiện xét duyệt.

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
ReviewType	VARCHAR(30)	Không rỗng	Kết quả xét duyệt đề tài.
Feedback	NVARCHAR(MAX)	Có thể rỗng	Nội dung nhận xét hoặc yêu cầu chỉnh sửa.
ReviewedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm xét duyệt.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo bản ghi.

Trường ReviewType có thể nhận các giá trị APPROVED, REJECTED hoặc REVISION_REQUESTED. Mỗi quyết định của giảng viên được lưu thành một bản ghi mới để giữ lại lịch sử xét duyệt.

4.6.7 Thiết kế bảng tiến độ và yêu cầu nộp bài

Bảng ProgressMilestones lưu các mốc quan trọng trong quá trình thực hiện đồ án. Giảng viên sử dụng các mốc này để định hướng công việc và giúp sinh viên theo dõi tiến độ.

Bảng SubmissionRequirements lưu các yêu cầu nộp bài trong từng lớp học phần. Mỗi yêu cầu có nội dung, thời gian bắt đầu, thời hạn kết thúc, số lần nộp tối đa và trạng thái hoạt động.

Bảng 4.14. Cấu trúc bảng ProgressMilestones

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
MilestoneId	INT	Khóa chính, tự tăng	Mã định danh mốc tiến độ.
CourseClassId	INT	Khóa ngoại đến CourseClasses	Lớp học phần áp dụng mốc tiến độ.
Title	NVARCHAR(200)	Không rỗng	Tên của mốc tiến độ.

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
Description	NVARCHAR(MAX)	Có thể rỗng	Nội dung công việc cần thực hiện.
PlannedDate	DATETIME2	Không rỗng	Thời gian dự kiến hoàn thành.
IsActive	BIT	Giá trị mặc định là 1	Trạng thái hoạt động của mốc tiến độ.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo mốc tiến độ.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật gần nhất.

Giảng viên chỉ được tạo hoặc cập nhật mốc tiến độ trong lớp học phần mà mình phụ trách. Những mốc không còn sử dụng có thể được chuyển sang trạng thái không hoạt động thay vì xóa khỏi cơ sở dữ liệu.

Bảng 4.15. Cấu trúc bảng SubmissionRequirements

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
RequirementId	INT	Khóa chính, tự tăng	Mã định danh yêu cầu nộp bài.
CourseClassId	INT	Khóa ngoại đến CourseClasses	Lớp học phần áp dụng yêu cầu.
Title	NVARCHAR(200)	Không rỗng	Tiêu đề yêu cầu nộp bài.
Description	NVARCHAR(MAX)	Có thể rỗng	Nội dung và hướng dẫn thực hiện.
StartDate	DATETIME2	Không rỗng	Thời điểm bắt đầu cho phép nộp bài.
DueDate	DATETIME2	Không rỗng	Thời hạn kết thúc nộp bài.
MaxAttempts	INT	Giá trị mặc định là 1	Số lần nộp tối đa

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
			được cho phép.
AllowLate	BIT	Giá trị mặc định là 0	Cho biết có chấp nhận bài nộp trễ hay không.
IsActive	BIT	Giá trị mặc định là 1	Trạng thái hoạt động của yêu cầu.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo yêu cầu.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật gần nhất.

DueDate phải lớn hơn StartDate và MaxAttempts phải lớn hơn không. Trước khi tiếp nhận bài nộp, backend kiểm tra trạng thái yêu cầu, thời gian hiện tại và số lần sinh viên đã nộp.

4.6.8 Thiết kế bảng bài nộp và lịch sử nộp

Dữ liệu bài nộp được tổ chức trong hai bảng gồm AssignmentSubmissions và SubmissionAttempts. Bảng AssignmentSubmissions lưu trạng thái hiện tại của bài nộp, trong khi bảng SubmissionAttempts lưu chi tiết từng lần sinh viên gửi bài.

Việc tách thành hai bảng giúp hệ thống truy xuất nhanh bài nộp mới nhất nhưng vẫn giữ được toàn bộ lịch sử nộp. Giảng viên có thể kiểm tra thời gian, tệp và nội dung ghi chú của từng lần nộp trước khi đánh giá.

Bảng 4.16. Cấu trúc bảng AssignmentSubmissions

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
SubmissionId	INT	Khóa chính, tự tăng	Mã định danh bài nộp.
RequirementId	INT	Khóa ngoại đến SubmissionRequirements	Yêu cầu nộp bài tương ứng.

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
GroupId	INT	Khóa ngoại đến StudentGroups	Nhóm sinh viên thực hiện bài nộp.
LatestAttemptId	INT	Có thể rỗng, khóa ngoại đến SubmissionAttempts	Lần nộp mới nhất của bài nộp.
Status	VARCHAR(20)	Không rỗng	Trạng thái hiện tại của bài nộp.
SubmittedAt	DATETIME2	Có thể rỗng	Thời điểm nộp gần nhất.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo bản ghi bài nộp.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật gần nhất.

Mỗi nhóm chỉ nên có một bản ghi AssignmentSubmissions đối với một yêu cầu nộp bài. Cặp RequirementId và GroupId cần được kiểm soát để tránh tạo nhiều bài nộp chính cho cùng một nhóm.

Trường Status có thể nhận các giá trị như chưa nộp, đã nộp hoặc đã chấm điểm. Khi sinh viên nộp phiên bản mới, hệ thống cập nhật LatestAttemptId và SubmittedAt.

Bảng 4.17. Cấu trúc bảng SubmissionAttempts

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
AttemptId	INT	Khóa chính, tự tăng	Mã định danh của lần nộp.
SubmissionId	INT	Khóa ngoại đến AssignmentSubmissions	Bài nộp chính chứa lần nộp này.
AttemptNo	INT	Không rỗng	Số thứ tự của lần nộp.
FilePath	NVARCHAR(500)	Không rỗng	Đường dẫn lưu trữ tệp bài nộp.

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
FileName	NVARCHAR(255)	Không rỗng	Tên tệp do sinh viên gửi.
FileSize	BIGINT	Có thể rỗng	Kích thước của tệp bài nộp.
SubmittedAt	DATETIME2	Không rỗng	Thời điểm sinh viên nộp bài.
SubmittedBy	INT	Khóa ngoại đến Users	Sinh viên thực hiện thao tác nộp.
Comment	NVARCHAR(MAX)	Có thể rỗng	Nội dung ghi chú cho lần nộp.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo bản ghi lịch sử.

Cặp SubmissionId và AttemptNo phải duy nhất để tránh trùng số lần nộp. Trước khi tạo bản ghi mới, backend kiểm tra thời hạn, trạng thái yêu cầu và số lần đã nộp so với MaxAttempts.

Những bản ghi lịch sử không nên bị ghi đè khi sinh viên nộp lại. Mỗi lần nộp mới được lưu thành một bản ghi riêng để bảo đảm khả năng theo dõi và đối chiếu.

4.6.9 Thiết kế bảng điểm và phản hồi

Bảng Grades lưu kết quả đánh giá bài nộp của sinh viên. Mỗi kết quả bao gồm điểm số, phản hồi của giảng viên, trạng thái công bố và thời điểm công bố.

Một bài nộp chỉ có một bản ghi điểm hiện tại. Giảng viên có thể cập nhật điểm và phản hồi trước khi công bố. Sinh viên chỉ được xem kết quả khi trường IsPublished có giá trị bằng 1.

Bảng 4.18. Cấu trúc bảng Grades

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
GradeId	INT	Khóa chính, tự tăng	Mã định danh kết

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
			qua đánh giá.
SubmissionId	INT	Khóa ngoại đến AssignmentSubmissions, duy nhất	Bài nộp được đánh giá.
LecturerId	INT	Khóa ngoại đến Users	Giảng viên thực hiện chấm điểm.
Score	DECIMAL(5,2)	Có thể rỗng	Điểm số của bài nộp.
Feedback	NVARCHAR(MAX)	Có thể rỗng	Nội dung nhận xét của giảng viên.
IsPublished	BIT	Giá trị mặc định là 0	Trạng thái công bố điểm.
PublishedAt	DATETIME2	Có thể rỗng	Thời điểm công bố điểm.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo kết quả đánh giá.
UpdatedAt	DATETIME2	Có thể cập nhật	Thời điểm cập nhật kết quả gần nhất.

Trường SubmissionId được thiết lập duy nhất để bảo đảm một bài nộp không có nhiều bản ghi điểm hiện tại. Khi giảng viên chấm lại, hệ thống cập nhật bản ghi đã tồn tại thay vì tạo thêm bản ghi mới.

Giá trị Score phải nằm trong phạm vi điểm mà hệ thống quy định. Giảng viên thực hiện chấm điểm phải là người đang phụ trách lớp học phần chứa bài nộp.

Khi công bố điểm, hệ thống cập nhật IsPublished thành 1 và ghi thời gian vào PublishedAt. Nếu giảng viên thu hồi kết quả, trạng thái công bố được chuyển về 0 để sinh viên tạm thời không xem được điểm.

4.6.10 Thiết kế bảng thông báo và sự kiện Outbox

Hệ thống sử dụng hai bảng Notifications và NotificationOutbox để quản lý quá trình gửi thông báo. Bảng Notifications lưu nội dung cuối cùng được hiển thị cho người dùng, còn NotificationOutbox lưu các sự kiện đang chờ xử lý bất đồng bộ.

Khi một nghiệp vụ quan trọng hoàn thành, backend ghi sự kiện vào NotificationOutbox. Tiến trình xử lý Outbox gửi sự kiện đến RabbitMQ, sau đó Notification Worker tạo thông báo cho người nhận trong bảng Notifications.

Bảng 4.19. Cấu trúc bảng Notifications

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
OutboxId	BIGINT	Khóa chính, tự tăng	Mã định danh sự kiện Outbox.
EventType	NVARCHAR(100)	Không rỗng	Loại sự kiện cần xử lý.
Payload	NVARCHAR(MAX)	Không rỗng	Dữ liệu sự kiện được lưu dưới dạng JSON.
Status	VARCHAR(20)	Không rỗng	Trạng thái xử lý của sự kiện.
RetryCount	INT	Giá trị mặc định là 0	Số lần hệ thống đã thử xử lý lại.
ErrorMessage	NVARCHAR(MAX)	Có thể rỗng	Nội dung lỗi khi xử lý sự kiện thất bại.
CreatedAt	DATETIME2	Giá trị mặc định là thời gian hiện tại	Thời điểm tạo sự kiện.
ProcessedAt	DATETIME2	Có thể rỗng	Thời điểm xử lý sự kiện thành công.

Trường Status có thể nhận các giá trị như:

- PENDING: sự kiện đang chờ xử lý.
- PROCESSING: sự kiện đang được xử lý.
- COMPLETED: sự kiện đã được xử lý thành công.
- FAILED: sự kiện xử lý thất bại.

Khi gửi sự kiện không thành công, hệ thống tăng RetryCount và lưu nội dung lỗi vào ErrorMessage. Khi sự kiện được xử lý thành công, trạng thái được cập nhật thành COMPLETED và thời gian hoàn thành được lưu trong ProcessedAt.

4.6.11 Thiết kế ràng buộc và chỉ mục cơ sở dữ liệu

Ngoài khóa chính và khóa ngoại, hệ thống sử dụng các ràng buộc để hạn chế dữ liệu trùng lặp, sai định dạng hoặc không phù hợp với quy tắc nghiệp vụ. Việc kiểm tra được thực hiện tại cả cơ sở dữ liệu và tầng service của backend.

Các trường có giá trị không được trùng như email, mã môn học và số thứ tự lần nộp cần được thiết lập ràng buộc duy nhất. Những trường trạng thái, vai trò và điểm số cần được kiểm soát để chỉ nhận các giá trị hợp lệ.

Một số ràng buộc quan trọng gồm:

- Email của người dùng không được trùng.
- Mã môn học không được trùng.
- Một sinh viên không được thêm nhiều lần vào cùng lớp học phần.
- Một sinh viên không được xuất hiện nhiều lần trong cùng nhóm.
- Một nhóm chỉ có một bài nộp chính cho mỗi yêu cầu.
- Số thứ tự lần nộp không được trùng trong cùng một bài nộp.
- Một bài nộp chỉ có một kết quả điểm hiện tại.
- Ngày kết thúc nộp bài phải lớn hơn ngày bắt đầu.
- Số lần nộp tối đa phải lớn hơn không.
- Điểm số phải nằm trong phạm vi cho phép.

Bên cạnh ràng buộc, hệ thống cần xây dựng chỉ mục cho các trường thường xuyên được sử dụng trong điều kiện tìm kiếm, lọc và liên kết. Chỉ mục giúp SQL Server giảm thời gian truy vấn khi số lượng tài khoản, lớp học phân, bài nộp và thông báo tăng lên.

Bảng 4.21. Các ràng buộc và chỉ mục quan trọng

Bảng	Trường hoặc nhóm trường	Loại thiết kế	Mục đích
Users	Email	UNIQUE	Không cho phép nhiều tài khoản sử dụng cùng một email.
Users	Role, IsActive	INDEX	Hỗ trợ lọc tài khoản theo vai trò và trạng thái.
Courses	CourseCode	UNIQUE	Bảo đảm mã môn học không bị trùng.
CourseClasses	CourseId, LecturerId	INDEX	Hỗ trợ tìm lớp theo môn học hoặc giảng viên phụ trách.
CourseClassEnrollments	CourseClassId, UserId	UNIQUE	Ngăn sinh viên được thêm nhiều lần vào cùng một lớp.
StudentGroups	CourseClassId, GroupName	UNIQUE	Hạn chế trùng tên nhóm trong cùng lớp học phân.
StudentGroupMembers	GroupId, UserId	UNIQUE	Ngăn một sinh viên xuất hiện nhiều lần trong

Bảng	Trường hoặc nhóm trường	Loại thiết kế	Mục đích
			cùng nhóm.
TopicRegistrations	GroupId, Status	INDEX	Hỗ trợ kiểm tra đề tài hiện tại của nhóm.
SubmissionRequirements	CourseClassId, IsActive, DueDate	INDEX	Hỗ trợ lọc yêu cầu theo lớp, trạng thái và thời hạn.
AssignmentSubmissions	RequirementId, GroupId	UNIQUE	Bảo đảm một nhóm chỉ có một bài nộp chính cho mỗi yêu cầu.
SubmissionAttempts	SubmissionId, AttemptNo	UNIQUE	Bảo đảm số thứ tự lần nộp không bị trùng.
Grades	SubmissionId	UNIQUE	Bảo đảm mỗi bài nộp chỉ có một kết quả điểm hiện tại.
Notifications	UserId, IsRead, CreatedAt	INDEX	Hỗ trợ lấy thông báo mới và thông báo chưa đọc.
NotificationOutbox	Status, CreatedAt	INDEX	Hỗ trợ tiến trình nền lấy nhanh các sự kiện đang chờ.

Các chỉ mục cần được lựa chọn phù hợp để tránh tạo quá nhiều chỉ mục không cần thiết. Mặc dù chỉ mục cải thiện tốc độ truy vấn, chúng cũng làm tăng chi phí khi thêm hoặc cập nhật dữ liệu.

4.7 Thiết kế RESTful API

RESTful API là thành phần trung gian giúp giao diện frontend trao đổi dữ liệu với backend. Frontend gửi yêu cầu thông qua giao thức HTTP, backend tiếp nhận yêu cầu, xác thực người dùng, kiểm tra quyền truy cập, xử lý nghiệp vụ và trả kết quả dưới định dạng JSON.

Trong website quản lý đồ án sinh viên, các API được chia theo từng nhóm chức năng như xác thực, tài khoản người dùng, môn học, lớp học phần, nhóm sinh viên, đề tài, tiến độ, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Việc tổ chức API theo từng nhóm tài nguyên giúp mã nguồn dễ quản lý, thuận tiện khi kiểm thử và hạn chế ảnh hưởng giữa các chức năng. Các API được mô tả bằng Swagger để hỗ trợ tra cứu và tích hợp giữa frontend với backend.

4.7.1 Nguyên tắc thiết kế API

Đường dẫn API được đặt tên theo tài nguyên cần quản lý và sử dụng cấu trúc thống nhất trong toàn hệ thống. Tên đường dẫn ưu tiên sử dụng danh từ để thể hiện rõ đối tượng đang được thao tác.

Phương thức HTTP được lựa chọn dựa trên mục đích của yêu cầu. Những thao tác lấy dữ liệu, tạo mới, cập nhật hoặc xóa được phân biệt thông qua phương thức HTTP thay vì đặt toàn bộ hành động trong đường dẫn.

Bảng 4.22. Các phương thức HTTP được sử dụng trong hệ thống

Phương thức HTTP	Mục đích sử dụng
GET	Lấy danh sách hoặc thông tin chi tiết của tài nguyên.
POST	Tạo mới tài nguyên hoặc thực hiện một nghiệp vụ mới.
PUT	Cập nhật toàn bộ thông tin hoặc trạng thái của tài nguyên.
PATCH	Cập nhật một phần thông tin của tài nguyên.
DELETE	Xóa hoặc ngừng sử dụng tài nguyên.

Đối với những API trả về danh sách, hệ thống hỗ trợ tìm kiếm, lọc, sắp xếp và phân trang. Các tham số được gửi kèm theo yêu cầu để backend chỉ truy xuất phạm vi dữ liệu cần thiết.

Các API phải thống nhất về cách đặt tên, dữ liệu đầu vào, dữ liệu trả về và mã trạng thái HTTP. Sự thống nhất này giúp frontend dễ xử lý dữ liệu và hạn chế lỗi trong quá trình tích hợp.

4.7.2 Xác thực và phân quyền API

Những API được bảo vệ yêu cầu người dùng phải đăng nhập và cung cấp JWT hợp lệ. Backend kiểm tra token trước khi chuyển yêu cầu đến tầng xử lý nghiệp vụ.

JWT chứa thông tin nhận diện người dùng và vai trò tương ứng. Hệ thống có ba vai trò chính gồm quản trị viên, giảng viên và sinh viên.

Quản trị viên được phép quản lý tài khoản, môn học, lớp học phần và các dữ liệu nền tảng. Giảng viên được phép quản lý các lớp học phần được phân công. Sinh viên chỉ được truy cập dữ liệu của lớp và nhóm mà mình tham gia.

Ngoài việc kiểm tra vai trò, backend còn kiểm tra phạm vi dữ liệu tại tầng service. Mỗi yêu cầu liên quan đến lớp học phần, nhóm sinh viên, đề tài, bài nộp hoặc điểm số đều phải được đối chiếu với tài khoản đang đăng nhập.

Khi token không tồn tại, không hợp lệ hoặc hết hạn, hệ thống từ chối yêu cầu. Khi người dùng đã đăng nhập nhưng không có quyền truy cập tài nguyên, backend trả về thông báo không được phép thực hiện thao tác.

4.7.3 Kiểm tra dữ liệu đầu vào

Dữ liệu gửi đến API phải được kiểm tra trước khi thực hiện nghiệp vụ và lưu vào cơ sở dữ liệu. Quá trình kiểm tra bao gồm trường bắt buộc, kiểu dữ liệu, độ dài, định dạng và phạm vi giá trị.

Các dữ liệu như email, tên đề tài, điểm số, thời gian bắt đầu, thời hạn nộp, số lần nộp và trạng thái phải đáp ứng đúng quy định của hệ thống.

Ngoài kiểm tra định dạng, tầng service còn kiểm tra các quy tắc nghiệp vụ như:

- Tài khoản phải tồn tại và đang hoạt động.
- Giảng viên phải được phân công phụ trách lớp học phần.
- Sinh viên phải thuộc lớp và nhóm liên quan.

- Nhóm không được có nhiều đề tài đang chờ xử lý hoặc đã được duyệt.
- Yêu cầu nộp bài phải còn hiệu lực.
- Số lần nộp không được vượt quá giới hạn.
- Điểm số phải nằm trong phạm vi cho phép.
- Sinh viên chỉ được xem điểm đã được công bố.

Khi dữ liệu không hợp lệ, backend không thực hiện thao tác lưu trữ và trả về thông báo để frontend hiển thị cho người dùng.

4.7.4 Thiết kế dữ liệu phản hồi

Dữ liệu phản hồi từ API được tổ chức theo cấu trúc thống nhất. Nội dung phản hồi có thể bao gồm trạng thái xử lý, thông báo, dữ liệu kết quả và thông tin phân trang.

Đối với yêu cầu thành công, backend trả về dữ liệu cần thiết cùng thông báo xác nhận. Đối với yêu cầu thất bại, backend trả về nội dung lỗi và mã trạng thái phù hợp.

Các API danh sách cần cung cấp thông tin về trang hiện tại, số bản ghi trên mỗi trang, tổng số bản ghi và tổng số trang. Dữ liệu này giúp frontend xây dựng chức năng phân trang và hiển thị danh sách chính xác.

Thông báo lỗi không được chứa các thông tin nhạy cảm như câu truy vấn SQL, mật khẩu, khóa JWT, địa chỉ kết nối cơ sở dữ liệu hoặc cấu hình máy chủ.

4.7.5 Mã trạng thái HTTP

Mã trạng thái HTTP được sử dụng để thể hiện kết quả xử lý của mỗi yêu cầu. Frontend dựa vào mã trạng thái để xác định yêu cầu thành công, dữ liệu không hợp lệ, người dùng chưa đăng nhập hoặc không có quyền truy cập.

Bảng 4.23. Các mã trạng thái HTTP được sử dụng trong hệ thống

Mã trạng thái	Ý nghĩa	Trường hợp sử dụng
200 OK	Yêu cầu được xử lý thành công.	Lấy dữ liệu hoặc cập nhật tài nguyên thành công.

Mã trạng thái	Ý nghĩa	Trường hợp sử dụng
201 Created	Tài nguyên được tạo thành công.	Hoàn thành thao tác thêm mới dữ liệu.
400 Bad Request	Dữ liệu đầu vào hoặc điều kiện nghiệp vụ không hợp lệ.	Thiếu thông tin, sai định dạng hoặc không đáp ứng quy tắc nghiệp vụ.
401 Unauthorized	Người dùng chưa được xác thực.	Không có token, token không hợp lệ hoặc token đã hết hạn.
403 Forbidden	Người dùng không có quyền truy cập.	Vai trò hoặc phạm vi dữ liệu không phù hợp.
404 Not Found	Không tìm thấy tài nguyên.	Tài nguyên được yêu cầu không tồn tại.
409 Conflict	Dữ liệu xảy ra xung đột.	Dữ liệu bị trùng hoặc trạng thái hiện tại không cho phép thao tác.
500 Internal Server Error	Lỗi xảy ra trong quá trình xử lý tại máy chủ.	Lỗi cơ sở dữ liệu hoặc lỗi hệ thống chưa được xử lý.

4.7.6 Tài liệu hóa và kiểm thử API

Các RESTful API được mô tả bằng Swagger. Tài liệu API trình bày đường dẫn, phương thức HTTP, dữ liệu đầu vào, dữ liệu trả về, yêu cầu xác thực và các mã trạng thái có thể xuất hiện.

Swagger giúp nhóm frontend hiểu cách sử dụng API và hỗ trợ kiểm tra endpoint trong quá trình phát triển. Bên cạnh đó, hệ thống còn sử dụng Postman và các bài kiểm thử tự động để kiểm tra tính đúng đắn của API.

Quá trình kiểm thử tập trung vào các trường hợp dữ liệu hợp lệ, dữ liệu không hợp lệ, thiếu token, token hết hạn, sai vai trò và truy cập dữ liệu ngoài phạm vi. Việc kiểm thử thường xuyên giúp phát hiện lỗi sớm và bảo đảm API hoạt động đúng với yêu cầu nghiệp vụ.

4.8 Thiết kế giao diện người dùng

Giao diện người dùng là thành phần trực tiếp tiếp nhận thao tác và hiển thị dữ liệu cho quản trị viên, giảng viên và sinh viên. Giao diện của website quản lý đồ án sinh viên được xây dựng bằng ReactJS, đồng thời kết nối với backend thông qua RESTful API.

Mỗi vai trò được cung cấp nhóm chức năng và khu vực làm việc riêng. Sau khi đăng nhập, hệ thống xác định vai trò của tài khoản và chuyển người dùng đến giao diện phù hợp.

Quá trình thiết kế giao diện tập trung vào tính rõ ràng, dễ sử dụng, thống nhất và hỗ trợ hiển thị trên nhiều kích thước màn hình. Những chức năng thường xuyên sử dụng được bố trí tại vị trí dễ tìm kiếm nhằm giảm số bước thao tác.

4.8.1 Nguyên tắc thiết kế giao diện

Giao diện của hệ thống được xây dựng dựa trên các nguyên tắc sau:

- Bố cục giữa các trang phải thống nhất.
- Chức năng được phân chia rõ theo từng vai trò.
- Nội dung quan trọng phải dễ nhận biết.
- Các nút thao tác cần sử dụng tên gọi rõ ràng.
- Trạng thái dữ liệu phải được thể hiện trực quan.
- Biểu mẫu phải thông báo lỗi tại trường dữ liệu liên quan.
- Các bảng danh sách cần hỗ trợ tìm kiếm và lọc.
- Giao diện phải thích nghi với nhiều kích thước màn hình.
- Người dùng phải nhận được phản hồi sau mỗi thao tác.
- Các chức năng nguy hiểm cần có bước xác nhận.

Màu sắc được sử dụng thống nhất để thể hiện trạng thái. Trạng thái thành công được trình bày bằng màu dễ nhận biết, trạng thái đang chờ được thể hiện bằng màu trung tính, còn lỗi hoặc thao tác từ chối được thể hiện bằng màu cảnh báo.

Nội dung dài như tên môn học, tên lớp học phần, tên đề tài và tên tệp cần được xử lý để không làm phá vỡ bố cục. Khi cần thiết, giao diện sử dụng xuống dòng, rút gọn nội dung hoặc hiển thị đầy đủ khi người dùng đưa con trỏ vào.

4.8.2 Thiết kế bố cục chung

Bố cục chung của hệ thống gồm thanh điều hướng, khu vực nội dung chính và các thành phần hỗ trợ như thông báo, hộp thoại xác nhận và trạng thái tải dữ liệu.

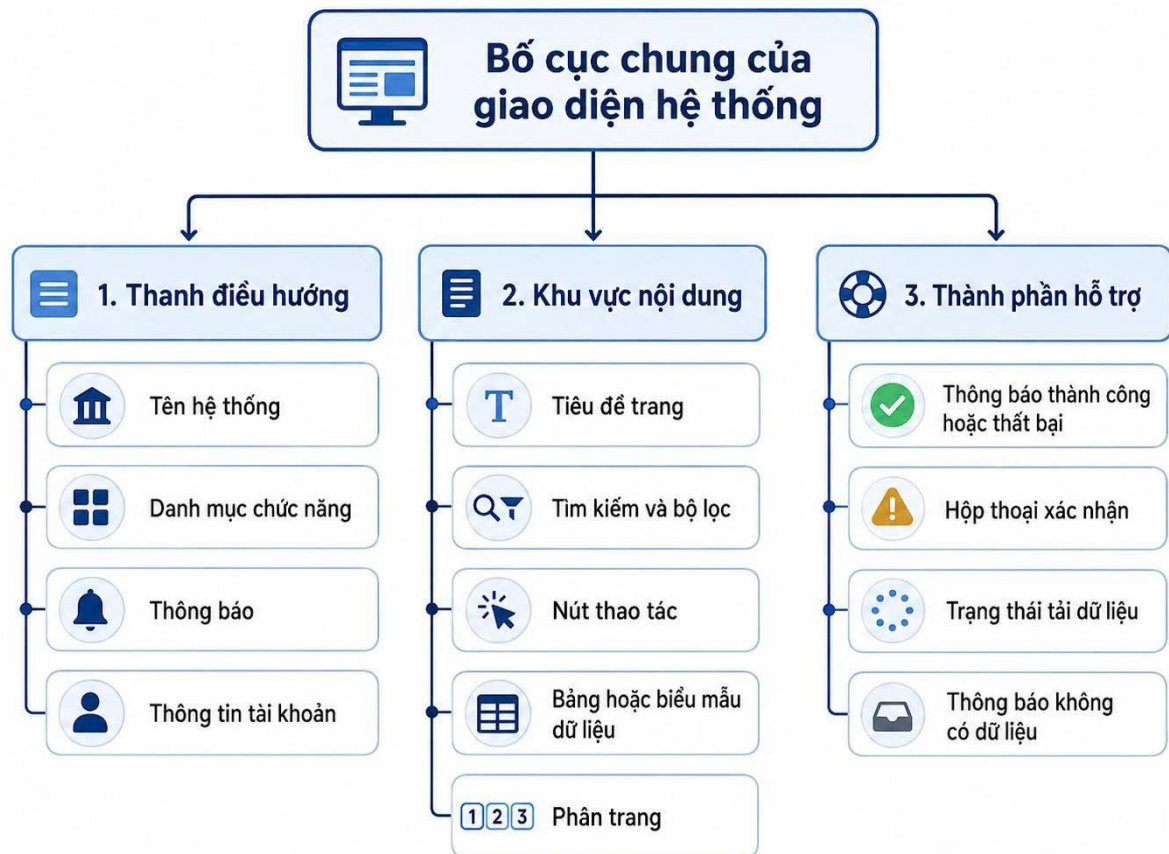
Thanh điều hướng hiển thị tên hệ thống, các chức năng thuộc vai trò hiện tại, thông báo và thông tin tài khoản. Các mục không thuộc quyền của người dùng sẽ không được hiển thị.

Khu vực nội dung chính thay đổi theo trang người dùng đang truy cập. Tiêu đề trang được đặt ở phía trên, bên dưới là thanh tìm kiếm, bộ lọc, nút thao tác và nội dung dữ liệu.

Đối với những trang hiển thị danh sách, dữ liệu được trình bày bằng bảng. Các cột chỉ hiển thị thông tin cần thiết, trong khi các thao tác xem chi tiết, cập nhật hoặc thay đổi trạng thái được đặt tại cột chức năng.

Khi hệ thống đang lấy dữ liệu từ backend, giao diện hiển thị trạng thái tải để người dùng biết yêu cầu đang được xử lý. Nếu không có dữ liệu, hệ thống hiển thị thông báo phù hợp thay vì để trống khu vực nội dung.

Khi xảy ra lỗi, giao diện hiển thị thông báo dễ hiểu và không đưa trực tiếp lỗi kỹ thuật từ backend cho người dùng. Những lỗi liên quan đến token hết hạn sẽ chuyển người dùng về trang đăng nhập.



Hình 4.18. Bố cục chung của giao diện hệ thống

4.8.3 Thiết kế giao diện đăng nhập

Trang đăng nhập là giao diện đầu tiên khi người dùng truy cập hệ thống mà chưa có phiên đăng nhập hợp lệ. Giao diện gồm tên hệ thống, trường nhập email, trường nhập mật khẩu và nút đăng nhập.

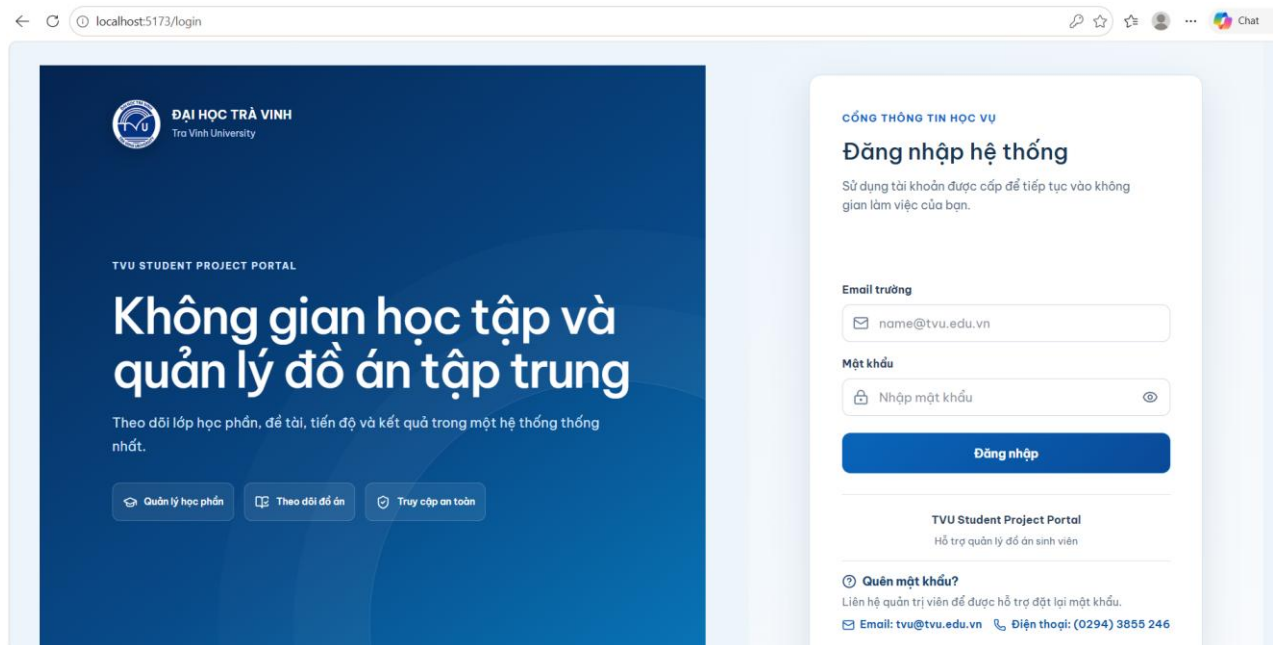
Người dùng phải nhập đầy đủ email và mật khẩu trước khi gửi yêu cầu. Nếu một trường bị bỏ trống hoặc sai định dạng, giao diện hiển thị thông báo ngay tại vị trí tương ứng.

Khi người dùng nhấn nút đăng nhập, giao diện chuyển sang trạng thái đang xử lý để hạn chế gửi nhiều yêu cầu liên tiếp. Frontend gửi thông tin đăng nhập đến backend và chờ kết quả xác thực.

Nếu thông tin không chính xác, tài khoản không tồn tại hoặc đã bị vô hiệu hóa, hệ thống hiển thị thông báo đăng nhập thất bại. Nội dung thông báo không tiết lộ chi tiết khiến tài khoản bị từ chối nhằm hạn chế rủi ro bảo mật.

Nếu đăng nhập thành công, frontend lưu JWT và thông tin người dùng. Hệ thống dựa vào vai trò để chuyển đến trang làm việc tương ứng:

- Quản trị viên được chuyển đến trang tổng quan quản trị.
- Giảng viên được chuyển đến trang quản lý lớp phụ trách.
- Sinh viên được chuyển đến trang tổng quan học tập.



Hình 4.19. Giao diện đăng nhập hệ thống

Giao diện đăng nhập cần bảo đảm đơn giản, dễ thao tác và không hiển thị các chức năng không cần thiết. Mật khẩu được che khi nhập và người dùng có thể lựa chọn hiển thị hoặc ẩn mật khẩu trong quá trình kiểm tra.

4.8.4 Thiết kế giao diện dành cho quản trị viên

Giao diện quản trị viên được xây dựng để hỗ trợ quản lý các dữ liệu nền tảng của hệ thống. Sau khi đăng nhập thành công, quản trị viên được chuyển đến trang tổng quan, nơi

hiển thị các thông tin khái quát như số lượng tài khoản, môn học, lớp học phần và người dùng đang hoạt động.

Thanh điều hướng của quản trị viên bao gồm những chức năng chính như quản lý tài khoản, quản lý môn học, quản lý lớp học phần và thông tin cá nhân. Các chức năng được phân chia thành từng mục riêng để người dùng dễ tìm kiếm và thao tác.

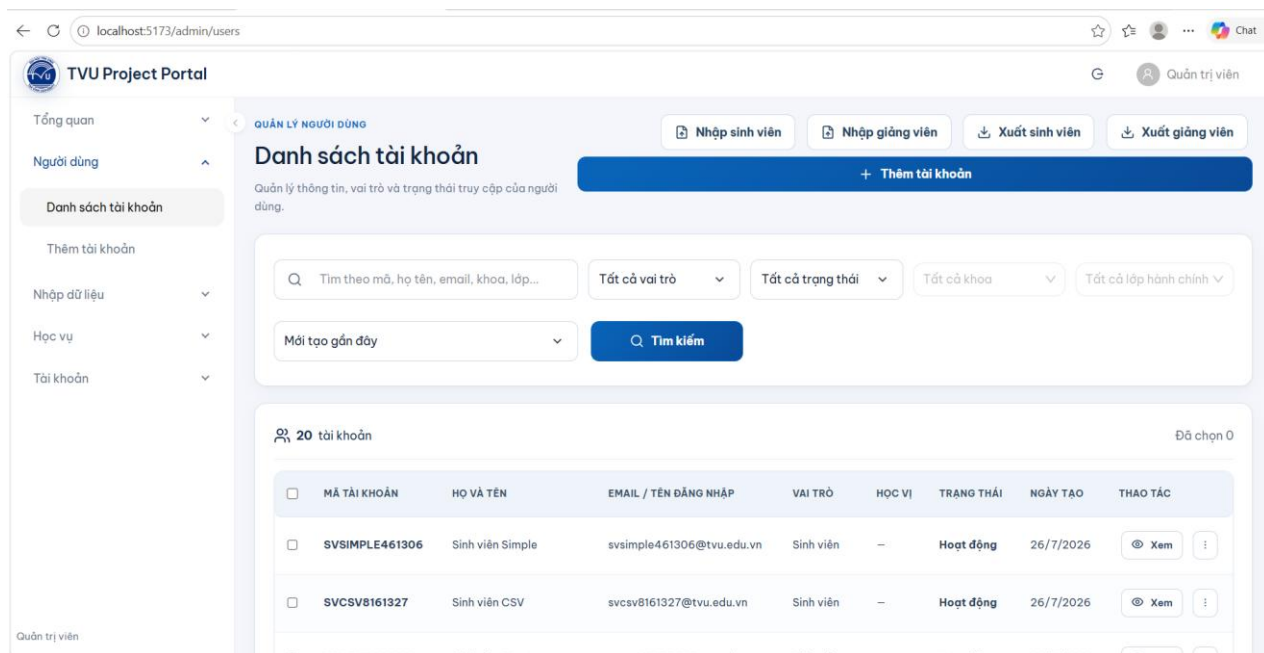
4.8.4.1 Giao diện quản lý tài khoản

Màn hình quản lý tài khoản hiển thị danh sách quản trị viên, giảng viên và sinh viên trong hệ thống. Thông tin cơ bản của mỗi tài khoản gồm họ tên, email, vai trò và trạng thái hoạt động.

Quản trị viên có thể tìm kiếm tài khoản theo tên hoặc email, đồng thời lọc danh sách theo vai trò và trạng thái. Các thao tác tạo mới, cập nhật thông tin và thay đổi trạng thái tài khoản được thực hiện thông qua biểu mẫu hoặc hộp thoại.

Khi tạo tài khoản mới, giao diện yêu cầu nhập đầy đủ những thông tin bắt buộc. Frontend kiểm tra dữ liệu trước khi gửi đến backend. Nếu email đã tồn tại hoặc thông tin không hợp lệ, hệ thống hiển thị thông báo để quản trị viên điều chỉnh.

Khi vô hiệu hóa tài khoản, giao diện hiển thị hộp thoại xác nhận nhằm hạn chế thao tác nhầm. Tài khoản bị vô hiệu hóa không thể đăng nhập nhưng dữ liệu liên quan vẫn được giữ trong hệ thống.



Hình 4.20. Giao diện quản lý tài khoản người dùng

4.8.4.2 Giao diện quản lý môn học

Màn hình quản lý môn học hiển thị mã môn, tên môn, mô tả và trạng thái sử dụng. Quản trị viên có thể tìm kiếm môn học, thêm môn học mới hoặc cập nhật thông tin của môn học đã tồn tại.

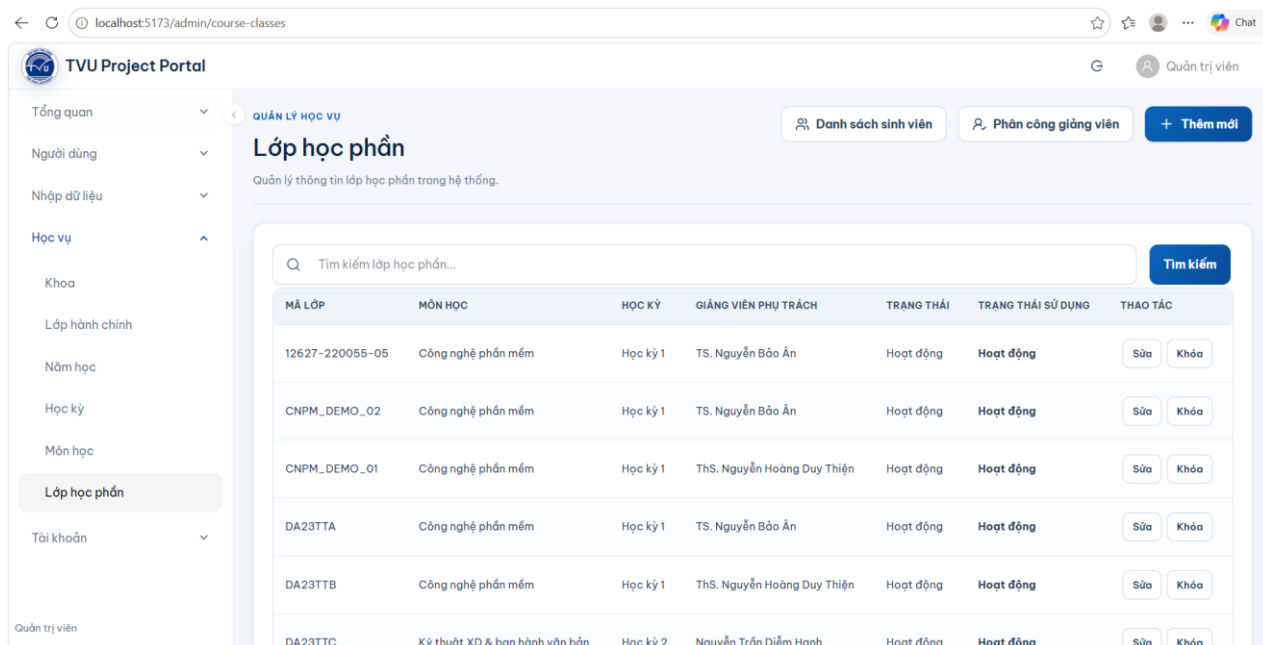
Mã môn học phải được kiểm tra để tránh trùng lặp. Những môn học đã được sử dụng trong lớp học phần không nên xóa trực tiếp mà có thể chuyển sang trạng thái không hoạt động.

4.8.4.3 Giao diện quản lý lớp học phần

Màn hình quản lý lớp học phần cho phép quản trị viên xem danh sách lớp theo môn học, học kỳ, năm học và giảng viên phụ trách. Người dùng có thể tìm kiếm hoặc lọc dữ liệu để nhanh chóng xác định lớp cần quản lý.

Khi tạo lớp học phần, quản trị viên lựa chọn môn học, giảng viên phụ trách, học kỳ và năm học. Backend kiểm tra tài khoản được phân công có đúng vai trò giảng viên hay không trước khi lưu dữ liệu.

Từ trang chi tiết lớp học phần, quản trị viên có thể xem thông tin môn học, giảng viên phụ trách và danh sách sinh viên. Các chức năng thêm hoặc loại sinh viên khỏi lớp được thực hiện tại khu vực này.



Hình 4.21. Giao diện quản lý lớp học phần

4.8.5 Thiết kế giao diện dành cho giảng viên

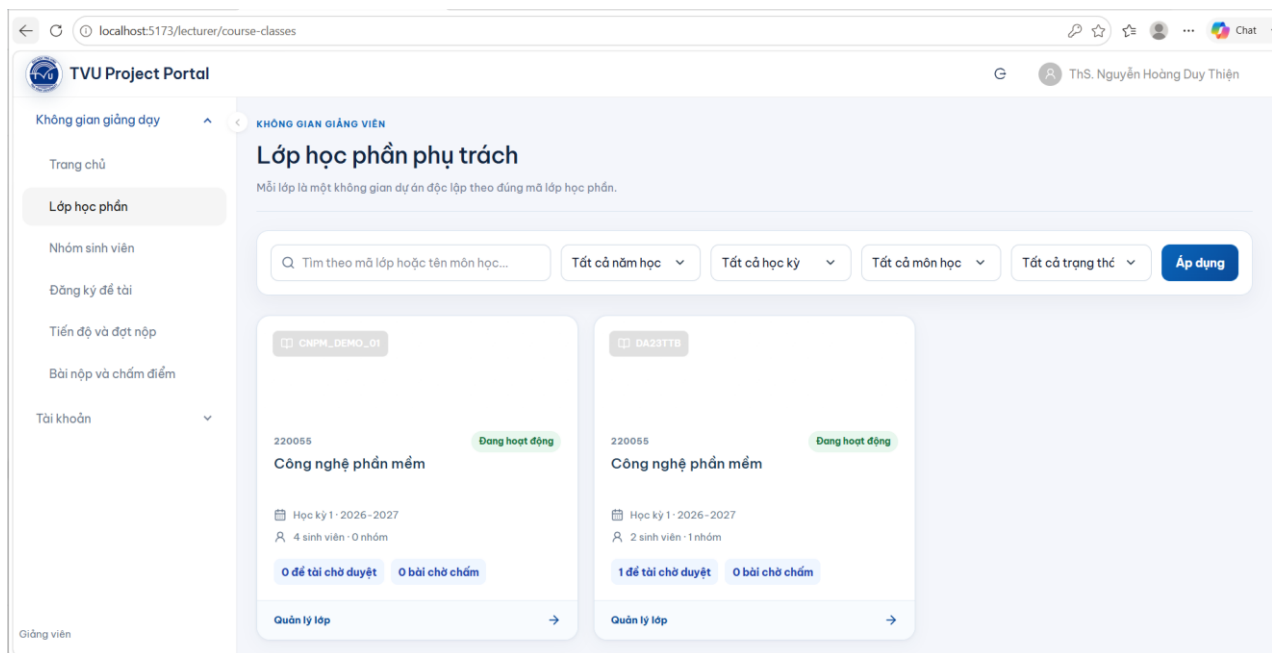
Giao diện giảng viên tập trung vào việc quản lý quá trình thực hiện đồ án trong các lớp học phần được phân công. Sau khi đăng nhập, giảng viên được chuyển đến trang tổng quan hoặc danh sách lớp phụ trách.

Mỗi lớp học phần được hiển thị với các thông tin như mã lớp, tên môn học, học kỳ và năm học. Giảng viên chọn một lớp để truy cập các chức năng quản lý sinh viên, nhóm, đề tài, tiến độ, yêu cầu nộp bài, bài nộp và điểm số.

4.8.5.1 Giao diện danh sách lớp phụ trách

Màn hình danh sách lớp phụ trách chỉ hiển thị những lớp học phần được phân công cho giảng viên hiện tại. Điều này giúp hạn chế hiển thị dữ liệu không liên quan và hỗ trợ kiểm soát phạm vi truy cập.

Giảng viên có thể tìm kiếm lớp theo mã lớp hoặc tên môn học. Khi chọn một lớp, hệ thống chuyển đến trang chi tiết và hiển thị các nhóm chức năng liên quan.



Hình 4.22. Giao diện danh sách lớp học phần giảng viên phụ trách

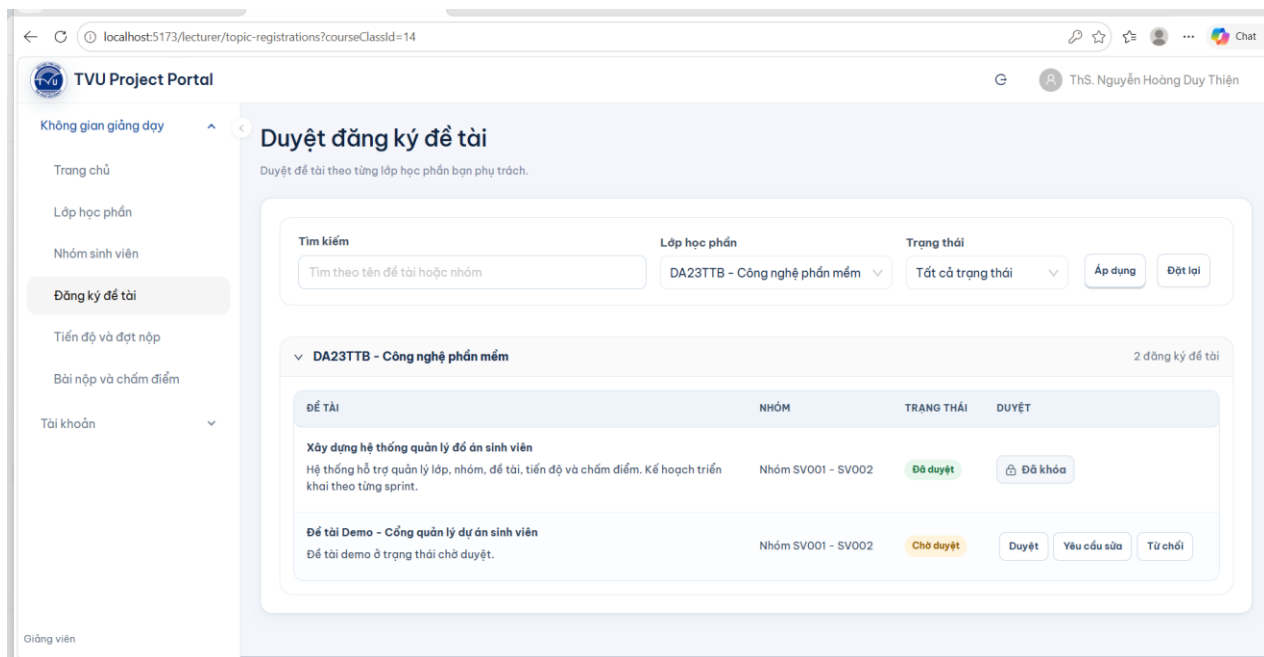
4.8.5.2 Giao diện quản lý đề tài

Màn hình quản lý đề tài hiển thị danh sách đăng ký của các nhóm trong lớp học phần. Mỗi bản ghi thể hiện tên nhóm, tên đề tài, thời điểm đăng ký và trạng thái xét duyệt.

Giảng viên có thể lọc đề tài theo trạng thái đang chờ duyệt, đã duyệt, bị từ chối hoặc cần chỉnh sửa. Khi mở chi tiết, giảng viên xem nội dung đề tài và đưa ra quyết định xử lý.

Các thao tác xét duyệt gồm duyệt đề tài, từ chối hoặc yêu cầu chỉnh sửa. Đối với trường hợp từ chối hoặc yêu cầu chỉnh sửa, giao diện nên yêu cầu giảng viên nhập phản hồi để sinh viên biết nguyên nhân và nội dung cần điều chỉnh.

Sau khi xử lý thành công, trạng thái đề tài được cập nhật trên bảng và hệ thống tạo thông báo cho nhóm sinh viên liên quan.



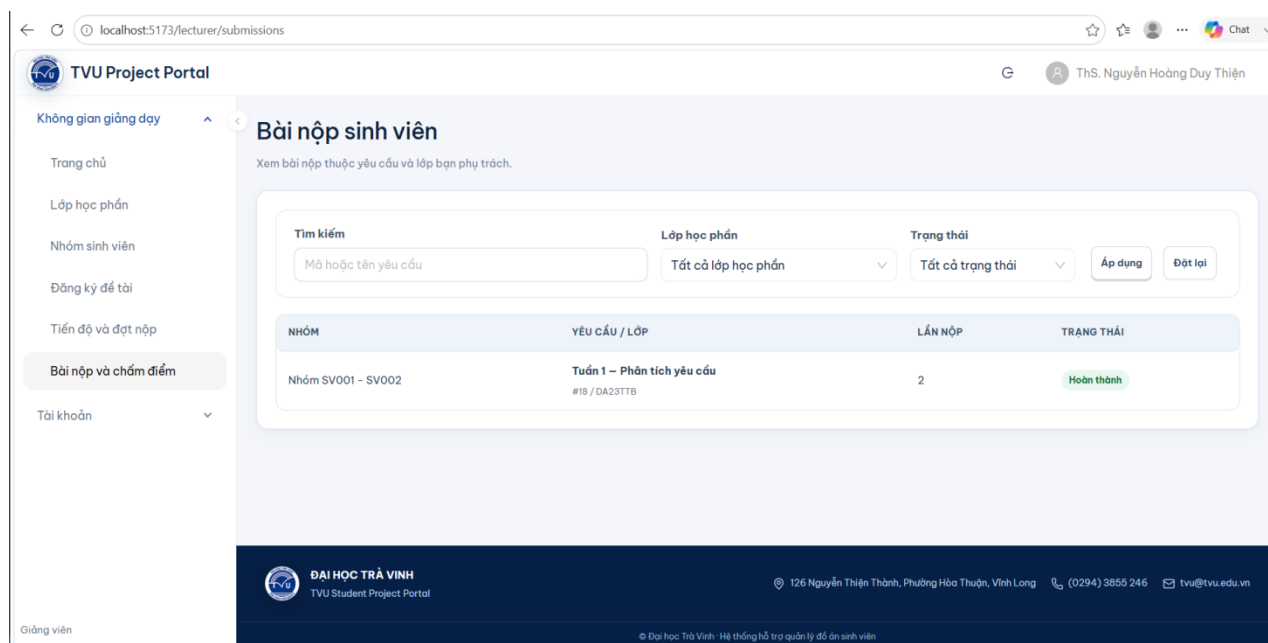
Hình 4.23. Giao diện xét duyệt đề tài của giảng viên

4.8.5.3 Giao diện quản lý yêu cầu và bài nộp

Giảng viên có thể tạo yêu cầu nộp bài trong từng lớp học phần. Biểu mẫu tạo yêu cầu bao gồm tiêu đề, nội dung, thời gian bắt đầu, thời hạn, số lần nộp tối đa và trạng thái hoạt động.

Màn hình bài nộp hiển thị danh sách nhóm đã nộp, thời gian nộp gần nhất, số lần nộp và trạng thái đánh giá. Giảng viên có thể mở chi tiết để xem tệp, lịch sử các lần nộp và thông tin người thực hiện.

Tại giao diện đánh giá, giảng viên nhập điểm và phản hồi, sau đó lưu kết quả ở trạng thái chưa công bố. Khi kiểm tra hoàn tất, giảng viên thực hiện công bố để sinh viên có thể xem kết quả.



Hình 4.24. Giao diện quản lý bài nộp và chấm điểm

4.8.6 Thiết kế giao diện dành cho sinh viên

Giao diện sinh viên được thiết kế nhằm hỗ trợ quá trình theo dõi và thực hiện đồ án trong các lớp học phần đang tham gia. Sau khi đăng nhập thành công, sinh viên được chuyển đến trang tổng quan, nơi hiển thị những thông tin liên quan đến lớp học phần, nhóm, đề tài, tiến độ, yêu cầu nộp bài và thông báo mới.

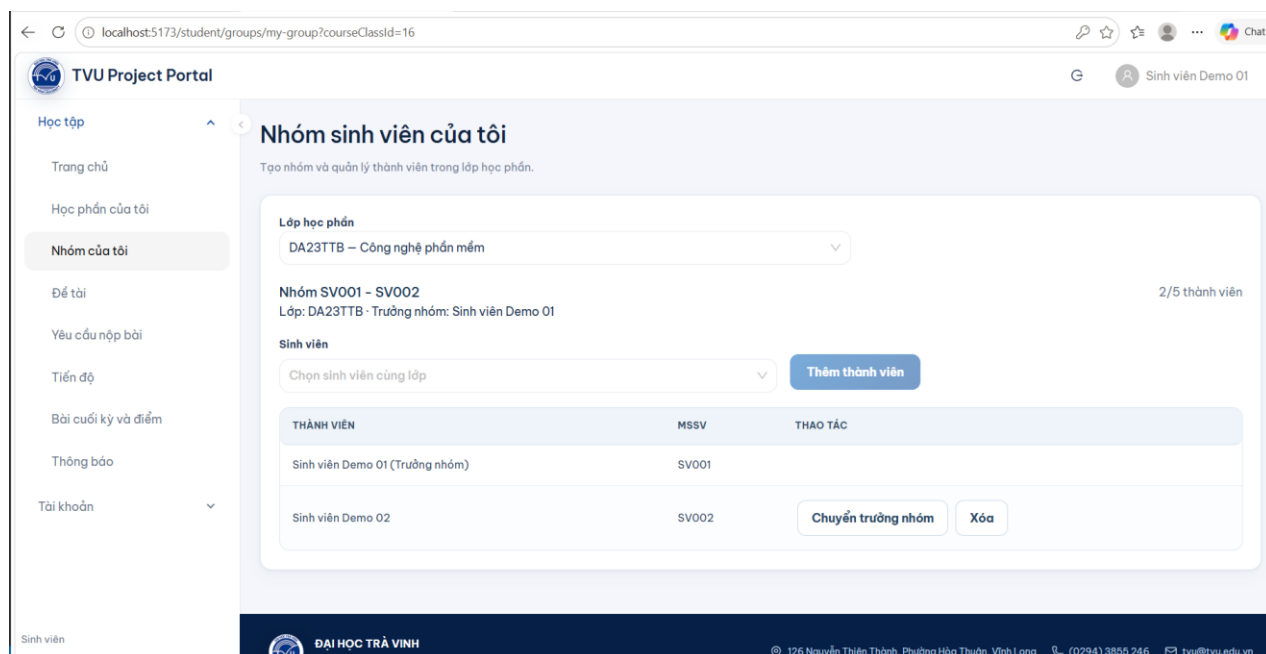
Thanh điều hướng của sinh viên chỉ hiển thị các chức năng phù hợp với vai trò. Sinh viên không thể truy cập những chức năng quản trị hoặc chức năng chuyên môn dành cho giảng viên.

4.8.6.1 Giao diện lớp học phần và nhóm sinh viên

Màn hình lớp học phần hiển thị danh sách các lớp mà sinh viên đang tham gia. Mỗi lớp có những thông tin cơ bản như mã lớp, tên môn học, học kỳ, năm học và giảng viên phụ trách.

Khi chọn một lớp học phần, sinh viên có thể xem thông tin chi tiết, danh sách thành viên trong nhóm và vai trò của từng thành viên. Nếu sinh viên chưa được phân vào nhóm, giao diện hiển thị trạng thái phù hợp thay vì để trống nội dung.

Dữ liệu lớp học phần và nhóm được giới hạn theo tài khoản đang đăng nhập. Sinh viên không thể truy cập thông tin nhóm của lớp học phần mà mình không tham gia.



Hình 4.25. Giao diện lớp học phần và nhóm sinh viên

4.8.6.2 Giao diện đăng ký đề tài

Màn hình đăng ký đề tài cho phép sinh viên đại diện nhóm nhập tên đề tài, nội dung mô tả và các thông tin cần thiết. Trước khi hiển thị biểu mẫu, hệ thống kiểm tra sinh viên có thuộc nhóm hợp lệ và nhóm đã có đề tài đang xử lý hay chưa.

Sau khi gửi đăng ký, giao diện hiển thị trạng thái hiện tại của đề tài. Các trạng thái có thể bao gồm đang chờ duyệt, đã được duyệt, bị từ chối hoặc cần chỉnh sửa.

Khi giảng viên yêu cầu chỉnh sửa, sinh viên có thể mở lại biểu mẫu, cập nhật nội dung và gửi lại. Nếu đề tài đã được duyệt, chức năng chỉnh sửa được khóa để bảo đảm nội dung không bị thay đổi tùy ý.

Phản hồi của giảng viên được hiển thị cùng trạng thái đề tài nhằm giúp sinh viên biết nguyên nhân bị từ chối hoặc nội dung cần điều chỉnh.

The screenshot displays the 'Đăng ký đề tài' (Topic Registration) page on the TVU Project Portal. The page has a light blue header with the TVU logo and the text 'TVU Project Portal'. Below the header, there's a sidebar on the left with navigation links: 'Học tập' (Study), 'Trang chủ' (Home), 'Học phần của tôi' (My Courses), 'Nhóm của tôi' (My Groups), 'Đề tài' (Topics), 'Yêu cầu nộp bài' (Assignment Requirements), 'Tiến độ' (Progress), 'Bài cuối kỳ và điểm' (Final Exam and Grades), 'Thông báo' (Notifications), and 'Tài khoản' (Account). The main content area is titled 'Đăng ký đề tài' and includes a sub-header 'Trạng thái đề tài' (Topic Status) with a 'Chờ duyệt' (Pending Approval) button. The form contains several text input fields: 'Đề tài Demo - Cổng quản lý dự án sinh viên', 'Đề tài demo ở trạng thái chờ duyệt', 'Node.js, React, SQL Server', 'Kiểm tra thủ công lượng nhóm và đăng ký đề tài', and a 'Ghi chú' (Notes) section. A blue 'Lưu đăng ký' (Save Registration) button is at the bottom of the form.

Hình 4.26. Giao diện đăng ký và theo dõi trạng thái đề tài

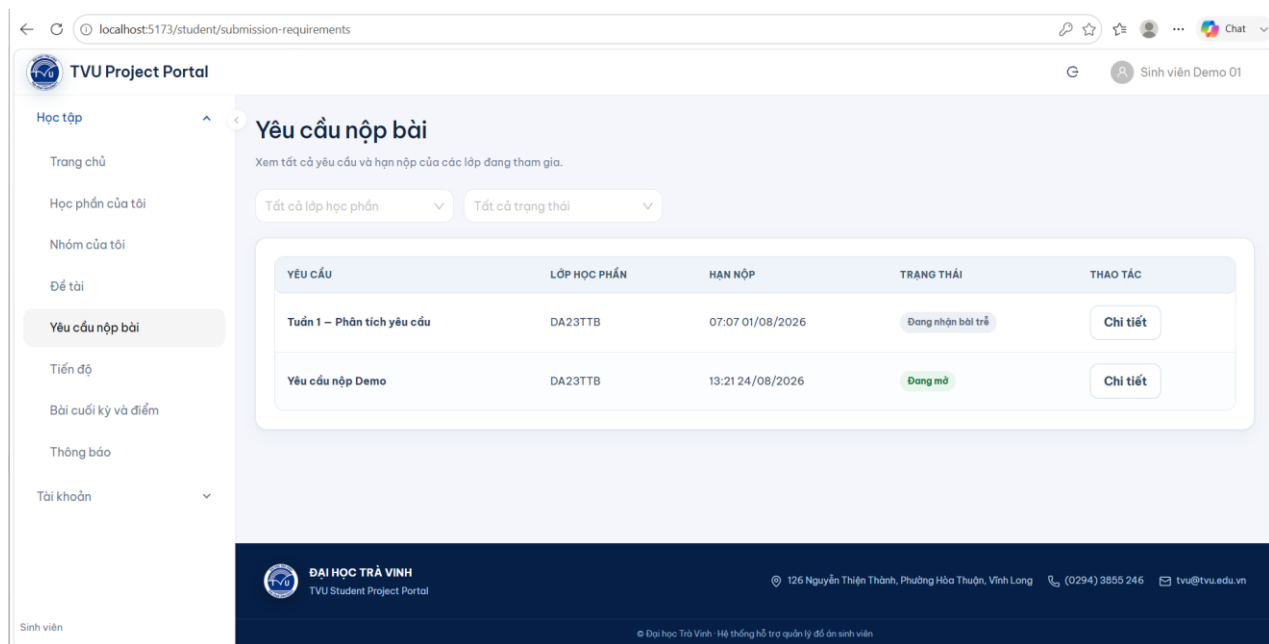
4.8.6.3 Giao diện yêu cầu và nộp bài

Màn hình yêu cầu nộp bài hiển thị danh sách các yêu cầu do giảng viên tạo trong lớp học phần. Mỗi yêu cầu có các thông tin như tiêu đề, nội dung, thời gian bắt đầu, thời hạn, số lần nộp tối đa và trạng thái hoạt động.

Sinh viên chọn một yêu cầu để xem chi tiết và thực hiện nộp bài. Giao diện hiển thị số lần đã nộp, số lần còn lại và tình trạng thời hạn để sinh viên biết mình có đủ điều kiện nộp hay không.

Khi nộp bài, sinh viên chọn tệp, nhập ghi chú nếu cần và xác nhận thao tác. Frontend kiểm tra dữ liệu trước khi gửi đến backend. Nếu bài nộp được tiếp nhận, giao diện hiển thị thông báo thành công và cập nhật lịch sử nộp.

Mỗi lần nộp được hiển thị với số thứ tự, tên tệp, người nộp và thời gian thực hiện. Sinh viên có thể theo dõi các phiên bản đã gửi nhưng không được tự ý sửa dữ liệu lịch sử.



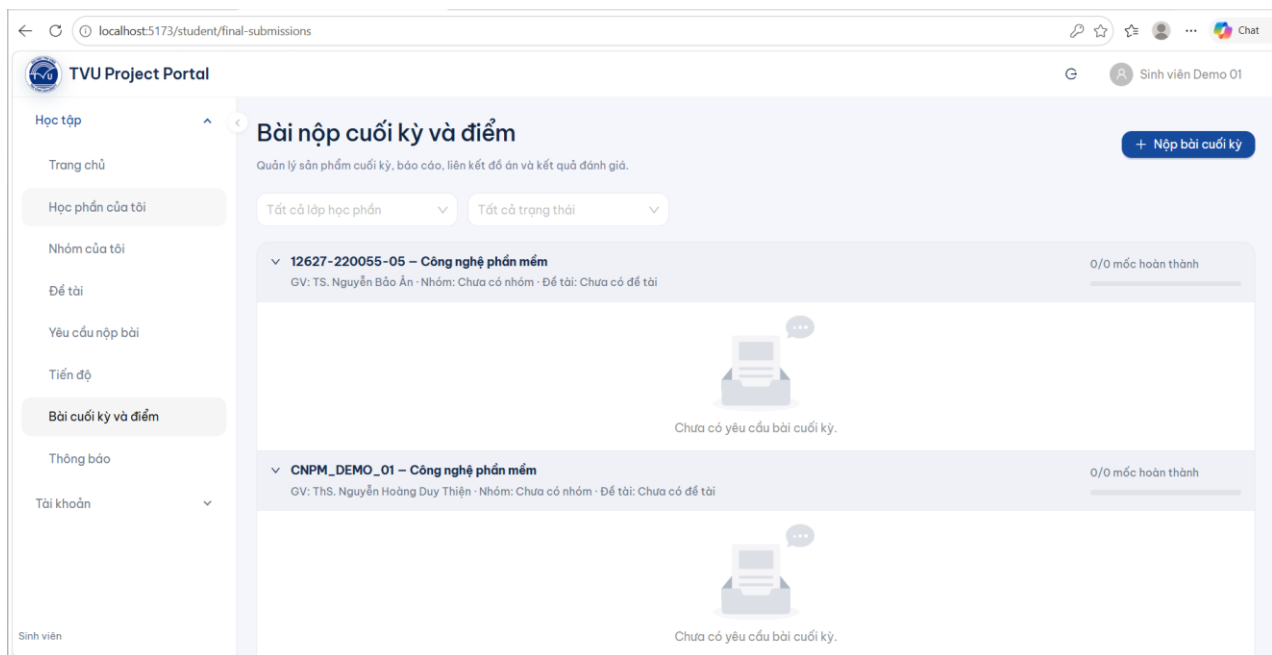
Hình 4.27. Giao diện yêu cầu nộp bài và lịch sử nộp

4.8.6.4 Giao diện xem điểm và phản hồi

Màn hình kết quả hiển thị điểm số và phản hồi của giảng viên đối với bài nộp. Sinh viên chỉ được xem kết quả khi giảng viên đã thực hiện công bố điểm.

Nếu điểm chưa được công bố, giao diện hiển thị trạng thái đang chờ đánh giá hoặc chưa công bố, không hiển thị dữ liệu điểm từ backend.

Khi kết quả được công bố, sinh viên có thể xem điểm, nhận xét và thời điểm công bố. Nếu giảng viên thu hồi trạng thái công bố để chỉnh sửa, kết quả tạm thời không còn hiển thị trên giao diện sinh viên.



Hình 4.28. Giao diện xem điểm và phản hồi của giảng viên

4.8.7 Thiết kế giao diện thông báo

Chức năng thông báo được sử dụng chung cho quản trị viên, giảng viên và sinh viên. Biểu tượng thông báo được đặt trên thanh điều hướng để người dùng dễ nhận biết khi có nội dung mới.

Danh sách thông báo hiển thị tiêu đề, nội dung tóm tắt, thời gian tạo và trạng thái đã đọc hoặc chưa đọc. Những thông báo chưa đọc được trình bày nổi bật hơn để người dùng dễ phân biệt.

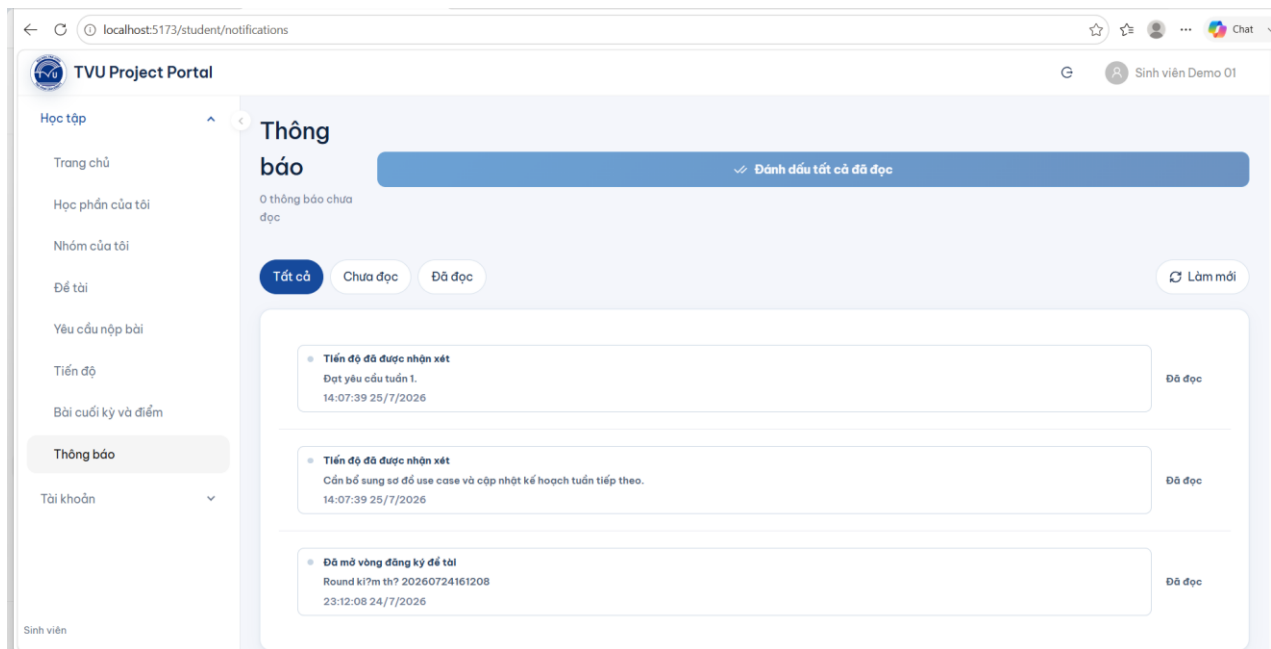
Khi người dùng chọn một thông báo, hệ thống hiển thị nội dung chi tiết và có thể chuyển đến chức năng liên quan thông qua đường dẫn được lưu trong thông báo. Sau khi mở, trạng thái thông báo được cập nhật thành đã đọc.

Các loại thông báo chính trong hệ thống gồm:

- Có đăng ký đề tài mới cần xét duyệt.
- Đề tài đã được duyệt, từ chối hoặc yêu cầu chỉnh sửa.
- Có yêu cầu nộp bài mới.

- Sinh viên đã gửi bài nộp.
- Điểm và phản hồi đã được công bố.
- Có thay đổi liên quan đến lớp học phần hoặc nhóm sinh viên.

Khi danh sách chưa có thông báo, giao diện hiển thị trạng thái không có dữ liệu. Nếu quá trình tải thông báo thất bại, hệ thống hiển thị thông báo lỗi và cho phép người dùng thử lại.



Hình 4.29. Giao diện danh sách thông báo

4.8.8 Thiết kế giao diện responsive và trạng thái hệ thống

Giao diện của website quản lý đồ án sinh viên được thiết kế để có thể hiển thị trên nhiều kích thước màn hình khác nhau. Khi kích thước cửa sổ thay đổi, các thành phần như thanh điều hướng, bảng dữ liệu, biểu mẫu và hộp thoại cần tự động điều chỉnh để không bị tràn hoặc chồng lấn nội dung.

Trên màn hình máy tính, thanh điều hướng được hiển thị ở bên trái và khu vực nội dung được bố trí ở bên phải. Khi màn hình có kích thước nhỏ, thanh điều hướng có thể được thu gọn để dành nhiều không gian hơn cho nội dung chính.

Đối với các bảng dữ liệu có nhiều cột, hệ thống cho phép cuộn theo chiều ngang hoặc ẩn bớt những cột ít quan trọng. Các nút thao tác cần được bố trí hợp lý để người dùng vẫn có thể sử dụng trên màn hình nhỏ.

Biểu mẫu nhập liệu được chia thành các nhóm thông tin rõ ràng. Trên màn hình rộng, các trường có thể được bố trí trên nhiều cột. Khi hiển thị trên thiết bị có kích thước nhỏ, các trường được chuyển thành một cột để thuận tiện cho việc nhập dữ liệu.

Ngoài khả năng hiển thị responsive, giao diện còn phải xử lý đầy đủ các trạng thái trong quá trình trao đổi với backend.

Các trạng thái chính gồm:

- Đang tải dữ liệu.
- Không có dữ liệu.
- Tải dữ liệu thành công.
- Thao tác thành công.
- Dữ liệu đầu vào không hợp lệ.
- Người dùng không có quyền truy cập.
- Không thể kết nối đến backend.
- Lỗi phát sinh trong quá trình xử lý.

Khi đang tải dữ liệu, hệ thống hiển thị biểu tượng hoặc nội dung chờ để người dùng biết yêu cầu đang được xử lý. Nếu danh sách không có bản ghi, giao diện hiển thị thông báo phù hợp thay vì để trống khu vực nội dung.

Khi thao tác thành công, hệ thống hiển thị thông báo xác nhận. Nếu xảy ra lỗi, giao diện hiển thị nội dung ngắn gọn và dễ hiểu, đồng thời cung cấp nút thử lại trong những trường hợp phù hợp.

Những lỗi kỹ thuật như câu truy vấn SQL, stack trace hoặc thông tin cấu hình không được hiển thị trực tiếp cho người dùng. Các thông tin chi tiết chỉ được ghi nhận trong log phục vụ quá trình kiểm tra và sửa lỗi.

← ↻ ⓘ localhost:5173/login



ĐẠI HỌC TRÀ VINH

CỔNG THÔNG TIN HỌC VỤ

Đăng nhập hệ thống

Sử dụng tài khoản được cấp để tiếp tục vào không gian làm việc của bạn.

Email trường

 sv001@tvu.edu.vn

Mật khẩu

Đăng nhập

TVU Student Project Portal

Hỗ trợ quản lý đồ án sinh viên

Hình 4.30. Giao diện hiển thị trên các kích thước màn hình khác nhau

4.9 Thiết kế triển khai hệ thống

Hệ thống được thiết kế để có thể vận hành trong cả môi trường phát triển cục bộ và môi trường cloud. Các thành phần frontend, backend, cơ sở dữ liệu và dịch vụ xử lý thông báo được triển khai riêng nhưng liên kết với nhau thông qua mạng và các biến môi trường.

Trong môi trường local, nhóm sử dụng Docker hoặc chạy trực tiếp từng dịch vụ. Frontend được khởi chạy bằng Vite, backend được chạy bằng Node.js, dữ liệu được lưu trong SQL Server và RabbitMQ được sử dụng để truyền sự kiện thông báo.

Trong môi trường cloud, frontend, backend và Notification Worker được triển khai thành các dịch vụ riêng. Cơ sở dữ liệu SQL Server và RabbitMQ được cung cấp bởi các nền tảng độc lập.

Việc tách các thành phần giúp hệ thống dễ theo dõi, cập nhật và xử lý khi một dịch vụ gặp sự cố. Backend có thể được triển khai lại mà không cần thay đổi frontend hoặc cơ sở dữ liệu.

4.9.1 Thiết kế môi trường triển khai

Môi trường triển khai của hệ thống gồm các thành phần chính:

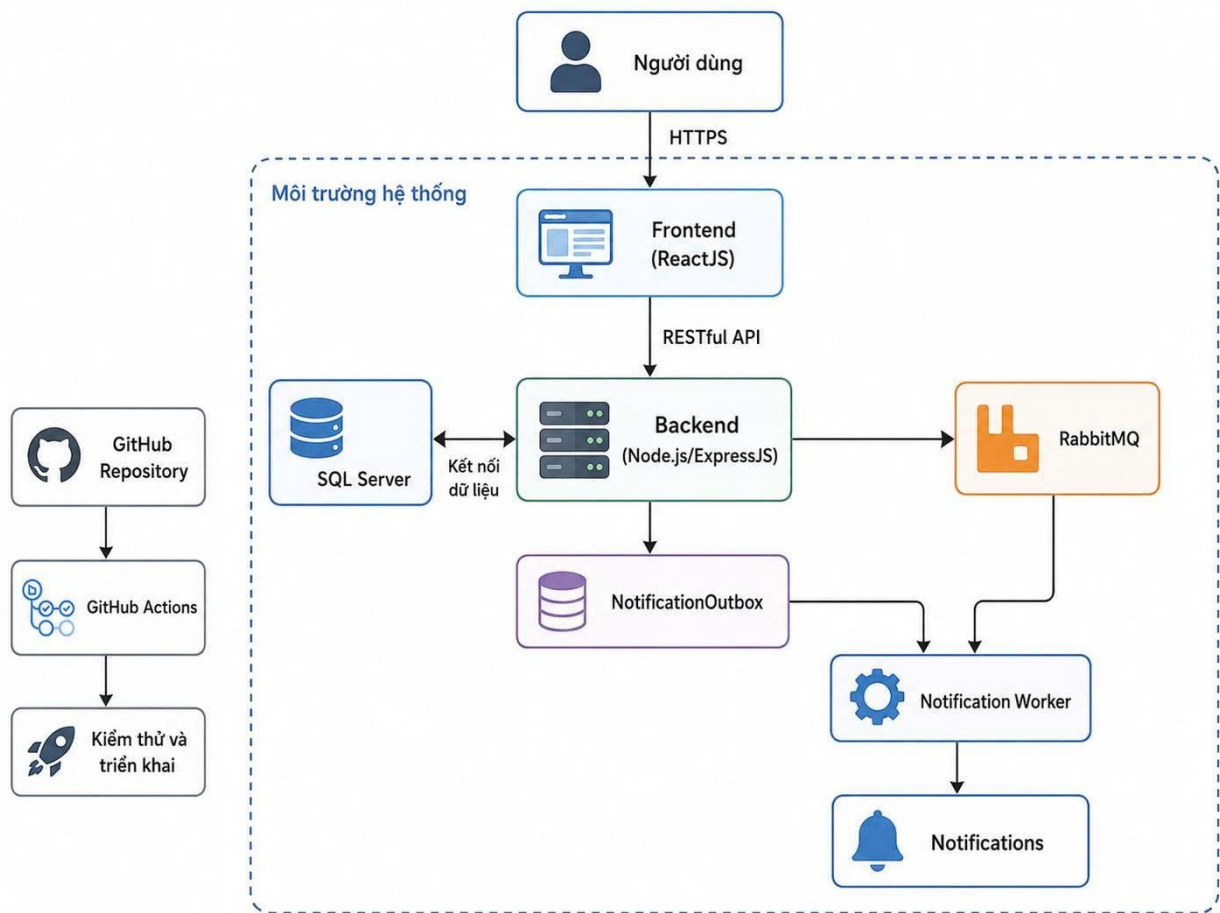
- Frontend ReactJS.
- Backend Node.js và ExpressJS.
- Notification Worker.
- SQL Server.
- RabbitMQ.
- GitHub Repository.
- GitHub Actions.
- Dịch vụ cloud triển khai ứng dụng.

Frontend gửi các yêu cầu HTTPS đến backend thông qua địa chỉ API được cấu hình trong biến môi trường. Backend kết nối với SQL Server để truy xuất dữ liệu và kết nối với RabbitMQ để gửi sự kiện thông báo.

Notification Worker hoạt động độc lập với backend. Worker nhận thông điệp từ RabbitMQ, xử lý nội dung và lưu thông báo vào cơ sở dữ liệu.

GitHub được sử dụng để lưu trữ mã nguồn. Khi mã nguồn được cập nhật lên nhánh chính, GitHub Actions thực hiện kiểm thử, kiểm tra frontend và build các thành phần trước khi kích hoạt triển khai.

Các thông tin nhạy cảm như mật khẩu cơ sở dữ liệu, khóa JWT, RabbitMQ URL và Deploy Hook được lưu trong biến môi trường của từng dịch vụ. Những thông tin này không được ghi trực tiếp trong mã nguồn hoặc đưa lên GitHub.



Hình 4.31. Mô hình triển khai các thành phần của hệ thống

4.9.2 Thiết kế triển khai bằng Docker và Docker Compose

Docker được sử dụng để đóng gói các thành phần của hệ thống cùng với môi trường và thư viện cần thiết. Mỗi thành phần có thể được triển khai trong một container riêng, giúp giảm sự khác biệt giữa máy phát triển và môi trường triển khai.

Backend Node.js được xây dựng thông qua Dockerfile. Trong quá trình tạo image, hệ thống cài đặt các thư viện cần thiết, sao chép mã nguồn và khai báo lệnh khởi động backend. Các thông tin nhạy cảm không được ghi trực tiếp trong Dockerfile mà được truyền vào thông qua biến môi trường.

Frontend ReactJS được build thành các tệp tĩnh trước khi triển khai. Tùy môi trường, frontend có thể được phục vụ bởi dịch vụ lưu trữ tĩnh hoặc web server trong container.

Notification Worker được đóng gói thành một dịch vụ riêng. Worker sử dụng cùng mã nguồn và cấu hình kết nối cơ sở dữ liệu với backend nhưng có lệnh khởi động khác để thực hiện nhiệm vụ tiếp nhận và xử lý thông điệp.

Docker Compose được sử dụng trong môi trường local để quản lý nhiều dịch vụ trong cùng một cấu hình. Các dịch vụ có thể bao gồm:

- Frontend ReactJS.
- Backend Node.js và ExpressJS.
- Notification Worker.
- SQL Server.
- RabbitMQ.

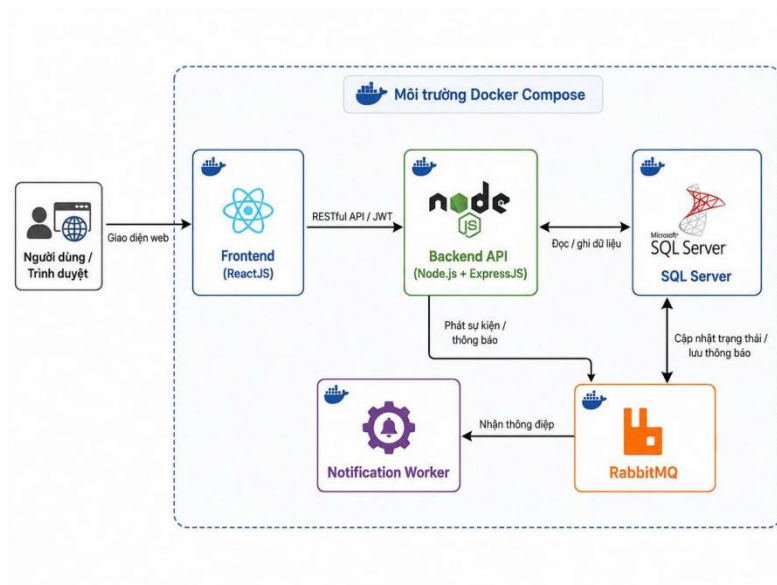
Docker Compose giúp nhóm khởi động hoặc dừng toàn bộ môi trường bằng một lệnh. Các container được kết nối qua mạng nội bộ và truy cập nhau thông qua tên dịch vụ thay vì địa chỉ IP cố định.

Dữ liệu của SQL Server và RabbitMQ cần được lưu trong volume để không bị mất khi container được khởi tạo lại. Những dịch vụ phụ thuộc phải có cơ chế kiểm tra trạng thái trước khi backend hoặc worker bắt đầu kết nối.

Các biến môi trường sử dụng trong Docker được tách khỏi mã nguồn. File chứa thông tin nhạy cảm phải được đưa vào .gitignore và không được lưu lên GitHub.

Bảng 4.24. Các thành phần triển khai bằng Docker

Thành phần	Vai trò	Yêu cầu cấu hình chính
Frontend	Hiển thị giao diện và gửi yêu cầu đến backend.	Địa chỉ RESTful API và cấu hình build.
Backend	Xử lý xác thực, nghiệp vụ và truy xuất dữ liệu.	SQL Server, JWT, RabbitMQ và cổng dịch vụ.
Notification Worker	Tiếp nhận sự kiện và tạo thông báo cho người dùng.	SQL Server, RabbitMQ và tên hàng đợi.
SQL Server	Lưu trữ dữ liệu nghiệp vụ của hệ thống.	Tài khoản, mật khẩu, database và volume.
RabbitMQ	Tiếp nhận và phân phối thông điệp bất đồng bộ.	Exchange, queue, routing key và tài khoản kết nối.



Hình 4.32. Mô hình các dịch vụ trong Docker Compose

4.9.3 Thiết kế quy trình CI/CD bằng GitHub Actions

GitHub Actions được sử dụng để tự động kiểm tra mã nguồn và hỗ trợ triển khai hệ thống. Quy trình tự động được kích hoạt khi mã nguồn được cập nhật lên nhánh chính hoặc khi nhóm tạo yêu cầu hợp nhất mã nguồn.

Workflow được chia thành các công việc riêng cho backend và frontend. Mỗi công việc được thực hiện trong một môi trường độc lập và chỉ được xem là thành công khi toàn bộ bước kiểm tra hoàn tất.

Đối với backend, quy trình có thể gồm các bước:

- Lấy mã nguồn từ GitHub Repository.
 - Cài đặt phiên bản Node.js phù hợp.
 - Cài đặt các thư viện của backend.
 - Kiểm tra cấu hình và mã nguồn.
 - Chạy toàn bộ bộ kiểm thử.
 - Xác nhận backend có thể build hoặc khởi động đúng cấu trúc.
- Đối với frontend, quy trình gồm:
- Lấy mã nguồn.
 - Cài đặt Node.js.
 - Cài đặt thư viện frontend.
 - Chạy kiểm tra lint.
 - Build ứng dụng ReactJS.
 - Xác nhận thư mục kết quả được tạo thành công.

Chỉ khi các bước kiểm thử và build đều thành công, workflow mới chuyển sang giai đoạn triển khai. Việc này giúp hạn chế đưa phiên bản có lỗi lên môi trường hoạt động.

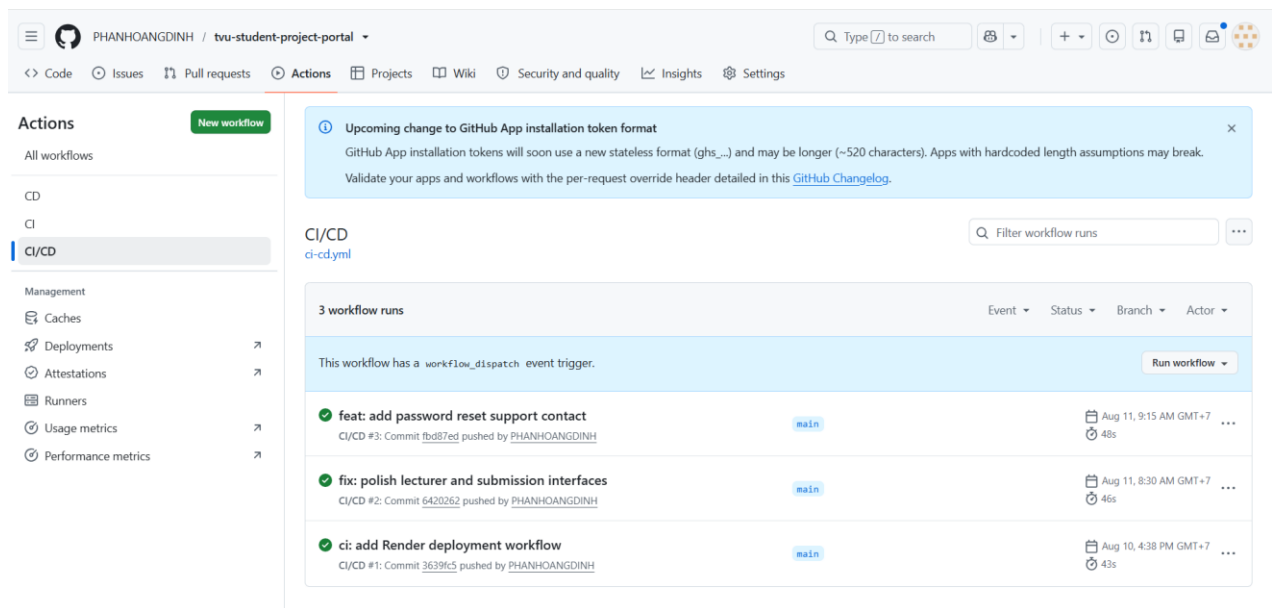
Frontend, backend và Notification Worker được triển khai thành các dịch vụ riêng. GitHub Actions có thể kích hoạt quá trình triển khai thông qua Deploy Hook hoặc cơ chế tích hợp của nền tảng cloud.

Những thông tin như Deploy Hook, khóa truy cập và biến môi trường production được lưu trong GitHub Secrets. Các giá trị này không xuất hiện trong file workflow hoặc log công khai.

Khi một bước thất bại, quy trình triển khai phải dừng lại và hiển thị công việc gây lỗi. Nhóm cần sửa mã nguồn, chạy kiểm tra lại và chỉ triển khai khi workflow chuyển sang trạng thái thành công.

Bảng 4.25. Các bước chính trong quy trình CI/CD

Giai đoạn	Nội dung thực hiện	Điều kiện hoàn thành
Checkout	Lấy phiên bản mã nguồn mới nhất từ repository.	Mã nguồn được tải thành công.
Cài đặt môi trường	Thiết lập Node.js và cài đặt thư viện.	Không phát sinh lỗi phụ thuộc.
Kiểm thử backend	Chạy các bài kiểm thử tự động của backend.	Toàn bộ bài kiểm thử đạt yêu cầu.
Kiểm tra frontend	Chạy lint và kiểm tra lỗi mã nguồn frontend.	Không có lỗi làm workflow thất bại.
Build frontend	Tạo phiên bản frontend dùng để triển khai.	Quá trình build hoàn thành thành công.
Triển khai	Kích hoạt triển khai các dịch vụ lên cloud.	Chỉ chạy khi các bước trước thành công.
Xác minh	Kiểm tra trạng thái dịch vụ sau triển khai.	Dịch vụ hoạt động và có thể truy cập.



Hình 4.33. Quy trình CI/CD của hệ thống

4.10 Tổng kết chương

Chương 4 đã trình bày quá trình thiết kế website quản lý đồ án sinh viên từ kiến trúc tổng thể đến các thành phần chi tiết. Hệ thống được xây dựng theo mô hình frontend và backend tách biệt, trong đó backend được tổ chức thành các tầng Routes, Controllers, Services và Repositories.

Các tác nhân và ca sử dụng của quản trị viên, giảng viên và sinh viên đã được xác định. Những quy trình nghiệp vụ quan trọng như đăng nhập, đăng ký đề tài, xét duyệt đề tài, tạo yêu cầu nộp bài, nộp bài, chấm điểm và xử lý thông báo đã được mô tả thông qua sơ đồ hoạt động và sơ đồ tuần tự.

Cơ sở dữ liệu SQL Server được thiết kế theo mô hình quan hệ, gồm các nhóm bảng quản lý người dùng, môn học, lớp học phần, nhóm sinh viên, đề tài, bài nộp, điểm số và thông báo. Các khóa chính, khóa ngoại, ràng buộc và chỉ mục được sử dụng để bảo đảm tính toàn vẹn và nâng cao hiệu quả truy vấn.

RESTful API được thiết kế theo cấu trúc thống nhất, kết hợp xác thực JWT và phân quyền theo vai trò cũng như phạm vi dữ liệu. Giao diện người dùng được tổ chức riêng cho quản trị viên, giảng viên và sinh viên, đồng thời xử lý các trạng thái tải, lỗi, không có dữ liệu và hiển thị responsive.

Hệ thống được thiết kế để triển khai bằng Docker, Docker Compose và quy trình CI/CD thông qua GitHub Actions. Kiến trúc triển khai tách riêng frontend, backend, Notification Worker, SQL Server và RabbitMQ, giúp thuận tiện cho việc vận hành, bảo trì và mở rộng.

Các nội dung thiết kế trong chương này là cơ sở để tiến hành xây dựng, kiểm thử, triển khai và đánh giá kết quả của hệ thống trong chương tiếp theo.

CHƯƠNG 5: TRIỂN KHAI VÀ CÔNG NGHỆ SỬ DỤNG

5.1 Các công nghệ sử dụng

Hệ thống được phát triển trên môi trường Windows. Visual Studio Code được sử dụng để quản lý mã nguồn, chạy terminal và hỗ trợ kiểm tra lỗi trong quá trình lập trình.

Backend được xây dựng bằng Node.js và ExpressJS. Frontend được phát triển bằng ReactJS kết hợp với Vite. Hai thành phần trao đổi dữ liệu thông qua RESTful API.

Microsoft SQL Server được sử dụng làm hệ quản trị cơ sở dữ liệu. SQL Server Management Studio hỗ trợ tạo cơ sở dữ liệu, thực hiện truy vấn và kiểm tra dữ liệu trong quá trình phát triển.

RabbitMQ được sử dụng để truyền các sự kiện thông báo giữa backend và Notification Worker. Docker và Docker Compose hỗ trợ đóng gói, khởi chạy các dịch vụ trong môi trường thống nhất.

Git được sử dụng để quản lý phiên bản mã nguồn, trong khi GitHub hỗ trợ lưu trữ repository và thực hiện quy trình kiểm thử, build, triển khai thông qua GitHub Actions.

Bảng 5.1. Môi trường và công cụ phát triển hệ thống

Công cụ hoặc công nghệ	Mục đích sử dụng
Visual Studio Code	Viết mã nguồn, chạy terminal và quản lý cấu trúc dự án.
Node.js	Cung cấp môi trường chạy JavaScript phía backend.
ExpressJS	Xây dựng RESTful API và xử lý request, response.
ReactJS	Xây dựng giao diện người dùng theo mô hình component.
Vite	Khởi tạo, chạy và build ứng dụng frontend.
Microsoft SQL Server	Lưu trữ dữ liệu nghiệp vụ của hệ thống.
SQL Server Management Studio	Quản lý cơ sở dữ liệu và thực hiện câu truy

Công cụ hoặc công nghệ	Mục đích sử dụng
	vấn SQL.
RabbitMQ	Truyền và phân phối các sự kiện thông báo bất đồng bộ.
Docker	Đóng gói các thành phần của hệ thống thành container.
Docker Compose	Quản lý và khởi chạy nhiều dịch vụ trong môi trường local.
Git	Theo dõi và quản lý lịch sử thay đổi mã nguồn.
GitHub	Lưu trữ repository và phối hợp phát triển.
GitHub Actions	Tự động kiểm thử, build và hỗ trợ triển khai hệ thống.
Swagger	Mô tả và kiểm tra các RESTful API.
Postman	Gửi yêu cầu và kiểm tra kết quả trả về của API.

Name	Date modified	Type	Size
.agents	7/12/2026 3:54 AM	File folder	
.github	7/29/2026 9:25 AM	File folder	
backend	8/8/2026 2:10 PM	File folder	
backups	8/14/2026 2:26 PM	File folder	
database	7/29/2026 7:44 AM	File folder	
deploy	8/8/2026 2:38 PM	File folder	
docker	7/29/2026 7:44 AM	File folder	
docs	8/13/2026 8:06 PM	File folder	
frontend	8/8/2026 2:10 PM	File folder	
.codex-backend-reg36.err.log	7/26/2026 9:45 PM	Text Document	0 KB
.codex-backend-reg36.out.log	7/26/2026 9:45 PM	Text Document	1 KB
.codex-frontend-reg36.err.log	8/8/2026 1:36 PM	Text Document	0 KB
.codex-frontend-reg36.out.log	8/11/2026 8:59 AM	Text Document	23 KB
.codex-sql-backend.err.log	8/23/2026 1:08 AM	Text Document	35 KB
.codex-sql-backend.out.log	8/23/2026 6:24 PM	Text Document	123 KB
.env	7/29/2026 7:13 AM	ENV File	1 KB
.env.docker	8/8/2026 12:43 AM	DOCKER File	1 KB
.env.docker.example	8/1/2026 9:53 AM	EXAMPLE File	1 KB
.env.production	8/9/2026 3:06 AM	PRODUCTION File	2 KB
.env.production.example	8/9/2026 3:06 AM	EXAMPLE File	2 KB
.gitattributes	8/1/2026 9:53 AM	Git Attributes Sour...	1 KB
.gitignore	8/8/2026 11:37 PM	Git Ignore Source ...	1 KB
.local-seed-accounts.txt	8/10/2026 6:43 AM	Text Document	1 KB
.presentation-seed-secret	8/7/2026 3:43 PM	PRESENTATION-SE...	1 KB
docker-compose.monitoring.yml	8/1/2026 11:41 AM	Yaml Source File	3 KB
docker-compose.production.yml	8/9/2026 3:06 AM	Yaml Source File	10 KB
docker-compose.yml	8/8/2026 2:10 PM	Yaml Source File	7 KB
package-lock.json	6/29/2026 8:54 PM	JSON Source File	1 KB
README.md	8/1/2026 11:41 AM	Markdown Source ...	1 KB

Hình 5.1. Cấu trúc thư mục của dự án

5.2 Xây dựng backend

Backend chịu trách nhiệm xác thực người dùng, kiểm tra quyền truy cập, xử lý các quy tắc nghiệp vụ, truy xuất cơ sở dữ liệu và cung cấp dữ liệu cho frontend thông qua RESTful API.

Mã nguồn backend được tổ chức theo kiến trúc phân tầng gồm Routes, Controllers, Services và Repositories. Mỗi tầng đảm nhận một nhiệm vụ riêng để giúp mã nguồn dễ đọc, kiểm thử và bảo trì.

5.2.1 Xây dựng tầng Routes và Middleware

Tầng Routes khai báo đường dẫn API và phương thức HTTP tương ứng. Mỗi nhóm chức năng có route riêng như xác thực, người dùng, môn học, lớp học phân, nhóm sinh viên, đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Middleware xác thực được sử dụng để kiểm tra JWT trước khi yêu cầu được chuyển đến controller. Khi token không tồn tại, không hợp lệ hoặc đã hết hạn, backend từ chối yêu cầu và trả về mã trạng thái phù hợp.

Middleware phân quyền kiểm tra vai trò của tài khoản. Những chức năng quản trị chỉ cho phép tài khoản có vai trò quản trị viên truy cập. Các chức năng giảng viên và sinh viên cũng được giới hạn theo vai trò tương ứng.

Ngoài kiểm tra vai trò, backend còn kiểm tra phạm vi dữ liệu. Giảng viên chỉ được thao tác với các lớp học phân được phân công, trong khi sinh viên chỉ được truy cập dữ liệu của lớp và nhóm mà mình tham gia.

5.2.2 Xây dựng tầng Controllers

Controllers tiếp nhận request từ Routes, lấy dữ liệu từ tham số, query hoặc request body và gọi phương thức tương ứng tại tầng Services.

Controllers không trực tiếp xử lý các quy tắc nghiệp vụ phức tạp hoặc thực hiện truy vấn SQL. Sau khi nhận kết quả từ service, controller tạo response và trả dữ liệu về frontend.

Các controller được xây dựng theo cấu trúc thống nhất để xử lý:

- Yêu cầu thành công.
- Dữ liệu đầu vào không hợp lệ.
- Người dùng chưa xác thực.
- Người dùng không có quyền.
- Không tìm thấy dữ liệu.
- Xung đột dữ liệu.
- Lỗi phát sinh tại máy chủ.

Việc xử lý lỗi tập trung giúp backend trả về thông báo và mã trạng thái thống nhất cho toàn bộ hệ thống.

5.2.3 Xây dựng tầng Services và Repositories

Services là tầng chứa các quy tắc nghiệp vụ chính của hệ thống. Trước khi thêm hoặc cập nhật dữ liệu, service kiểm tra quyền của người dùng, trạng thái của đối tượng và các điều kiện liên quan.

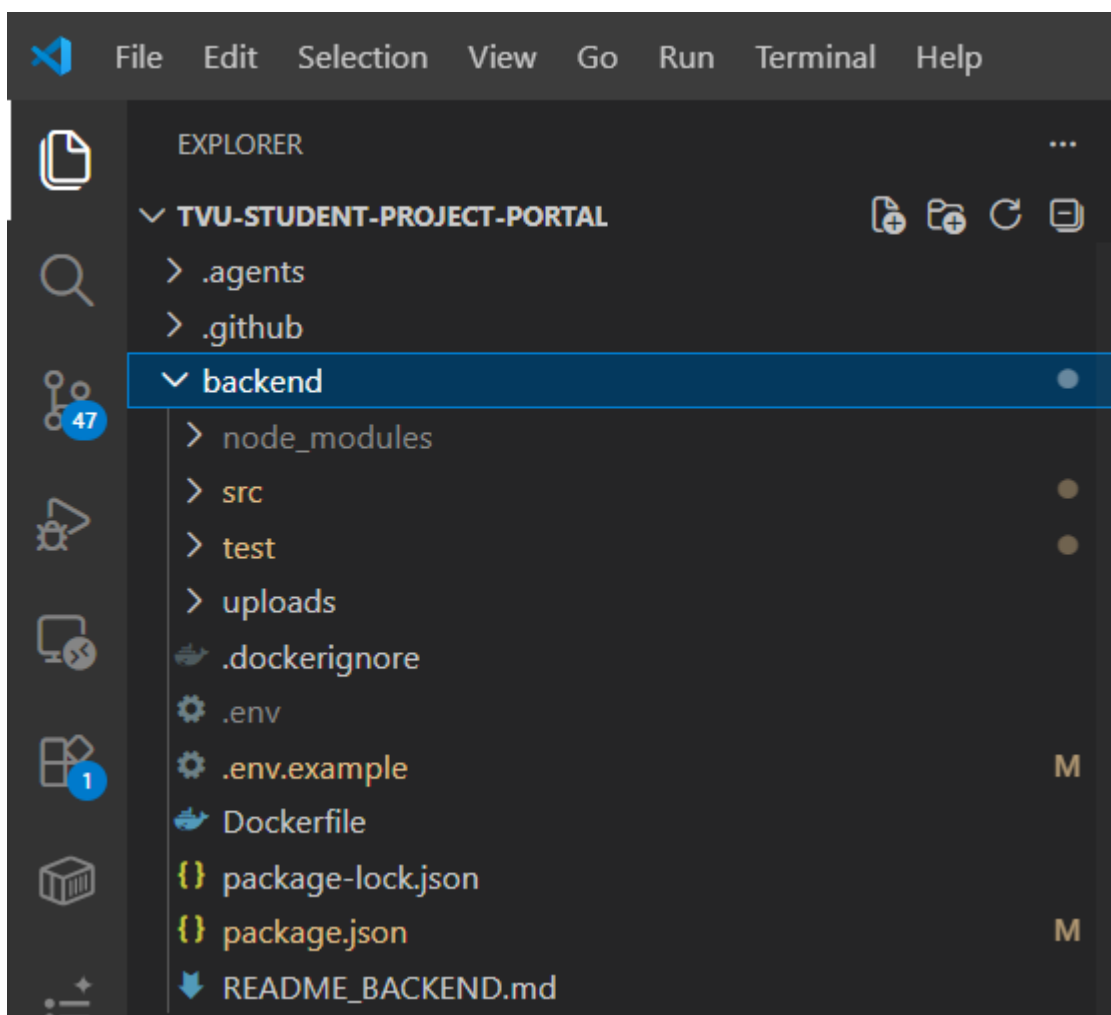
Một số quy tắc nghiệp vụ được xử lý tại tầng service gồm:

- Kiểm tra tài khoản còn hoạt động.
- Kiểm tra giảng viên có phụ trách lớp học phần.
- Kiểm tra sinh viên thuộc lớp và nhóm.
- Không cho phép nhóm có nhiều đề tài đang xử lý.
- Khóa đề tài sau khi được duyệt.
- Kiểm tra thời hạn và số lần nộp bài.

- Kiểm tra phạm vi điểm.
- Chỉ hiển thị điểm đã được công bố.

Repositories chịu trách nhiệm thực hiện các truy vấn đến SQL Server. Các thao tác thêm, cập nhật, xóa và lấy dữ liệu được tập trung tại tầng này.

Đối với những nghiệp vụ cần cập nhật nhiều bảng, backend sử dụng transaction. Nếu một thao tác thất bại, toàn bộ thay đổi trong transaction được hoàn tác để tránh dữ liệu không nhất quán.



Hình 5.2. Cấu trúc phân tầng của backend

5.2.4 Xây dựng chức năng xác thực và phân quyền

Hệ thống sử dụng JSON Web Token để xác thực người dùng. Khi đăng nhập, frontend gửi email và mật khẩu đến backend. Backend tìm kiếm tài khoản trong cơ sở dữ liệu, kiểm tra trạng thái hoạt động và so sánh mật khẩu với giá trị đã được băm.

Nếu thông tin hợp lệ, backend tạo JWT chứa mã người dùng và vai trò. Token được gửi về frontend và đính kèm trong các yêu cầu tiếp theo thông qua tiêu đề Authorization.

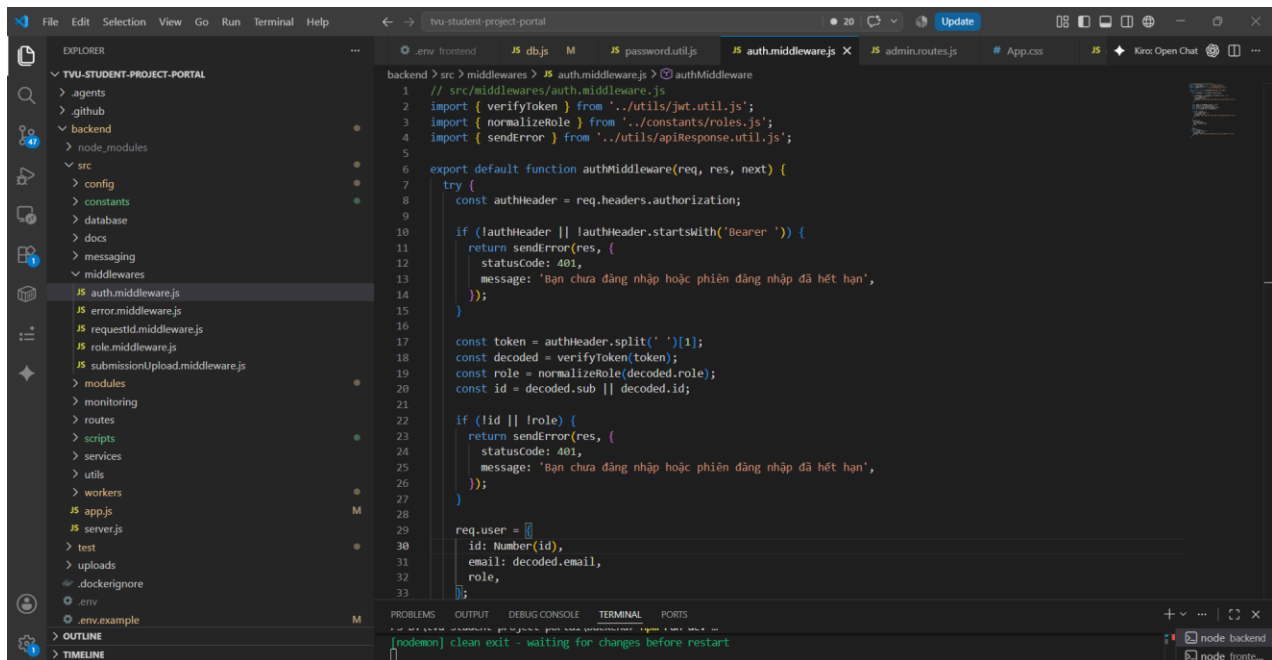
Middleware xác thực tiếp nhận token, kiểm tra chữ ký và thời hạn sử dụng. Khi token không hợp lệ hoặc đã hết hạn, backend từ chối yêu cầu và trả về mã trạng thái phù hợp.

Sau khi xác thực thành công, middleware phân quyền kiểm tra vai trò của người dùng. Hệ thống có ba vai trò chính:

- ADMIN: quản lý tài khoản, môn học, lớp học phân và dữ liệu nền tảng.
- LECTURER: quản lý lớp phụ trách, đề tài, yêu cầu nộp bài, bài nộp và điểm số.
- STUDENT: xem lớp học phân, nhóm, đăng ký đề tài, nộp bài và xem kết quả.

Ngoài kiểm tra vai trò, backend còn xác định phạm vi dữ liệu. Một giảng viên có vai trò hợp lệ nhưng không phụ trách lớp học phân vẫn không được phép truy cập dữ liệu của lớp đó. Tương tự, sinh viên chỉ được xem dữ liệu thuộc lớp và nhóm mà mình tham gia.

Mật khẩu được băm trước khi lưu vào cơ sở dữ liệu. Backend không trả trường mật khẩu hoặc PasswordHash trong các API cung cấp thông tin người dùng.



Hình 5.3. Mã nguồn middleware xác thực và phân quyền

5.2.5 Xây dựng và tài liệu hóa RESTful API

Các RESTful API được xây dựng theo từng nhóm chức năng. Mỗi nhóm có route, controller, service và repository riêng để thuận tiện trong quá trình phát triển và bảo trì.

Các nhóm API chính của hệ thống gồm:

- API xác thực và thông tin tài khoản.
- API quản lý người dùng.
- API quản lý môn học và lớp học phần.
- API quản lý sinh viên trong lớp.
- API quản lý nhóm sinh viên.
- API đăng ký và xét duyệt đề tài.
- API quản lý tiến độ.
- API quản lý yêu cầu nộp bài.

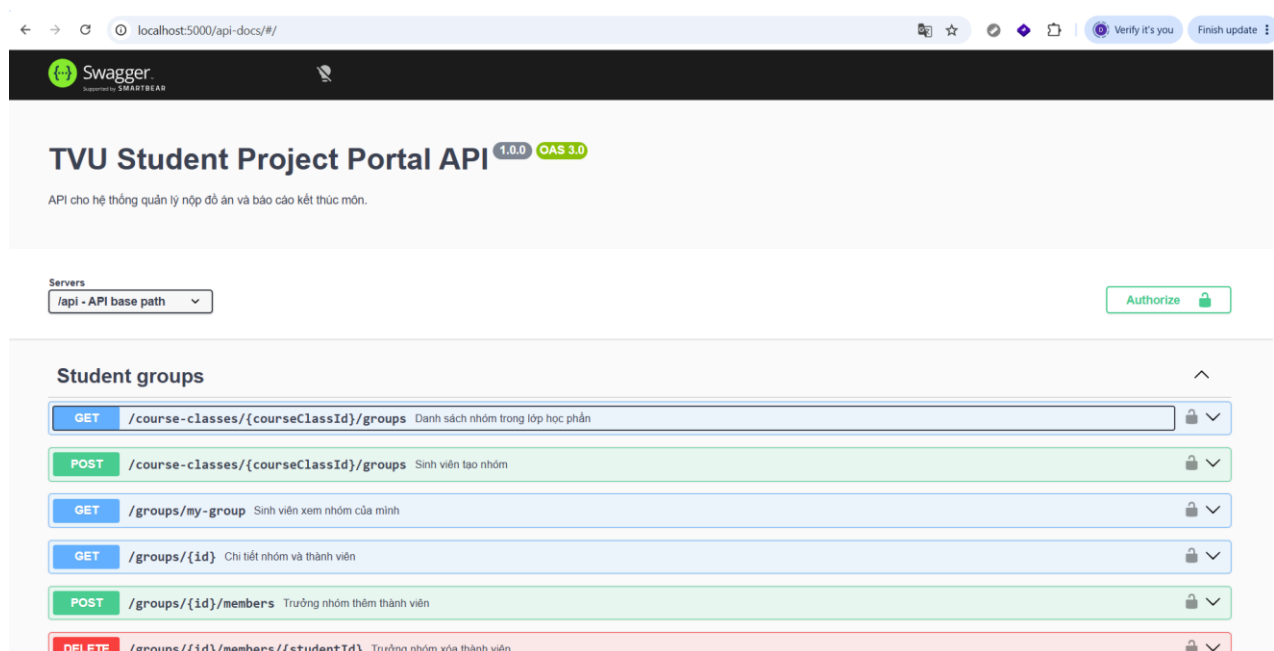
- API quản lý bài nộp và lịch sử nộp.
- API chấm điểm và công bố kết quả.
- API quản lý thông báo.

Dữ liệu đầu vào được kiểm tra trước khi chuyển đến tầng service. Khi xảy ra lỗi, backend trả về cấu trúc phản hồi và mã trạng thái HTTP phù hợp.

Các API được tài liệu hóa bằng Swagger. Giao diện Swagger giúp nhóm xem đường dẫn, phương thức HTTP, tham số, dữ liệu đầu vào, yêu cầu xác thực và cấu trúc phản hồi.

Swagger cũng hỗ trợ nhập JWT để kiểm tra những endpoint được bảo vệ. Nhờ đó, nhóm có thể kiểm tra API của quản trị viên, giảng viên và sinh viên mà không cần thao tác trực tiếp qua giao diện frontend.

Trong môi trường local, Swagger UI được cung cấp tại đường dẫn tài liệu API của backend. Swagger JSON cũng được sử dụng để mô tả toàn bộ endpoint theo tiêu chuẩn OpenAPI.



Hình 5.4. Giao diện tài liệu RESTful API bằng Swagger

5.3 Xây dựng frontend

Frontend được phát triển bằng ReactJS kết hợp với Vite. Giao diện được tổ chức theo mô hình component, giúp tái sử dụng các thành phần và hạn chế lặp lại mã nguồn.

Frontend chịu trách nhiệm hiển thị dữ liệu, tiếp nhận thao tác của người dùng, kiểm tra dữ liệu cơ bản và gửi yêu cầu đến backend. Các quy tắc nghiệp vụ quan trọng vẫn được xử lý và kiểm tra lại tại backend.

Cấu trúc frontend được chia thành các nhóm như trang, component dùng chung, layout, route, API service, hook và tiện ích hỗ trợ. Mỗi vai trò có hệ thống menu và màn hình riêng nhưng vẫn sử dụng chung những thành phần như bảng dữ liệu, biểu mẫu, hộp thoại và thông báo.

5.3.1 Xây dựng layout và điều hướng

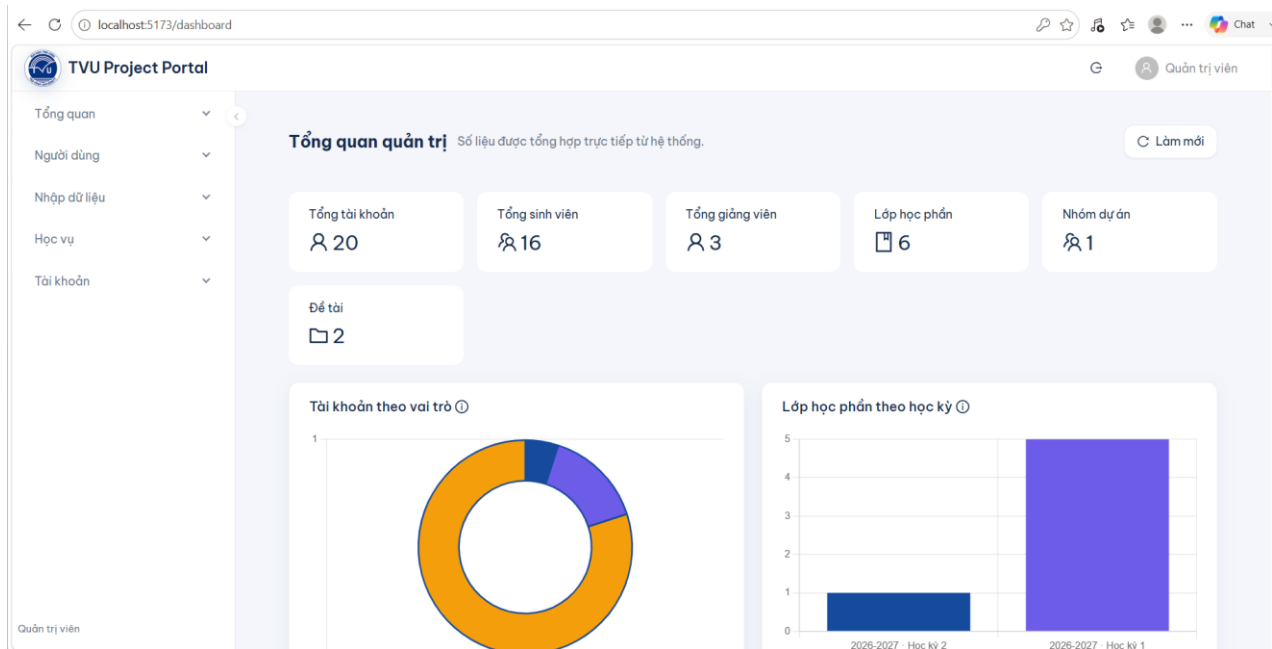
Layout chung của hệ thống gồm thanh điều hướng phía trên, menu chức năng bên trái, khu vực nội dung chính và phần chân trang.

Sau khi người dùng đăng nhập, frontend đọc vai trò từ thông tin phiên đăng nhập và hiển thị menu tương ứng. Quản trị viên, giảng viên và sinh viên không nhìn thấy các mục chức năng ngoài phạm vi của mình.

React Router được sử dụng để quản lý đường dẫn và chuyển trang. Các route được bảo vệ bằng thành phần kiểm tra đăng nhập và vai trò trước khi hiển thị nội dung.

Khi người dùng chưa đăng nhập, hệ thống chuyển về trang đăng nhập. Nếu người dùng đã đăng nhập nhưng truy cập đường dẫn không đúng vai trò, giao diện hiển thị thông báo không có quyền hoặc chuyển đến trang phù hợp.

Thanh điều hướng còn hiển thị tên người dùng, chức năng đăng xuất và khu vực thông báo. Khi đăng xuất, frontend xóa thông tin phiên đăng nhập và chuyển người dùng về trang đăng nhập.



Hình 5.5. Cấu trúc layout và điều hướng của frontend

5.3.2 Xây dựng giao diện dành cho quản trị viên

Giao diện quản trị viên được xây dựng để hỗ trợ quản lý các dữ liệu nền tảng của hệ thống. Sau khi đăng nhập, quản trị viên được chuyển đến trang tổng quan và có thể truy cập các chức năng quản lý tài khoản, môn học, lớp học phần, giảng viên phụ trách và sinh viên trong lớp.

Trang quản lý người dùng hiển thị danh sách tài khoản với các thông tin như họ tên, email, vai trò và trạng thái hoạt động. Giao diện hỗ trợ tìm kiếm theo tên hoặc email, lọc theo vai trò, trạng thái tài khoản và sắp xếp dữ liệu.

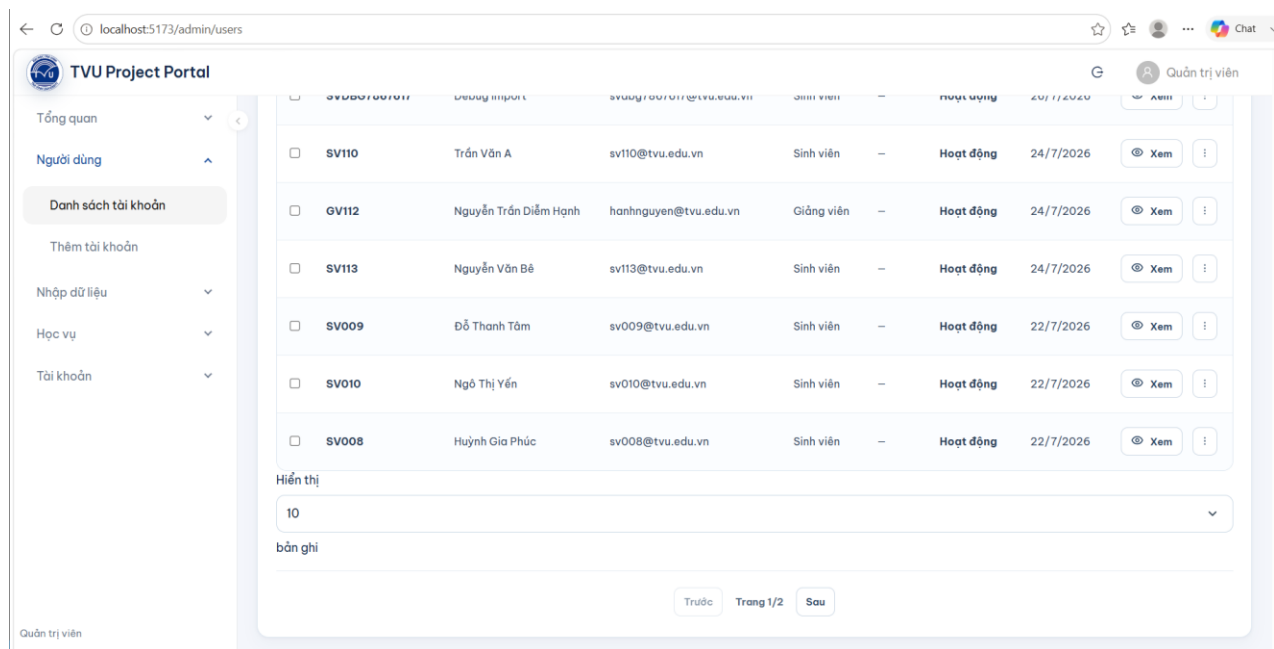
Khi quản trị viên tạo tài khoản mới, frontend kiểm tra các trường bắt buộc trước khi gửi dữ liệu đến backend. Nếu email đã tồn tại hoặc dữ liệu không hợp lệ, hệ thống hiển thị thông báo tại vị trí phù hợp để người dùng điều chỉnh.

Đối với tài khoản đã tồn tại, quản trị viên có thể cập nhật thông tin hoặc thay đổi trạng thái hoạt động. Trước những thao tác có thể ảnh hưởng đến khả năng đăng nhập của người dùng, giao diện hiển thị hộp thoại xác nhận nhằm hạn chế sai sót.

Trang quản lý môn học cho phép quản trị viên theo dõi mã môn học, tên môn học, mô tả và trạng thái sử dụng. Các môn học đã được liên kết với lớp học phần được giữ lại trong hệ thống để bảo đảm tính toàn vẹn dữ liệu.

Trang quản lý lớp học phần hỗ trợ tạo lớp, lựa chọn môn học, học kỳ, năm học và phân công giảng viên phụ trách. Từ trang chi tiết lớp, quản trị viên có thể xem và quản lý danh sách sinh viên tham gia.

Frontend chỉ hiển thị các nút thao tác phù hợp với vai trò quản trị viên. Tuy nhiên, toàn bộ quyền truy cập vẫn được backend kiểm tra lại trước khi thực hiện nghiệp vụ.



Hình 5.6. Giao diện quản lý người dùng của quản trị viên

5.3.3 Xây dựng giao diện dành cho giảng viên

Giao diện giảng viên được xây dựng theo phạm vi lớp học phần được phân công. Sau khi đăng nhập, giảng viên có thể xem danh sách lớp phụ trách và truy cập các chức năng quản lý nhóm, đề tài, tiến độ, yêu cầu nộp bài, bài nộp và điểm số.

Trang danh sách lớp phụ trách hiển thị mã lớp, tên môn học, học kỳ và năm học. Khi chọn một lớp, hệ thống tải các dữ liệu liên quan đến lớp đó và cung cấp các chức năng quản lý tương ứng.

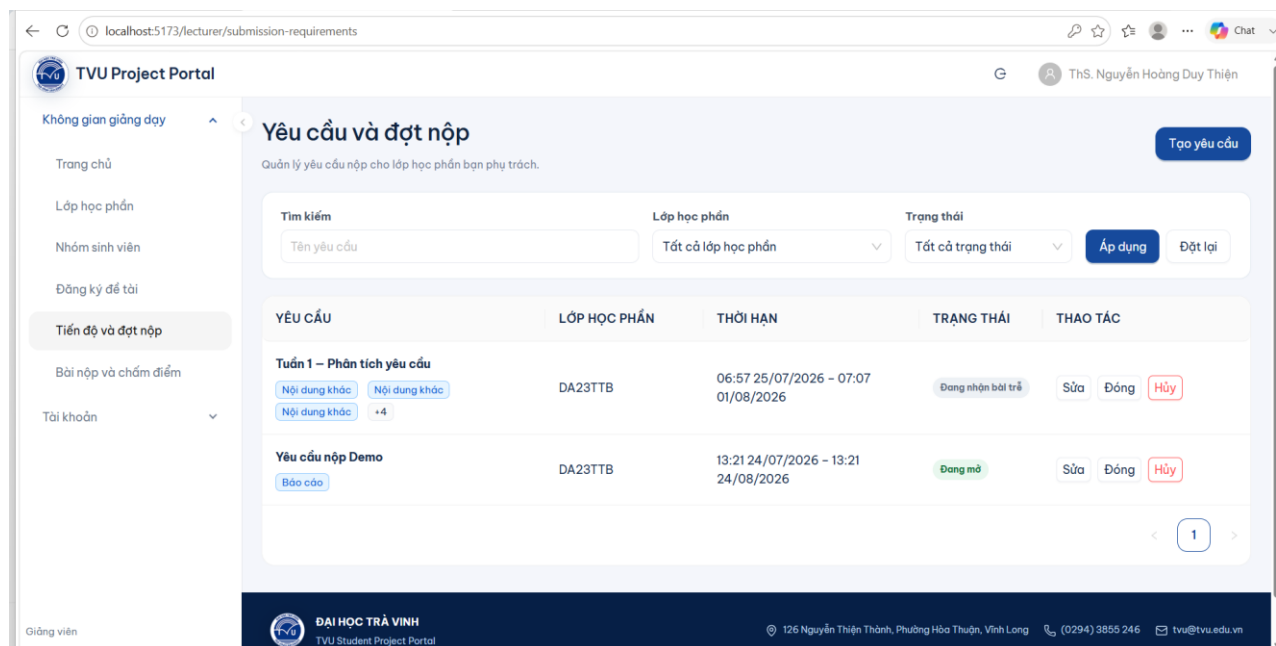
Đối với chức năng xét duyệt đề tài, giao diện hiển thị tên nhóm, tên đề tài, thời điểm đăng ký và trạng thái xử lý. Giảng viên có thể duyệt, từ chối hoặc yêu cầu chỉnh sửa. Khi lựa chọn từ chối hoặc yêu cầu chỉnh sửa, hệ thống yêu cầu nhập phản hồi để sinh viên biết nội dung cần điều chỉnh.

Trang quản lý yêu cầu nộp bài cho phép giảng viên nhập tiêu đề, nội dung, thời gian bắt đầu, thời hạn và số lần nộp tối đa. Frontend kiểm tra thời gian và các trường bắt buộc trước khi gửi yêu cầu đến backend.

Trang bài nộp và chấm điểm hiển thị danh sách nhóm hoặc sinh viên đã nộp bài, thời gian nộp gần nhất, số lần nộp và trạng thái đánh giá. Giảng viên có thể mở chi tiết để xem tệp, lịch sử các lần nộp và thực hiện chấm điểm.

Điểm và phản hồi được lưu ở trạng thái chưa công bố. Khi giảng viên hoàn tất kiểm tra, chức năng công bố điểm được sử dụng để cho phép sinh viên xem kết quả.

Các trang của giảng viên đều kiểm tra trạng thái tài, danh sách rỗng và lỗi từ backend. Khi chưa có bài nộp, giao diện hiển thị thông báo phù hợp thay vì phát sinh lỗi hoặc hiển thị bảng trống.



Hình 5.7. Giao diện quản lý bài nộp và chấm điểm của giảng viên

5.3.4 Xây dựng giao diện dành cho sinh viên

Giao diện sinh viên được thiết kế để hỗ trợ theo dõi và thực hiện các công việc liên quan đến đồ án. Sau khi đăng nhập, sinh viên có thể xem lớp học phân đang tham gia, thông tin nhóm, đề tài, yêu cầu nộp bài, lịch sử nộp, điểm số và thông báo.

Trang lớp học phân chỉ hiển thị những lớp mà sinh viên đã được thêm vào. Khi chọn một lớp, sinh viên có thể xem môn học, giảng viên phụ trách, nhóm hiện tại và danh sách thành viên.

Trang đề tài cho phép sinh viên đại diện nhóm nhập tên đề tài và nội dung mô tả. Sau khi gửi, giao diện hiển thị trạng thái chờ duyệt. Nếu giảng viên yêu cầu chỉnh sửa, sinh viên có thể cập nhật và gửi lại nội dung.

Khi đề tài được duyệt, giao diện khóa chức năng chỉnh sửa. Phản hồi của giảng viên được hiển thị cùng trạng thái để sinh viên dễ theo dõi kết quả xét duyệt.

Trang yêu cầu nộp bài hiển thị tiêu đề, nội dung, thời hạn, số lần nộp tối đa và số lần sinh viên đã thực hiện. Trước khi gửi tệp, frontend kiểm tra dữ liệu đầu vào và hiển thị cảnh báo nếu yêu cầu đã hết hạn hoặc không còn lượt nộp.

Sau khi nộp thành công, danh sách lịch sử được cập nhật với số lần nộp, tên tệp, thời gian và người thực hiện. Dữ liệu lịch sử chỉ được xem, sinh viên không thể tự ý sửa hoặc xóa.

Trang kết quả chỉ hiển thị điểm và phản hồi khi giảng viên đã công bố. Nếu điểm chưa được công bố, giao diện hiển thị trạng thái đang chờ đánh giá.

Trang thông báo lấy dữ liệu thật từ backend và hiển thị các thông tin liên quan đến đề tài, yêu cầu nộp bài, bài nộp và điểm số. Sinh viên có thể lọc thông báo theo trạng thái và đánh dấu đã đọc.

The screenshot shows a web application titled 'TVU Project Portal'. On the left is a sidebar with navigation links: 'Học tập' (Study), 'Trang chủ' (Home), 'Học phần của tôi' (My Courses), 'Nhóm của tôi' (My Group), 'Đề tài' (Topic), 'Yêu cầu nộp bài' (Submit Assignment), 'Tiến độ' (Progress), 'Bài cuối kỳ và điểm' (Final Exam and Grade), 'Thông báo' (Notification), and 'Tài khoản' (Account). The main content area is titled 'Đăng ký đề tài' (Topic Registration) and includes a sub-header 'Trưởng nhóm gửi đề tài để giảng viên phụ trách duyệt.' (Group leader sends topic for lecturer review). Below this is a 'Trạng thái đề tài' (Topic Status) section with a 'Chờ duyệt' (Waiting for review) button. The main form contains several input fields: 'Cổng quản lý dự án sinh viên', 'Cổng thông tin quản lý nộp đề tài và báo cáo tiến độ', 'Node.js, React, SQL Server', and a 'Kiểm tra thủ công công lương nhóm và đăng ký đề tài.' (Check group salary manually and register topic) checkbox. A 'Ghi chú' (Note) field is at the bottom. A blue 'Lưu đăng ký' (Save Registration) button is at the bottom right.

Hình 5.8. Giao diện nộp bài và theo dõi lịch sử nộp của sinh viên

5.3.5 Xử lý trạng thái và lỗi trên giao diện

Trong quá trình trao đổi dữ liệu với backend, frontend cần xử lý đầy đủ các trạng thái để người dùng biết hệ thống đang thực hiện thao tác nào. Mỗi màn hình được thiết kế với trạng thái tải dữ liệu, hiển thị thành công, không có dữ liệu và xử lý lỗi.

Khi frontend gửi yêu cầu đến backend, giao diện hiển thị trạng thái đang tải và tạm thời vô hiệu hóa các nút có thể gửi yêu cầu lặp lại. Sau khi nhận được kết quả, trạng thái tải được kết thúc và nội dung tương ứng được hiển thị.

Đối với các trang danh sách, khi API trả về mảng rỗng, giao diện hiển thị thông báo chưa có dữ liệu phù hợp. Trường hợp này không được xem là lỗi hệ thống.

Khi thao tác thêm, cập nhật, nộp bài hoặc công bố điểm thành công, giao diện hiển thị thông báo xác nhận. Dữ liệu trên màn hình được tải lại hoặc cập nhật trực tiếp để phản ánh kết quả mới nhất.

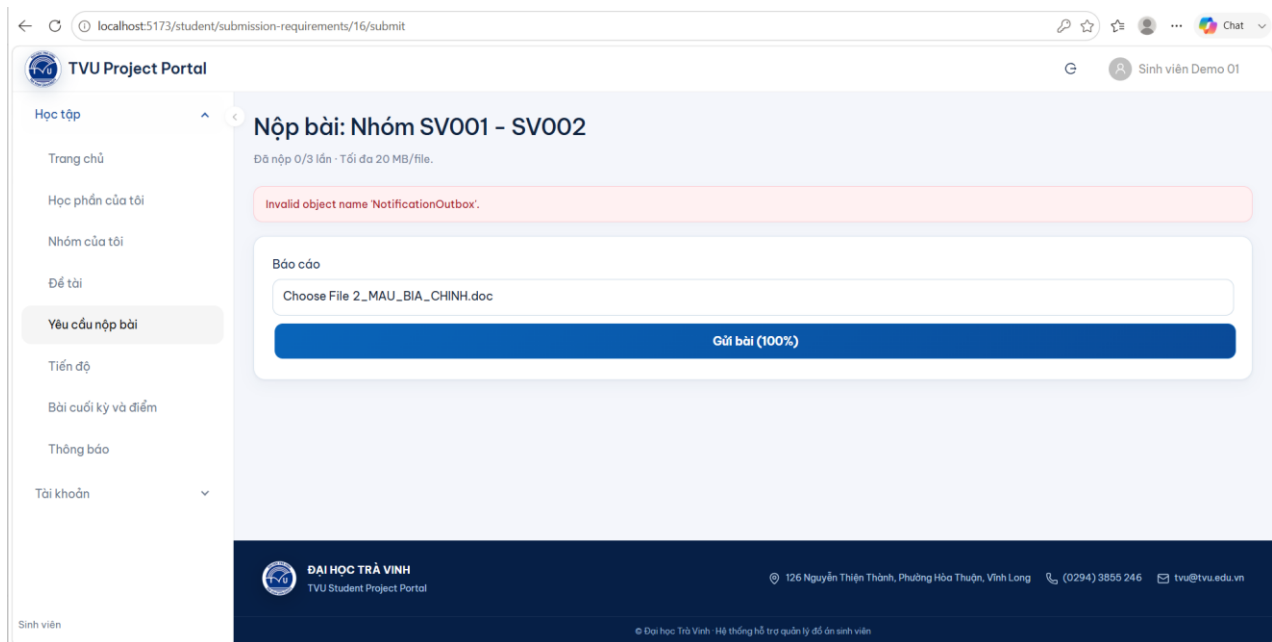
Frontend xử lý lỗi dựa trên mã trạng thái HTTP do backend trả về. Các trường hợp xác thực thất bại, không có quyền, không tìm thấy dữ liệu hoặc lỗi máy chủ được hiển thị bằng nội dung phù hợp với người dùng.

Bảng 5.2. Cách xử lý các trạng thái trên frontend

Trạng thái	Cách xử lý trên giao diện
Đang tải	Hiển thị biểu tượng hoặc thông báo đang xử lý và hạn chế gửi yêu cầu lặp lại.
Thành công	Hiển thị thông báo xác nhận và cập nhật dữ liệu trên màn hình.
Không có dữ liệu	Hiển thị nội dung chưa có dữ liệu phù hợp thay vì bảng trống.
Dữ liệu không hợp lệ	Hiển thị lỗi tại trường nhập hoặc thông báo hướng dẫn điều chỉnh.
Chưa xác thực	Xóa phiên đăng nhập không hợp lệ và chuyển về trang đăng nhập.
Không có quyền	Hiển thị thông báo người dùng không được phép truy cập chức năng.
Lỗi kết nối	Thông báo không thể kết nối đến backend và cung cấp nút thử lại.
Lỗi máy chủ	Hiển thị thông báo lỗi chung, không công khai thông tin kỹ thuật.

Hệ thống sử dụng Error Boundary để tiếp nhận những lỗi runtime không được xử lý trong cây component. Trong môi trường phát triển, lỗi được ghi vào Console để hỗ trợ kiểm tra. Trên giao diện dành cho người dùng, hệ thống chỉ hiển thị thông báo ngắn gọn và nút tải lại trang.

Các thông tin như stack trace, câu truy vấn SQL, JWT, mật khẩu hoặc chuỗi kết nối không được hiển thị trực tiếp trên giao diện.



Hình 5.9. Giao diện xử lý trạng thái dữ liệu và thông báo lỗi

5.4 Xây dựng cơ sở dữ liệu

Cơ sở dữ liệu được xây dựng bằng Microsoft SQL Server với tên TvuStudentProjectPortal. Các bảng được tạo dựa trên mô hình quan hệ đã thiết kế, đồng thời bổ sung khóa chính, khóa ngoại, ràng buộc duy nhất và chỉ mục.

Cơ sở dữ liệu lưu toàn bộ dữ liệu nghiệp vụ của hệ thống, bao gồm tài khoản, môn học, lớp học phần, sinh viên trong lớp, nhóm thực hiện, đề tài, yêu cầu nộp bài, bài nộp, điểm số và thông báo.

Việc tạo và cập nhật dữ liệu được thực hiện thông qua backend. Người dùng không thao tác trực tiếp với cơ sở dữ liệu. Tầng repository chịu trách nhiệm gửi truy vấn đến SQL Server, còn tầng service kiểm tra quyền và điều kiện nghiệp vụ trước khi truy vấn được thực hiện.

5.4.1 Khởi tạo cấu trúc và dữ liệu

Các bảng được tạo theo thứ tự phù hợp với quan hệ khóa ngoại. Những bảng dữ liệu nền tảng như Users và Courses được tạo trước, sau đó mới đến các bảng lớp học phần, nhóm sinh viên, đề tài, bài nộp và thông báo.

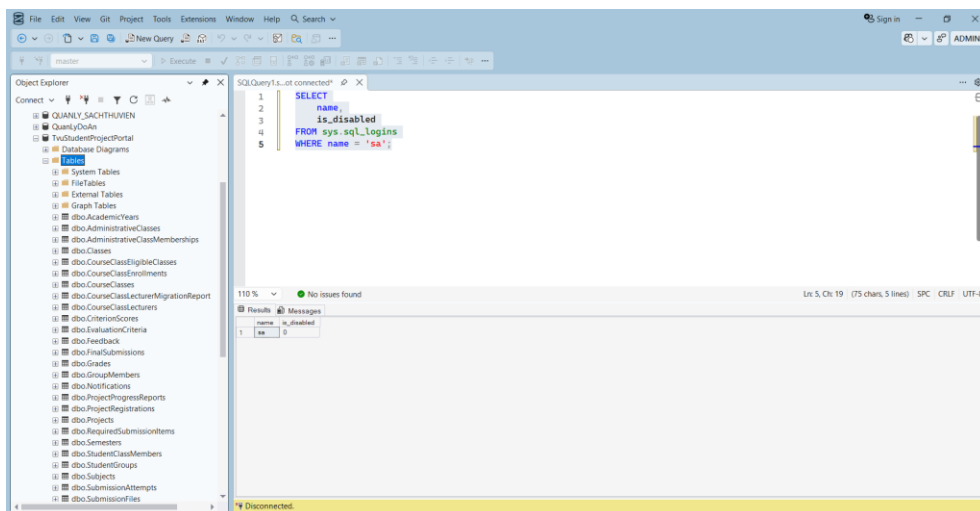
Sau khi tạo cấu trúc, nhóm sử dụng dữ liệu mẫu để phục vụ quá trình phát triển và kiểm thử. Dữ liệu mẫu bao gồm tài khoản thuộc ba vai trò, môn học, lớp học phân, danh sách sinh viên, nhóm, đề tài, yêu cầu nộp bài và điểm số.

Các script khởi tạo dữ liệu cần bảo đảm:

- Không tạo bản ghi trùng khi chạy lại.
- Không lưu mật khẩu dưới dạng văn bản rõ.
- Không tạo dữ liệu vi phạm khóa ngoại.
- Dữ liệu được phân bố đủ cho ba vai trò.
- Các bản ghi có trạng thái đa dạng để kiểm tra bộ lọc.
- Không sử dụng thông tin nhạy cảm của người dùng thật.

Đối với tài khoản demo, mật khẩu được băm bằng cùng cơ chế mà backend sử dụng khi tạo tài khoản. Script chỉ lưu giá trị đã băm vào cột mật khẩu.

Cơ sở dữ liệu local được kiểm tra bằng SQL Server Management Studio. Nhóm thực hiện các truy vấn đếm bản ghi, kiểm tra quan hệ và xác nhận không tồn tại dữ liệu mở còi trước khi chạy hệ thống.



Hình 5.10. Danh sách các bảng trong cơ sở dữ liệu SQL Server

5.4.2 Kết nối backend với SQL Server

Thông tin kết nối cơ sở dữ liệu được quản lý bằng biến môi trường trong file cấu hình local. Backend sử dụng các thông tin gồm địa chỉ máy chủ, cổng, tên database, tài khoản và mật khẩu để tạo connection pool.

Trong môi trường phát triển hiện tại, backend kết nối SQL Server local qua cổng tĩnh 1433. Tên database được sử dụng đúng theo database thực tế là TvuStudentProjectPortal.

Khi backend khởi động, hệ thống kiểm tra các biến môi trường bắt buộc và thử kết nối đến SQL Server. Sau khi connection pool được tạo, backend thực hiện một truy vấn kiểm tra đơn giản để xác nhận kết nối hoạt động.

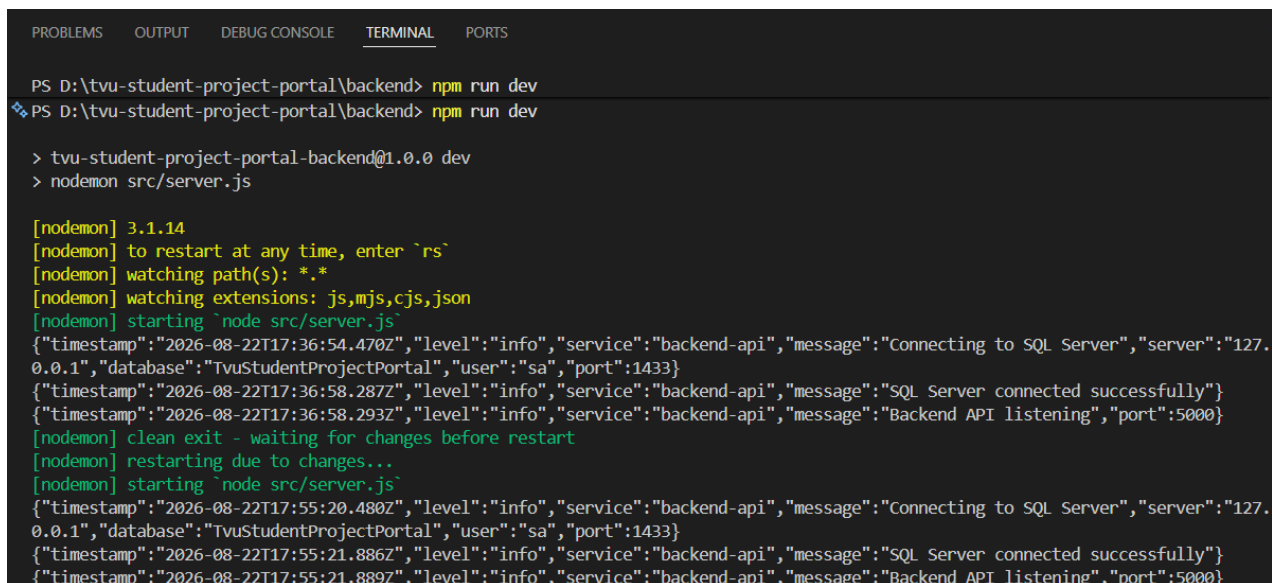
Nếu kết nối thành công, terminal hiển thị thông báo:

SQL Server connected successfully

Nếu đăng nhập không hợp lệ, không tìm thấy server hoặc sai tên database, backend ghi thông báo lỗi phù hợp và dừng quá trình khởi động. Mật khẩu cơ sở dữ liệu không được ghi ra terminal.

Connection pool được tái sử dụng cho nhiều request nhằm hạn chế việc tạo kết nối mới liên tục. Khi ứng dụng dừng, kết nối được đóng đúng cách để giải phóng tài nguyên.

Các biến môi trường thật được lưu trong backend/.env. File này được đưa vào .gitignore và không được commit lên GitHub. File .env.example chỉ chứa tên biến và giá trị minh họa, không chứa mật khẩu thật.



```
PS D:\tvu-student-project-portal\backend> npm run dev
PS D:\tvu-student-project-portal\backend> npm run dev

> tvu-student-project-portal-backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
{"timestamp":"2026-08-22T17:36:54.470Z","level":"info","service":"backend-api","message":"Connecting to SQL Server","server":"127.0.0.1","database":"TvuStudentProjectPortal","user":"sa","port":1433}
{"timestamp":"2026-08-22T17:36:58.287Z","level":"info","service":"backend-api","message":"SQL Server connected successfully"}
{"timestamp":"2026-08-22T17:36:58.293Z","level":"info","service":"backend-api","message":"Backend API listening","port":5000}
[nodemon] clean exit - waiting for changes before restart
[nodemon] restarting due to changes...
[nodemon] starting `node src/server.js`
{"timestamp":"2026-08-22T17:55:20.480Z","level":"info","service":"backend-api","message":"Connecting to SQL Server","server":"127.0.0.1","database":"TvuStudentProjectPortal","user":"sa","port":1433}
{"timestamp":"2026-08-22T17:55:21.886Z","level":"info","service":"backend-api","message":"SQL Server connected successfully"}
{"timestamp":"2026-08-22T17:55:21.889Z","level":"info","service":"backend-api","message":"Backend API listening","port":5000}
```

Hình 5.11. Kết quả kết nối backend với SQL Server

5.5 Xây dựng chức năng thông báo bất đồng bộ

Chức năng thông báo được xây dựng nhằm cung cấp cho người dùng các thông tin mới liên quan đến đề tài, yêu cầu nộp bài, bài nộp và kết quả đánh giá. Thay vì tạo thông báo trực tiếp trong luồng xử lý chính, hệ thống sử dụng cơ chế bất đồng bộ gồm NotificationOutbox, RabbitMQ và Notification Worker.

Khi một nghiệp vụ quan trọng hoàn thành, backend ghi nhận sự kiện thông báo vào cơ sở dữ liệu. Sự kiện sau đó được gửi đến RabbitMQ và được Notification Worker tiếp nhận để tạo thông báo cho người dùng.

Cách tổ chức này giúp giảm thời gian phản hồi của API, đồng thời hạn chế trường hợp dữ liệu nghiệp vụ đã được lưu nhưng thông báo bị mất khi RabbitMQ hoặc worker tạm thời gặp sự cố.

5.5.1 Xây dựng cơ chế Transactional Outbox

Transactional Outbox được sử dụng để lưu sự kiện thông báo trước khi gửi đến RabbitMQ. Sự kiện được ghi vào bảng NotificationOutbox trong cùng transaction với dữ liệu nghiệp vụ.

Khi sinh viên đăng ký đề tài, nộp bài hoặc khi giảng viên tạo yêu cầu và công bố điểm, backend đồng thời thực hiện hai thao tác:

- Lưu hoặc cập nhật dữ liệu nghiệp vụ.
- Ghi sự kiện thông báo vào NotificationOutbox.

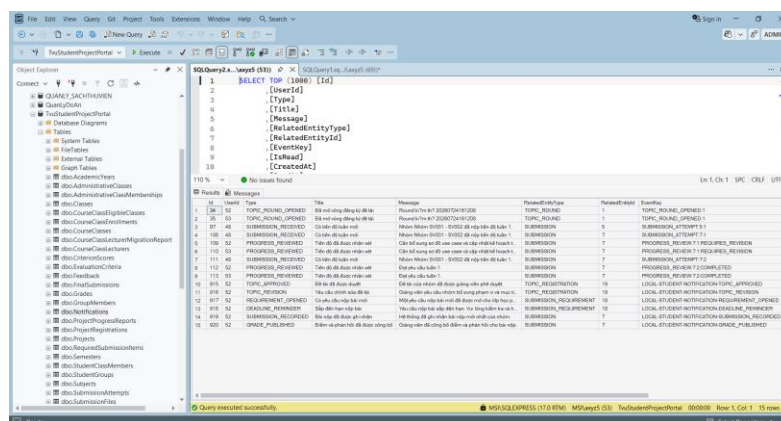
Nếu một thao tác trong transaction thất bại, toàn bộ thay đổi được hoàn tác. Điều này giúp bảo đảm dữ liệu nghiệp vụ và sự kiện thông báo luôn nhất quán.

Mỗi sự kiện Outbox chứa các thông tin như loại sự kiện, dữ liệu JSON, trạng thái xử lý, số lần thử lại và nội dung lỗi. Trạng thái ban đầu của sự kiện là chờ xử lý.

Tiến trình xử lý Outbox định kỳ lấy những sự kiện chưa được gửi, chuyển chúng đến RabbitMQ và cập nhật trạng thái sau khi gửi thành công. Nếu xảy ra lỗi, hệ thống tăng số lần thử và giữ lại nội dung lỗi để phục vụ kiểm tra.

Các trạng thái xử lý chính gồm:

- PENDING: sự kiện đang chờ xử lý.
- PROCESSING: sự kiện đang được gửi.
- COMPLETED: sự kiện đã được xử lý thành công.
- FAILED: sự kiện xử lý thất bại.



Id	UserId	Type	Title	Message	RelatedEntityType	RelatedEntityId	EventType	Title	CreatedAt
1	1	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	1	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
2	2	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	2	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
3	3	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	3	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
4	4	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	4	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
5	5	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	5	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
6	6	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	6	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
7	7	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	7	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
8	8	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	8	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
9	9	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	9	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
10	10	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	10	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
11	11	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	11	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
12	12	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	12	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
13	13	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	13	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
14	14	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	14	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10
15	15	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	Phản hồi của giảng viên	TOPIC_ROUND	15	TOPIC_ROUND_OPENED	Đề tài đang được đăng ký	2023-10-10 10:10:10

Hình 5.12. Dữ liệu sự kiện trong bảng NotificationOutbox

5.5.2 Tích hợp RabbitMQ

RabbitMQ được sử dụng làm hệ thống hàng đợi trung gian giữa backend và Notification Worker. Backend không trực tiếp yêu cầu worker xử lý mà gửi sự kiện vào exchange của RabbitMQ.

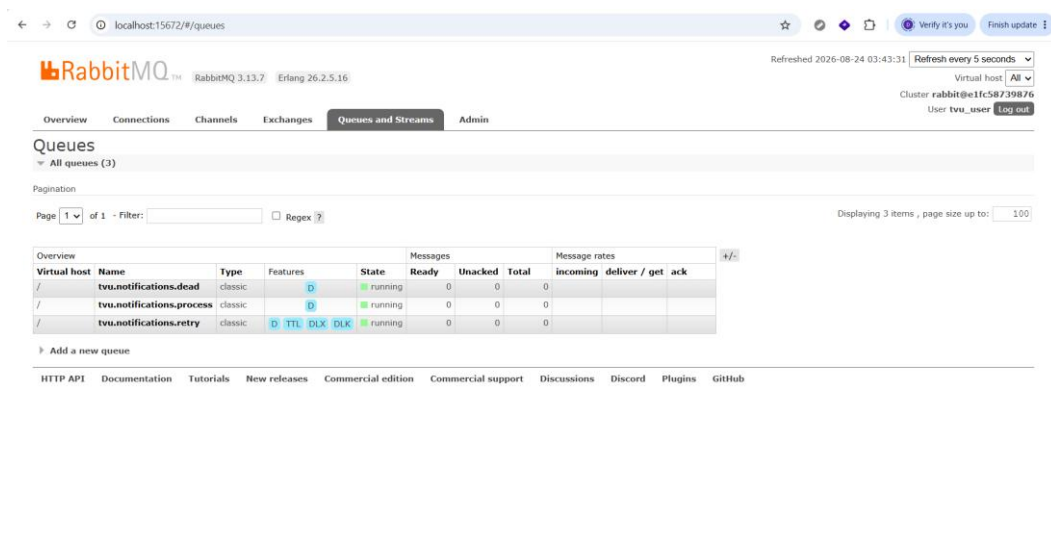
Cấu hình RabbitMQ gồm exchange, routing key, queue chính, queue thử lại và dead-letter queue. Các giá trị cấu hình được lấy từ biến môi trường để có thể thay đổi giữa môi trường local và môi trường cloud.

Khi nhận sự kiện từ Outbox, tiến trình publisher thực hiện các bước:

- Kết nối đến RabbitMQ.
- Khai báo exchange và queue cần thiết.
- Chuyển dữ liệu sự kiện thành thông điệp.
- Gửi thông điệp theo routing key.
- Chờ xác nhận từ RabbitMQ.
- Cập nhật trạng thái Outbox khi gửi thành công.

Nếu quá trình gửi thất bại, sự kiện không bị xóa khỏi cơ sở dữ liệu. Hệ thống có thể thực hiện gửi lại trong lần xử lý tiếp theo.

Queue thử lại được sử dụng cho các thông điệp chưa thể xử lý ngay. Khi vượt quá số lần thử quy định, thông điệp được chuyển đến dead-letter queue để tránh gây vòng lặp xử lý liên tục.



Hình 5.13. Giao diện quản lý queue trên RabbitMQ

5.5.3 Xây dựng Notification Worker

Notification Worker là tiến trình hoạt động độc lập với backend API. Worker kết nối đến RabbitMQ, lắng nghe queue thông báo và xử lý các thông điệp được gửi đến.

Khi nhận một thông điệp, worker thực hiện các bước:

- Đọc loại sự kiện và dữ liệu đi kèm.
- Kiểm tra cấu trúc thông điệp.
- Xác định người dùng cần nhận thông báo.
- Tạo tiêu đề, nội dung và đường dẫn liên quan.
- Lưu thông báo vào bảng Notifications.
- Xác nhận thông điệp đã được xử lý.

Nếu xử lý thành công, worker gửi tín hiệu xác nhận để RabbitMQ loại bỏ thông điệp khỏi queue. Nếu xảy ra lỗi tạm thời, thông điệp được đưa vào cơ chế thử lại.

Worker xử lý các sự kiện chính như:

- Có đăng ký đề tài mới.

- Đề tài đã được duyệt hoặc yêu cầu chỉnh sửa.
- Giảng viên tạo yêu cầu nộp bài mới.
- Sinh viên đã gửi bài nộp.
- Điểm và phản hồi đã được công bố.

Notification Worker sử dụng cùng database với backend nhưng có tiến trình khởi động riêng. Việc tách worker khỏi API giúp quá trình xử lý thông báo không làm chậm các yêu cầu của người dùng.

The screenshot displays a development environment for a project named 'tvu-student-project-portal'. The Explorer sidebar on the left lists various directories and files, including configuration files like '.env', '.env.docker', and '.env.example'. The main editor window shows the content of '.env.example', which contains placeholders for database credentials, JWT secrets, and RabbitMQ connection details. Below the editor, the Terminal panel shows the execution of Docker commands. The 'docker compose ps' command lists the services, and the 'docker compose logs notification-worker --tail 100' command shows the logs for the notification-worker service, indicating successful connections to the database and RabbitMQ.

Hình 5.14. Kết quả khởi động Notification Worker

5.6 Kiểm thử hệ thống

Kiểm thử được thực hiện trong suốt quá trình xây dựng nhằm phát hiện lỗi sớm và bảo đảm các chức năng hoạt động đúng với yêu cầu đã phân tích. Hệ thống được kiểm tra ở cả backend, RESTful API, giao diện frontend, cơ sở dữ liệu và cơ chế phân quyền.

Các trường hợp kiểm thử bao gồm luồng xử lý thành công và các trường hợp thất bại như dữ liệu không hợp lệ, thiếu JWT, token hết hạn, sai vai trò, truy cập dữ liệu ngoài phạm vi hoặc tài nguyên không tồn tại.

Dữ liệu demo được sử dụng để kiểm tra hệ thống với ba vai trò gồm quản trị viên, giảng viên và sinh viên. Mỗi vai trò được kiểm tra trên các chức năng và phạm vi dữ liệu tương ứng.

Bảng 5.3. Các nhóm kiểm thử được thực hiện

Nhóm kiểm thử	Nội dung kiểm tra	Kết quả cần đạt
Kiểm thử đơn vị	Kiểm tra các hàm xử lý nghiệp vụ, xác thực dữ liệu và tiện ích hỗ trợ.	Các hàm trả về kết quả đúng theo từng trường hợp đầu vào.
Kiểm thử tích hợp	Kiểm tra sự phối hợp giữa controller, service, repository và SQL Server.	Dữ liệu được xử lý và lưu trữ chính xác.
Kiểm thử RESTful API	Kiểm tra endpoint, phương thức HTTP, request, response và mã trạng thái.	API trả đúng dữ liệu và mã trạng thái phù hợp.
Kiểm thử xác thực	Kiểm tra đăng nhập, JWT, token thiếu, sai hoặc hết hạn.	Chỉ tài khoản hợp lệ mới truy cập được chức năng bảo vệ.
Kiểm thử phân quyền	Kiểm tra quyền của quản trị viên, giảng viên và sinh viên.	Người dùng chỉ thao tác được trong phạm vi cho phép.
Kiểm thử giao diện	Kiểm tra điều hướng, biểu mẫu, bảng dữ liệu, bộ lọc và thông báo.	Giao diện hoạt động ổn định và dễ sử dụng.
Kiểm thử cơ sở dữ liệu	Kiểm tra khóa ngoại, dữ liệu trùng và tính toàn vẹn.	Không xuất hiện bản ghi mờ côi hoặc dữ liệu không hợp lệ.

Nhóm kiểm thử	Nội dung kiểm tra	Kết quả cần đạt
Kiểm thử thông báo	Kiểm tra Outbox, RabbitMQ, Worker và bảng Notifications.	Sự kiện được xử lý và thông báo được tạo đúng người nhận.
Kiểm thử hồi quy	Chạy lại toàn bộ bài kiểm thử sau các thay đổi quan trọng.	Chức năng cũ không bị ảnh hưởng bởi mã nguồn mới.

5.6.1 Kiểm thử backend và RESTful API

Backend được kiểm thử bằng các bài kiểm thử tự động kết hợp với Swagger và Postman. Các bài kiểm thử tập trung vào những nghiệp vụ quan trọng như đăng nhập, quản lý người dùng, lớp học phần, nhóm sinh viên, đăng ký đề tài, bài nộp, chấm điểm và thông báo.

Đối với chức năng đăng nhập, hệ thống kiểm tra tài khoản hợp lệ, mật khẩu sai, tài khoản bị vô hiệu hóa và dữ liệu đầu vào bị thiếu. Khi thông tin chính xác, API trả về JWT và vai trò của người dùng.

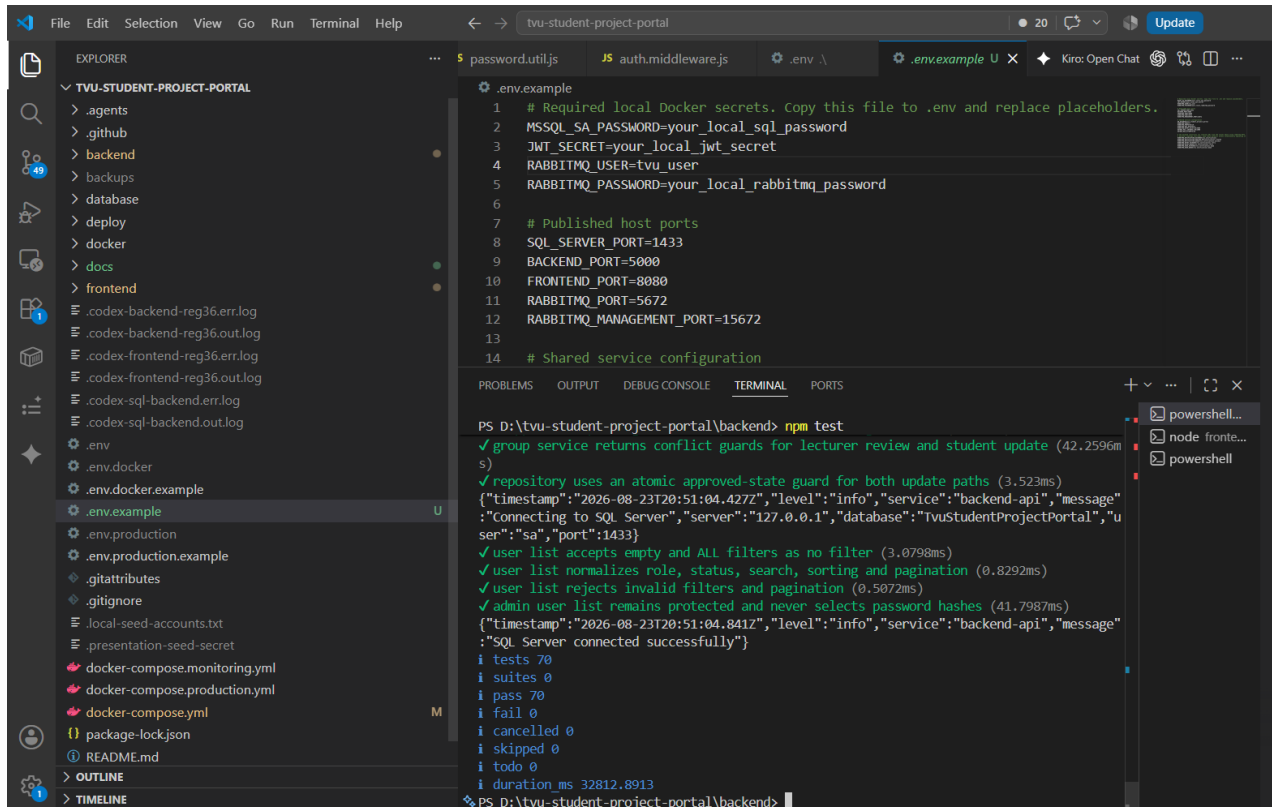
Các API được bảo vệ được kiểm tra trong trường hợp không có token, token không hợp lệ và token hết hạn. Hệ thống phải trả về mã trạng thái xác thực phù hợp và không cho phép request tiếp tục đến tầng nghiệp vụ.

Kiểm thử phân quyền được thực hiện bằng tài khoản thuộc ba vai trò. Quản trị viên có thể truy cập chức năng quản lý dữ liệu nền tảng. Giảng viên chỉ được quản lý các lớp được phân công. Sinh viên chỉ được xem dữ liệu thuộc lớp và nhóm đang tham gia.

Đối với nghiệp vụ nộp bài, các trường hợp được kiểm tra gồm yêu cầu còn hiệu lực, yêu cầu đã hết hạn, vượt quá số lần nộp và sinh viên không thuộc lớp học phần. Chỉ những bài nộp đáp ứng đầy đủ điều kiện mới được lưu vào cơ sở dữ liệu.

Đối với nghiệp vụ chấm điểm, hệ thống kiểm tra quyền của giảng viên, phạm vi điểm và trạng thái công bố. Điểm chưa công bố không được trả về cho sinh viên.

Kết quả chạy bộ kiểm thử backend tại thời điểm hoàn thiện đạt 58/58 trường hợp kiểm thử thành công. Backend kết nối SQL Server ổn định và không phát sinh lỗi trong quá trình thực hiện các bài kiểm thử hiện có.



Hình 5.15. Kết quả chạy bộ kiểm thử backend

5.6.2 Kiểm thử giao diện và phân quyền người dùng

Giao diện được kiểm tra bằng cách đăng nhập lần lượt với tài khoản quản trị viên, giảng viên và sinh viên. Các chức năng, menu và đường dẫn được đối chiếu với vai trò của tài khoản đang sử dụng.

Đối với quản trị viên, quá trình kiểm thử tập trung vào trang tổng quan, danh sách người dùng, tìm kiếm, lọc vai trò, thêm tài khoản và quản lý lớp học phần. Danh sách rỗng phải được hiển thị bằng trạng thái không có dữ liệu, không được xem là lỗi máy chủ.

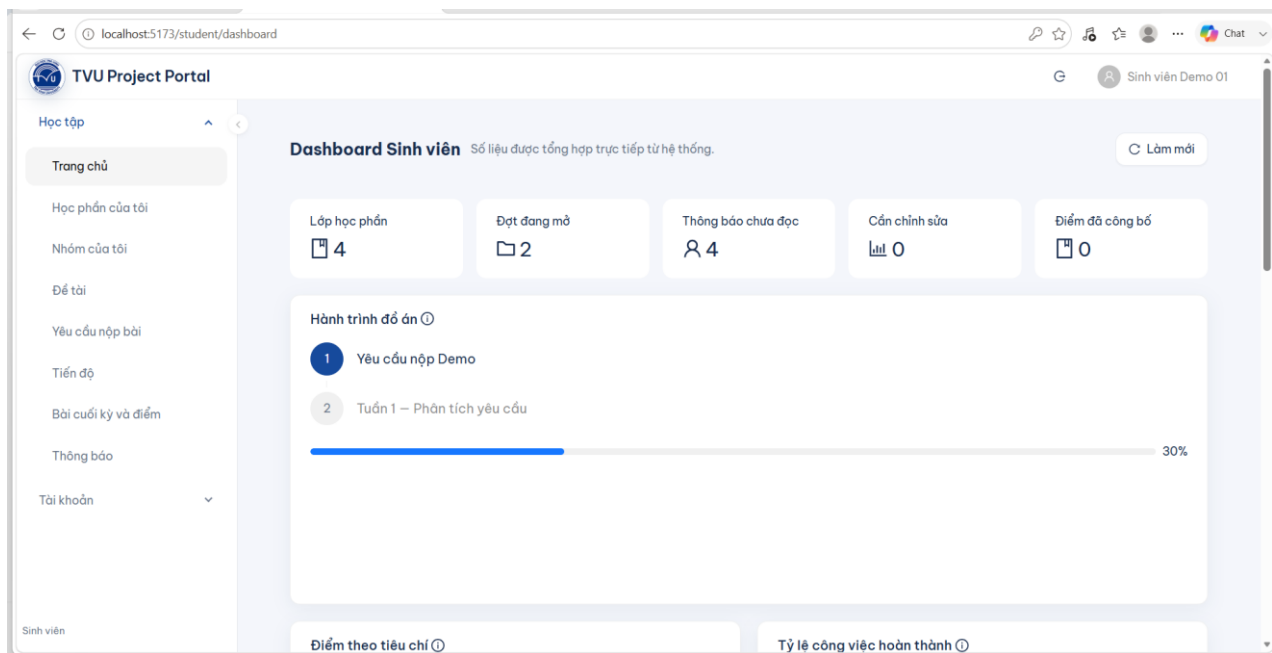
Đối với giảng viên, hệ thống được kiểm tra trên các chức năng lớp phụ trách, nhóm sinh viên, xét duyệt đề tài, tạo yêu cầu nộp bài, xem bài nộp và chấm điểm. Giảng viên không được truy cập dữ liệu của lớp không được phân công.

Đối với sinh viên, quá trình kiểm thử gồm xem lớp học phần, nhóm, đăng ký đề tài, yêu cầu nộp bài, lịch sử nộp, điểm số và thông báo. Sinh viên không được truy cập các route quản trị hoặc giảng viên.

Giao diện cũng được kiểm tra trong các trường hợp API trả dữ liệu rỗng, lỗi xác thực, không có quyền và mất kết nối backend. Thông báo hiển thị cho người dùng phải rõ ràng và không chứa stack trace hoặc nội dung lỗi SQL.

Bảng 5.4. Kết quả kiểm thử theo vai trò

Vai trò	Chức năng được kiểm tra	Kết quả
Quản trị viên	Đăng nhập, quản lý tài khoản, môn học, lớp học phần và sinh viên trong lớp.	Các chức năng được hiển thị và thực hiện đúng quyền hạn.
Giảng viên	Xem lớp phụ trách, quản lý nhóm, xét duyệt đề tài, bài nộp và điểm số.	Chỉ truy cập được dữ liệu thuộc lớp được phân công.
Sinh viên	Xem lớp, nhóm, đăng ký đề tài, nộp bài, xem điểm và thông báo.	Chỉ truy cập được dữ liệu của lớp và nhóm đang tham gia.
Người chưa đăng nhập	Truy cập trực tiếp các route được bảo vệ.	Hệ thống chuyển về trang đăng nhập hoặc trả lỗi xác thực.
Người sai vai trò	Truy cập chức năng không thuộc quyền hạn.	Hệ thống từ chối truy cập và hiển thị thông báo phù hợp.



Hình 5.16. Kết quả kiểm thử chức năng theo vai trò người dùng

5.6.3 Kiểm thử dữ liệu và tính toàn vẹn cơ sở dữ liệu

Bên cạnh kiểm thử chức năng, nhóm tiến hành kiểm tra tính toàn vẹn của dữ liệu trong SQL Server. Mục tiêu của quá trình này là bảo đảm các bản ghi được liên kết đúng, không xuất hiện dữ liệu mờ côi và không vi phạm các ràng buộc đã thiết kế.

Các quan hệ khóa ngoại được kiểm tra giữa người dùng, lớp học phần, nhóm sinh viên, đề tài, yêu cầu nộp bài, bài nộp và điểm số. Khi một dữ liệu tham chiếu không tồn tại, SQL Server hoặc backend phải từ chối thao tác.

Những điều kiện chính được kiểm tra gồm:

- Sinh viên trong lớp phải tham chiếu đến tài khoản và lớp học phần tồn tại.
- Thành viên nhóm phải thuộc lớp học phần chứa nhóm.
- Đăng ký đề tài phải liên kết với nhóm và đề tài hợp lệ.
- Bài nộp phải thuộc một yêu cầu nộp bài đang tồn tại.
- Lịch sử nộp phải liên kết với bài nộp chính.

- Điểm số phải liên kết với một bài nộp hợp lệ.
- Thông báo phải thuộc về người dùng tồn tại trong hệ thống.
- Một sinh viên không được xuất hiện nhiều lần trong cùng một lớp hoặc cùng một nhóm.
- Một nhóm chỉ có một bài nộp chính đối với mỗi yêu cầu.
- Một bài nộp chỉ có một kết quả đánh giá hiện tại.

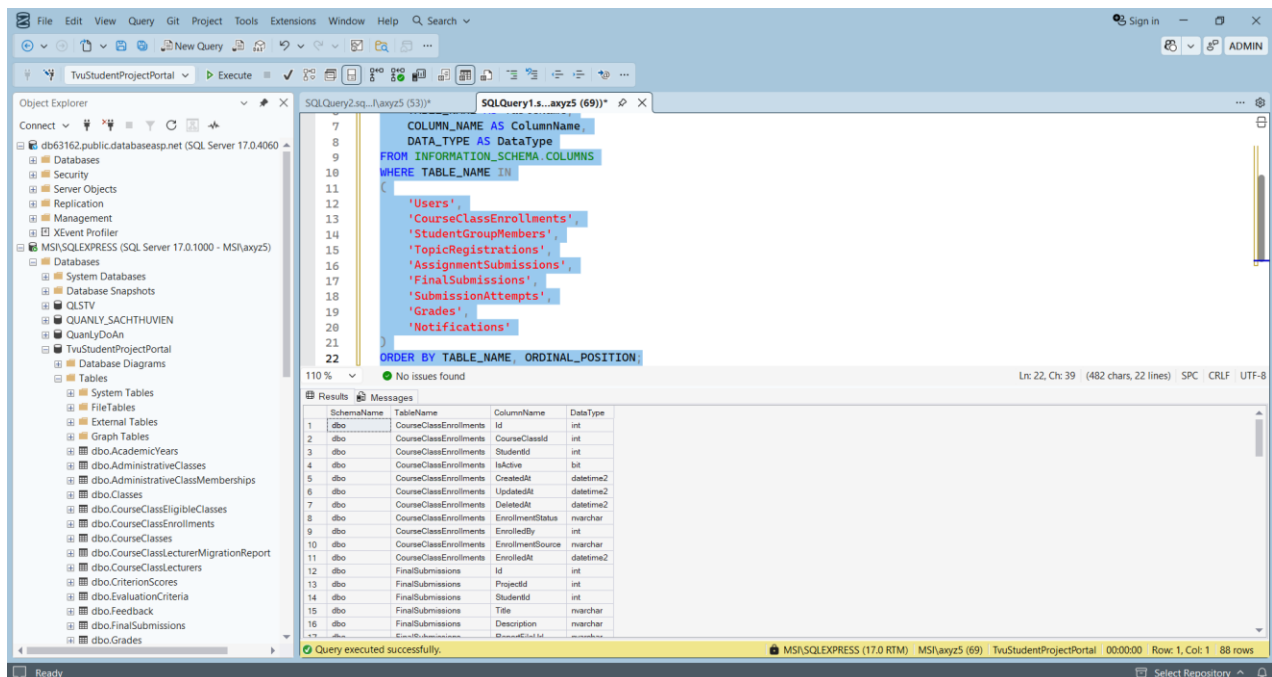
Nhóm sử dụng dữ liệu demo có số lượng lớn để kiểm tra các chức năng tìm kiếm, lọc, phân trang và truy xuất dữ liệu theo phạm vi lớp học phần. Việc kiểm tra với nhiều bản ghi giúp phát hiện các lỗi không xuất hiện khi chỉ sử dụng một số lượng dữ liệu nhỏ.

Các truy vấn kiểm tra được thực hiện trực tiếp trong SQL Server Management Studio để phát hiện bản ghi thiếu liên kết, dữ liệu trùng lặp hoặc giá trị trạng thái không hợp lệ.

Bảng 5.5. Nội dung kiểm tra tính toàn vẹn dữ liệu

Nội dung kiểm tra	Điều kiện cần bảo đảm	Kết quả
Tài khoản người dùng	Email không trùng và vai trò thuộc danh sách cho phép.	Dữ liệu tài khoản đáp ứng ràng buộc.
Sinh viên trong lớp	Mỗi sinh viên chỉ xuất hiện một lần trong một lớp học phần.	Không phát hiện bản ghi trùng.
Thành viên nhóm	Thành viên thuộc đúng lớp học phần của nhóm.	Các quan hệ nhóm hợp lệ.
Đăng ký đề tài	Nhóm và đề tài tham chiếu phải tồn tại.	Không có đăng ký đề tài mồ côi.
Bài nộp	Bài nộp liên kết đúng yêu cầu và nhóm sinh viên.	Các bài nộp có đầy đủ dữ liệu liên quan.
Lịch sử nộp	Số lần nộp không trùng	Lịch sử được lưu theo đúng

Nội dung kiểm tra	Điều kiện cần bảo đảm	Kết quả
	trong cùng một bài nộp.	thứ tự.
Điểm số	Mỗi bài nộp chỉ có một kết quả đánh giá hiện tại.	Không xuất hiện nhiều bản ghi điểm cho một bài nộp.
Thông báo	Người nhận phải tồn tại và trạng thái đọc hợp lệ.	Thông báo được liên kết đúng tài khoản.



Hình 5.17. Kết quả kiểm tra dữ liệu trong SQL Server

5.6.4 Kiểm thử cơ chế thông báo bất đồng bộ

Cơ chế thông báo được kiểm thử từ thời điểm nghiệp vụ phát sinh sự kiện đến khi người dùng nhận được thông báo trên giao diện. Quá trình kiểm thử bao gồm NotificationOutbox, RabbitMQ, Notification Worker và bảng Notifications.

Nhóm thực hiện các nghiệp vụ có tạo thông báo như đăng ký đề tài, xét duyệt đề tài, tạo yêu cầu nộp bài, nộp bài và công bố điểm. Sau mỗi thao tác, hệ thống được kiểm tra theo trình tự:

Dữ liệu nghiệp vụ được lưu thành công.

- Sự kiện được ghi vào NotificationOutbox.

- Publisher gửi sự kiện đến RabbitMQ.
- Queue tiếp nhận thông điệp.
- Notification Worker nhận và xử lý thông điệp.
- Thông báo được lưu vào bảng Notifications.
- Người nhận xem được thông báo trên giao diện.

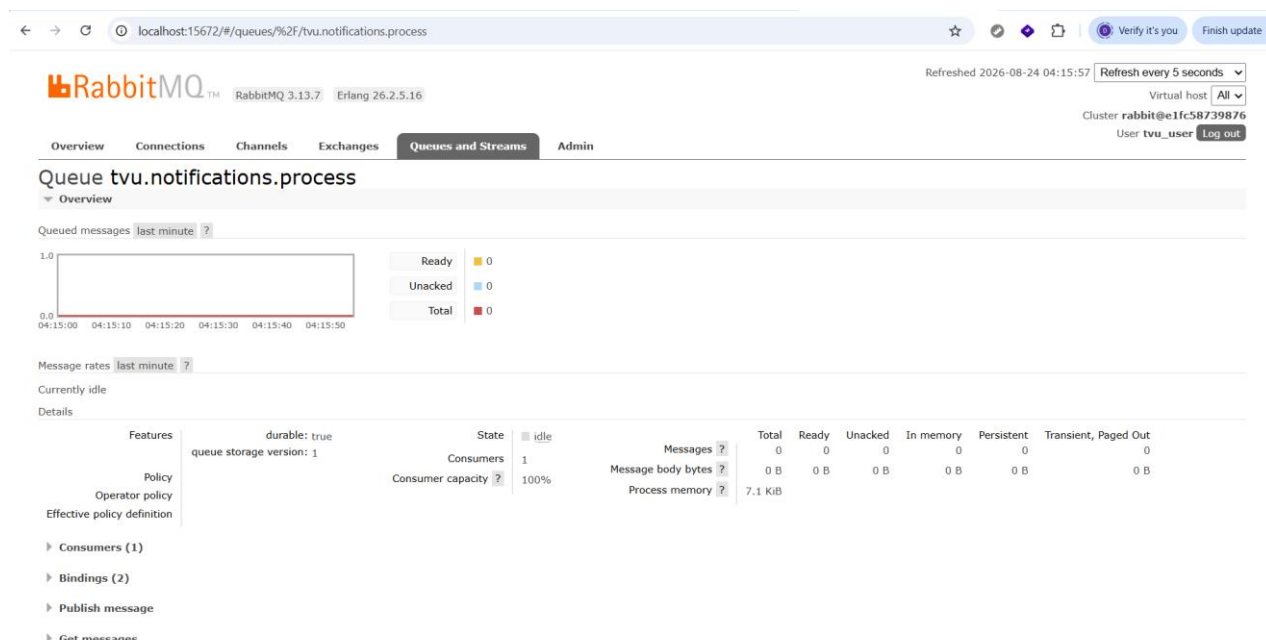
Trong quá trình kiểm thử, RabbitMQ và Notification Worker đều kết nối thành công. Queue xử lý thông báo có một consumer đang hoạt động, trong khi các queue thử lại và dead-letter không có thông điệp tồn đọng.

Các trường hợp lỗi cũng được kiểm tra nhằm bảo đảm sự kiện không bị mất. Khi RabbitMQ hoặc worker tạm thời chưa sẵn sàng, sự kiện vẫn được giữ trong Outbox để xử lý lại sau đó.

Bảng 5.6. Kết quả kiểm thử chức năng thông báo

Bước kiểm thử	Nội dung kiểm tra	Kết quả cần đạt
Ghi sự kiện Outbox	Nghiệp vụ hoàn thành và tạo bản ghi sự kiện.	Sự kiện được lưu với trạng thái chờ xử lý.
Gửi đến RabbitMQ	Publisher lấy và gửi sự kiện đến exchange.	Thông điệp được chuyển vào đúng queue.
Worker nhận thông điệp	Notification Worker lắng nghe queue xử lý.	Queue chính có consumer hoạt động.
Xử lý thông báo	Worker xác định người nhận và nội dung thông báo.	Thông báo được tạo đúng người dùng.
Xác nhận thông điệp	Worker xác nhận sau khi xử lý thành công.	Message không còn tồn đọng trong queue.
Thử lại khi lỗi	Kiểm tra queue retry và dead-letter.	Thông điệp lỗi được quản lý, không tạo vòng lặp liên tục.
Hiển thị frontend	Người dùng mở trang thông	Danh sách hiển thị dữ liệu

Bước kiểm thử	Nội dung kiểm tra	Kết quả cần đạt
	báo.	thật từ SQL Server.
Đánh dấu đã đọc	Người dùng đọc một hoặc toàn bộ thông báo.	Trạng thái và thời gian đọc được cập nhật.



Hình 5.18. Kết quả xử lý thông báo từ RabbitMQ đến người dùng

5.7 Triển khai hệ thống

Sau khi các chức năng chính được xây dựng và kiểm thử, hệ thống được triển khai để có thể hoạt động ngoài môi trường phát triển. Quá trình triển khai gồm đóng gói các dịch vụ, cấu hình biến môi trường, thiết lập cơ sở dữ liệu, RabbitMQ và kiểm tra kết nối giữa các thành phần.

Hệ thống được tổ chức thành các thành phần triển khai độc lập gồm:

- Frontend ReactJS.
- Backend Node.js và ExpressJS.
- Notification Worker.
- SQL Server.

- RabbitMQ.

Việc tách thành các dịch vụ riêng giúp nhóm dễ dàng cập nhật, theo dõi log và xử lý sự cố mà không cần triển khai lại toàn bộ hệ thống.

5.7.1 Triển khai trong môi trường local bằng Docker Compose

Trong môi trường local, Docker Compose được sử dụng để khởi chạy SQL Server, RabbitMQ, backend và Notification Worker. Các dịch vụ được kết nối qua mạng nội bộ của Docker.

SQL Server sử dụng volume để lưu dữ liệu lâu dài. Việc dừng hoặc khởi tạo lại container không làm mất database nếu volume vẫn được giữ nguyên.

RabbitMQ cung cấp cổng AMQP cho backend và worker, đồng thời cung cấp giao diện quản trị tại cổng Management. Notification Worker kết nối đồng thời đến SQL Server và RabbitMQ để xử lý các sự kiện thông báo.

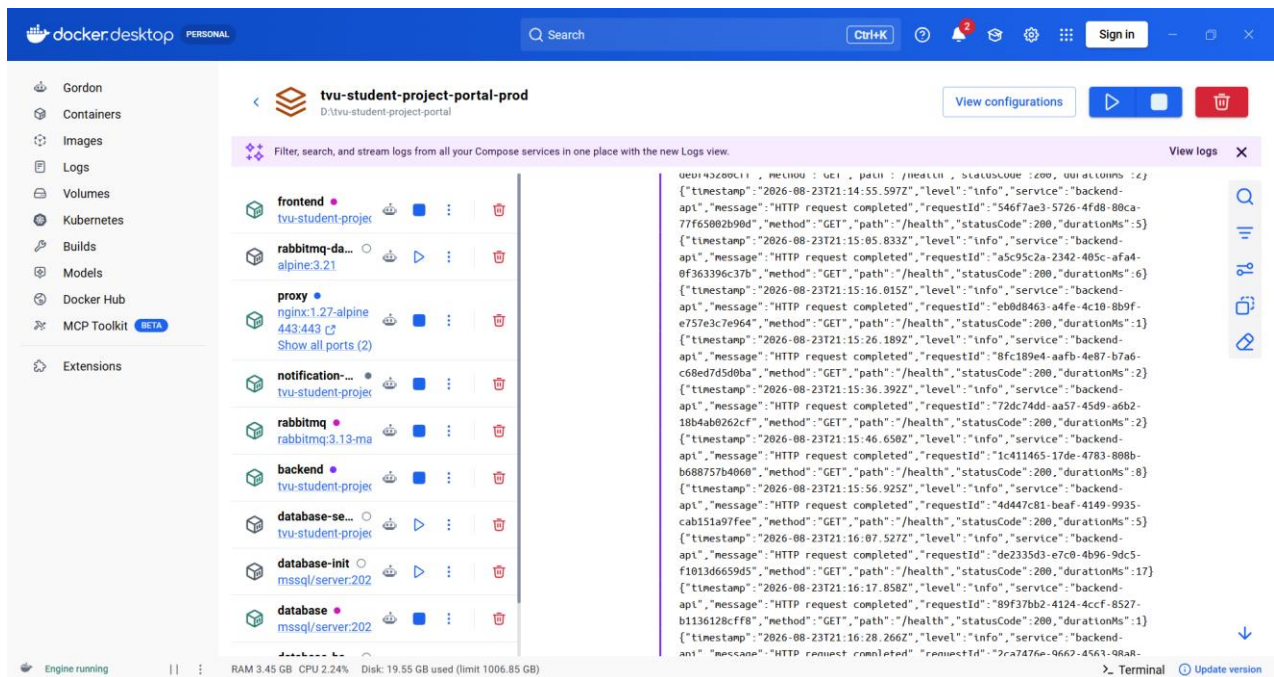
Các thông tin như mật khẩu SQL Server và RabbitMQ được khai báo trong file .env ở thư mục gốc. File này chỉ được sử dụng trong môi trường local và được loại trừ khỏi Git.

Sau khi cấu hình, nhóm sử dụng các lệnh Docker Compose để kiểm tra và khởi động hệ thống. Kết quả xác minh cho thấy SQL Server, RabbitMQ và Notification Worker đều ở trạng thái hoạt động ổn định. Worker kết nối thành công đến hai dịch vụ và không xảy ra tình trạng khởi động lại liên tục.

Bảng 5.7. Trạng thái các dịch vụ trong môi trường Docker Compose

Dịch vụ	Vai trò	Trạng thái cần đạt
SQL Server	Lưu trữ dữ liệu nghiệp vụ.	Container hoạt động và health check thành công.
RabbitMQ	Quản lý hàng đợi thông báo.	Container hoạt động, cổng AMQP và Management khả dụng.
Backend	Cung cấp RESTful API.	Kết nối SQL Server thành

Dịch vụ	Vai trò	Trạng thái cần đạt
		công và lắng nghe đúng cổng.
Notification Worker	Xử lý thông báo bất đồng bộ.	Kết nối SQL Server, RabbitMQ và lắng nghe queue.
Frontend	Cung cấp giao diện cho người dùng.	Kết nối đúng backend và tải được dữ liệu.



Hình 5.19. Trạng thái các dịch vụ trong Docker Compose

5.7.2 Triển khai hệ thống lên môi trường cloud

Sau khi kiểm tra hệ thống trong môi trường local, các thành phần được triển khai lên môi trường cloud để người dùng có thể truy cập thông qua Internet. Frontend, backend và Notification Worker được triển khai thành các dịch vụ riêng biệt nhằm thuận tiện cho việc quản lý, cập nhật và theo dõi trạng thái hoạt động.

Frontend ReactJS được build thành các tệp tĩnh và triển khai trên Render. Trong quá trình build, địa chỉ RESTful API được cung cấp thông qua biến môi trường để frontend gửi yêu cầu đến đúng backend.

Backend Node.js và ExpressJS được triển khai dưới dạng Web Service. Dịch vụ backend thực hiện kết nối đến SQL Server, xác thực JWT, xử lý nghiệp vụ và cung cấp API cho frontend.

Notification Worker được triển khai thành một dịch vụ riêng. Worker không cung cấp giao diện HTTP cho người dùng mà hoạt động nền để nhận thông điệp từ RabbitMQ và tạo thông báo trong cơ sở dữ liệu.

Cơ sở dữ liệu SQL Server được triển khai trên MonsterASP. Chuỗi kết nối được lưu trong biến môi trường của backend và Notification Worker. Những thông tin như tài khoản, mật khẩu, địa chỉ máy chủ và tên database không được ghi trực tiếp trong mã nguồn.

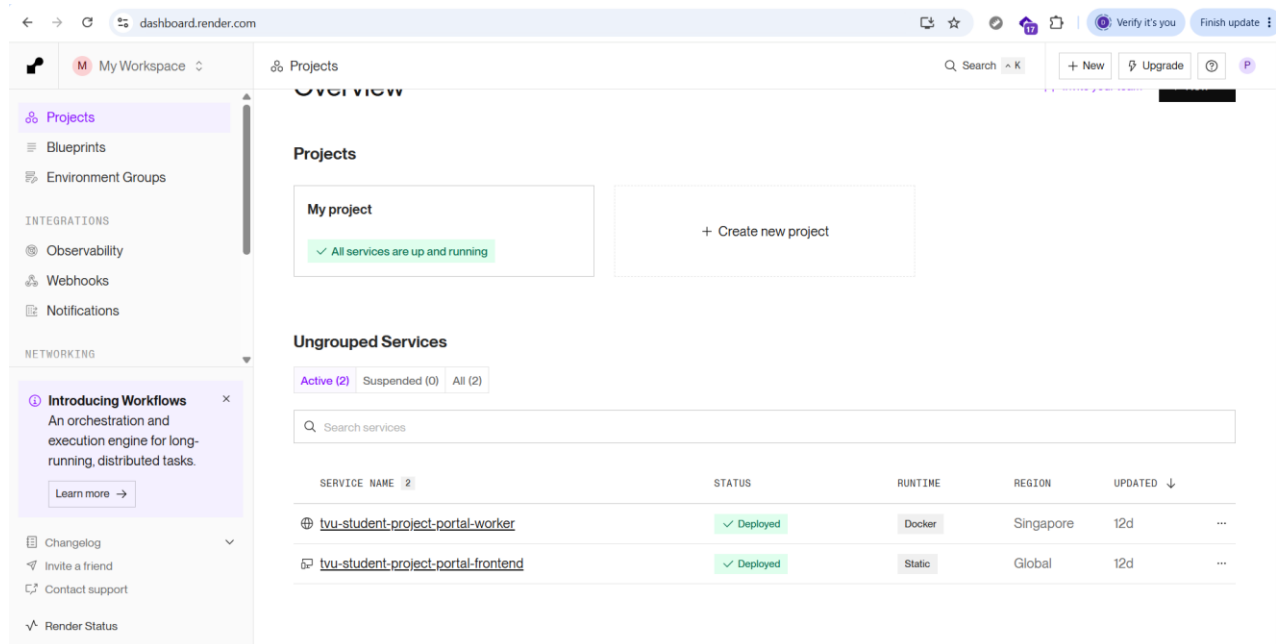
RabbitMQ được cung cấp thông qua CloudAMQP. Backend gửi sự kiện thông báo đến RabbitMQ, trong khi Notification Worker tiếp nhận và xử lý các thông điệp từ queue.

Bảng 5.8. Các thành phần được triển khai trên môi trường cloud

Thành phần	Nền tảng triển khai	Chức năng
Frontend	Render	Cung cấp giao diện website cho người dùng.
Backend API	Render	Xử lý nghiệp vụ và cung cấp RESTful API.
Notification Worker	Render	Xử lý thông báo bất đồng bộ từ RabbitMQ.
SQL Server	MonsterASP	Lưu trữ dữ liệu nghiệp vụ của hệ thống.
RabbitMQ	CloudAMQP	Tiếp nhận và phân phối thông điệp thông báo.
Mã nguồn	GitHub	Lưu trữ mã nguồn và quản lý phiên bản.
CI/CD	GitHub Actions	Kiểm thử, build và kích hoạt triển khai tự động.

Sau khi triển khai, nhóm kiểm tra khả năng truy cập frontend, trạng thái health của backend, tài liệu Swagger và khả năng kết nối của Notification Worker. Các dịch vụ phải sử dụng đúng biến môi trường và không được chứa cấu hình local trong phiên bản production.

Hình 5.20. Trạng thái các dịch vụ được triển khai trên môi trường cloud



5.7.3 Triển khai tự động bằng GitHub Actions

GitHub Actions được sử dụng để tự động kiểm tra mã nguồn và hỗ trợ quá trình triển khai. Workflow được kích hoạt khi mã nguồn được cập nhật lên nhánh chính hoặc khi người phát triển thực hiện chạy thủ công.

Quy trình CI/CD thực hiện các công việc chính gồm:

- Lấy mã nguồn mới nhất từ repository.
- Thiết lập môi trường Node.js.
- Cài đặt thư viện cho backend và frontend.
- Chạy các bài kiểm thử backend.
- Kiểm tra lint của frontend.

- Build ứng dụng frontend.

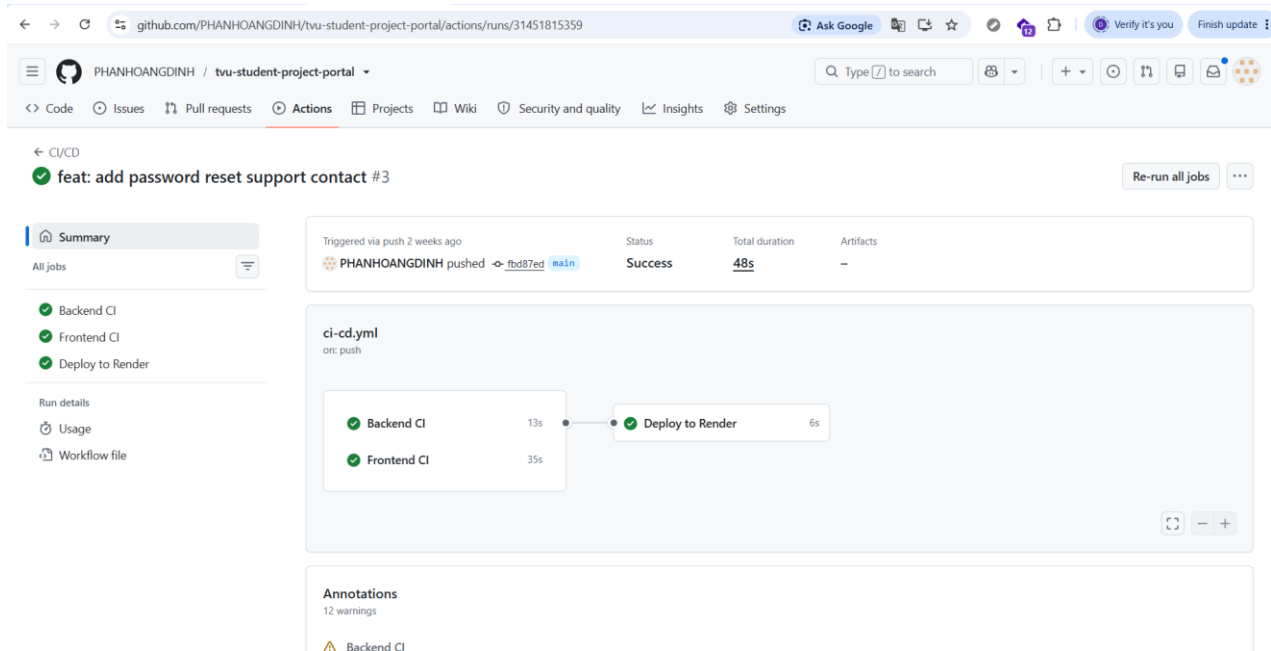
- Kích hoạt triển khai các dịch vụ khi toàn bộ bước kiểm tra thành công.

Đối với backend, workflow chạy bộ kiểm thử tự động để bảo đảm các thay đổi mới không làm ảnh hưởng đến chức năng đã hoàn thành. Tại thời điểm kiểm tra, bộ kiểm thử backend đạt 58/58 trường hợp thành công.

Đối với frontend, workflow thực hiện lint và build. Nếu xuất hiện lỗi cú pháp, lỗi import hoặc lỗi làm quá trình build thất bại, workflow dừng lại và không tiếp tục triển khai.

Các thông tin dùng để kích hoạt triển khai như Deploy Hook hoặc khóa truy cập được lưu trong GitHub Secrets. Những giá trị này không xuất hiện trong file workflow hoặc log công khai.

Khi một bước thất bại, GitHub Actions hiển thị rõ job và bước gây lỗi. Nhóm cần sửa mã nguồn, cập nhật repository và chạy lại workflow trước khi triển khai phiên bản mới.



Hình 5.21. Kết quả thực hiện quy trình CI/CD trên GitHub Actions

5.7.4 Kiểm tra hệ thống sau khi triển khai

Sau khi các dịch vụ được triển khai, nhóm tiến hành kiểm tra lại toàn bộ hệ thống trên môi trường thực tế. Mục tiêu của quá trình này là bảo đảm frontend, backend, cơ sở dữ liệu, RabbitMQ và Notification Worker có thể kết nối và phối hợp đúng với nhau.

Trước tiên, nhóm truy cập địa chỉ frontend để kiểm tra khả năng tải giao diện, điều hướng và đăng nhập. Sau khi đăng nhập bằng từng vai trò, hệ thống được kiểm tra trên các chức năng chính như quản lý người dùng, lớp học phần, đề tài, bài nộp, điểm số và thông báo.

Backend được kiểm tra thông qua health endpoint. Khi dịch vụ hoạt động bình thường, endpoint trả về mã trạng thái HTTP 200. Swagger UI và Swagger JSON cũng được truy cập để xác nhận tài liệu API vẫn hoạt động sau khi triển khai.

Notification Worker được kiểm tra thông qua log của dịch vụ. Worker phải kết nối thành công đến SQL Server và RabbitMQ, đồng thời duy trì trạng thái hoạt động ổn định mà không bị khởi động lại liên tục.

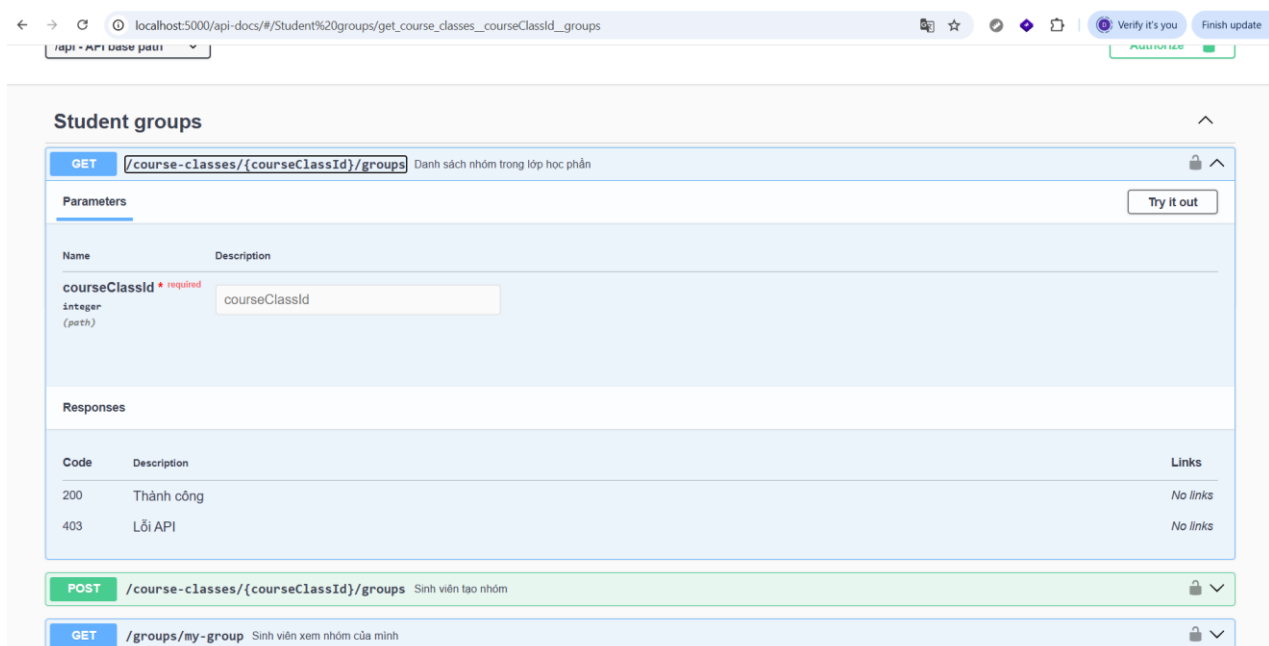
Quá trình kiểm tra sau triển khai tập trung vào các nội dung:

- Frontend có thể truy cập qua trình duyệt.
- Đăng nhập thành công với ba vai trò.
- Frontend gọi đúng địa chỉ backend production.
- Backend health endpoint trả HTTP 200.
- Swagger UI có thể truy cập.
- Backend kết nối được SQL Server.
- Notification Worker kết nối được RabbitMQ.
- Các nghiệp vụ tạo thông báo hoạt động.

- Dữ liệu được lưu và hiển thị đúng sau khi tải lại trang.
- Không xuất hiện lỗi cấu hình local trong môi trường cloud.

Bảng 5.9. Nội dung kiểm tra hệ thống sau triển khai

Thành phần	Nội dung kiểm tra	Kết quả cần đạt
Frontend	Truy cập website và tải giao diện.	Giao diện hiển thị bình thường, không lỗi tài nguyên.
Xác thực	Đăng nhập bằng quản trị viên, giảng viên và sinh viên.	Người dùng được chuyển đến đúng khu vực theo vai trò.
Backend	Gọi health endpoint và các API chính.	Backend trả phản hồi hợp lệ và không bị gián đoạn.
Swagger	Truy cập Swagger UI và Swagger JSON.	Tài liệu API hiển thị đầy đủ.
SQL Server	Kiểm tra kết nối và truy xuất dữ liệu.	Backend và worker truy cập được cơ sở dữ liệu.
RabbitMQ	Kiểm tra exchange, queue và kết nối.	Thông điệp được chuyển đến đúng queue.
Notification Worker	Kiểm tra trạng thái và log xử lý.	Worker hoạt động ổn định và xử lý được thông báo.
Phân quyền	Thử truy cập chức năng ngoài vai trò.	Hệ thống từ chối yêu cầu không hợp lệ.
Dữ liệu	Thêm, cập nhật và tải lại dữ liệu.	Dữ liệu được lưu và hiển thị chính xác.



Hình 5.22. Kết quả kiểm tra hệ thống sau khi triển khai

5.8 Đánh giá kết quả xây dựng hệ thống

Sau quá trình phân tích, thiết kế, xây dựng, kiểm thử và triển khai, website quản lý đồ án sinh viên đã hoàn thành các nhóm chức năng chính theo mục tiêu đề ra.

Hệ thống hỗ trợ ba vai trò gồm quản trị viên, giảng viên và sinh viên. Mỗi vai trò được cung cấp giao diện và chức năng riêng, đồng thời được giới hạn quyền truy cập tại cả frontend và backend.

Quản trị viên có thể quản lý tài khoản, môn học, lớp học phần, giảng viên phụ trách và danh sách sinh viên. Giảng viên có thể quản lý nhóm, đề tài, tiến độ, yêu cầu nộp bài, bài nộp và kết quả đánh giá. Sinh viên có thể theo dõi lớp học phần, nhóm, đề tài, yêu cầu nộp bài, lịch sử nộp, điểm số và thông báo.

Backend được tổ chức theo kiến trúc phân tầng, giúp tách biệt xử lý HTTP, nghiệp vụ và truy vấn dữ liệu. RESTful API được tài liệu hóa bằng Swagger và được bảo vệ bằng JWT cùng cơ chế phân quyền theo vai trò và phạm vi dữ liệu.

Cơ sở dữ liệu SQL Server lưu trữ tập trung dữ liệu của hệ thống. Các quan hệ khóa ngoại và ràng buộc hỗ trợ hạn chế dữ liệu mờ côi, dữ liệu trùng và các trạng thái không hợp lệ.

Chức năng thông báo được xử lý theo cơ chế bất đồng bộ thông qua Transactional Outbox, RabbitMQ và Notification Worker. Cách tổ chức này giúp tách việc tạo thông báo khỏi luồng xử lý chính và hỗ trợ lưu lại sự kiện khi dịch vụ hàng đợi tạm thời gặp sự cố.

Hệ thống đã được đóng gói và kiểm tra bằng Docker Compose. Quy trình GitHub Actions hỗ trợ tự động kiểm thử, kiểm tra frontend và triển khai khi các bước đều thành công.

5.8.1 Kết quả đạt được

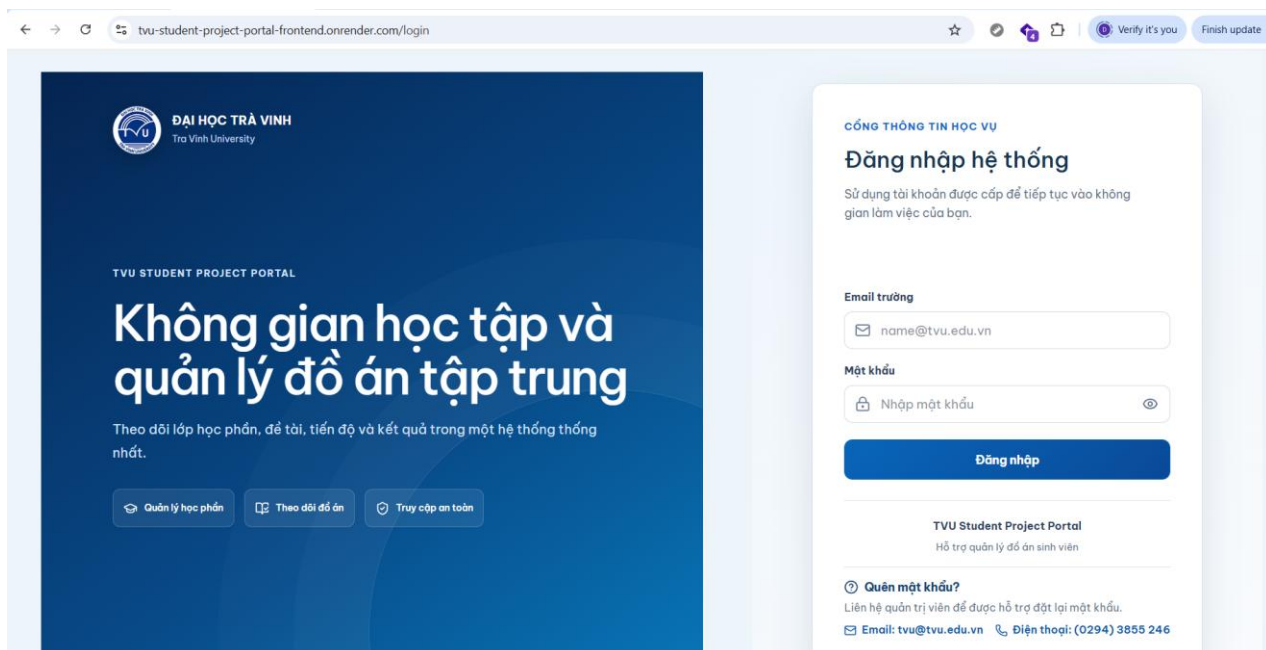
Các kết quả chính đạt được trong quá trình thực hiện đề tài gồm:

- Xây dựng được website quản lý đồ án sinh viên theo mô hình frontend và backend tách biệt.
- Hoàn thiện hệ thống đăng nhập và phân quyền cho ba vai trò.
- Quản lý được tài khoản, môn học, lớp học phần và danh sách sinh viên.
- Quản lý được nhóm sinh viên và đăng ký đề tài.
- Hỗ trợ giảng viên xét duyệt, từ chối hoặc yêu cầu chỉnh sửa đề tài.
- Quản lý được tiến độ và yêu cầu nộp bài.
- Lưu được bài nộp và lịch sử nhiều lần nộp.
- Hỗ trợ chấm điểm, phản hồi, công bố và thu hồi kết quả.
- Xây dựng được chức năng thông báo đã đọc và chưa đọc.
- Tích hợp RabbitMQ, Transactional Outbox và Notification Worker.
- Tài liệu hóa RESTful API bằng Swagger.
- Triển khai được môi trường local bằng Docker Compose.
- Xây dựng được quy trình CI/CD bằng GitHub Actions.

- Bộ kiểm thử backend đạt 58/58 trường hợp thành công tại thời điểm kiểm tra.

Bảng 5.10. Tổng hợp kết quả đạt được của hệ thống

Nhóm nội dung	Kết quả đạt được
Kiến trúc	Frontend và backend tách biệt, backend được tổ chức phân tầng.
Xác thực	Đăng nhập bằng JWT và kiểm tra trạng thái tài khoản.
Phân quyền	Giới hạn chức năng và dữ liệu theo ADMIN, LECTURER và STUDENT.
Quản lý học vụ	Quản lý môn học, lớp học phần, giảng viên và sinh viên.
Quản lý đồ án	Quản lý nhóm, đề tài, tiến độ và quá trình xét duyệt.
Nộp bài	Hỗ trợ yêu cầu nộp, nhiều lần nộp và lưu lịch sử.
Đánh giá	Hỗ trợ điểm, phản hồi, công bố và thu hồi kết quả.
Thông báo	Xử lý bất đồng bộ bằng Outbox, RabbitMQ và Worker.
Cơ sở dữ liệu	Dữ liệu được lưu trong SQL Server với các quan hệ và ràng buộc.
Kiểm thử	Bộ kiểm thử backend đạt 58/58 trường hợp thành công.
Triển khai	Hỗ trợ Docker Compose, GitHub Actions và môi trường cloud.



Hình 5.23. Giao diện tổng thể của hệ thống sau khi hoàn thiện

5.8.2 Hạn chế của hệ thống

Mặc dù đã hoàn thành các chức năng chính, hệ thống vẫn còn một số hạn chế cần tiếp tục cải thiện.

Thứ nhất, hệ thống mới tập trung vào quy trình quản lý đồ án trong phạm vi một đơn vị đào tạo. Một số quy trình đặc thù như phân công hội đồng, tổ chức bảo vệ, quản lý biên bản và tổng hợp điểm hội đồng chưa được triển khai đầy đủ.

Thứ hai, chức năng lưu trữ tệp hiện chủ yếu phục vụ việc ghi nhận đường dẫn và thông tin bài nộp. Hệ thống chưa tích hợp dịch vụ lưu trữ chuyên dụng để quản lý dung lượng, phiên bản tệp và thời gian lưu trữ lâu dài.

Thứ ba, chức năng thông báo mới được triển khai trong website. Hệ thống chưa hỗ trợ gửi thông báo qua email, thiết bị di động hoặc các nền tảng nhắn tin khác.

Thứ tư, cơ chế tìm kiếm hiện chủ yếu dựa trên các trường thông tin cơ bản. Khi số lượng đề tài, bài nộp và người dùng tăng lớn, hệ thống cần bổ sung tìm kiếm nâng cao và tối ưu truy vấn.

Thứ năm, hệ thống chưa có chức năng phân tích dữ liệu chuyên sâu. Các số liệu trên trang tổng quan mới dừng ở mức thống kê cơ bản, chưa hỗ trợ dự đoán tiến độ, phát hiện nhóm có nguy cơ trễ hạn hoặc đánh giá xu hướng kết quả học tập.

Thứ sáu, quá trình kiểm thử hiện tập trung nhiều vào backend và các luồng nghiệp vụ chính. Kiểm thử tự động cho frontend, kiểm thử tải và kiểm thử bảo mật chuyên sâu vẫn cần được bổ sung.

Thứ bảy, hệ thống phụ thuộc vào nhiều dịch vụ gồm SQL Server, RabbitMQ, backend và Notification Worker. Khi một dịch vụ gặp sự cố, một số chức năng có thể bị gián đoạn nếu chưa có cơ chế giám sát và khôi phục tự động phù hợp.

Bảng 5.11. Các hạn chế hiện tại của hệ thống

Hạn chế	Ảnh hưởng	Hướng khắc phục
Chưa quản lý hội đồng bảo vệ	Chưa hỗ trợ đầy đủ giai đoạn nghiệm thu đồ án.	Bổ sung hội đồng, lịch bảo vệ, biên bản và điểm thành viên.
Chưa có lưu trữ tệp chuyên dụng	Khó quản lý dung lượng và phiên bản tệp.	Tích hợp dịch vụ lưu trữ đám mây hoặc object storage.
Thông báo chỉ hiển thị trên website	Người dùng có thể bỏ lỡ thông tin khi không truy cập hệ thống.	Bổ sung email và thông báo đẩy.
Tìm kiếm còn cơ bản	Khó truy xuất khi dữ liệu tăng lớn.	Bổ sung tìm kiếm nâng cao và lập chỉ mục.
Thống kê chưa chuyên sâu	Chưa hỗ trợ dự báo và cảnh báo sớm.	Xây dựng dashboard phân tích và báo cáo trực quan.
Kiểm thử frontend còn hạn chế	Một số lỗi giao diện có thể chưa được phát hiện tự động.	Bổ sung kiểm thử component và end-to-end.
Chưa có giám sát tập trung	Khó phát hiện sớm lỗi dịch	Tích hợp hệ thống logging,

Hạn chế	Ảnh hưởng	Hướng khắc phục
	vụ.	monitoring và cảnh báo.

5.8.3 Hướng phát triển

Trong thời gian tới, hệ thống có thể được mở rộng theo các hướng sau:

- Bổ sung chức năng quản lý hội đồng, lịch bảo vệ và kết quả nghiệm thu đồ án.
- Xây dựng chức năng phân công giảng viên hướng dẫn và phản biện.
- Tích hợp dịch vụ lưu trữ tệp trên cloud để quản lý bài nộp an toàn hơn.
- Hỗ trợ kiểm tra định dạng, dung lượng và loại tệp trước khi nộp.
- Bổ sung chức năng gửi thông báo qua email và thông báo đẩy.
- Phát triển ứng dụng di động để sinh viên và giảng viên theo dõi tiến độ thuận tiện hơn.
- Xây dựng dashboard thống kê theo học kỳ, lớp học phần, giảng viên và trạng thái đồ án.
- Bổ sung cảnh báo nhóm chậm tiến độ hoặc sắp hết hạn nộp bài.
- Tích hợp chức năng xuất báo cáo dưới dạng Excel và PDF.
- Bổ sung nhật ký hoạt động để theo dõi các thao tác quan trọng.
- Tăng cường kiểm thử bảo mật, kiểm thử hiệu năng và kiểm thử tải.
- Xây dựng cơ chế sao lưu, phục hồi dữ liệu và giám sát dịch vụ tự động.

Nghiên cứu tích hợp trí tuệ nhân tạo để hỗ trợ gợi ý đề tài, phân loại nội dung và phát hiện bài nộp có mức độ tương đồng cao.

Việc phát triển các chức năng trên sẽ giúp hệ thống đáp ứng tốt hơn quy trình quản lý đồ án trong thực tế, đồng thời nâng cao khả năng mở rộng và độ tin cậy khi số lượng người dùng tăng lên.

5.9 Tổng kết chương

Trong chương này, nhóm đã trình bày quá trình xây dựng, kiểm thử và triển khai website quản lý đồ án sinh viên.

Backend được phát triển bằng Node.js và ExpressJS theo kiến trúc phân tầng, trong khi frontend được xây dựng bằng ReactJS và Vite. SQL Server được sử dụng để lưu trữ dữ liệu, JWT được áp dụng cho xác thực và phân quyền người dùng.

Hệ thống thông báo được xây dựng theo cơ chế bất đồng bộ với Transactional Outbox, RabbitMQ và Notification Worker. Cơ chế này giúp tách quá trình xử lý thông báo khỏi luồng nghiệp vụ chính và hạn chế nguy cơ mất sự kiện khi dịch vụ trung gian tạm thời gặp sự cố.

Hệ thống đã được kiểm thử trên các nhóm chức năng chính gồm đăng nhập, phân quyền, quản lý người dùng, lớp học phần, nhóm sinh viên, đề tài, bài nộp, chấm điểm và thông báo. Bộ kiểm thử backend đạt 58/58 trường hợp thành công tại thời điểm hoàn thiện.

Các dịch vụ được kiểm tra trong môi trường local bằng Docker Compose và được hỗ trợ triển khai tự động thông qua GitHub Actions. Kết quả cho thấy hệ thống đáp ứng được các yêu cầu chức năng chính đã đề ra.

Bên cạnh những kết quả đạt được, chương cũng chỉ ra các hạn chế về lưu trữ tệp, thông báo đa kênh, kiểm thử frontend, thống kê nâng cao và giám sát dịch vụ. Đây là cơ sở để tiếp tục phát triển và hoàn thiện hệ thống trong tương lai.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đề tài “Xây dựng website quản lý đồ án sinh viên” được thực hiện nhằm hỗ trợ số hóa quá trình quản lý đồ án, từ khâu tổ chức lớp học phần, thành lập nhóm, đăng ký đề tài, theo dõi tiến độ, nộp bài đến chấm điểm và công bố kết quả.

Trong quá trình thực hiện, nhóm đã tiến hành khảo sát yêu cầu, phân tích nghiệp vụ, thiết kế cơ sở dữ liệu, xây dựng giao diện, phát triển RESTful API, kiểm thử và triển khai hệ thống. Kết quả cho thấy hệ thống đã đáp ứng được phần lớn các yêu cầu chức năng đặt ra ban đầu.

Hệ thống được xây dựng theo kiến trúc tách biệt giữa frontend và backend. Frontend sử dụng ReactJS và Vite để cung cấp giao diện cho người dùng. Backend sử dụng Node.js và ExpressJS, được tổ chức theo các tầng Routes, Controllers, Services và Repositories. Cách tổ chức này giúp mã nguồn rõ ràng, giảm sự phụ thuộc giữa các thành phần và thuận tiện cho quá trình bảo trì.

SQL Server được sử dụng để lưu trữ dữ liệu tập trung. Các bảng được thiết kế để quản lý người dùng, môn học, lớp học phần, sinh viên trong lớp, nhóm sinh viên, đề tài, yêu cầu nộp bài, lịch sử nộp, điểm số và thông báo. Các mối quan hệ khóa ngoại hỗ trợ duy trì tính nhất quán giữa các dữ liệu.

Cơ chế đăng nhập được thực hiện bằng JWT. Sau khi xác thực thành công, người dùng được cấp quyền theo một trong ba vai trò gồm quản trị viên, giảng viên và sinh viên. Việc kiểm tra quyền được thực hiện tại backend nhằm hạn chế người dùng truy cập hoặc thay đổi dữ liệu không thuộc phạm vi được phép.

Bên cạnh các chức năng quản lý cơ bản, hệ thống còn sử dụng Transactional Outbox, RabbitMQ và Notification Worker để xử lý thông báo bất đồng bộ. Cơ chế này giúp việc tạo thông báo không làm chậm luồng nghiệp vụ chính và hạn chế nguy cơ mất sự kiện khi dịch vụ trung gian tạm thời không hoạt động.

Hệ thống đã được kiểm thử trên các nhóm chức năng quan trọng như đăng nhập, phân quyền, quản lý người dùng, lớp học phần, nhóm sinh viên, đề tài, bài nộp, chấm điểm và thông báo. Tại thời điểm hoàn thiện, bộ kiểm thử backend đạt 58/58 trường hợp kiểm thử thành công.

Qua quá trình thực hiện đề tài, nhóm đã vận dụng được kiến thức về phân tích và thiết kế hệ thống, lập trình web, thiết kế cơ sở dữ liệu, xây dựng RESTful API, bảo mật JWT, xử lý hàng đợi thông điệp, Docker và CI/CD. Đề tài không chỉ tạo ra một sản phẩm có khả năng ứng dụng mà còn giúp nhóm nâng cao kinh nghiệm phát triển một hệ thống phần mềm có nhiều thành phần phối hợp với nhau.

6.2 Kết quả đạt được

6.2.1 Kết quả về mặt chức năng

Hệ thống đã hoàn thành các nhóm chức năng chính dành cho ba vai trò người dùng.

Đối với quản trị viên, hệ thống hỗ trợ:

- Đăng nhập và quản lý thông tin tài khoản.
- Quản lý danh sách người dùng.
- Quản lý tài khoản giảng viên và sinh viên.
- Quản lý môn học.
- Quản lý lớp học phần.
- Phân công giảng viên phụ trách lớp.
- Quản lý danh sách sinh viên trong từng lớp học phần.
- Theo dõi dữ liệu tổng quan của hệ thống.
- Đối với giảng viên, hệ thống hỗ trợ:
- Xem danh sách lớp học phần được phân công.

- Quản lý các nhóm sinh viên trong lớp.
- Theo dõi thành viên của từng nhóm.
- Quản lý danh sách đề tài.
- Xét duyệt, từ chối hoặc yêu cầu chỉnh sửa đề tài đăng ký.
- Tạo các yêu cầu nộp bài.
- Theo dõi bài nộp và lịch sử nộp của sinh viên.
- Chấm điểm và nhập nhận xét.
- Công bố hoặc thu hồi kết quả đánh giá.
- Theo dõi các thông báo liên quan đến lớp học phần.
- Đối với sinh viên, hệ thống hỗ trợ:
 - Xem thông tin lớp học phần đang tham gia.
 - Xem nhóm và danh sách thành viên.
 - Đăng ký đề tài cho nhóm.
 - Theo dõi trạng thái xét duyệt đề tài.
 - Xem các yêu cầu nộp bài.
 - Thực hiện nộp bài và xem lịch sử các lần nộp.
 - Xem điểm và nhận xét sau khi giảng viên công bố.
 - Nhận và đánh dấu trạng thái thông báo.

Nhờ phân chia chức năng theo vai trò, người dùng có thể tập trung vào các nghiệp vụ liên quan trực tiếp đến nhiệm vụ của mình. Đồng thời, hệ thống hạn chế việc hiển thị các chức năng không cần thiết hoặc không thuộc quyền truy cập của tài khoản.

6.2.2 Kết quả về mặt kỹ thuật

Về mặt kỹ thuật, hệ thống đã đạt được các kết quả sau:

- Xây dựng frontend bằng ReactJS và Vite.
- Xây dựng backend bằng Node.js và ExpressJS.
- Tổ chức backend theo kiến trúc phân tầng.
- Xây dựng RESTful API phục vụ trao đổi dữ liệu giữa frontend và backend.
- Tài liệu hóa API bằng Swagger.
- Sử dụng SQL Server để lưu trữ dữ liệu.
- Xây dựng cơ chế xác thực bằng JWT.
- Áp dụng phân quyền theo vai trò và phạm vi dữ liệu.
- Tích hợp Transactional Outbox để lưu sự kiện cùng với nghiệp vụ.
- Tích hợp RabbitMQ để truyền thông điệp.
- Xây dựng Notification Worker để xử lý thông báo.
- Sử dụng Docker Compose để quản lý các dịch vụ trong môi trường local.
- Sử dụng GitHub để quản lý phiên bản mã nguồn.
- Xây dựng GitHub Actions để hỗ trợ kiểm thử, build và triển khai.
- Hoàn thiện bộ kiểm thử backend với 58 trường hợp kiểm thử thành công.

Việc kết hợp các công nghệ trên giúp hệ thống có cấu trúc tương đối rõ ràng và thuận lợi cho việc mở rộng. Frontend, backend và worker có thể được cập nhật độc lập mà không cần thay đổi toàn bộ hệ thống.

6.2.3 Kết quả về khả năng ứng dụng

Sản phẩm có thể được sử dụng làm nền tảng hỗ trợ quản lý đồ án hoặc báo cáo kết thúc môn tại các khoa, bộ môn và cơ sở đào tạo.

Hệ thống giúp giảm việc quản lý thủ công bằng bảng tính, biểu mẫu giấy hoặc tin nhắn rời rạc. Dữ liệu về lớp học phần, nhóm sinh viên, đề tài, bài nộp và điểm số được lưu trữ tập trung, giúp giảng viên và quản trị viên dễ dàng theo dõi hơn.

Sinh viên có thể chủ động kiểm tra yêu cầu, thời hạn nộp bài, trạng thái đề tài và kết quả đánh giá. Giảng viên có thể quản lý nhiều nhóm và nhiều bài nộp trong cùng một hệ thống, hạn chế tình trạng thất lạc dữ liệu hoặc nhầm lẫn giữa các phiên bản bài nộp.

Mặc dù sản phẩm hiện vẫn còn một số hạn chế, kết quả đạt được cho thấy hệ thống có khả năng tiếp tục phát triển thành một giải pháp quản lý đồ án hoàn chỉnh và có thể áp dụng trong môi trường thực tế.

6.3 Hạn chế của đề tài

Bên cạnh những kết quả đã đạt được, hệ thống vẫn còn một số hạn chế cần tiếp tục nghiên cứu và hoàn thiện.

Trước hết, hệ thống hiện chủ yếu tập trung vào quá trình quản lý đồ án trong phạm vi lớp học phần. Các nghiệp vụ ở giai đoạn bảo vệ như thành lập hội đồng, phân công giảng viên phản biện, lập lịch bảo vệ, quản lý biên bản và tổng hợp điểm hội đồng chưa được xây dựng đầy đủ. Vì vậy, hệ thống chưa thể quản lý trọn vẹn toàn bộ quy trình từ lúc đăng ký đề tài đến khi hoàn thành bảo vệ.

Chức năng quản lý tệp của hệ thống vẫn còn hạn chế. Bài nộp hiện chưa được quản lý bằng một dịch vụ lưu trữ chuyên dụng nên việc kiểm soát dung lượng, định dạng, phiên bản, sao lưu và phân quyền truy cập tệp chưa thật sự tối ưu. Khi số lượng người dùng và bài nộp tăng lên, vấn đề lưu trữ có thể ảnh hưởng đến hiệu năng và khả năng mở rộng.

Hệ thống thông báo hiện chủ yếu hoạt động bên trong website. Người dùng cần đăng nhập mới có thể xem thông báo, trong khi hệ thống chưa hỗ trợ gửi email, thông báo đẩy hoặc thông báo qua ứng dụng di động. Điều này có thể khiến sinh viên hoặc giảng viên bỏ lỡ những thông tin quan trọng nếu không truy cập hệ thống thường xuyên.

Các chức năng thống kê hiện mới dừng ở mức tổng hợp dữ liệu cơ bản. Hệ thống chưa hỗ trợ phân tích tiến độ theo thời gian, so sánh kết quả giữa các lớp học phần, đánh giá tỷ lệ nộp bài đúng hạn hoặc cảnh báo sớm các nhóm có nguy cơ chậm tiến độ.

Quá trình kiểm thử tự động hiện tập trung nhiều vào backend và các nghiệp vụ chính. Frontend chưa có bộ kiểm thử component và kiểm thử end-to-end đầy đủ. Hệ thống cũng chưa được kiểm thử tải với số lượng lớn người dùng truy cập đồng thời nên khả năng hoạt động trong môi trường quy mô lớn vẫn cần được đánh giá thêm.

Ngoài ra, hệ thống sử dụng nhiều thành phần độc lập như frontend, backend, SQL Server, RabbitMQ và Notification Worker. Việc theo dõi log, giám sát trạng thái và cảnh báo sự cố giữa các dịch vụ hiện chưa được thực hiện tập trung. Khi một thành phần gặp lỗi, người quản trị có thể mất nhiều thời gian để xác định nguyên nhân và khôi phục hệ thống.

Giao diện của hệ thống đã đáp ứng các chức năng chính nhưng vẫn cần tiếp tục tối ưu trên nhiều loại thiết bị và kích thước màn hình khác nhau. Một số bảng dữ liệu hoặc biểu mẫu có thể chưa hiển thị thuận tiện trên màn hình nhỏ.

6.4 Hướng phát triển

Từ những hạn chế đã nêu, hệ thống có thể tiếp tục được mở rộng theo nhiều hướng nhằm nâng cao khả năng ứng dụng trong thực tế.

6.4.1 Hoàn thiện quy trình quản lý đồ án

Trong các phiên bản tiếp theo, hệ thống cần bổ sung các chức năng liên quan đến giai đoạn bảo vệ đồ án. Các chức năng có thể bao gồm thành lập hội đồng, phân công giảng viên hướng dẫn và phản biện, lập lịch bảo vệ, quản lý phòng bảo vệ, ghi nhận nhận xét của từng thành viên hội đồng và tổng hợp điểm cuối cùng.

Hệ thống cũng có thể bổ sung chức năng xác định sinh viên đủ điều kiện bảo vệ, lưu biên bản hội đồng và xuất danh sách kết quả theo lớp học phần. Khi hoàn thiện các chức năng này, hệ thống có thể quản lý toàn bộ quá trình từ lúc đăng ký đề tài đến khi công bố kết quả cuối cùng.

6.4.2 Phát triển chức năng lưu trữ tệp

Hệ thống có thể tích hợp dịch vụ lưu trữ đám mây hoặc object storage để quản lý bài nộp hiệu quả hơn. Việc tích hợp này giúp tách phần lưu trữ tệp khỏi backend, giảm tải cho máy chủ và nâng cao khả năng mở rộng.

Các chức năng cần được bổ sung gồm kiểm tra loại tệp, giới hạn dung lượng, lưu phiên bản bài nộp, cho phép tải lại phiên bản cũ, phân quyền truy cập tệp và sao lưu dữ liệu định kỳ. Hệ thống cũng có thể bổ sung cơ chế quét tệp nhằm hạn chế nguy cơ người dùng tải lên nội dung không an toàn.

6.4.3 Mở rộng hệ thống thông báo

Ngoài thông báo hiển thị trên website, hệ thống có thể phát triển thêm các kênh gửi thông báo như email, thông báo đẩy trên trình duyệt hoặc ứng dụng di động.

Các thông báo có thể được gửi khi giảng viên tạo yêu cầu nộp bài mới, khi đề tài được duyệt hoặc từ chối, khi sắp đến hạn nộp bài, khi bài nộp được chấm điểm hoặc khi kết quả được công bố.

Hệ thống cũng có thể cho phép người dùng lựa chọn loại thông báo muốn nhận. RabbitMQ và Notification Worker hiện có thể tiếp tục được mở rộng để xử lý các kênh thông báo mới mà không làm ảnh hưởng đến luồng nghiệp vụ chính.

6.4.4 Xây dựng chức năng thống kê và cảnh báo

Hệ thống có thể phát triển dashboard thống kê chi tiết hơn cho quản trị viên và giảng viên. Các nội dung thống kê có thể bao gồm số lượng đề tài theo trạng thái, tỷ lệ đề tài được duyệt, tỷ lệ nộp bài đúng hạn, phân bố điểm, số lần nộp trung bình và tiến độ của từng nhóm.

Bên cạnh thống kê, hệ thống có thể tự động phát hiện và cảnh báo các trường hợp như nhóm chưa đăng ký đề tài, nhóm chưa cập nhật tiến độ, bài nộp sắp hết hạn hoặc sinh viên thường xuyên nộp bài trễ.

Những thông tin này sẽ giúp giảng viên theo dõi lớp học phần hiệu quả hơn và có biện pháp hỗ trợ sinh viên kịp thời.

6.4.5 Nâng cao khả năng kiểm thử và bảo mật

Trong tương lai, hệ thống cần bổ sung kiểm thử component cho frontend và kiểm thử end-to-end cho các luồng quan trọng như đăng nhập, đăng ký đề tài, nộp bài và công bố điểm.

Hệ thống cũng cần thực hiện kiểm thử tải để đánh giá khả năng xử lý khi có nhiều người dùng truy cập đồng thời. Bên cạnh đó, cần tăng cường kiểm thử bảo mật đối với JWT, API, phân quyền, dữ liệu đầu vào và các thư viện phụ thuộc.

Việc bổ sung các bước kiểm thử trên sẽ giúp phát hiện lỗi sớm, giảm nguy cơ mất an toàn dữ liệu và nâng cao độ ổn định của hệ thống.

6.4.6 Hoàn thiện giám sát và vận hành

Hệ thống có thể tích hợp giải pháp thu thập log tập trung để theo dõi frontend, backend và Notification Worker. Các thông tin như lỗi API, thời gian phản hồi, số lượng thông điệp trong queue và trạng thái kết nối cơ sở dữ liệu cần được theo dõi thường xuyên.

Ngoài ra, hệ thống có thể bổ sung cơ chế cảnh báo khi backend ngừng hoạt động, worker mất kết nối RabbitMQ hoặc số lượng thông điệp tồn đọng vượt quá giới hạn cho phép.

Cơ chế sao lưu và phục hồi dữ liệu cũng cần được xây dựng để hạn chế thiệt hại khi xảy ra lỗi phần cứng, lỗi phần mềm hoặc thao tác nhầm của người quản trị.

6.4.7 Nghiên cứu tích hợp trí tuệ nhân tạo

Trí tuệ nhân tạo có thể được nghiên cứu để hỗ trợ một số nghiệp vụ trong hệ thống như gợi ý đề tài dựa trên chuyên ngành và kỹ năng của sinh viên, phân loại đề tài theo lĩnh vực, phát hiện các đề tài có nội dung tương tự và tóm tắt báo cáo.

Hệ thống cũng có thể sử dụng dữ liệu tiến độ và lịch sử nộp bài để nhận diện các nhóm có nguy cơ không hoàn thành đúng hạn. Một chatbot có thể được tích hợp để giải đáp các câu hỏi thường gặp về quy trình đăng ký đề tài, nộp bài và xem kết quả.

Các chức năng trí tuệ nhân tạo chỉ nên đóng vai trò hỗ trợ. Việc xét duyệt đề tài, đánh giá bài nộp và chấm điểm vẫn cần do giảng viên trực tiếp thực hiện.

6.5 Tổng kết chương

Chương này đã tổng hợp các kết quả đạt được sau quá trình xây dựng website quản lý đồ án sinh viên. Hệ thống đã hoàn thành các chức năng chính dành cho quản trị viên, giảng viên và sinh viên, đồng thời áp dụng các công nghệ như ReactJS, Node.js, ExpressJS, SQL Server, JWT, RabbitMQ, Docker và GitHub Actions.

Hệ thống đã hỗ trợ quản lý người dùng, môn học, lớp học phần, nhóm sinh viên, đề tài, yêu cầu nộp bài, lịch sử nộp, điểm số và thông báo. Cơ chế phân quyền giúp mỗi người dùng chỉ có thể truy cập các chức năng và dữ liệu phù hợp với vai trò của mình.

Kết quả kiểm thử cho thấy các chức năng quan trọng của backend hoạt động ổn định, với 58/58 trường hợp kiểm thử thành công tại thời điểm hoàn thiện. Cơ chế thông báo bất đồng bộ sử dụng Transactional Outbox, RabbitMQ và Notification Worker cũng đã được tích hợp và kiểm tra thành công.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế về quản lý hội đồng bảo vệ, lưu trữ tệp, thông báo đa kênh, kiểm thử frontend, thống kê nâng cao và giám sát dịch vụ. Đây là những nội dung cần tiếp tục nghiên cứu và phát triển trong các phiên bản tiếp theo.

Nhìn chung, đề tài đã đạt được mục tiêu xây dựng một website hỗ trợ quản lý đồ án sinh viên theo hướng tập trung, thuận tiện và có khả năng mở rộng. Sản phẩm có thể tiếp

tục được hoàn thiện để áp dụng vào công tác quản lý đồ án và báo cáo kết thúc môn tại các cơ sở đào tạo.

TÀI LIỆU THAM KHẢO

- [1] React Team, “React Documentation,” *React*. [Online]. Available: <https://react.dev/>. [Accessed: Aug. 24, 2026].
- [2] Vite Team, “Vite Documentation,” *Vite*. [Online]. Available: <https://vite.dev/guide/>. [Accessed: Aug. 24, 2026].
- [3] OpenJS Foundation, “Node.js Documentation,” *Node.js*. [Online]. Available: <https://nodejs.org/api/>. [Accessed: Aug. 24, 2026].
- [4] Express.js Team, “Express.js Documentation,” *Express*. [Online]. Available: <https://expressjs.com/>. [Accessed: Aug. 24, 2026].
- [5] Microsoft, “SQL Server Documentation,” *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/sql/sql-server/>. [Accessed: Aug. 24, 2026].
- [6] Microsoft, “Transact-SQL Reference,” *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/sql/t-sql/language-reference>. [Accessed: Aug. 24, 2026].
- [7] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.
- [8] RabbitMQ Team, “RabbitMQ Documentation,” *RabbitMQ*. [Online]. Available: <https://www.rabbitmq.com/docs>. [Accessed: Aug. 24, 2026].
- [9] Docker Inc., “Docker Documentation,” *Docker Docs*. [Online]. Available: <https://docs.docker.com/>. [Accessed: Aug. 24, 2026].
- [10] Docker Inc., “Docker Compose Documentation,” *Docker Docs*. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Aug. 24, 2026].
- [11] GitHub Inc., “GitHub Actions Documentation,” *GitHub Docs*. [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: Aug. 24, 2026].
- [12] SmartBear Software, “Swagger UI Documentation,” *Swagger*. [Online]. Available: <https://swagger.io/tools/swagger-ui/>. [Accessed: Aug. 24, 2026].
- [13] OpenAPI Initiative, “OpenAPI Specification,” *OpenAPI Initiative*. [Online]. Available: <https://spec.openapis.org/oas/latest.html>. [Accessed: Aug. 24, 2026].
- [14] OWASP Foundation, “OWASP API Security Top 10 – 2023,” *OWASP*. [Online]. Available: <https://owasp.org/API-Security/>. [Accessed: Aug. 24, 2026].
- [15] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley Professional, 2002.

- [16] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.
- [17] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015.
- [18] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

PHỤ LỤC

Hướng dẫn sử dụng cài đặt và chạy ứng dụng, cùng với code của hệ thống điều được đẩy lên Github với liên kết: <https://github.com/PHANHOANGDINH/tvu-student-project-portal>